

Informationssysteme

Sommersemester 2026

Prof. Dr.-Ing. Sebastian Michel
RPTU Kaiserslautern-Landau

sebastian.michel@cs.rptu.de

Grundlagen der Informationssuche

Grundlagen der Informationssuche

- Dokumentverarbeitung, Distanzmaße auf Zeichenketten
- Precision und Recall
- Boolesches Modell, Verarbeitung von Dokumenten
- Vektorraummodell, $TF \cdot IDF$
- Vielfalt und Neuheit, Ähnlichkeit zwischen Dokumenten
- Latent-Semantic-Indexing (LSI)
- Pagerank
- Frequent-Itemset-Mining
- K-Means-Clustering

Rothkäppchen schlug die Augen auf, und als es sah wie die Sonnenstrahlen durch die Bäume hin und her tanzten, und alles voll schöner Blumen stand, dachte es 'wenn ich der Großmutter einen frischen Strauß mitbringe, der wird ihr auch Freude machen; es ist so früh am Tag, daß ich doch zu rechter Zeit ankomme,' lief vom Wege ab in den Wald hinein und suchte Blumen. Und wenn es eine gebrochen hatte, meinte es weiter hinaus stände eine schönere, und lief darnach, und gerieth immer tiefer in den Wald hinein. Der Wolf aber gieng geradeswegs nach dem Haus der Großmutter, und klopfte an die Thüre. Wer ist draußen?' 'Rothkäppchen, das bringt Kuchen und Wein, mach auf.' 'Drück nur auf die Klinke,' rief die Großmutter, 'ich bin zu schwach und kann nicht aufstehen.' Der Wolf drückte auf die Klinke, die Thüre sprang auf und er ging, ohne ein Wort zu sprechen, gerade zum Bett der Großmutter und verschluckte sie. Dann that er ihre Kleider an, setzte ihre Haube auf, legte sich in ihr Bett und zog die Vorhänge vor. Rothkäppchen aber war nach den Blumen herum gelaufen, und als es so viel zusammen hatte, daß es keine mehr tragen konnte, fiel ihm die Großmutter wieder ein und es machte sich auf den Weg zu ihr. Es wunderte sich daß die Thüre aufstand, und wie es in die Stube trat, so kam es ihm so seltsam darin vor, daß es dachte 'ei, du mein

Fragen

- Was ist ein Dokument?
- Was sind gültige “Treffer” ?
- Was ist mit Dingen wie Tippfehlern?
- Ist jedes Wort gleich wichtig?
- Ist es relevant, ob ein Wort mehrfach auftritt?
- Sind sehr lange Dokumente ein Problem?
- Was ist mit dem Dokument selbst, sind z.B. manche Webseiten glaubwürdiger oder besser als andere?

Was ist ein Dokument?

Falls ein Dokument nicht in einem einfachen Textformat (z.B. ASCII, UTF-8) vorliegt muss es konvertiert werden (z.B. aus PDF, Word, HTML)

Was ist die gewünschte Granularität der Daten/Dokumente?

- Soll das Buch “Grimms Kinder- und Hausmärchen” als ein Dokument betrachtet werden, oder sollte jedes Märchen als ein separates Dokument angesehen werden?
- Sollen HTML Seiten, die jeweils nur einen Paragraph eines Dokuments darstellen aneinanderhängt werden?

Tokenisierung (Tokenization)

- Beim Tokenisieren wird Text in einzelne Tokens aufgeteilt.
- Wir betrachten ganz allgemein Terme, normalerweise einfache Worte, die möglicherweise noch normalisiert sind.
- Beispiel:

Aber	Großmutter	was	hast	du	für	ein
entsetzlich	großes	Maul	!			

Mögliche Probleme

- can't $\Rightarrow$ can t
- was ist mit Web- oder Emailadressen wie <http://www.cs.uni-kl.de> oder support@ebay.com?
- Los Angeles
- Lebensversicherungsgesellschaftsangestellter
- Oder Sprachen, die gar keine Leerzeichen haben (viele Ostasiatische Sprachen)

Stopwörter

- **Stopwörter sind sehr häufig auftretende Worte, die somit kaum oder keine Informationen enthalten.**
- Daher werden diese oftmals komplett ignoriert.
- **Beispiele:** ein, eine, der, die das, von, aus
- Gegeben entweder durch explizite Liste oder implizit durch z.B. Betrachtung der häufigsten k Worte der Kollektion)

Stammformreduktion (Stemming)

- **Verschiedene Variationen eines Wortes können zusammen betrachtet werden, z.B. Plural, Adverbien, verschiedene Zeiten eines Verbs.**
- **Beispiele*:** kaufen → kauf, käufer → kauf, kategorie → kategori, consistency → consist, knives → knife, consolidating → consolid.
- Eine solche **“Grundform”** eines Wortes ist nicht notwendigerweise ein richtiges (korrektes) Wort.
- Idealerweise werden Variationen des gleichen Wortes auf eine einzige Grundform reduziert.

* <http://snowball.tartarus.org/algorithms/english/stemmer.html>

Andere Ideen

- Behandlung von **diakritischen Zeichen**, z.B. ü, á, ç, ø
 - Anfragen enthalten oftmals diese Zeichen gar nicht, z.B. les miserables vs. les misérables
 - Manchmal werden solche Buchstaben auch umgeschrieben, wie für ⇒ fuer
- **Groß-/ Kleinschreibung**, z.B. United States vs. united states

Tippfehler etc – Distanzmaße auf Zeichenketten

- Oft enthalten Texte aber auch die von Benutzern gestellten Anfragen **Tippfehler**.
- Wenn ein Benutzer ein dem System unbekanntes Wort eingibt, so kann anstelle dessen mit einem korrekt geschriebenen Wort, welches möglichst nah am inkorrekten Wort liegt, weitergearbeitet werden.
- Oder man kann Vorschläge zu ähnlichen Suchbegriffen erhalten, im Sinne von “Meinten Sie”
- Was für dazu brauchen ist ein **Distanzmaß auf Zeichenketten**
- Diese sollten z.B. berücksichtigen:
 - **Extra Zeichen**, wie in Haus vs. Hauus
 - **Vergessene Zeichen**, wie in Haus vs. Hus
 - **Fehlerhafte Zeichen**, wie in Haus vs. Hius

Hamming-Editier-Distanz

- **Die Hamming-Editier-Distanz** beschreibt die Anzahl an Positionen, an denen die beide Strings x und y verschieden sind.
- **Strings unterschiedlicher Länge** werden verglichen in dem der kürzere String mit “NULL” aufgefüllt wird (z.B. uni vs. universell $\rightarrow$ uni_____ vs. universell)

Beispiele:

- $d(\text{car}, \text{cat})=1$
- $d(\text{house}, \text{hot})=3$
- $d(\text{house}, \text{hooose})=4$

Dreiecksungleichung

- Für Strings x, y, z und Distanzmaß d gilt

$$d(x, z) \leq d(x, y) + d(y, z)$$

Längste Gemeinsame Teilsequenz

Eine **Teilsequenz** ist eine Sequenz, die von einer anderen Sequenz durch weglassen von Zeichen aber unter Beibehaltung der Reihenfolge der Zeichen entsteht.

Eine **gemeinsame** Teilsequenz zweier Strings x und y ist ein String s , so dass **alle Zeichen von s in x und y in der gleichen Reihenfolge auftreten** (aber eben **nicht notwendigerweise zusammenhängend**).

Längste-Gemeinsame-Teilsequenz-Distanz

$$d(x,y) = \max(|x|, |y|) - \max_{s \in S(x,y)} |s|$$

wobei $S(x,y)$ die Menge aller gemeinsamer Teilsequenzen von x und y ist.

Beispiele:

- $d(\text{house}, \text{huse}) = 1$

Längste Gemeinsame Teilsequenz: Beispiel

house vs. huse

Alle Teilsequenzen von house: {h, o, u, s, e, ho, hu, hs, he, ou, os, oe, us, ue, se, hou, hos, hoe, hus, hue, hse, ous, oue, ose, use, hous, houe, hose, huse, ouse, house}

Alle Teilsequenzen von huse: {h, u, s, e, hu, hs, he, us, ue, se, hus, hue, hse, use, huse}

Gemeinsame Teilsequenzen: {h, u, s, e, hu, hs, he, us, ue, se, hus, hue, hse, use, huse}

Die längste dieser Teilsequenzen ist huse mit Länge 4.

Also ist $d(\text{house}, \text{huse}) = 5 - 4 = 1$

Levenshtein-Editier-Distanz

- Die **Levenshtein-Editier-Distanz** zwischen zwei Strings x und y ist die minimale Anzahl von **Änderungsoperationen** (**insert**, **replace**, **delete**), die benötigt werden, um x in y zu transformieren.
- Die **minimale Anzahl an Operationen** $m[i,j]$, um die Präfix-Zeichenkette $x[1:i]$ in $y[1:j]$ zu transformieren ist definiert durch

$$m[i,j] = \min \begin{cases} m[i-1,j-1] + (x[i] = y[j]?0:1) & \text{(ersetze } x[i]) \\ m[i-1,j] + 1 & \text{(lösche } x[i]) \\ m[i,j-1] + 1 & \text{(füge ein } y[i]) \end{cases}$$

Beispiele:

- $d(\text{hooouse}, \text{house}) = 1$
- $d(\text{house}, \text{rose}) = 2$
- $d(\text{house}, \text{hot}) = 3$

Die Änderungsoperationen können auch gewichtet werden, z.B. nach Zeichen. Oder man kann unterschiedlich gewichtete Penalties für die Operationen haben.

Levenshtein-Editier-Distanz

Die **Levenshtein-Editier-Distanz** zwischen zwei Zeichenketten x und y entspricht $m[|x|, |y|]$ und kann mittels **dynamischer Programmierung** in $O(|x| |y|)$ berechnet werden.

Anmerkung: Die Kosten für Löschen, Einfügen und Ersetzen können auch unterschiedlich sein.

Initialisierung

$y[j]$

	-	s	c	h	e	i	n
-	0	1	2	3	4	5	6
s	1						
t	2						
e	3						
i	4						
n	5						

$x[i]$

	-	s	c	h	e	i	n
-	0	1	2	3	4	5	6
s	1	?					
t	2						
e	3						
i	4						
n	5						

Was ist die Distanz von $x[1:1]='s'$ zu $y[1:1]='s'$?

Drei Möglichkeiten:

- ↖ Distanz von " zu " plus Kosten 0 (weil $y[1]=x[1]$) = $0+0=0$
- ↑ Distanz von " zu 's' plus Kosten 1 = $m[0,1]+1 = 2$
- ← Distanz von 's' zu " plus Kosten 1 = $m[1,0]+1 = 2$

$$m[i,j] = \min \begin{cases} m[i-1,j-1] + (x[i] = y[j]?0:1) & \text{(ersetze } x[i]) \quad \swarrow \\ m[i-1,j] + 1 & \text{(lösche } x[i]) \quad \uparrow \\ m[i,j-1] + 1 & \text{(füge ein } y[i]) \quad \leftarrow \end{cases}$$

	-	s	c	h	e	i	n
-	0	1	2	3	4	5	6
s	1	0	?				
t	2						
e	3						
i	4						
n	5						

Was ist die Distanz von $x[1:1]='s'$ zu $y[1:2]='sc'$?

Drei Möglichkeiten:

- ↖ Distanz von " und 's' (also $m[0,1]=1$) plus Kosten 1 für Ersetzung von 'c' durch 's'
- ↑ Distanz von " und 'sc' (also 2) plus Kosten 1 für Löschen von 's'.
- ← Distanz von 's' und 's' (also 0) plus Kosten für Einfügen von 'c'.

$$m[i,j] = \min \begin{cases} m[i-1,j-1] + (x[i] = y[j]?0:1) & \text{(ersetze } x[i]) \quad \swarrow \\ m[i-1,j] + 1 & \text{(lösche } x[i]) \quad \uparrow \\ m[i,j-1] + 1 & \text{(füge ein } y[i]) \quad \leftarrow \end{cases}$$

	-	s	c	h	e	i	n
-	0	1	2	3	4	5	6
s	1	0	1				
t	2						
e	3						
i	4						
n	5						

	-	s	c	h	e	i	n
-	0	1	2	3	4	5	6
s	1	0	1	2	3	4	5
t	2	1	1	2	3	4	5
e	3	2	2	2	2	3	4
i	4	3	3	3	3	2	3
n	5	4	4	4	4	3	2

Die Levenshtein-Distanz zwischen “stein” und “schein” ist also 2.

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0			
a	2				
t	3				
z	4				
e	5				

Gibt nichts zu tun

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1		
a	2				
t	3				
z	4				
e	5				

Einfügen von a

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	
a	2				
t	3				
z	4				
e	5				

Einfügen von t

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2				
t	3				
z	4				
e	5				

Einfügen von e

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1			
t	3				
z	4				
e	5				

Löschen

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0		
t	3				
z	4				
e	5				

Gibt nichts zu tun

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	
t	3				
z	4				
e	5				

Einfügen von t

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3				
z	4				
e	5				

Einfügen von e

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2			
z	4				
e	5				

Löschen

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1		
z	4				
e	5				

Löschen

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	
z	4				
e	5				

Gibt nichts zu tun

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	1
z	4				
e	5				

Einfügen von e

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	1
z	4	3			
e	5				

Löschen

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	1
z	4	3	2		
e	5				

Löschen

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	1
z	4	3	2	1	
e	5				

Löschen

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	1
z	4	3	2	1	1
e	5				

Ersetzen z durch e

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	1
z	4	3	2	1	1
e	5	4			

Löschen

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	1
z	4	3	2	1	1
e	5	4	3		

Löschen

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	1
z	4	3	2	1	1
e	5	4	3	2	

Löschen

Weiteres Beispiel

	-	k	a	t	e
-	0	1	2	3	4
k	1	0	1	2	3
a	2	1	0	1	2
t	3	2	1	0	1
z	4	3	2	1	1
e	5	4	3	2	1

Gibt nichts zu tun

C++ Code

```
1  int s1_size = s1.size();
2  int s2_size = s2.size();
3  int m[s1_size+1][s2_size+1];
4  //init
5  for (int i=0; i<=s1_size; i++) m[i][0] = i;
6
7  for (int j=0; j<=s2_size; j++) m[0][j] = j;
8
9  for (int i=1; i<=s1_size; i++) {
10     for (int j=1; j<=s2_size; j++) {
11         int min1 = m[i-1][j-1] + ((s1[i-1]==s2[j-1])?0:1);
12         //replace
13         int min2 = m[i-1][j] + 1; //delete
14         int min3 = m[i][j-1] + 1; //insert
15         m[i][j] = min(min1, min(min2, min3));
16     }
17 }
```

Retrieval Methoden

- **Das Boolesche Modell**
- **TF*IDF**
- **Vektorraummodell**
- **Berücksichtigung von Vielfalt und Neuheit**
- **Dokumentenähnlichkeit, Shingling**
- **Latent-Semantic-Indexing (LSI)**

Boolesches Modell: Term-Dokument-Matrix (Incidence-Matrix)

	Froschk.	Hänsel& G.	Rapunzel	Rotkä.	7 Zwerge
Vater	1	1	0	0	1
Mutter	0	1	1	1	0
König/in	1	0	1	0	1
Zwerge	0	0	0	0	1
Königstochter	1	0	0	0	1
Wolf	0	0	0	1	0
Gold	0	0	1	0	1

Ein einfaches Boolesches Modell zur Suche

- Variablen beschreiben ob ein Wort in einem Dokument auftritt
- **Boolesche Operatoren AND, OR, und NOT**
- Anfragen können beliebig komplizierte (verschachtelte) Konstrukte aus diesen Operatoren sein.
- Z.B.
 - Haus **AND** Hexe **AND** Wald
 - Wolf **AND** Haus **AND NOT** Schweinchen
 - Frosch **OR** Gold

Das Ergebnis ist eine **nicht geordnete Menge von Dokumenten, die diese Anfrage erfüllen.**

Wieso ist das nicht gut?

Im Gegensatz zum Booleschen Modell sehen wir unten die **Häufigkeit** mit der die einzelnen Terme in den Dokumenten auftreten.

	Froschk.	Hänsel& G.	Rapunzel	Rotkä.	7 Zwerge
Vater	3	7	0	0	1
Mutter	0	2	1	2	0
König/in	9	0	5	0	19
Zwerge	0	0	0	0	10
Königstochter	6	0	0	0	1
Wolf	0	0	0	6	0
Gold	0	0	1	0	2

Vektorraummodell

Beobachtung

- Boolesches Modell hat keine (oder nur sehr einfache) Möglichkeit Resultate nach Güte zu Ordnen.
- Diese Einschränkung ist insbesondere bei großen Datenmengen (Google!) ein Problem.

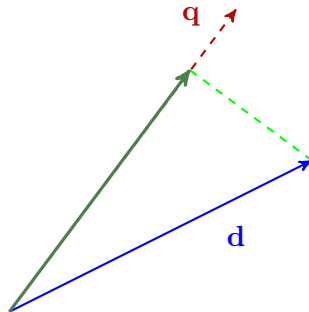
Vektorraummodell

- Im Vektorraummodell wird **jedes Dokument als v -dimensionaler Vektor** betrachtet, wobei $v = |V|$ die Anzahl an Worten ist (V =Vokabular), d.h. eine Dimension pro Wort.
- Beispiel aus den Daten von zuvor: $d_7 \text{ Zwerge} = \langle 1, 0, 19, 10, 1, 0, 2 \rangle$
- Ebenso wird die **Anfrage als Vektor** ausgedrückt, z.B. die Anfrage {Zwerge, Gold} entspricht dem Vektor $q = \langle 0, 0, 0, 1, 0, 0, 1 \rangle$

Vektorraummodell: Kosinus-Ähnlichkeit

Die **Kosinus-Ähnlichkeit** zwischen zwei Vektoren $\mathbf{q}$ und $\mathbf{d}$ ist der **Kosinus des Winkels zwischen den Vektoren**:

$$\begin{aligned} \text{sim}(\mathbf{q}, \mathbf{d}) &= \frac{\mathbf{q} \cdot \mathbf{d}}{||\mathbf{q}|| \ ||\mathbf{d}||} \\ &= \frac{\sum_{i=1}^{|V|} \mathbf{q}_i \mathbf{d}_i}{\sqrt{\sum_{i=1}^{|V|} \mathbf{q}_i^2} \sqrt{\sum_{i=1}^{|V|} \mathbf{d}_i^2}} \\ &= \frac{\mathbf{q}}{||\mathbf{q}||} \frac{\mathbf{d}}{||\mathbf{d}||} \end{aligned}$$



Mit $\mathbf{d} = \langle 1, 0, 19, 10, 1, 0, 2 \rangle$ und $\mathbf{q} = \langle 0, 0, 0, 1, 0, 0, 1 \rangle$ haben wir $\mathbf{q} \cdot \mathbf{d} = 12$, $||\mathbf{d}|| = \sqrt{467}$ und $||\mathbf{q}|| = \sqrt{2}$ und erhalten $\text{sim}(\mathbf{q}, \mathbf{d}) = 0.392652$.

TF*IDF

- Wie können die einzelnen Einträge q_t und d_t für einen Term t berechnet werden?
- Im Booleschen-Modell hätten wir Einträge aus $\{0,1\}$
- Im Beispiel zuvor haben wir die Anzahl des Auftretens eines Terms benutzt, die sogenannte Termfrequenz.
- **Termfrequenz** $tf_{t,d}$ als die Anzahl des Auftretens von Term t in Dokument d
- **Termfrequenz sollte gewichtet werden und nicht direkt benutzt werden. Wieso?**

TF*IDF

- Wie können die einzelnen Einträge q_t und d_t für einen Term t berechnet werden?
- Im Booleschen-Modell hätten wir Einträge aus $\{0,1\}$
- Im Beispiel zuvor haben wir die Anzahl des Auftretens eines Terms benutzt, die sogenannte Termfrequenz.
- **Termfrequenz** $tf_{t,d}$ als die Anzahl des Auftretens von Term t in Dokument d
- **Termfrequenz sollte gewichtet werden und nicht direkt benutzt werden. Wieso?**
- **Dokumentfrequenz** df_t als die Anzahl der Dokumente, in denen Term t auftritt
- **Inverse Dokumentfrequenz** idf_t als

$$idf_t = \frac{|D|}{df_t}$$

wobei $|D|$ die Anzahl der Dokumente in der Kollektion sind.

TF*IDF

- Das *tf.idf* **Gewicht** von Term t in Dokument d ist dann definiert als

$$tf.idf_{t,d} = tf_{t,d} \times idf_t$$

- Beobachtungen: $tf.idf_{t,d}$ ist
 - größer falls t oft in d auftritt**
 - kleiner falls t nicht oft in d auftritt und/oder t in vielen Dokumenten auftritt**
- Nun können wir im **Vektorraummodell** die einzelnen Komponenten der Vektoren bestimmen:

$$\mathbf{d}_t = tf.idf_{t,d} \quad \mathbf{q}_t = tf.idf_{t,q}$$

- Eine einfachere Möglichkeit die Güte von Dokument d zu berechnen ist durch eine einfache **Summe der einzelnen $tf.idf$ Werte**:

$$score(\mathbf{q}, \mathbf{d}) = \sum_{t \in \mathbf{q}} tf.idf_{t,d}$$

Dämpfung, Längennormalisierung etc.

Es gibt verschiedene Variationen des klassischen tf.idf Maßes.

Logarithmische Dämpfung der inversen Dokumentfrequenz

$$idf_t = \log \frac{|D|}{df_t}$$

verhindert, dass zu sehr exotische Terme zu viel Gewicht erhalten.

Sublineare Skalierung der Termfrequenz

$$wf_{t,d} = \begin{cases} 1 + \log tf_{t,d} & \text{falls } tf_{t,d} > 0 \\ 0 & \text{sonst} \end{cases}$$

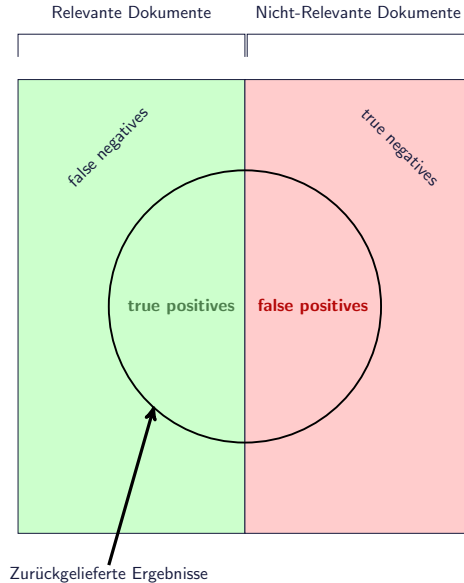
Längennormalisierung und max-tf Normalisierung

$$rtf_{t,d} = \frac{tf_{t,d}}{\sum_{v \in \mathbf{d}} tf_{v,d}} \qquad ntf_{t,d} = \frac{tf_{t,d}}{\max_{v \in \mathbf{d}} tf_{v,d}}$$

verhindern, dass sehr lange Dokumente zu sehr favorisiert werden.

Bewertung der Resultatgüte

- Es gibt also verschiedene Möglichkeiten zu berechnen wie gut bzw. wie schlecht ein Dokument als Ergebnis einer Anfrage geeignet ist (und es gibt noch zahlreiche andere, die hier nicht betrachtet wurden).
- Wie kann nun gesagt werden, dass eine Möglichkeit besser als eine andere ist?



Klassifizierung der Ergebnisse

Ein System gibt eine Menge von Dokumenten als Antwort zu einer Anfrage zurück. Wir unterscheiden zwischen den folgenden vier Arten von Dokumenten:

	relevant	nicht relevant
zurückgegeben	true positives (tp)	false positives (fp)
nicht zurückgegeben	false negatives (fn)	true negatives (tn)

Was sollte ein gutes Informationssystem erfüllen?

- Viele (alle) der zurückgelieferten Dokumente sollten relevant sein, d.h. viele true positives und wenig (keine) false positives.
- Viele (alle) der relevanten Dokumente sollten zurückgeliefert werden, d.h. wenig (keine) false negatives.

Präzision (Precision) und Ausbeute (Recall)

Precision P ist der Anteil der relevanten Dokumente in den zurückgegebenen Dokumenten

$$P = \frac{tp}{tp + fp}$$

Recall R ist der Anteil der relevanten zurückgegebenen Dokumente an allen relevanten Dokumenten

$$R = \frac{tp}{tp + fn}$$

Was ist wichtiger, Precision oder Recall?

F-Measure

F-measure kombiniert Precision und Recall

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

wobei mit dem Parameter β zwischen Precision und Recall gewichtet werden kann.

- $\beta = 1$ ist balanciert
- $\beta < 1$ höherer Einfluss von Precision
- $\beta > 1$ höherer Einfluss von Recall

Average-Precision und Mean-Average-Precision

- Precision, Recall und auch das F-Maß ignorieren die Ordnung der Resultate.
- **Average Precision (AP)**
 - Sei $\{d_1, \dots, d_m\}$ die Menge der relevanten Dokumente der Anfrage q
 - Sei R_k die Menge der geordneten Ergebnisse für Anfrage von oben bis zum relevanten Ergebnis d_k .

$$AP(q) = \frac{1}{m} \sum_{d_k \in \{d_1, \dots, d_m\}} \text{Precision}(R_k)$$

Ist ein relevantes Dokument d_k nicht zurückgeliefert worden, wird $\text{Precision}(R_k)$ als 0 angenommen.

- Für eine Menge Q von Anfrage kann **Mean-Average-Precision (MAP)** berechnet werden durch

$$MAP(Q) = \frac{1}{|Q|} \sum_{q \in Q} AP(q)$$

Average Precision: Beispiel

Rang	Relevant?
1	ja
2	ja
3	nein
4	nein
5	ja
6	nein
7	nein
8	nein
9	nein
10	ja
11	nein
12	nein
13	ja
14	nein
15	nein

d_i sei das Dokument auf Rang i .

Wir haben also die Menge der relevanten Dokumente $\{d_1, d_2, d_5, d_{10}, d_{13}\}$

- $R_1 = \{d_1\}$ und $\text{Precision}(R_1) = 1/1 = 1$
- $R_2 = \{d_1, d_2\}$ und $\text{Precision}(R_2) = 2/2 = 1$
- $R_5 = \{d_1, d_2, d_3, d_4, d_5\}$ und $\text{Precision}(R_5) = 3/5 = 0.6$
- $R_{10} = \{d_1, d_2, d_3, d_4, d_5, d_6, d_7, d_8, d_9, d_{10}\}$ und $\text{Precision}(R_{10}) = 4/10 = 0.4$
- $R_{13} = \{d_1, d_2, d_3, d_4, d_5, d_6, d_7, d_8, d_9, d_{10}, d_{11}, d_{12}, d_{13}\}$ und $\text{Precision}(R_{13}) = 5/13 \approx 0.384$

Wir haben also eine Average-Precision von

$$AP = (1+1+3/5+4/10+5/13)/5 = 44/65 \approx 0.677$$

Precision@k

- Es ist unrealistisch anzunehmen, dass Anwender tatsächlich die gesamte Ergebnisliste anschauen.
- Vielmehr werden Anwender vielleicht nur die top-10 oder top-20 Ergebnisse betrachten, wie z.B. in der Websuche üblich.
- **Precision@k (=P@k) ist die Precision, die in den ersten k (also top-k) Ergebnissen erreicht wird.**

Precision@k: Beispiel

Rang	Relevant?
1	ja
2	ja
3	nein
4	nein
5	ja
6	nein
7	nein
8	nein
9	nein
10	ja
11	nein
12	nein
13	ja
14	nein
15	nein

- $\text{Precision@1} = 1/1 = 1$
- $\text{Precision@5} = 3/5 = 0.6$
- $\text{Precision@10} = 4/10 = 0.4$
- $\text{Precision@15} = 5/15 = 1/3 = 0.\bar{3}$

Vielfalt und Neuheit

- Die **betrachteten Methoden** (Boolesches-Modell und tf.idf) gehen davon aus, dass die **Relevanz eines Dokuments unabhängig von der Relevanz anderer Dokumente** ist.
- **In der Realität ist diese Annahme allerdings problematisch**, da zurückgegebene Ergebnisse gleiche oder sehr ähnliche Informationen enthalten können, sogar Duplikate sein können.
- **Idee:** Wir versuchen, dass jedes weitere Ergebnis möglichst verschieden ist zu den zuvor gelesenen Dokumenten (angenommen Dokumente sind sortiert nach Güte und der Anwender liest von oben nach unten).

Maximale Marginale Relevanz (MMR)

Das nächste zurückzugebende Dokument d_i soll relevant zur Anfrage q sein, aber auch verschieden zu den bislang zurückgegebenen Dokumenten $d_1, \dots, d_{i-1}$

$$\operatorname{argmax}_{d_i \in D} (\lambda \operatorname{sim}(q, d_i) - (1 - \lambda) \max_{d_j: 1 \leq j < i} \operatorname{sim}(d_i, d_j))$$

mit Tuning-Parameter λ und Ähnlichkeitsmaß sim .

Ergebnisliste nach Güte

	$\operatorname{sim}(q, d_1) = 0.9$
	$\operatorname{sim}(q, d_2) = 0.8$
	$\operatorname{sim}(q, d_3) = 0.7$
	$\operatorname{sim}(q, d_4) = 0.6$
	$\operatorname{sim}(q, d_5) = 0.5$

$$\operatorname{sim}(d, d') = \begin{cases} 1.0 & \text{gleiche Farbe} \\ 0.0 & \text{sonst} \end{cases}$$

$$\lambda = 0.5$$

Finale Ergebnisse

	$\operatorname{mmr}(q, d_1) = 0.45$
	$\operatorname{mmr}(q, d_3) = 0.35$
	$\operatorname{mmr}(q, d_5) = 0.25$
	$\operatorname{mmr}(q, d_2) = -0.10$
	$\operatorname{mmr}(q, d_4) = -0.2$

Ähnlichkeitsmaß Jaccard-Koeffizient

Bekanntes Maß zur **Berechnung von Ähnlichkeiten zweier Mengen**.

Gegeben zwei Mengen A und B von Elementen. Der Jaccard-Koeffizient $J(A,B)$ dieser Mengen ist

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Zur Berechnung der **Ähnlichkeit von Text-Dokumenten** zwecks Duplikaterkennung werden in der Regel nicht die einzelnen Worte, sondern Wortsequenzen, sogenannte **k-grams** oder **k-shingles**, betrachtet.

Wieso ist dies i.d.R. aussagekräftiger?

Shingling

- Shingling berechnet für ein Dokument d eine **Menge von Sequenzen jeweils bestehend k Worten**, die in dem Dokument **nebeneinander** auftreten.
- Solch eine Sequenz wird **k-shingle** oder auch **k-gram** genannt.
- **Beispiel:** Für ein Dokument $[a\ b\ b\ c]$, wobei a , b und c Worte sind, haben wir die folgende Menge an 2-shingles, die dieses Dokument beschreiben: $\{ab, bb, bc\}$.
- Die k-shingles eines Dokuments d_i werden mit $S(d_i)$ bezeichnet.

Shingling

- Shingling berechnet für ein Dokument d eine **Menge von Sequenzen jeweils bestehend k Worten**, die in dem Dokument **nebeneinander** auftreten.
- Solch eine Sequenz wird **k-shingle** oder auch **k-gram** genannt.
- **Beispiel:** Für ein Dokument $[a\ b\ b\ c]$, wobei a , b und c Worte sind, haben wir die folgende Menge an 2-shingles, die dieses Dokument beschreiben: $\{ab, bb, bc\}$.
- Die k-shingles eines Dokuments d_i werden mit $S(d_i)$ bezeichnet.

Um die **Ähnlichkeit zweier Dokumente** d_1 und d_2 zu berechnen kann nun der Jaccard-Koeffizient zwischen den Mengen der Shingles von d_1 und d_2 berechnet werden:

$$\text{sim}(d_1, d_2) = \frac{|S(d_1) \cap S(d_2)|}{|S(d_1) \cup S(d_2)|}$$

Latent-Semantic-Indexing (LSI)

Beobachtung:

Bislang betrachtete Ansätze (z.B. TF*IDF) können nicht mit **Synonymie** (z.B. Auto und PKW) und **Polysemie** (z.B. Java) umgehen.

- Das gemeinsame Auftreten von Worten kann hier helfen, z.B.
 - Auto und PKW treten in Dokumenten über Abgase oder Parkplatz, etc. auf
 - Java tritt gemeinsam mit Worten wie *Klasse* oder *Methode* auf, aber auch mit Worten wie *Kaffee* und *Bohnen*.

Latent Topic Models

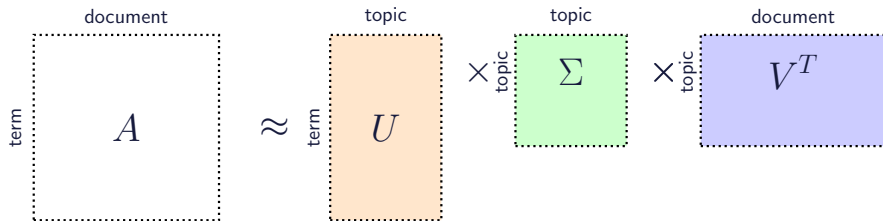
machen die Annahme, dass Dokumente aus eine kleinen Anzahl von versteckten (latent) Konzepten (topics) bestehen.

Wir betrachten hier **Latent Semantic Indexing (LSI)** [Deerwester et al. '90], aber es gibt weitere Ansätze. <http://lsa.colorado.edu/papers/JASIS.lsi.90.pdf>

Latent-Semantic-Indexing (LSI)

Idee

Betrachte **Singulärwertzerlegung** (SVD, Englisch: Singular Value Decomposition) der $m \times n$ **Term-Dokument-Matrix** A .



- U bildet Terme auf Topics ab.
- V bildet Dokumente auf Topics ab

Wiederholung: Vektoroperationen

- $\mathbf{v}^T$ transponiert einen Zeilenvektor in einen Spaltenvektor und umgekehrt.
- Für Vektoren $\mathbf{v}, \mathbf{w} \in R^n$, $\mathbf{v} + \mathbf{w}$ ist ein Vektor mit $(\mathbf{v} + \mathbf{w})_i = \mathbf{v}_i + \mathbf{w}_i$
- Für Vektor $\mathbf{v}$ und Skalar α , $(\alpha \mathbf{v})_i = \alpha \mathbf{v}_i$
- Das Skalarprodukt zweier Vektoren $\mathbf{v}, \mathbf{w} \in R^n$ ist

$$\mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^n v_i w_i$$

Wiederholung: Matrixoperationen

- Die transponierte Matrix $\mathbf{A}^T$ hat die Zeilen von $\mathbf{A}$ als Spalten
- Sind $\mathbf{A}$ und $\mathbf{B}$ zwei $n \times m$ Matrizen, dann ist $\mathbf{A} + \mathbf{B}$ eine $n \times m$ Matrix mit $(\mathbf{A} + \mathbf{B})_{ij} = a_{ij} + b_{ij}$
- Ist $\mathbf{A}$ eine $n \times k$ und $\mathbf{B}$ eine $k \times m$ Matrix, dann ist $\mathbf{AB}$ eine $n \times m$ Matrix mit

$$(\mathbf{AB})_{ij} = \sum_{l=1}^k a_{il}b_{lj}$$

Die “innere” Dimension (k) muss übereinstimmen

SVD Beispiel

$$A =$$

	d_1	d_2	d_3	d_4	d_5	d_6
Schiff	1	0	1	0	0	0
Boot	0	1	0	0	0	0
Ozean	1	1	0	0	0	0
Reise	1	0	0	1	1	0
Ausflug	0	0	0	1	0	1

$$U =$$

	1	2	3	4	5
Schiff	0.44	-0.30	-0.57	0.58	-0.25
Boot	0.13	-0.33	0.59	0.00	-0.73
Ozean	0.48	-0.51	0.37	0.00	0.61
Reise	0.70	0.35	-0.15	-0.58	-0.16
Ausflug	0.26	0.65	0.41	0.58	0.09

$$V^T =$$

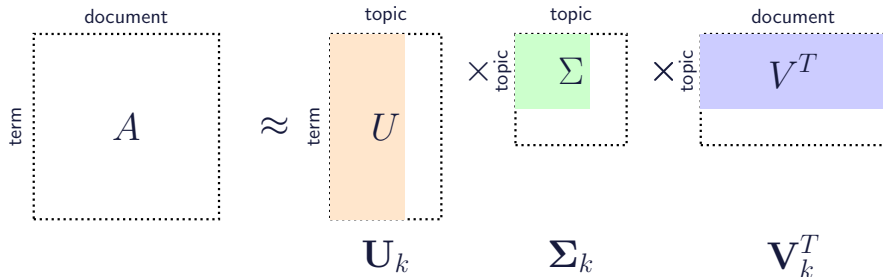
	1	2	3	4	5	6
1	0.75	0.28	0.20	0.45	0.33	0.12
2	-0.29	-0.53	-0.19	0.63	0.22	0.41
3	-0.28	0.75	-0.45	0.20	-0.12	0.33
4	0.00	0.00	0.58	0.00	-0.58	0.58
5	0.53	-0.29	-0.63	-0.19	-0.41	0.22

Die Singulärwerte von Σ auf der Diagonalen sind 2.16, 1.59, 1.28, 1.00, 0.39

Rang-k Approximation

- Man möchte A gar nicht korrekt wiederherstellen
- Sondern approximieren
- Durch Auswahl der wichtigsten (latenten) Topics
- **In Spalten von U stehen Topics beschrieben durch Terme.**
Interpretation: Alle gleiche Vorzeichen $\rightarrow$ treten gemeinsam auf.
Mischung aus $+/-$ $\rightarrow$ Terme mit positivem Gewicht zusammen, aber ohne Terme mit neg. Gewicht.
- Wir wählen die k **größten Singulärwerte aus Σ aus.**
- D.h. wir setzen die übrigen Singulärwerte auf 0
- Und entsprechend die dazugehörigen Einträge in U und V^T .

Latent-Semantic-Indexing (LSI)



- U_k, V_k^T, Σ_k enthalten die ersten k Singularwerte und Werte
- **Wie viele Singulärwerte sollten erhalten bleiben?** Faustregel*: so viele, dass mindestens 90% der Summe der Quadrate der Singulärwerte erhalten bleibt.

*) <http://infolab.stanford.edu/~ullman/mmds/ch11.pdf>

Beispiel: Approximation für $k = 2$

für Matrix $A =$

	d_1	d_2	d_3	d_4	d_5	d_6
Schiff	1	0	1	0	0	0
Boot	0	1	0	0	0	0
Ozean	1	1	0	0	0	0
Reise	1	0	0	1	1	0
Ausflug	0	0	0	1	0	1

Zur Erinnerung:

- U_k **Term-Topic-Matrix**
- V_k^T **ist Topic-Dokument-Matrix**

$$\mathbf{U}_2 = \begin{pmatrix} 0.44 & -0.30 \\ 0.13 & -0.33 \\ 0.48 & -0.51 \\ 0.70 & 0.35 \\ 0.26 & 0.65 \end{pmatrix}$$

$$\mathbf{\Sigma}_2 = \begin{pmatrix} 2.16 & 0.00 \\ 0.00 & 1.59 \end{pmatrix}$$

$$\mathbf{V}_2^T = \begin{pmatrix} 0.75 & 0.28 & 0.20 & 0.45 & 0.33 & 0.12 \\ -0.29 & -0.53 & -0.19 & 0.63 & 0.22 & 0.41 \end{pmatrix}$$

Weiteres Beispiel

$n = 5$ **Dokumente**

d_1 : how to bake bread without recipes

d_2 : the classic art of viennese pastry

d_3 : numerical recipes: the art of scientific computing

d_4 : breads, pastries, pies and cakes: quantity baking recipes

d_5 : pastry: a book of the best french recipes

m=6 Terme

t_1 : bak(e,ing)

t_2 : recipe(s)

t_3 : bread

t_4 : cake

t_5 : pastr(y,ies)

t_6 : pie

$$A_{raw} = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Weiteres Beispiel

$n = 5$ **Dokumente**

d_1 : how to bake bread without recipes

d_2 : the classic art of viennese pastry

d_3 : numerical recipes: the art of scientific computing

d_4 : breads, pastries, pies and cakes: quantity baking recipes

d_5 : pastry: a book of the best french recipes

m=6 Terme

t_1 : bak(e,ing)

t_2 : recipe(s)

t_3 : bread

t_4 : cake

t_5 : pastr(y,ies)

t_6 : pie

$$A_{\text{normalisiert}} = \begin{pmatrix} 0.5774 & 0 & 0 & 0.4082 & 0 & 0 \\ 0.5774 & 0 & 1 & 0.4082 & 0.7071 & 0 \\ 0.5774 & 0 & 0 & 0.4082 & 0 & 0 \\ 0 & 0 & 0 & 0.4082 & 0 & 0 \\ 0 & 1 & 0 & 0.4082 & 0.7071 & 0 \\ 0 & 0 & 0 & 0.4082 & 0 & 0 \end{pmatrix}$$

Rang 3 Approximation

$$\begin{aligned}
 \mathbf{A}_3 = \mathbf{U}_3 \times \mathbf{\Sigma}_3 \times \mathbf{V}_3^T = & \begin{pmatrix} -0.2670 & 0.2567 & -0.5308 \\ -0.7479 & 0.3980 & 0.5249 \\ -0.2670 & 0.2567 & -0.5308 \\ -0.1182 & 0.0127 & -0.2774 \\ -0.5197 & -0.8423 & -0.0839 \\ -0.1182 & 0.0127 & -0.2774 \end{pmatrix} \\
 & \times \begin{pmatrix} 1.6950 & 0.00000 & 0.0000 \\ 0.0000 & 1.1158 & 0.0000 \\ 0.0000 & 0.0000 & 0.8403 \end{pmatrix} \\
 & \times \begin{pmatrix} -0.4367 & -0.3066 & -0.4413 & -0.4908 & -0.5288 \\ 0.4717 & -0.7549 & 0.3567 & 0.0346 & -0.2815 \\ -0.3688 & -0.0998 & 0.6247 & -0.5710 & 0.3711 \end{pmatrix}
 \end{aligned}$$

Rang 3 Approximation

Die approximierte Matrix A_3 brauchen wir gar nicht zu betrachten. Sie sieht aber wie folgt aus:

$$A_3 = \begin{pmatrix} 0.4972 & -0.0330 & 0.0232 & 0.4867 & -0.0069 \\ 0.6004 & 0.0094 & 0.9933 & 0.3857 & 0.7091 \\ 0.4972 & -0.0330 & 0.0232 & 0.4867 & -0.0069 \\ 0.1801 & 0.0740 & -0.0522 & 0.2319 & 0.0155 \\ -0.0326 & 0.9866 & 0.0094 & 0.4401 & 0.7043 \\ 0.1801 & 0.0740 & -0.0522 & 0.2319 & 0.0155 \end{pmatrix}$$

Operationen im Topic-Raum

- Wir können eine **Anfrage aus dem m-dimensionalen Term-Raum in den k-dimensionalen Topic-Raum abbilden** durch

$$q \rightarrow \mathbf{U}_k^T q = q'$$

Operationen im Topic-Raum (2)

- Die Eignung (Score) der Dokumente wird dann im Topic-Raum berechnet, in dem q' **mit den Spalten der Matrix V_k^T verglichen** wird, anhand der Cosinus-Ähnlichkeit oder des Skalarprodukts.

Anfragen in LSI

Anfrage: baking bread

- $q = (1, 0, 1, 0, 0, 0)^T$

Nun Abbildung von q in den Topic-Raum:

- $q' = \mathbf{U}_3^T q = (-0.5340, 0.5134, -1.0616)^T$

Skalarprodukt als Ähnlichkeitsmaß der Vektoren im Topic-Raum:

- $\text{sim}(q', d'_1) \approx 0.86$
- $\text{sim}(q', d'_2) \approx -0.12$
- $\text{sim}(q', d'_3) \approx -0.24$

Zur Erinnerung:

t_1 : bak(e,ing)

t_2 : recipe(s)

t_3 : bread

t_4 : cake

t_5 : pastr(y,ies)

t_6 : pie

Kurzeinführung in R (<https://www.r-project.org/>)

<pre>> x <- 10</pre>	Eine einfache Zuweisung (Enter drücken)
<pre>> x</pre>	Enter drücken und
<pre>[1] 10</pre>	... Wert von x wird ausgegeben
<pre>>v <- c(1,2,3,4,5)</pre>	Definition eines Vektors
<pre>>v</pre>	
<pre>[1] 1 2 3 4 5</pre>	
<pre>>vv <- 2*v</pre>	Vektor multipliziert mit Skalar
<pre>>vv</pre>	
<pre>[1] 2 4 6 8 10</pre>	

Kurzeinführung in R

Matrizen....

```
>x <- c(1,2,3,4)
>y <- c(5,6,7,8)
>mat1 <- cbind(x,y)
>mat1
```

	x	y
[1,]	1	5
[2,]	2	6
[3,]	3	7
[4,]	4	8

```
mat2 <- rbind(x,y)
mat2
```

	[,1]	[,2]	[,3]	[,4]
x	1	2	3	4
y	5	6	7	8

Kurzeinführung in R

`>mat2[1,]` Wählt erste Zeile aus `[1] 1 2 3 4`

`>mat2[1,1]` Wählt Eintrag aus

`>t(mat2)` Berechnet Transponierte

	x	y
<code>[1,]</code>	1	5
<code>[2,]</code>	2	6
<code>[3,]</code>	3	7
<code>[4,]</code>	4	8

	<code>[,1]</code>	<code>[,2]</code>	<code>[,3]</code>	<code>[,4]</code>
x	1	2	3	4
y	5	6	7	8

Kurzeinführung in R

```
>2*mat1
```

	x	y
[1,]	2	10
[2,]	4	12
[3,]	6	14
[4,]	8	16

```
>mat1 %*% mat2
```

Multiplikation zweier Matrizen

	[,1]	[,2]	[,3]	[,4]
[1,]	26	32	38	44
[2,]	32	40	48	56
[3,]	38	48	58	68
[4,]	44	56	68	80

LSI in R

```
A <- matrix(c(0.5774,0,0,0.4082,0,0.5774,0,1,0.4082,
  0.7071,0.5774,0,0,0.4082,0, 0,0,0,0.4082,
  0,0,1,0,0.4082,0.7071,0,0,0,0.4082,0),
  nrow=6, ncol=5, byrow=TRUE)
A_svd = svd(A)

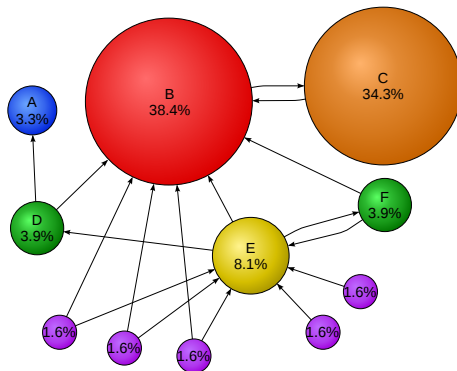
> A_svd
$d
1] 1.694953e+00 1.115800e+00 8.402774e-01 4.194943e-01 4.371327e-17 \\5mm]
$u
      [,1]      [,2]      [,3]      [,4]      [,5]
[1,] -0.2669599  0.25673561 -0.5308244  0.28468355 -7.071068e-01
[2,] -0.7479234  0.39803133  0.5249084 -0.08156498  1.110223e-16
[3,] -0.2669599  0.25673561 -0.5308244  0.28468355  7.071068e-01
[4,] -0.1182043  0.01265192 -0.2773859 -0.63947222 -1.526557e-16
[5,] -0.5197412 -0.84227346 -0.0838825  0.11579410  1.942890e-16
[6,] -0.1182043  0.01265192 -0.2773859 -0.63947222 -3.191891e-16
$V
      [,1]      [,2]      [,3]      [,4]      [,5]
[1,] -0.4366707  0.47168093 -0.36882335  0.67142025  0.0000000
[2,] -0.3066405 -0.75486063 -0.09982715  0.27603260  0.4999976
[3,] -0.4412650  0.35672284  0.62468462 -0.19443645  0.4999976
[4,] -0.4908152  0.03458356 -0.57099734 -0.65716544  0.0000000
[5,] -0.5288440 -0.28152323  0.37112672  0.05769664 -0.7071102
```

LSI in R: Rang-k Approximation

```
u3 <- A_svd$u[,1:3]
s3 <- diag(A_svd$d)[1:3,1:3]
vt3 <- t(A_svd$v)[1:3,]
A3 <- u3 %*% s3 %*% vt3
```

PageRank

- **PageRank sagt, dass eine Webseite v wichtig ist, wenn viele wichtige Seiten auf v verweisen**
- Beschrieben in einem Papier von **Brin & Page aus 1998**: **The PageRank Citation Ranking: Bringing Order to the Web**
<http://ilpubs.stanford.edu:8090/422/1/1999-66.pdf>.



PageRank

- **PageRank sagt, dass eine Webseite v wichtig ist, wenn viele wichtige Seiten auf v verweisen**

Random-Surfer-Modell

- **folgt zufällig den Links, die von einer Seite ausgehen**, mit Wahrscheinlichkeit $(1 - \varepsilon)$ oder
- **springt zu einer zufällig ausgewählten Seite**, mit Wahrscheinlichkeit ε

Intuition: Wichtige Webseiten werden häufiger besucht.

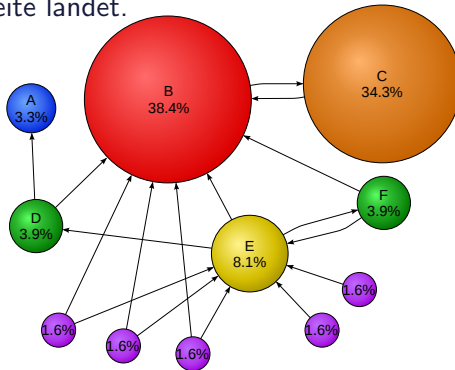
$$\text{PR}(p) = (1 - \varepsilon) \times \sum_{q \rightarrow p} \frac{\text{PR}(q)}{\text{out}(q)} + \varepsilon \times \frac{1}{N}$$

Schreibweise: $\sum_{q \rightarrow p}$, summiert über alle Knoten q , die auf p verweisen.

$\text{out}(q)$ ist die Anzahl der von q ausgehenden Links ("outdegree")

$$\text{PR}(p) = (1 - \varepsilon) \times \sum_{q \rightarrow p} \frac{\text{PR}(q)}{\text{out}(q)} + \varepsilon \times \frac{1}{N}$$

PageRank beschreibt die Wahrscheinlichkeitsverteilung, dass ein Random Surfer auf einer Seite landet.

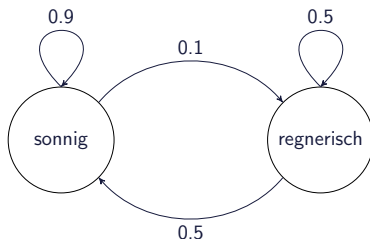


Verwandte Frage: Wie hoch ist die Wahrscheinlichkeit, dass ein Random Surfer auf Seite X landet, wenn er vorher auf Seite Y war?

Quelle der Abbildung: Wikipedia

Markov-Ketten: Wetter Beispiel

Wie wird das Wetter morgen sein, wenn es heute regnet?



$$P = \begin{pmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \end{pmatrix}$$

$(P)_{ij}$ ist die Wahrscheinlichkeit, dass der folgende Tag vom Typ j ist, wenn der heutige Tag vom Typ i ist.

Sagen wir zu Beginn haben wir einen sonnigen Tag: $x^{(0)} = (1 \ 0)$
Dann ist das Wetter am nachfolgenden Tag:

$$x^{(1)} = x^{(0)} P = (1 \ 0) \begin{pmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \end{pmatrix} = (0.9 \ 0.1)$$

Stochastischer Prozess & Markov-Ketten

- Ein **diskreter stochastischer Prozess** ist eine Familie von **Zufallsvariablen**

$$\{X_t | t \in T\}$$

wobei $T = \{0, 1, 2, \dots\}$ die diskrete Zeit-Domäne ist.

- Ein stochastischer Prozess ist eine **Markov-Kette** falls gilt

$$P[X_t = x | X_{t-1} = w, \dots, X_0 = a] = P[X_t = x | X_{t-1} = w]$$

D.h., der Prozess ist **gedächtnislos** (English: memoryless).

- Eine Markov-Kette ist **zeithomogen**, falls für alle Zeitpunkte t

$$P[X_{t+1} = x | X_t = w] = P[X_t = x | X_{t-1} = w]$$

gilt, d.h. die **Übergangswahrscheinlichkeiten hängen nicht von der Zeit ab**.

Zustandsraum & Übergangswahrscheinlichkeitsmatrix

- **Zustandsraum** einer Markov-Kette $\{X_t | t \in T\}$
- Markov-Kette ist in Zustand s zur Zeit t falls $X_t = s$
- Markov-Kette $\{X_t | t \in T\}$ ist endlich, falls der Zustandsraum **endlich** ist
- Falls eine Markov-Kette $\{X_t | t \in T\}$ endlich und zeithomogen ist, so können ihre **Zustandsübergangswahrscheinlichkeiten** durch eine Matrix $P = (p_{ij})$ mit

$$P = (p_{ij}) \text{ mit } p_{ij} = P[X_t = j | X_{t-1} = i]$$

beschrieben werden.

- Für n Zustände ist die **Zustandsübergangsmatrix** P eine $n \times n$ zeilenstochastische **Matrix** (d.h. ihre Zeilen summieren sich auf zu 1):

$$\forall i : \sum_j p_{ij} = 1$$

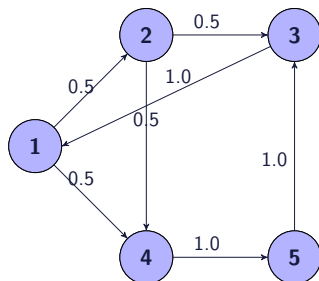
Eigenschaften von Markov-Ketten

Es gibt noch ein paar weitere Eigenschaften, die eine Markov-Kette haben kann, auf die wir aber in dieser Vorlesung nicht weiter eingehen.

Man sagt eine Markov-Kette ist **ergodisch** falls sie irreduzibel, positiv rekurrent und aperiodisch ist.

Theorem: Falls eine Markov-Kette endlich und ergodisch ist, dann existiert eine stationäre Zustandsverteilung π

Die Markov-Kette, die wir später für PageRank definieren werden hat diese Eigenschaften!



$$P = \begin{pmatrix} 0.0 & 0.5 & 0.0 & 0.5 & 0.0 \\ 0.0 & 0.0 & 0.5 & 0.5 & 0.0 \\ 1.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.0 \\ 0.0 & 0.0 & 1.0 & 0.0 & 0.0 \end{pmatrix}$$

$$\pi^{(0)} = (1.0 \quad 0.0 \quad 0.0 \quad 0.0 \quad 0.0)$$

$$\pi^{(1)} = (0.0 \quad 0.5 \quad 0.0 \quad 0.5 \quad 0.0)$$

$$\pi^{(2)} = (0.0 \quad 0.0 \quad 0.25 \quad 0.25 \quad 0.5)$$

$$\pi^{(3)} = (0.25 \quad 0.0 \quad 0.5 \quad 0.0 \quad 0.25)$$

$$\pi^{(4)} = (0.5 \quad 0.125 \quad 0.25 \quad 0.125 \quad 0.0)$$

$$\pi^{(5)} = (0.25 \quad 0.25 \quad 0.0625 \quad 0.3125 \quad 0.125)$$

....

$$\pi = (0.25 \quad 0.125 \quad 0.25 \quad 0.1875 \quad 0.1875)$$

Berechnung von π mit “Power Iteration”-Methode

Idee: Berechne schrittweise Zustandsverteilungen π bis diese konvergieren

Algorithmus

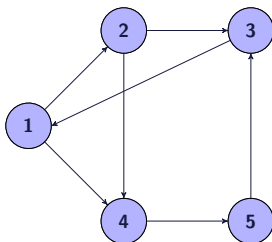
- wähle initiale Zustandsverteilung $\pi^{(0)}$ mit $\|\pi^{(0)}\|_1 = 1$
- berechne $\pi^{(k)} = \pi^{(k-1)} P$ bis zur Konvergenz (z.B. bis $\|\pi^{(k)} - \pi^{(k-1)}\|_1 < \xi$ mit $\xi = 0.000001$)
- gebe die zuletzt berechnete Verteilung $\pi^{(k)}$ als stationäre Zustandsverteilung π zurück

Wir berechnen hier also einen **Linkseigenvektor**.

PageRank als Markov-Kette

• Random-Surfer-Modell

- Folgt einem zufälligen ausgehenden Verweis mit Wahrscheinlichkeit $(1 - \varepsilon)$
- Springt zu einer zufälligen Webseite mit Wahrscheinlichkeit ε
- Sei **A** die Adjazenzmatrix des Webgraphen $G = (V, E)$ mit Kanten E und Knoten V .



$$A = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Zur Erinnerung: $PR(p) = (1 - \varepsilon) \times \sum_{q \rightarrow p} \frac{PR(q)}{out(q)} + \varepsilon \times \frac{1}{N}$

Daher: Aus Matrix **A** erzeugen wir Matrix **T** indem wir die Kantengewichte 1 durch die Anzahl der ausgehenden Links teilen, also

$$(\mathbf{T})_{ij} = \begin{cases} 1/out(i) & \text{falls } (i,j) \in E \\ 1/|V| & \text{falls } out(i) = 0 \\ 0 & \text{sonst} \end{cases}$$

$$\mathbf{T} = \begin{pmatrix} 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 1/1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/1 \\ 0 & 0 & 1/1 & 0 & 0 \end{pmatrix}$$

Nun fügen wir noch die “random jumps” hinzu....

Zur Erinnerung:
$$\text{PR}(p) = (1 - \varepsilon) \times \sum_{q \rightarrow p} \frac{\text{PR}(q)}{\text{out}(q)} + \varepsilon \times \frac{1}{N}$$

Also haben wir für unser Beispiel mit 5 Seiten für $\varepsilon = 0.15$

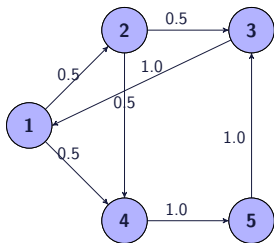
$$\mathbf{P} = (1 - \varepsilon)\mathbf{T} + \varepsilon \begin{pmatrix} 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \\ 1/5 & 1/5 & 1/5 & 1/5 & 1/5 \end{pmatrix}$$

oder allgemein ausgedrückt

$$\mathbf{P} = (1 - \varepsilon)\mathbf{T} + \varepsilon[1 \dots 1]^T \mathbf{j}$$

mit $(\mathbf{j})_i = 1/|V|$

Wir erhalten also für unseren Webgraphen die Matrix P



$$P = \begin{pmatrix} 0.03 & 0.455 & 0.030 & 0.455 & 0.03 \\ 0.03 & 0.030 & 0.455 & 0.455 & 0.03 \\ 0.88 & 0.030 & 0.030 & 0.030 & 0.03 \\ 0.03 & 0.030 & 0.030 & 0.030 & 0.88 \\ 0.03 & 0.030 & 0.880 & 0.030 & 0.03 \end{pmatrix}$$

Wir beginnen mit Startvektor $p^{(0)} = (0.2, 0.2, 0.2, 0.2, 0.2)$ und berechnen $p^{(k)} = p^{(k-1)} P$ bis $\|p^{(k)} - p^{(k-1)}\|_1 \leq 0.0001$

Bemerkungen

$$(0.2, 0.2, 0.2, 0.2, 0.2) * \begin{pmatrix} 0.03 & 0.455 & 0.030 & 0.455 & 0.03 \\ 0.03 & 0.030 & 0.455 & 0.455 & 0.03 \\ 0.88 & 0.030 & 0.030 & 0.030 & 0.03 \\ 0.03 & 0.030 & 0.030 & 0.030 & 0.88 \\ 0.03 & 0.030 & 0.880 & 0.030 & 0.03 \end{pmatrix}$$

Für Webseite 1 haben wir also nach dieser Iteration den neuen Wert im ersten Eintrag des Vektors berechnet durch

$$0.2 \times 0.03 + 0.2 \times 0.03 + 0.2 \times 0.88 + 0.2 \times 0.03 + 0.2 \times 0.03$$

da in **Spalte 1** der Matrix (rot markiert) steht wie viel Anteil des PR-Wertes der einzelnen Seiten auf Seite 1 propagiert wird.

Die Summe der Zeilen von P ist 1, d.h. P ist zeilenstochastisch. Man kann auch P^T betrachten und $p^{(k)T} = P^T p^{(k-1)T}$ berechnen. P^T ist entsprechend spaltenstochastisch.

```
T<-matrix(c(0,0.5,0,0.5,0,0,0,0.5,0.5,0,1,0,0,
0,0,0,0,0,0,1,0,0,1,0,0), nrow=5, ncol=5, byrow=TRUE)
```

```
#die Matrix für die zufälligen Sprünge
R <- c(rep(1/5,5)) %*% (t(c(rep(1,5))))
eps <- 0.15          #random jump Wahrscheinlichkeit
P <- (1-eps)*T + (eps)*R    #die Matrix P
p <- c(rep(1/5,5))      #so geht es los
```

```
stop <- 0.0001      #stop wenn kaum noch Aenderung
diff <- 1
p_old = p
while (diff>=stop) {
  p <- p_old %*% P
  diff <- (sum(abs(p_old-p)))
  p_old <-p
}
p    #finalen Vektor ausgeben
```

$$p^{(0)} = (0.2, 0.2, 0.2, 0.2, 0.2)$$

$$p^{(1)} = (0.2, 0.115, 0.285, 0.2, 0.2)$$

$$p^{(2)} = (0.2723, 0.115, 0.2489, 0.1639, 0.2)$$

$$p^{(3)} = (0.2415, 0.1457, 0.2489, 0.1946, 0.1693)$$

$$p^{(4)} = (0.2415, 0.1327, 0.2358, 0.1946, 0.1954)$$

$$p^{(5)} = (0.2305, 0.1327, 0.2525, 0.189, 0.1954)$$

$$p^{(6)} = (0.2446, 0.1279, 0.2525, 0.1843, 0.1907)$$

$$p^{(7)} = (0.2446, 0.134, 0.2465, 0.1883, 0.1867)$$

$$p^{(8)} = (0.2395, 0.134, 0.2456, 0.1909, 0.1901)$$

$$p^{(9)} = (0.2388, 0.1318, 0.2485, 0.1887, 0.1923)$$

...

$$p^{(19)} = (0.2409, 0.1323, 0.248, 0.1886, 0.1902)$$

$$p^{(20)} = (0.2408, 0.1324, 0.2479, 0.1886, 0.1903)$$

$$p^{(21)} = (0.2407, 0.1323, 0.248, 0.1886, 0.1903)$$

PageRank und Anfragen

PageRank berechnet ein statisches Ranking von Webseiten und ist unabhängig von Anfragen.

Eine Möglichkeit PageRank und “TF*IDF” Scores zu kombinieren ist eine einfache Linearkombination:

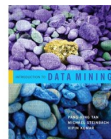
$$\alpha \times \text{sim}(q,d) + (1 - \alpha) \times \text{PR}(d)$$

Zusammenfassung PageRank

- Idee hinter PageRank: Webseiten haben verschiedenes Maß an Autorität
- Webseiten sollten nicht nur anhand der Nähe zur Anfrage sortiert werden sondern auch anhand der Autorität.
- PageRank modelliert einen random surfer, der zufällig das Web durchstöbert, dabei zufällig Links folgt oder zufällig irgendwelche URLS eingeben kann (random jumps).
- Autorität einer Webseite ist dann die Wahrscheinlichkeit, dass dieser Surfer auf der Seite landet.
- Das Verhalten des Surfers wird als Markov-Kette beschrieben.
- Man kann zeigen, dass diese eine stationäre Zustandsverteilung besitzt, also konvergiert (Details haben wir nicht angeschaut).
- Durch z.B. Power-Iteration kann man diese Verteilung berechnen

Literatur zu Data-Mining

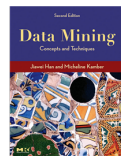
Pang-Ning Tan, Michael Steinbach, Vipin Kumar. **Introduction to Data Mining.** Ein paar relevante Kapitel sind frei verfügbar unter
<http://www-users.cs.umn.edu/~kumar/dmbook/index>



Mohammed J. Zaki, Wagner Meira Jr. **Data Mining and Analysis.**
<http://www.dataminingbook.info>



Jiawei Han, Micheline Kamber. **Data Mining. Concepts and Techniques.**



Warenkorbanalyse

Objekte sind: Brot, Milch, Windeln, Bier, Eier

Transaktionen sind: 1:{Brot, Milch}, 2:{Brot, Windeln, Bier, Eier}, 3:{Milch, Windeln, Bier}, 4:{Brot, Milch, Windeln, Bier} und 5:{Brot, Milch, Windeln}

TID	Brot	Milch	Windeln	Bier	Eier
1	1	1			
2	1		1	1	1
3		1	1	1	
4	1	1	1	1	
5	1	1	1		

Welche Objekte (Items) werden häufig zusammen gekauft?

Können wir Regeln angeben der Form: Kunden die Windeln kaufen kaufen auch meist Bier?

Darstellung als Binärmatrix

TID	Brot	Milch	Windeln	Bier	Eier
1	1	1	0	0	0
2	1	0	1	1	1
3	0	1	1	1	0
4	1	1	1	1	0
5	1	1	1	0	0

Itemsets

{Brot, Milch}

{Brot, Windeln, Bier, Eier}

{Milch, Windeln, Bier}

{Brot, Milch, Windeln, Bier}

{Brot, Milch, Windeln}

- **Ein Itemset ist eine Menge von Objekten**

- Eine **Transaktion** t ist ein Itemset mit dazugehöriger Transaktions ID, $t = (tid, I)$ wobei I das Itemset der Transaktion ist
- Eine Transaktion $t = (tid, I)$ **enthält ein Itemset** X falls $X \subseteq I$
- **Der Support von Itemset** X in einer Datenbank D ist die Anzahl der Transaktionen in D , die X enthalten:

$$\text{supp}(X, D) = |\{t \in D : t \text{ enthält } X\}|$$

- Die **relative Häufigkeit** von Itemset X in Datenbank D ist der Support relativ zur Größe der Datenbank, $\text{supp}(X, D)/|D|$
- **Ein Itemset ist häufig (frequent), falls** dessen relative Häufigkeit über einem bestimmten Schwellwert **minfreq** liegt.
- Alternativ kann man auch einen Schwellwert **minsupp** bzgl. des Supports betrachten.

Beispiel

TID	Brot	Milch	Windeln	Bier	Eier
1	1	1	0	0	0
2	1	0	1	1	1
3	0	1	1	1	0
4	1	1	1	1	0
5	1	1	1	0	0

Itemset {Brot, Milch} hat Support 3 und relative Häufigkeit $3/5$

Itemset {Brot, Milch, Eier} hat Support und relative Häufigkeit 0.

Für $\text{minfreq} = 1/2$ haben wir die folgenden frequent itemsets:

{Brot}, {Milch}, {Windeln}, {Bier}, {Brot, Milch}, {Brot, Windeln}, {Milch, Windeln} und {Windeln, Bier}.

Assoziationsregeln und Konfidenz

- Eine **Assoziationsregel** ist eine Regel der Form $X \rightarrow Y$, wobei X und Y disjunkte Itemsets sind (d.h. $X \cap Y = \emptyset$)
- **Idee:** Eine Transaktion, die Itemset X enthält, enthält (vermutlich) auch Itemset Y
- Der **Support einer Regel** $X \rightarrow Y$ in Datenbank D ist

$$\text{supp}(X \rightarrow Y, D) = \text{supp}(X \cup Y, D)$$

- Die **Konfidenz der Regel** $X \rightarrow Y$ in Datenbank D ist

$$\text{conf}(X \rightarrow Y, D) = \text{supp}(X \cup Y, D) / \text{supp}(X, D)$$

Mit anderen Worten: Die Konfidenz ist die bedingte Wahrscheinlichkeit, dass eine Transaktion Y enthält, wenn sie X enthält.

Beispiel

TID	Brot	Milch	Windeln	Bier	Eier
1	1	1	0	0	0
2	1	0	1	1	1
3	0	1	1	1	0
4	1	1	1	1	0
5	1	1	1	0	0

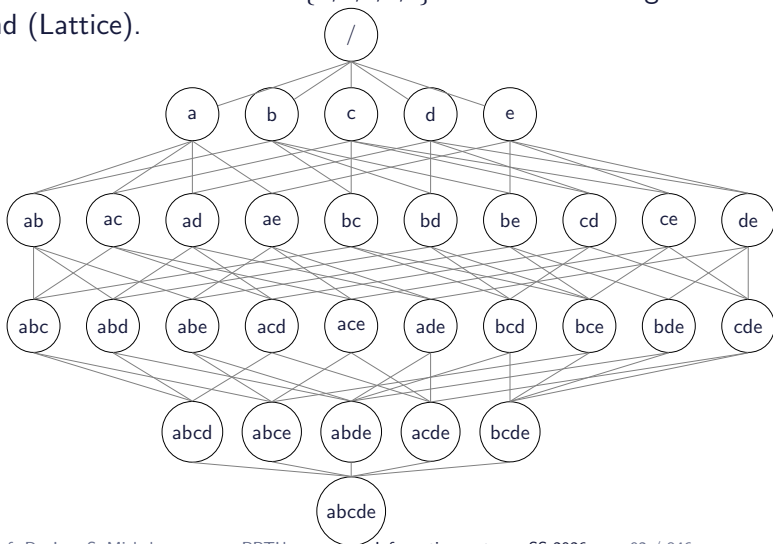
{**Brot, Milch**} → {**Windeln**} hat Support 2 und Konfidenz 2/3

{**Windeln**} → {**Brot, Milch**} hat Support 2 und Konfidenz 1/2

{**Eier**} → {**Brot, Windeln, Bier**} hat Support 1 und Konfidenz 1

Mögliche Itemset

- Was sind mögliche Itemset?
- Hier alle Itemsets für die Items $\{a,b,c,d,e\}$ in der Darstellung als Verband (Lattice).



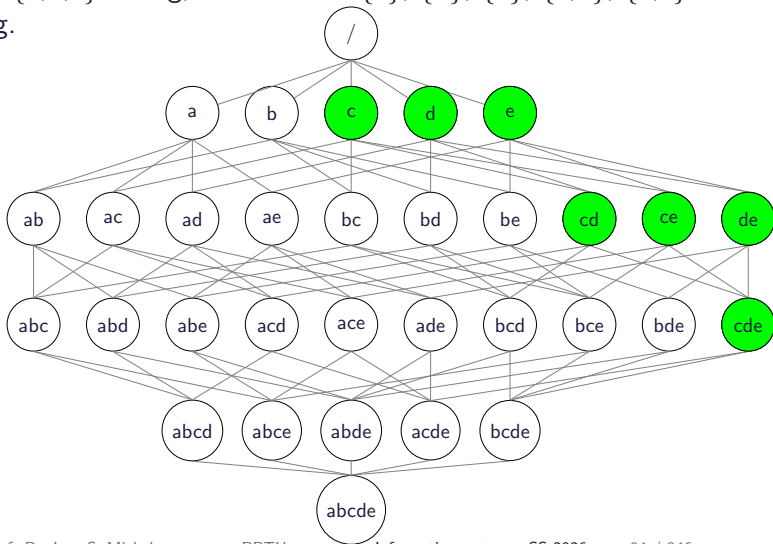
Ein naiver Algorithms

- **Betrachte jedes mögliche Itemset und teste ob es häufig ist.**
- **Wie berechnet man den Support?**
Zähle für jedes Itemset in welchen Transaktionen es enthalten ist
- Berechnen des Support dauert $O(|I| \times |D|)$ und es gibt $2^{|I|}$ mögliche Itemsets, also im Worstcase: $O(|I| \times |D| \times 2^{|I|})$

Das Apriori-Prinzip

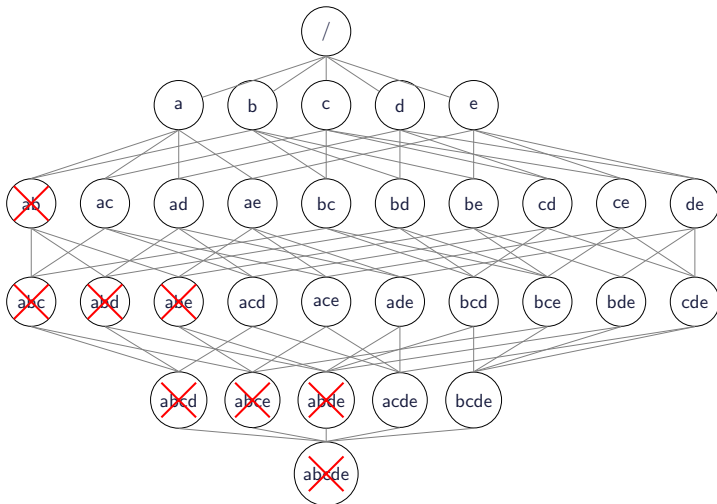
Falls ein Itemset häufig ist, so sind alle Teilmengen ebenfalls häufig.

Beispiel: Ist $\{c,d,e\}$ häufig, so sind auch $\{c\}$, $\{d\}$, $\{e\}$, $\{c,d\}$, $\{c,e\}$ und $\{d,e\}$ häufig.



Das Apriori-Prinzip

Umgekehrt: falls $\{a,b\}$ nicht häufig ist (Englisch: infrequent), so sind auch alle Supermengen von $\{a,b\}$ nicht häufig.



Anti-Monotonie

Sei I eine Menge von Items und sei $J = 2^I$ die Potenzmenge von I . Ein Maß f ist **monoton** (oder **aufwärts geschlossen**) falls

$$\forall X, Y \in J : (X \subseteq Y) \Rightarrow f(X) \leq f(Y)$$

Im Gegensatz, f ist **anti-monoton** (oder **abwärts geschlossen**) falls

$$\forall X, Y \in J : (X \subseteq Y) \Rightarrow f(Y) \leq f(X)$$

Ist Support monoton oder anti-monoton?

Anti-Monotonie

Sei I eine Menge von Items und sei $J = 2^I$ die Potenzmenge von I . Ein Maß f ist **monoton** (**oder aufwärts geschlossen**) falls

$$\forall X, Y \in J : (X \subseteq Y) \Rightarrow f(X) \leq f(Y)$$

Im Gegensatz, f ist **anti-monoton** (oder abwärts geschlossen) falls

$$\forall X, Y \in J : (X \subseteq Y) \Rightarrow f(Y) \leq f(X)$$

Support ist anti-monoton:

Für Itemsets X und Y mit $X \subseteq Y$ gilt $\text{supp}(X) \geq \text{supp}(Y)$. D.h. wenn X nicht häufig ist (infrequent), dann sind auch alle Obermengen von X nicht häufig.

Beispiel

Minimum Support Schwellwert = 3

Kandidaten 1-Itemsets

Item	Count
Bier	3
Brot	4
Cola	2
Windeln	4
Eier	2
Milch	4

Beispiel

Minimum Support Schwellwert = 3

Rot markierte Itemsets sind unter Schwellwert und werden eliminiert.

Kandidaten 1-Itemsets

Item	Count
Bier	3
Brot	4
Cola	2
Windeln	4
Eier	2
Milch	4

Beispiel

Minimum Support Schwellwert = 3

Rot markierte Itemsets sind unter Schwellwert und werden eliminiert.

Kandidaten 1-Itemsets

Item	Count
Bier	3
Brot	4
Cola	2
Windeln	4
Eier	2
Milch	4

Kandidaten 2-Itemsets

Itemset	Count
{Bier, Brot}	2
{Bier, Windeln}	3
{Bier, Milch}	2
{Brot, Windeln}	3
{Brot, Milch}	3
{Windeln, Milch}	3

Beispiel

Minimum Support Schwellwert = 3

Rot markierte Itemsets sind unter Schwellwert und werden eliminiert.

Kandidaten 1-Itemsets

Item	Count
Bier	3
Brot	4
Cola	2
Windeln	4
Eier	2
Milch	4

Kandidaten 2-Itemsets

Itemset	Count
{Bier, Brot}	2
{Bier, Windeln}	3
{Bier, Milch}	2
{Brot, Windeln}	3
{Brot, Milch}	3
{Windeln, Milch}	3

Beispiel

Minimum Support Schwellwert = 3

Rot markierte Itemsets sind unter Schwellwert und werden eliminiert.

Kandidaten 1-Itemsets

Item	Count
Bier	3
Brot	4
Cola	2
Windeln	4
Eier	2
Milch	4

Kandidaten 2-Itemsets

Itemset	Count
{Bier, Brot}	2
{Bier, Windeln}	3
{Bier, Milch}	2
{Brot, Windeln}	3
{Brot, Milch}	3
{Windeln, Milch}	3

Kandidaten 3-Itemsets

Itemset	Count
{Brot, Windeln, Milch}	3

Der Apriori-Algorithms

Der **Apriori-Algorithmus** benutzt die **Anti-Monotonie des Support-Maßes**, um die Menge an zu betrachtenden Itemsets einzuschränken.

- **Apriori generiert niemals ein Kandidaten-Itemset, dass nicht-häufige Teilmengen besitzt.**

Der Apriori-Algorithms: Pseudocode

```
/* Notation: mit  $\sigma$  bezeichnen wir den Support eines Itemsets */
1.  $k = 1$ 
2.  $F_k = \{ \{i\} \mid i \in I \wedge \sigma(\{i\}) \geq \text{minsupp} \}$  /*Häufige 1-Itemsets*/
3. repeat
4.    $k = k + 1$ 
5.    $C_k = \text{apriori-gen}(F_{k-1})$  /*Generiere Kandidaten*/
6.   for each transaction  $t \in T$  do
7.      $C_t = \text{subset}(C_k, t)$  /*Betrachte Kandidaten die in TA*/
8.     for each candidate itemset  $c \in C_t$  do
9.        $\sigma(c) = \sigma(c) + 1$  /*Erhöhe Support-Zähler*/
10.    end for
11.  end for
12.  $F_k = \{c \mid c \in C_k \wedge \sigma(c) \geq \text{minsupp}\}$  /*Finde häufige Itemsets*/
13. until  $F_k = \emptyset$ 
14.  $\text{Result} = \bigcup F_k$ 
```

Der Apriori-Algorithmus: Pseudocode (2)

- C_k ist die Menge der k -Itemsets
- F_k ist die Menge der häufigen k -Itemsets

Zuerst wird ein Mal über die Daten gelaufen, um den **Support jedes einzelnen Items** zu finden (Schritt 2). Dann kennen wir also F_1 .

Danach werden iterativ **neue Kandidaten k -Itemsets berechnet, basierend auf den häufigen $(k - 1)$ -Itemsets** (Schritt 5). Die Methode dafür nennt man **apriori-gen(...)**

Nun wird **für jedes Kandidaten-Itemset der Support berechnet**, indem ein Mal über die Daten (Transaktionen) gelaufen wird (Schritt 6–10).

Anschließend werden nicht-häufige Itemsets entfernt (Schritt 12).

Der Algorithmus terminiert sobald $F_k = \emptyset$ (Schritt 13).

Generierung und Eliminierung von Kandidaten

1. Generierung von Kandidaten

Generiert k -Itemsets (d.h. Itemsets der Länge k) basierend auf den Itemsets der vorherigen Iteration(en).

2. Eliminierung von Kandidaten

Finde und eliminiere unnütze Kandidaten k -Itemsets.

Generierung von Kandidaten: Ziele

Ziele:

- **Vollständigkeit:** Es müssen alle häufigen k-Itemsets erzeugt werden.
- **Effizienz:** Es sollte vermieden werden unnütze Itemsets zu erzeugen, d.h. solche die ein nicht-häufiges Itemset enthalten.
- Ebenso sollten **Itemsets nicht mehrfach generiert** werden.

Erfüllen die nachfolgenden Ansätze diese Ziele?

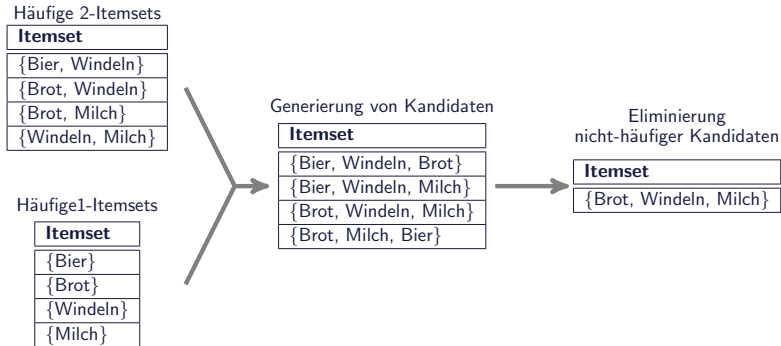
Generierung von Kandidaten: Brute-Force Ansatz

- **Schritt 1: Generiere alle Kandidaten.** Dies sind $\binom{d}{k}$ viele für d verschiedene Items.
- **Schritt 2:** Dann **entferne die nicht-häufigen Itemsets.**

Verbesserte Variante: Betrachte nur die Items aus F_1

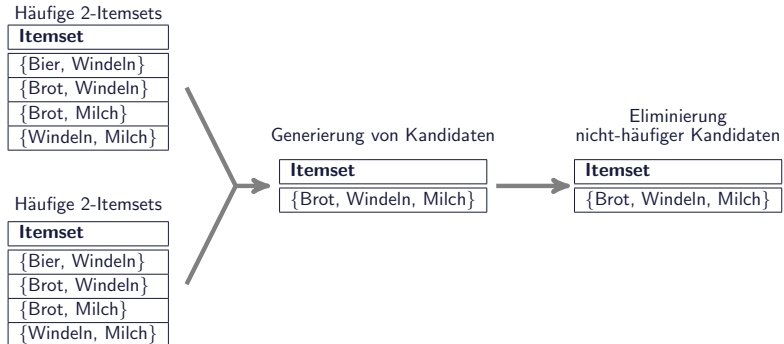
Generierung von Kandidaten: " $F_{k-1} \times F_1$ "

- **Verknüpfe die häufigen $(k-1)$ -Itemsets mit häufigen Items.**
- Dann entferne die nicht häufigen resultierenden Itemsets.
- Verbesserung: Erlaube nur Ergänzung durch 1-Itemset, wenn dies lexikographisch größer als Items des $(k-1)$ -Itemsets ist.



Generierung von Kandidaten: " $F_{k-1} \times F_{k-1}$ "

- **Kombiniere Paare von häufigen $(k-1)$ -Itemsets** falls diese in den ersten $(k-2)$ Items übereinstimmen.
- Betrachte lexikographische Sortierung der Itemsets.
- D.h., $A = \{a_1, a_2, \dots, a_{k-1}\}$ und $B = \{b_1, b_2, \dots, b_{k-1}\}$ können kombiniert werden falls $a_i = b_i$ (für $i = 1, 2, \dots, k-2$) und $a_{k-1} \neq b_{k-1}$



Assoziationsregeln (Association Rules)

- **Basierend auf den häufigen Itemsets** können wir nun **Assoziationsregeln** generieren.
- Falls Z ein häufiges Itemset ist und $X \subset Z$ ist eine echte Teilmenge von Z , dann haben wir eine Regel $X \rightarrow Y$, mit $Y = Z \setminus X$.
- **Diese Regeln sind häufig da**

$$\text{supp}(X \rightarrow Y) = \text{supp}(X \cup Y) = \text{supp}(Z)$$

Assoziationsregeln: Konfidenz

- Für eine Regel $X \rightarrow Y$ betrachten wir die **Konfidenz**, die wie folgt definiert ist

$$\text{conf}(X \rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)}$$

- Eine Regel $X \rightarrow Y$ mit $\text{conf}(X \rightarrow Y) \geq \text{minconf}$ wird als **confident** bezeichnet.
- Ist eine Regel $X \rightarrow Z \setminus X$ nicht confident, so kann keine Regel $W \rightarrow Z \setminus W$ mit $W \subseteq X$ confident sein.

Assoziationsregeln: Berechnung

Input: Menge F von häufigen Itemsets, minconf Schwellwert.

```
1. foreach  $Z \in F$  mit  $|Z| \geq 2$  do
2.    $A = \{X \mid X \subset Z, X \neq \emptyset\}$ 
3.   while  $A \neq \emptyset$  do
4.      $X = \text{größtes Itemset aus } A$ 
5.      $A = A \setminus X$ 
6.      $c = \text{supp}(Z) / \text{supp}(X)$     /* Berechnung Konfidenz */
7.     if  $c \geq \text{minconf}$  then
8.        $\text{print } X \rightarrow Y, \text{supp}(Z), c$     /* Ausgabe, wobei  $Y = Z \setminus X$  */
9.     else
10.       $A = A \setminus \{W \mid W \subset X\}$ 
11.    end
12.  end
13. end
```

Beispiel: Daten

Wir haben folgende Transaktionen

- 1: {Brot, Milch},
- 2: {Brot, Windeln, Bier, Eier},
- 3: {Milch, Windeln, Bier},
- 4: {Brot, Milch, Windeln, Bier}
- 5: {Brot, Milch, Windeln}

Mit minfreq=0.25 haben wir die folgenden häufigen Itemsets:

{Brot}, {Milch}, {Windeln}, {Bier}, {Brot, Milch}, {Brot, Windeln}, {Brot, Bier}, {Milch, Windeln}, {Milch, Bier}, {Windeln, Bier}, {Brot, Milch, Windeln}, {Brot, Windeln, Bier}, {Milch, Windeln, Bier}

Notation: Auf der folgenden Folie ist $A^{(i)}$ die Menge A in Iteration i

minfreq = 0.25, minconf = 0.5 und für $Z = \{\text{Milch, Windeln, Bier}\}$

$A^{(0)} = \{\{\text{Milch}\}, \{\text{Windeln}\}, \{\text{Bier}\}, \{\text{Milch, Windeln}\}, \{\text{Milch, Bier}\}, \{\text{Windeln, Bier}\}\}$

$X = \{\text{Windeln, Bier}\}$

Ausgabe: $\{\text{Windeln, Bier}\} \rightarrow \{\text{Milch}\} \quad 2 \quad 0.667$

$A^{(1)} = \{\{\text{Milch}\}, \{\text{Windeln}\}, \{\text{Bier}\}, \{\text{Milch, Windeln}\}, \{\text{Milch, Bier}\}\}$

$X = \{\text{Milch, Bier}\}$

Ausgabe: $\{\text{Milch, Bier}\} \rightarrow \{\text{Windeln}\} \quad 2 \quad 1.0$

$A^{(2)} = \{\{\text{Milch}\}, \{\text{Windeln}\}, \{\text{Bier}\}, \{\text{Milch, Windeln}\}\}$

$X = \{\text{Milch, Windeln}\}$

Ausgabe: $\{\text{Milch, Windeln}\} \rightarrow \{\text{Bier}\} \quad 2 \quad 0.667$

$A^{(3)} = \{\{\text{Milch}\}, \{\text{Windeln}\}, \{\text{Bier}\}\}$

$X = \{\text{Milch}\}$

Ausgabe: $\{\text{Milch}\} \rightarrow \{\text{Windeln, Bier}\} \quad 2 \quad 0.5$

$A^{(4)} = \{\{\text{Windeln}\}, \{\text{Bier}\}\}$

$X = \{\text{Windeln}\}$

Ausgabe: $\{\text{Windeln}\} \rightarrow \{\text{Milch, Bier}\} \quad 2 \quad 0.5$

$A^{(5)} = \{\{\text{Bier}\}\}$

$X = \{\text{Bier}\}$

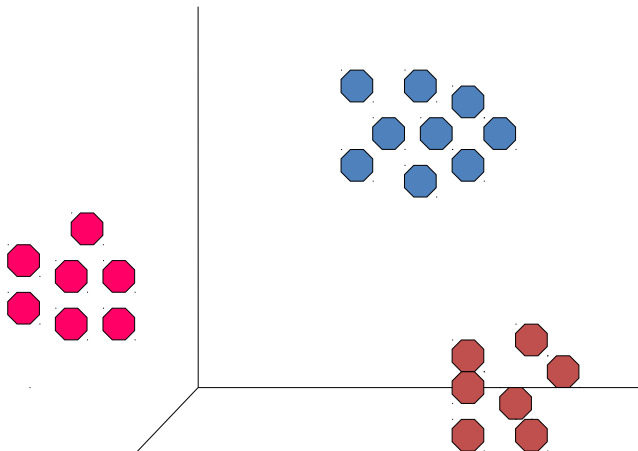
Ausgabe: $\{\text{Bier}\} \rightarrow \{\text{Milch, Windeln}\} \quad 2 \quad 0.667$

Zusammenfassung Itemsetmining

- **Warenkorbanalyse ist klassisches Beispiel für Data-Mining**
- Dabei werden **Transaktionen** bestehend aus Items auf **häufig zusammen auftretende Items** sowie nach **Assoziationsregeln** der Form “Wer Brot kauft kauft auch Bier” durchsucht.
- **Apriori Algorithmus** generiert Itemsets bottom-up basierend auf häufigen Itemsets kleinerer Länge.
- Dies funktioniert aufgrund der “**Anti-Monotonie**” des Supports.
- Assoziationsregeln werden basierend auf häufigen Itemsets berechnet.

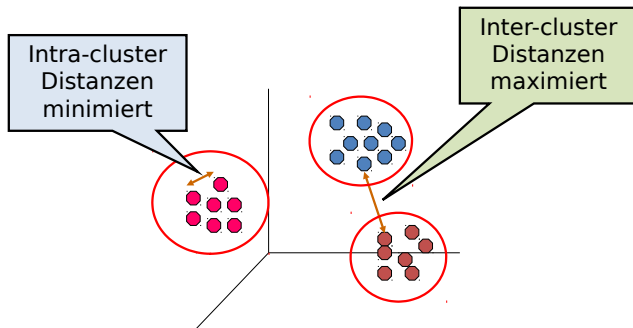
Clustering

Gegeben eine Menge von Objekten. Ziel: Finden eines guten Clusterings (Gruppierung) der Objekte anhand ihrer Eigenschaften. Hier, anhand ihrer 3D Koordinaten.



Das Clustering-Problem (2)

- Gegeben eine **Menge U von Objekten** und eine **Distanzfunktion** $d : U \times U \rightarrow \mathbb{R}^+$
- **Gruppiere Objekte** aus U in Cluster (Teilmengen), so dass die Distanz zwischen den Punkten eines Clusters klein ist und die Distanz zwischen den einzelnen Clustern groß ist.



Partitionen und Prototypen

Wir betrachten hier **exklusives Clustering**, d.h. **ein Objekt ist genau einem Cluster zugeordnet**:

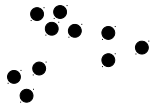
- Die Menge U ist partitioniert in k Clusters $C_1, C_2, \dots, C_k$ mit

$$\bigcup_i C_i = U \quad \text{und} \quad C_i \cap C_j = \emptyset \quad \text{für } i \neq j$$

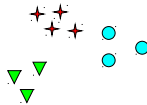
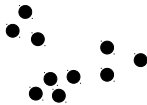
- Jedes Cluster C_i wird von einem sogenannten **Prototypen** μ_i repräsentiert (aka. **Schwerpunkt/Centroid oder Mitte/Durchschnitt**)
 - Dieser Prototyp μ_i muss nicht notwendigerweise eines der Objekte aus C_i sein
- Die **Qualität des Clusterings** wird dann in der Regel berechnet als den **quadratischen Fehler** zwischen den Objekten eines Clusters und dem Prototypen eines Clusters (hier für d -dimensionale Daten):

$$\sum_{i=1}^k \sum_{x_j \in C_i} \|x_j - \mu_i\|_2^2 = \sum_{i=1}^k \sum_{x_j \in C_i} \sum_{l=1}^d (x_{jl} - \mu_{il})^2$$

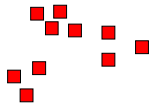
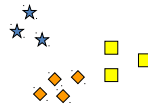
Clustering nicht (immer) eindeutig



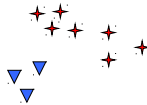
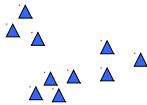
Wie viele Cluster?



Sechs Cluster



Zwei Cluster



Vier Cluster



Abbildung nach Tan, Steinbach, Kumar

Ein Naiver (Brute-Force) Ansatz

1. Generiere **alle möglichen Clusterings**, eins nach dem anderen
2. **Berechne den quadratischen Fehler**
3. **Wähle das Cluster mit dem kleinsten Fehler aus**

Dieser Ansatz ist leider unbrauchbar: Es gibt viel zu viele mögliche Clusterings, die ausprobiert werden müssen.

- Es gibt k^n Möglichkeiten k Cluster zu erzeugen bei n Objekten. Davon können einige Cluster leer sein. Also für 50 Objekte und 3 Cluster gibt es $3^{50} = 717897987691852588770249$ Möglichkeiten.
- Die Anzahl der Möglichkeiten diese n Punkte in k nicht leere Cluster aufzuteilen ist die Stirling-Zahl der zweiten Art.

$S(50,3) = 119649664052358811373730$.

Nur zur Info, hier die Definition Stirling-Zahl der zweiten Art:

$$S(n,k) = \left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \frac{1}{k!} \sum_{j=0}^k (-1)^j \binom{k}{j} (k-j)^n$$

K-Means Clustering

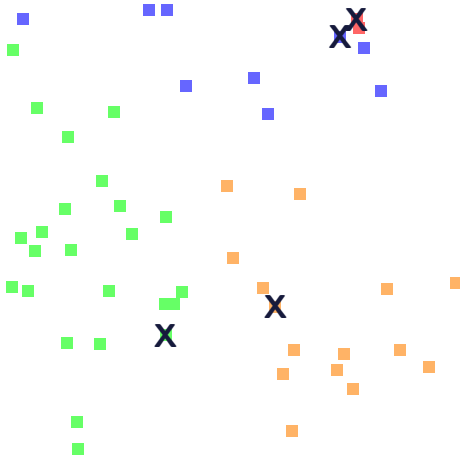
- Jedes Cluster wird durch einen Mittelpunkt (Centroid) repräsentiert
- Ein Objekt wird dem Centroid mit der geringsten Distanz zugewiesen
- Es gibt k Cluster. k ist ein Parameter.

Algorithmus:

1. Wähle zufällig k Objekte als initiale Centroids aus.
2. **repeat**
3. Ordne Objekte dem jeweils nächstgelegenen Centroid zu
4. Berechne für jedes Cluster den neuen Centroid.
5. **until** die Centroids ändern sich nicht mehr

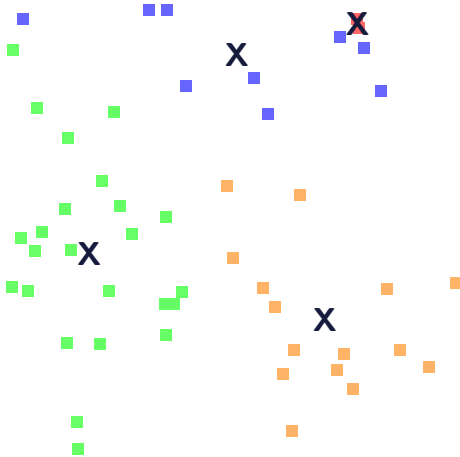
K-Means: Beispiel

Wähle zufällig $k = 4$ Centroids aus und ordne Objekte zu



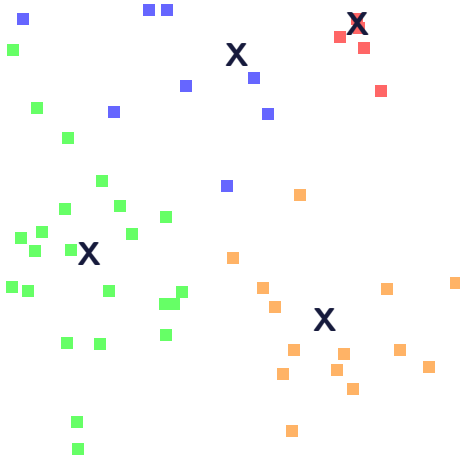
K-Means: Beispiel

Berechne Centroid jedes Clusters neu



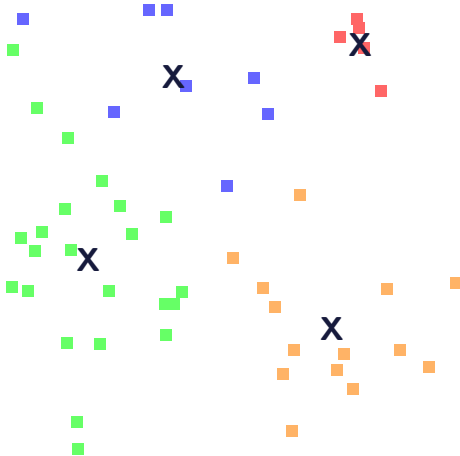
K-Means: Beispiel

Ordne Objekte neu zu



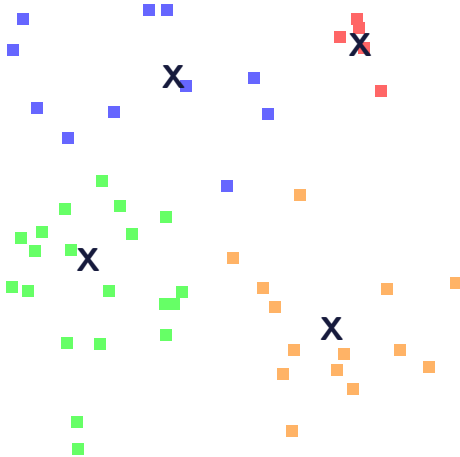
K-Means: Beispiel

Berechne Centroid jedes Clusters neu



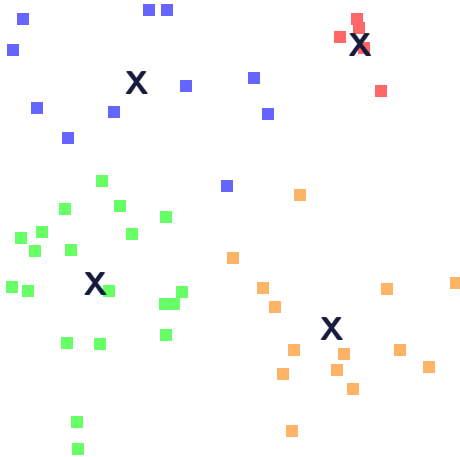
K-Means: Beispiel

Ordne Objekte neu zu



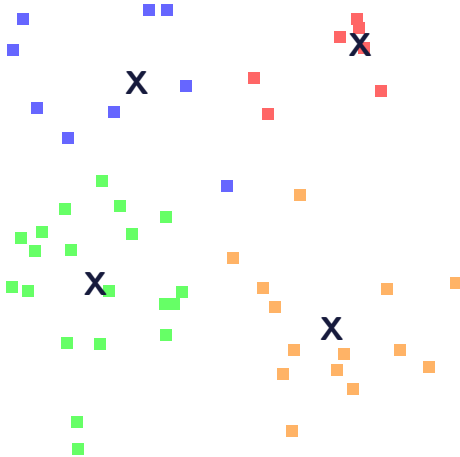
K-Means: Beispiel

Berechne Centroid jedes Clusters neu



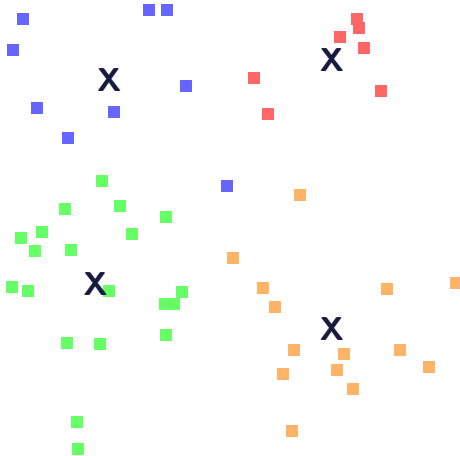
K-Means: Beispiel

Ordne Objekte neu zu



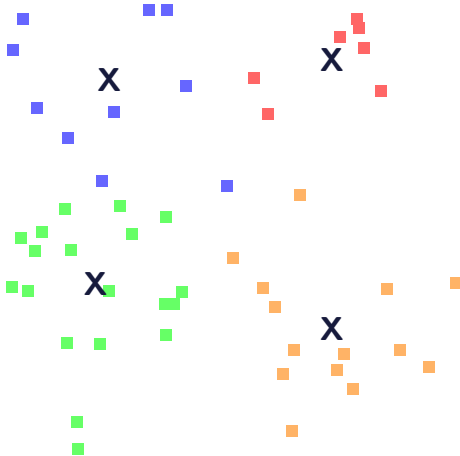
K-Means: Beispiel

Berechne Centroid jedes Clusters neu



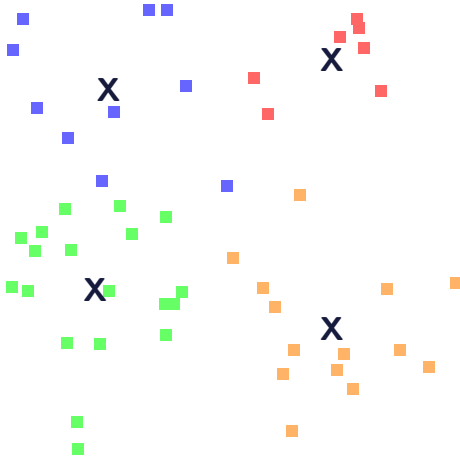
K-Means: Beispiel

Ordne Objekte neu zu



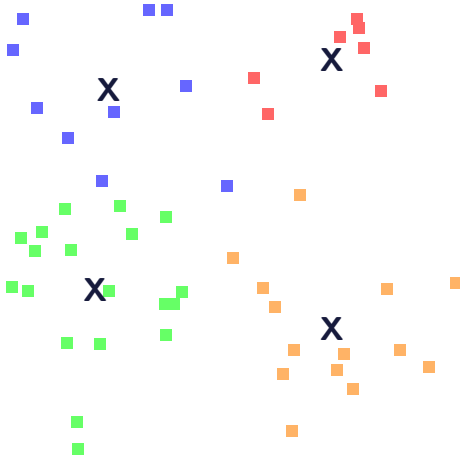
K-Means: Beispiel

Berechne Centroid jedes Clusters neu



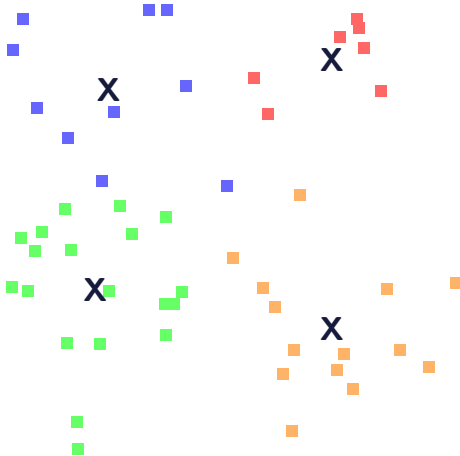
K-Means: Beispiel

Ordne Objekte neu zu



K-Means: Beispiel

Berechne Centroid jedes Clusters neu: Hat sich nichts geändert!



K-Means-Clustering

- Die initialen Centroids werden normalerweise zufällig ausgewählt. Dadurch können verschiedene Durchläufe auf den gleichen Daten unterschiedliche Cluster erzeugen.
- Als Centroid benutzt man typischerweise den Mittelwert (Mean) der Objekte eines Clusters.
- Als Distanzmaß wird z.B. die Euklidische Distanz benutzt
- Der K-Means-Algorithmus konvergiert
- In den ersten Iterationen sind die Änderungen des Clusterings am deutlichsten
- Abbruchkriterium auch: "Bis nur noch sehr wenige Objekte das Cluster wechseln"
- Komplexität ist $O(n \times k \times I \times d)$. n =Anzahl Objekte, k =Anzahl Cluster, I =Anzahl Iterationen, d =Dimensionalität der Daten.

Wiederholung: Betriebliche Informationssysteme

- spiegeln Geschäftsmodell eines Unternehmens wider
- organisieren und unterstützen Arbeitsabläufe
- integrieren eine Vielzahl von Datenquellen
- **Gewährleistung von Konsistenz (ACID)** essenziell
- im Kern: **traditionelle Datenbanksysteme** (Oracle, DB2, ...)

Studenten	
<u>Matr</u>	Name
26120	Fichte
25403	Jonas
...	...

hören	
<u>Matr</u>	<u>VorlNr</u>
25403	5022
26120	5001
...	...

Vorlesungen	
<u>VorlNr</u>	Titel
5001	Grundzüge
5022	Glaube und Wissen
...	...

Grober Überblick zu kommenden Vorlesungen

Entity/Relationship-Modellierung

Relationale Algebra und Kalküle

Tabellen und SQL

Normalformen

Indexierungstechniken

Transaktionen und Recovery

So geht es los ...

Abbilden eines Teilaspekts der realen Welt

- Was wollen wir Abbilden?
- Welcher Grad an Details?
- Welche Entitäten spielen eine Rolle?
- Und welche Rolle spielen sie?
- Wie sind Entitäten miteinander verbunden?

Anforderungsanalyse

Reale Welt: Universität

→ Anforderungsanalyse

Pflichtenheft

- „Studenten hören Vorlesungen“
- „Professoren halten Vorlesungen“
- „Studenten können anhand ihrer Matrikelnummer eindeutig identifiziert werden“
- ...

Anforderungsanalyse

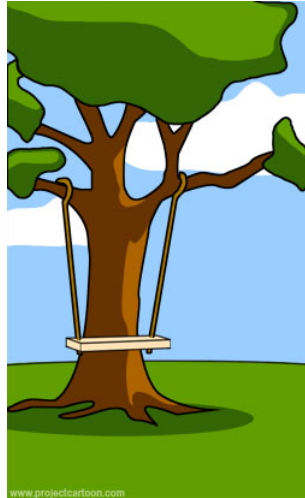
- Identifikation von Organisationseinheiten
- Identifikation von Beziehungen
- Identifikation von Prozessen
- Formalisierung
- Ziel: Anforderungskatalog/Pflichtenheft

Modellierung

Wie es der Kunde
beschrieben hat



Wie es der Projektmanager
verstanden hat



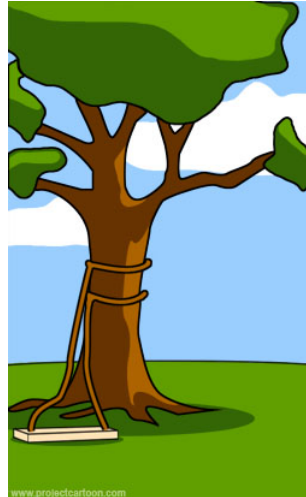
Quelle: <http://projectcartoon.com/cartoon/2>

Modellierung

Wie es der Analyst
designed hat.



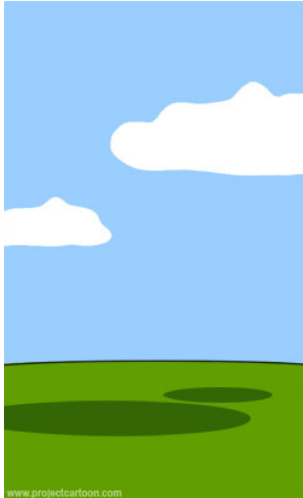
Wie der Programmierer es
implementiert hat.



Quelle: <http://projectcartoon.com/cartoon/2>

Modellierung

Wie es dokumentiert wurde.



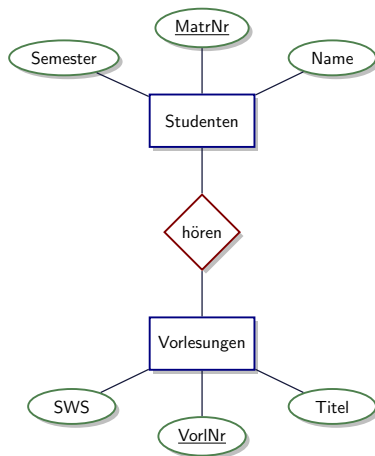
Was der Kunde wirklich gebraucht hätte.



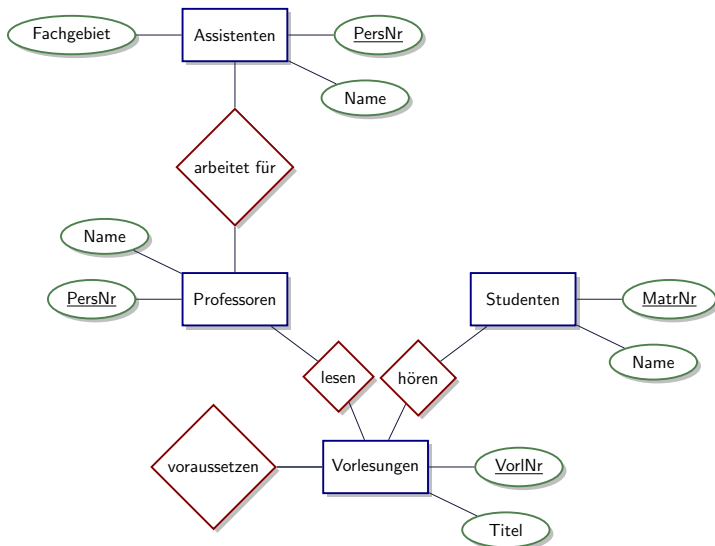
Quelle: <http://projectcartoon.com/cartoon/2>

Entity/Relationship-Modellierung

1. Entität → Entitätstyp
2. Beziehung → Beziehungstyp
3. Attribut (Eigenschaft)
4. Schlüssel (Identifikation)
5. Rolle



Beispiel: Universitätsschema



Relationales Modell

Abbilden des ER-Modells in Relationen:

Zum Beispiel:

- **Kunden**(ID, Telefon, Adresse, Name)
- **Kaufen**(Wert, Datum, Preis, Verkäufer, Auto, Kunde)
- **Verkaufen**(Datum, Wert, Kommission, Kunde, Verkäufer, Auto)
- ...

oder ...

- **Studenten** (MatrNr, Name)
- **Hören** (MatrNr, VorlNr)
- **Vorlesungen** (VorlNr, Titel)

Anfragen im Relationalen Modell

Relationale Algebra

- Selektion σ
- Projektion π
- Kreuzprodukt $\times$
- Join (Verbund) $\bowtie$
- Umbenennung ρ
- Differenz $-$
-

Beispiel:

- $\sigma_{Kunden.plz=66123}(Kunden)$
- $\sigma_{Kaufen.wert>50000}(Kaufen \bowtie_{Kaufen.kunde=Kunden.id} Kunden)$
- $\pi_{name}(Kunden)$

Tabellen und SQL

Studenten	
<u>Matr</u>	Name
26120	Fichte
25403	Jonas
...	...

hören	
<u>Matr</u>	<u>VorlNr</u>
25403	5022
26120	5001
...	...

Vorlesungen	
<u>VorlNr</u>	Titel
5001	Grundzüge
5022	Glaube und Wissen
...	...

SELECT Name

FROM Studenten, hören, Vorlesungen

WHERE Studenten.MatrNr = hören.MatrNr **and**

hören.VorlNr = Vorlesungen.VorlNr **and**

Vorlesungen.Titel = 'Grundzüge'

Tabellen und SQL

Studenten	
<u>Matr</u>	Name
26120	Fichte
25403	Jonas
...	...

hören	
<u>Matr</u>	<u>VorINr</u>
25403	5022
26120	5001
...	...

Vorlesungen	
<u>VorINr</u>	Titel
5001	Grundzüge
5022	Glaube und Wissen
...	...

UPDATE Vorlesungen

SET Titel = 'Grundzüge der Logik'

WHERE VorINr = 5001

SQL

Deklarative Anfragen

- Benutzer beschreiben WAS sie haben möchten.
- ... und nicht WIE es berechnet werden soll.
- Kein Wissen über die Implementierung erforderlich, d.h., wie und wo die Daten gespeichert sind.

→ weniger anfällig für Fehler!

Integritätsbedingungen

- ...sind ein zusätzliches Sicherheitssystem.
- **Ziel: Dateninkonsistenzen vermeiden.**
- Strategie: versuche zu verhindern, dass inkonsistente Daten in die DB eingefügt werden.
- Bedingungen an die möglichen Ausprägungen der Datenbank.

CREATE TABLE Studenten

(MatrNr integer primary key,

Name varchar(30) not null,

Semester integer **check Semester between 1 and 13)**

Tabellen

Tabelle mit Rechnungen:

Rechnungs-Nr.	Kunden-Name	Kunde-Straße	Kunde-Telefon	Kunde-PLZ
3422341	Meier	Bahnhofstr.	0681 12345	66123
3459235	Meier	Bahnhofstr.	0681 12345	66123
3512344	Meier	Bahnhofstr.	0681 12345	66123
4197515	Meier	Bahnhofstr.	0681 12345	66123
4205235	Meier	Bahnhofstr.	0681 12345	66123

Was passiert wenn Meier eine neue Telefonnummer bekommt?

Normalformen

Vermeiden von Redundanzen

- Regeln für einen guten relationalen Entwurf. Verschiedene Stufen
- z.B. Erste Normalform: keine Mengenwertige Attribute (z.B. {rot, geld, blau, grün})

StudentenBelegung			
MatrNr	VorlNr	Name	Semester
26120	5001	Fichte	10
27550	5001	Schopenhauer	6
27550	4052	Schopenhauer	6
28106	5041	Carnap	3
28106	5052	Carnap	3
28106	5216	Carnap	3
28106	5259	Carnap	3
...	...	...	...

Einfügeanomalien, Updateanomalien, Löschanomalien

Transaktionen

Typisches Beispiel

1. Lese den Kontostand von A in die Variable a : $read(A,a);$
2. Reduziere den Kontostand um 50 Euro: $a := a - 50;$
3. Schreibe den neuen Kontostand in die Datenbasis: $write(A,a);$
4. Lese den Kontostand von B in die Variable b : $read(B,b);$
5. Erhöhe den Kontostand um 50 Euro: $b := b + 50;$
6. Schreibe den neuen Kontostand in die Datenbasis: $write(B,b);$

ACID-Paradigma: Atomarität, Konsistenz (Consistency), Isolation, Dauerhaftigkeit

Transaktionen

Neulich am Geldautomaten ... ?

	Transaktion 1	Transaktion 2
1.	$x := \text{kontostand Kunde A}$	
2.	$x := x + 2000$ //Gehalt	$y := \text{kontostand Kunde A}$
3.	$\text{kontostand} := x$	$y = y - 100$ //Geld abheben
4.		$\text{kontostand} := y$

Dies ist ein Beispiel eines **“Lost Updates”**

Es gibt noch viele weitere Anomalien.

Recovery

Fehlersituation!

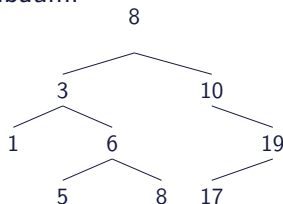
Zum Zeitpunkt t

- Wirkungen der Transaktionen die zuvor erfolgreich beendet wurden (commit) müssen vorhanden sein.
- Wirkungen der Transaktionen die abgebrochen wurden müssen vollständig eliminiert werden.

Indexierungstechniken

Was ist mit Performance?

- **Problem:** Effiziente Suche in großen Datenmengen
- Abbildung: Schlüssel (z.B., Matrikelnummer) $\rightarrow$ Menge von Einträgen
- Daten passen nicht in den Hauptspeicher
- Festplattenzugriffe sind teuer (insbesondere die wahlfreien (=zufälligen) Zugriffe)
- Beispiel binärer Suchbaum:



ca. 10ms für jeden Zugriff auf standard Festplatten (nicht SSD)

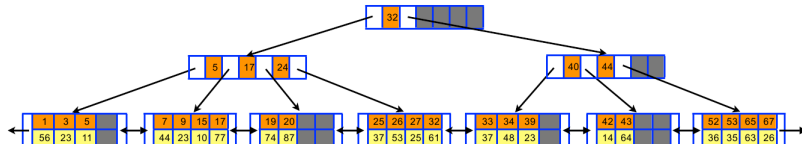
B+ Baum

Lesen geschieht nicht bitwise sondern in ganzen Blöcken.

Also, viel besser als binärer Baum:

- m -ary Baum ($m \gg 2$)
- Eine Seite auf der Festplatte = ein Knoten/Blatt
- Daten werden nur in Blättern gespeichert
- Sehr starker “fan-out” (=Ausfächerung)

→ sehr flach → wenige Zugriffe auf Festplatte



Basis Konzept einiger Dateisysteme: NTFS, ReiserFS, XFS, ...

Anfrageoptimierung

Wie werden Anfragen überhaupt ausgeführt?

- überführen der Anfrage in einen ausführbaren Plan
- jetzt wird über Implementierungen gesprochen und wo die Daten liegen und wie sie bearbeitet werden
- d.h. wie teuer die Ausführung ist → Kostenmodell
- **Idee: finde den günstigsten Plan**

Wie wird optimiert?

- “nach unten schieben” von Selektionen
- Reihenfolge von Verbundoperatoren

...($\sigma_{wert > 50000}(Kaufen \bowtie_{kaufen.kunde=kunden.id} Kunden)$)

besser: ...($\sigma_{wert > 50000}(Kaufen) \bowtie_{kaufen.kunde=kunden.id} Kunden$)

Last but not least: Die “Systeme”

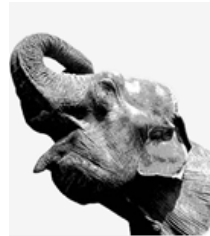
- Oracle
 - Microsoft SQL Server
 - IBM DB2
 - Sybase
-
- MySQL
 - Postgresql
 - ...

In dieser Vorlesung wird PostgreSQL verwendet

... in Beispielen in den Folien sowie in den Übungen

Postgresql

- “The world’s most advanced open source database”
- Einfach zu Installieren
- Gute SQL Unterstützung
- Transaktionen
- Gute Dokumentation (es gibt auch Bücher dazu)



<http://www.postgresql.org/>

PGAdmin User Interface

Download unter <http://www.pgadmin.org/>

Datenbankentwurf

Schritt 1: Anforderungsanalyse

Reale Welt: Universität

→ Anforderungsanalyse

- Identifikation von Organisationseinheiten
- Identifikation von Beziehungen
- Identifikation von Prozessen
- Formalisierung
- Ziel: Anforderungskatalog/Pflichtenheft

Pflichtenheft

- „Studenten hören Vorlesungen“
- „Professoren halten Vorlesungen“
- „Studenten können anhand ihrer Matrikelnummer eindeutig identifiziert werden“
- ...

Objektbeschreibung

Uni-Angestellte

Anzahl: 1000

Attribute:

- **PersonalNummer**

- Typ: char
- Länge: 10
- Wertebereich: 0...999.999.99
- Anzahl Wiederholung: 0
- Definiertheit: 100%
- Identifizierend: ja

- **Gehalt**

- Typ: dezimal
- Länge: (8,2)
- Anzahl Wiederholung: 0
- Definiertheit: 10%
- Identifizierend: nein

- **Rang**

- Typ: String
- Länge: 4
- Anzahl Wiederholung: 0
- Definiertheit: 100%
- Identifizierend: nein

Beziehungsbeschreibung: prüfen

Beteiligte Objekte:

- Professor als Prüfer
- Student als Prüfling
- Vorlesung als Prüfungsstoff

Attribute der prüfen-Beziehung:

- Datum
- Uhrzeit
- Note

Anzahl: 100 000 pro Jahr

Prozessbeschreibung*

Zeugnisausstellung

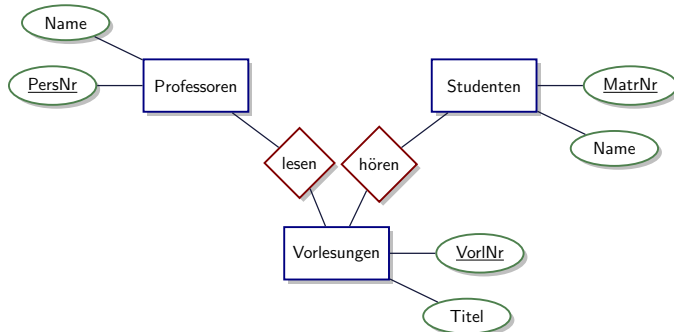
- Häufigkeit: halbjährlich
- benötigte Daten
 - Prüfungen
 - Studienordnungen
 - Studenteninformationen
 - ...
- Priorität: hoch
- Zu verarbeitende Datenmenge
 - 500 Studenten
 - 3000 Prüfungen
 - 10 Studienordnungen

*)Prozesse werden nicht weiter betrachtet.

Schritt 2: Abbildung auf konzeptuelles Modell

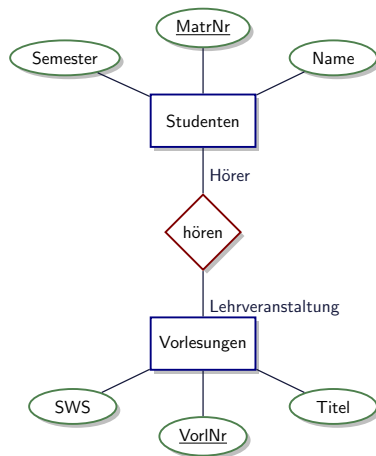
Pflichtenheft

- „Studenten hören Vorlesungen“
- „Professoren halten Vorlesungen“
- „Studenten können anhand ihrer Matrikelnummer eindeutig identifiziert werden“
- ...



Entity-Relationship-Modellierung

1. Entität → Entitätstyp
2. Beziehung → Beziehungstyp
3. Attribut (Eigenschaft)
4. Schlüssel (Identifikation)
5. Rolle

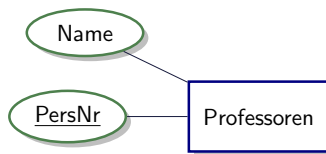


Schlüssel

Schlüssel

Eine **minimale** Menge von Attributen, deren Werte das zugeordnete Entity eindeutig innerhalb aller Entities seines Typs identifiziert, nennt man Schlüssel. **Beispiel:** Matrikelnummer oder Personalnummer.

- Werden dargestellt durch unterstrichenes Attribut:



- Schlüssel können auch aus mehreren Attributen bestehen.
- Es kann auch mehrere Möglichkeiten geben Schlüssel zu bilden: Dann wählt man aus diesen **Kandidatschlüsseln** einen **Primärschlüssel** aus.

Beziehungstypen und Rollen

Charakterisierung von Beziehungstypen

Ein Beziehungstyp R zwischen den Entitytypen $E_1, E_2, \dots, E_n$ kann als Relation im mathematischen Sinn angesehen werden. R ist also eine Teilmenge des kartesischen Produkts der beteiligten Entitytypen:

$$R \subseteq E_1 \times E_2 \times \dots \times E_n$$

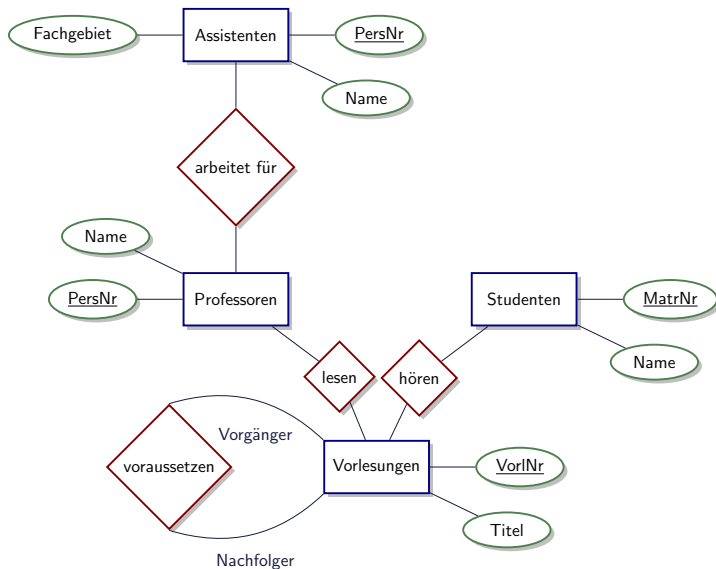
n ist der **Grad der Beziehung**.

Beispiel für eine binäre Beziehung (Grad=2):

$$\text{voraussetzen} \subseteq \text{Vorlesungen} \times \text{Vorlesungen}$$

In diesem Fall sollte man noch die **Rollen**, die die Entitäten spielen, angeben. Hier wäre für $(e_1, e_2) \in R$ die Entität e_1 der Vorgänger und die Entität e_2 der Nachfolger.

Universitätsschema



Funktionalitäten der Beziehungen

Eine binäre Beziehung R zwischen Entitytypen E_1 und E_2 heißt

1:1 Beziehung

falls jedem Entity e_1 aus E_1 höchstens ein Entity e_2 aus E_2 zugeordnet ist und umgekehrt. Es kann auch Entities aus E_1 (bzw. E_2) geben, denen kein "Partner" aus E_2 (bzw. E_1) zugeordnet ist.

1:N Beziehung

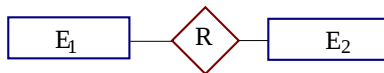
falls jedem Entity e_1 aus E_1 beliebig viele (also mehrere oder gar keine) Entities aus E_2 zugeordnet sein können, aber jedes Entity e_2 aus E_2 mit maximal einem Entity aus E_1 in Beziehung stehen kann. **Analog für N:1 Beziehung.**

N:M Beziehung

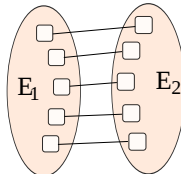
wenn keinerlei Restriktionen gelten müssen, d.h. jedes Entity aus E_1 kann mit beliebig vielen Entities aus E_2 assoziiert sein und umgekehrt.

Funktionalitäten (Chen-Notation)

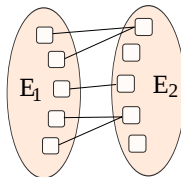
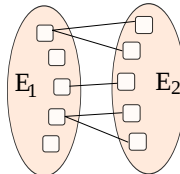
$$R \subseteq E_1 \times E_2$$



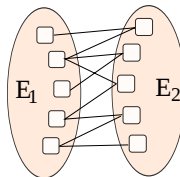
1:1



1:N



N:1



N:M

Partielle Funktionen*

Eine 1:1 oder 1:N Beziehung $R \subseteq E_1 \times E_2$ kann auch als partielle Funktion aufgefasst werden:

Für den Fall der 1:1 Beziehung kann die Funktion sowohl als $R : E_1 \rightarrow E_2$ wie auch als $R^{-1} : E_2 \rightarrow E_1$ gesehen werden. Beispiel: Mit *verheiratet* als Beispiel einer 1:1-Beziehung haben wir die partiellen Funktionen

Ehemann : Frauen $\rightarrow$ Männer

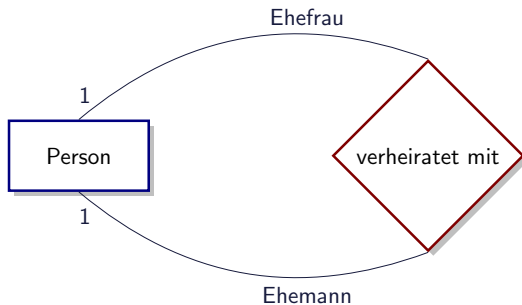
Ehefrau : Männer $\rightarrow$ Frauen

Bei einer 1:N Beziehung ist die “Richtung der Funktion” zwingend. Beispiel: Die Beziehung *beschäftigen* ist eine partielle Funktion von Personen nach Firmen, also:

beschäftigen : Personen $\rightarrow$ Firmen

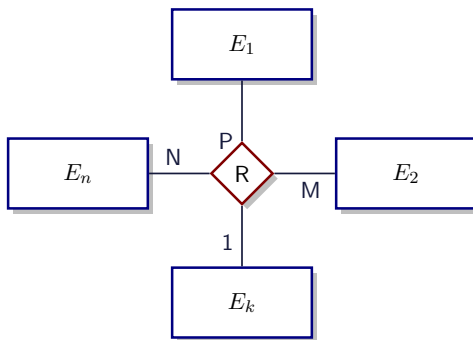
*) Partielle Funktion $f : X \rightarrow Y$, jedem Wert aus X wird höchstens ein Element aus Y zugeordnet.

Rekursive Beziehungen



Funktionalitäten bei n-stelligen Beziehungen

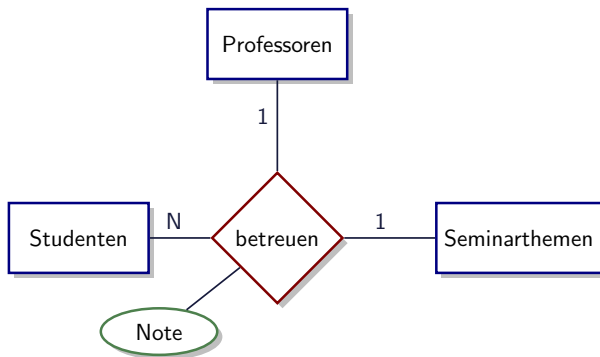
Sei R eine n -stellige Beziehung zwischen den Entitymengen $E_1, \dots, E_n$, wobei die Funktionalität bei der Entitymenge E_k mit einer "1" spezifiziert ist, bei den übrigen beliebig spezifiziert (1 oder N bzw. irgendein Buchstabe).



Dann muss folgende partielle Funktion gelten:

$$R : E_1 \times E_2 \times \dots \times E_{k-1} \times E_{k+1} \times \dots \times E_n \rightarrow E_k$$

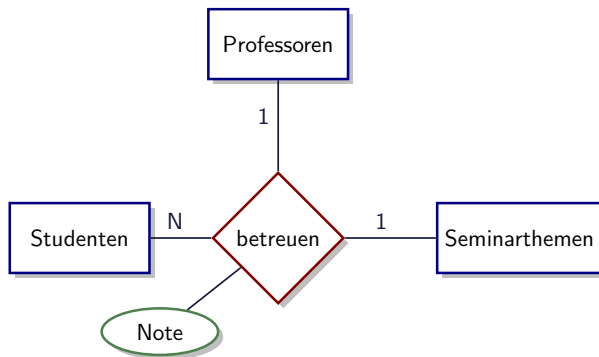
Beispiel-Beziehung: betreuen



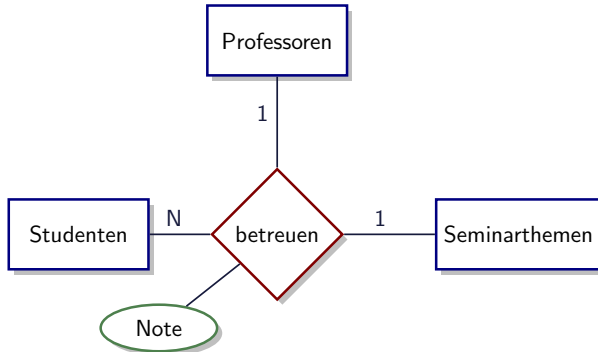
betreuen: Professoren $\times$ Studenten $\rightarrow$ Seminarthemen
betreuen: Seminarthemen $\times$ Studenten $\rightarrow$ Professoren

Dadurch erzwungene Konsistenzbedingungen

1. **Studenten dürfen bei demselben Professor bzw. derselben Professorin nur ein Seminarthema belegen.**
2. **Studenten dürfen das selbe Seminarthema nur einmal bearbeiten.** Also nicht bei einem anderen Professor nochmal das gleiche Thema bearbeiten.



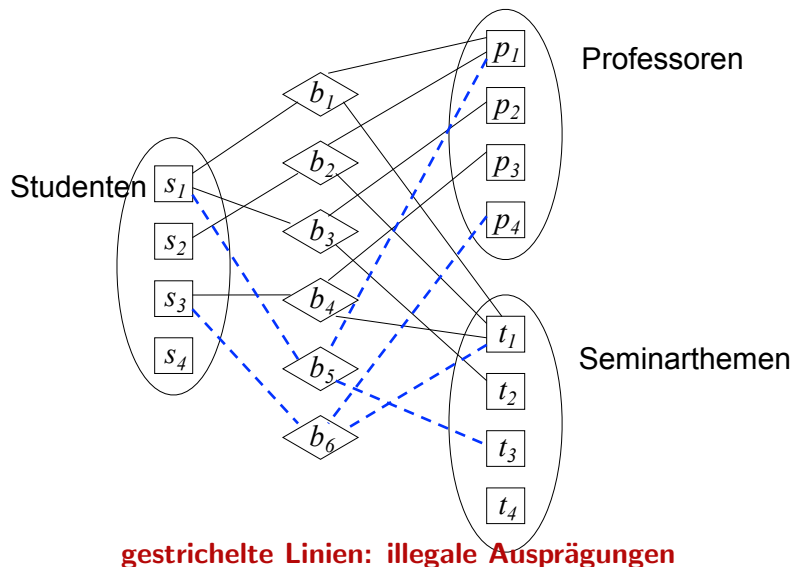
Dadurch erzwungene Konsistenzbedingungen



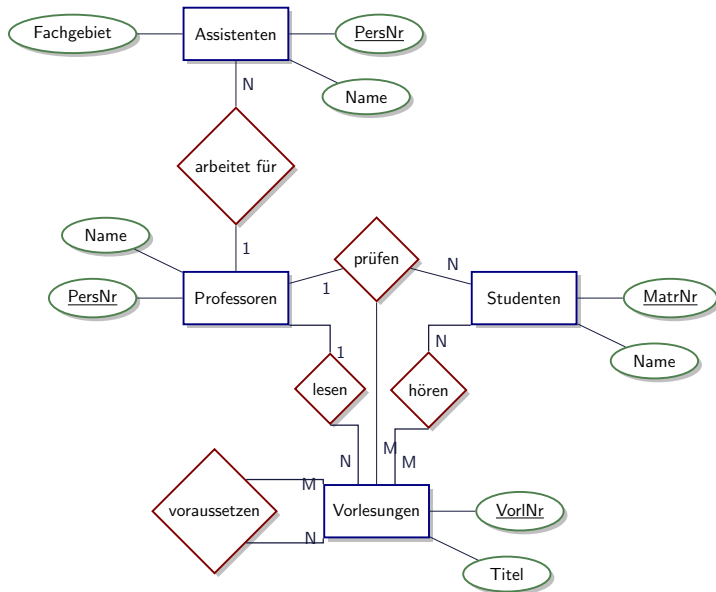
Folgende Datenbankzustände sind allerdings möglich:

- **Professoren können das selbe Seminarthema wiederverwenden**, d.h., das selbe Thema auch an mehrere Studenten geben.
- **Ein Thema kann von mehreren Professoren vergeben werden (aber an unterschiedliche Studenten).**

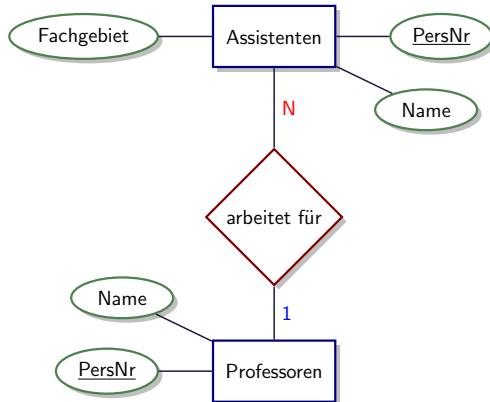
Äußerung der Beziehung *betreuen*



Universitätsschema mit Funktionalitäten

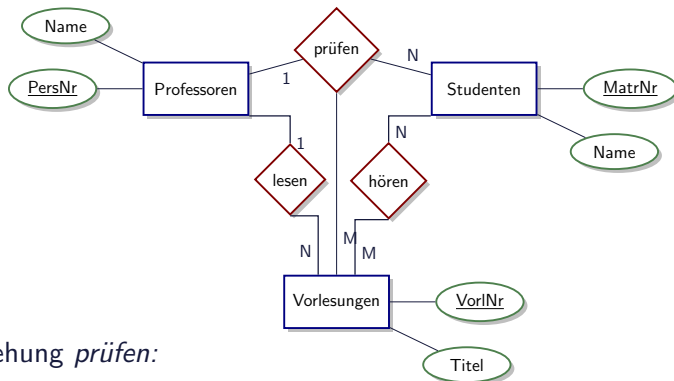


Universitätsschema mit Funktionalitäten



- "1": wenn ich eine Entität aus Assistenten herausnehme, dann bestimmt diese **eindeutig** den Professor.
- "N": wenn ich eine Entität aus Professoren herausnehme, dann bestimmt diese **nicht eindeutig** den Assistenten.

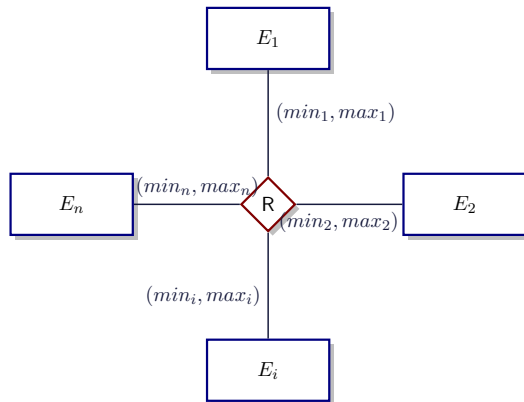
Universitätsschema mit Funktionalitäten



Für Beziehung *prüfen*:

- “M”: wenn ich eine Entität aus Studenten und eine aus Professoren herausnehme, dann bestimmt diese **nicht eindeutig** die Vorlesung.
- “N”: wenn ich eine Entität aus Vorlesungen und eine aus Professoren herausnehme, dann bestimmt diese **nicht eindeutig** den Studenten.
- “1”: wenn ich eine Entität aus Studenten und eine aus Vorlesungen herausnehme, dann bestimmt diese **eindeutig** den Professor.

(min,max)-Notation

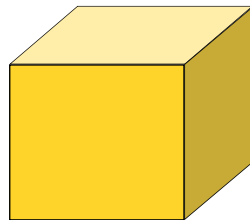


$$R \subseteq E_1 \times E_2 \times \dots \times E_i \times \dots \times E_n$$

Für jedes $e_i \in E_i$ gibt es

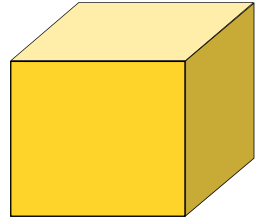
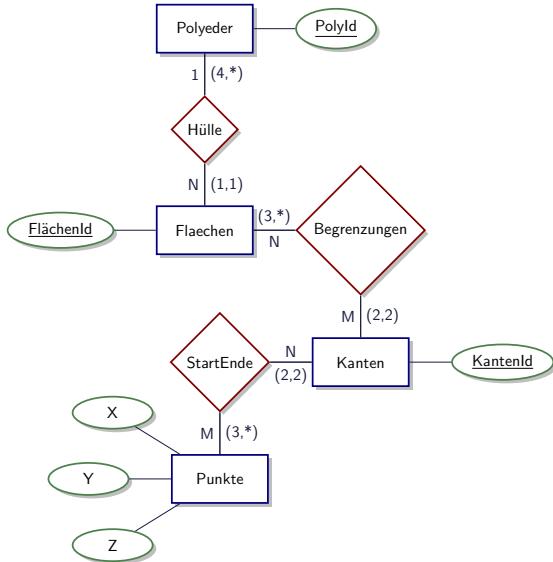
- **Mindestens** min_i Tupel der Art $(\dots, e_i, \dots)$ und
- **Höchstens** max_i viele Tupel der Art $(\dots, e_i, \dots)$

Beispiel: Begrenzungsflächendarstellung

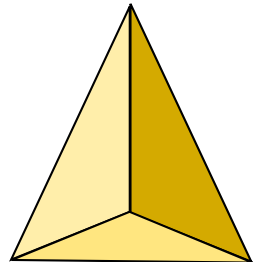


Beispiel-Polyeder

Beispiel: Begrenzungsflächendarstellung

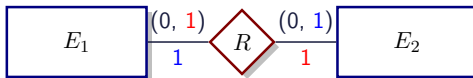


Beispiel-Polyeder

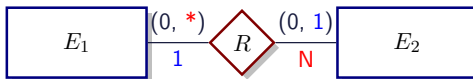


Chen- versus min/max Notation

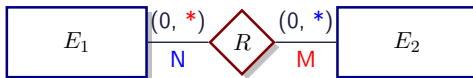
1:1 Beziehungen



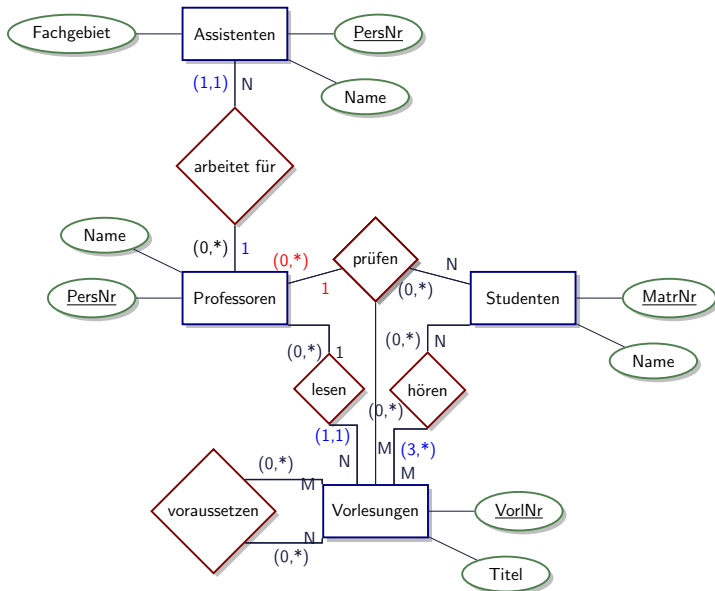
1:N Beziehungen



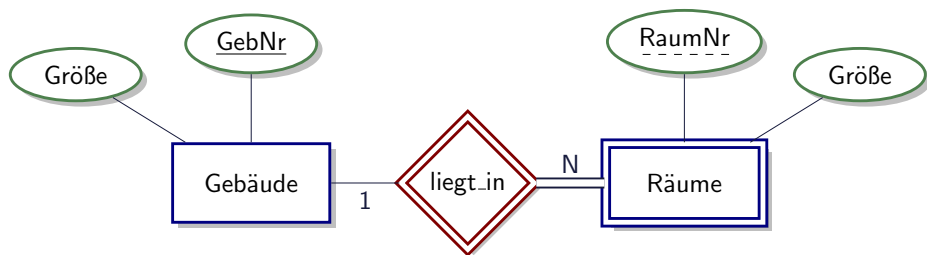
N:M Beziehungen



Funktionalitäten vs. min/max

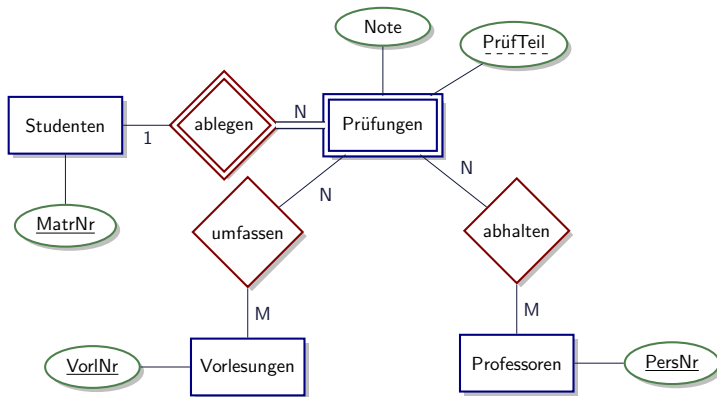


Schwache, existenzabhängige Entitäten



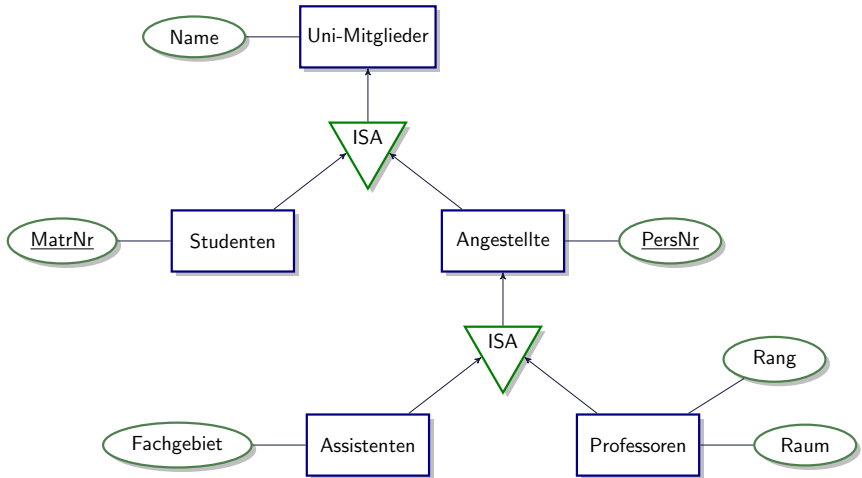
- Beziehung zwischen "starken" und schwachem Typ ist immer 1 : N (oder 1 : 1 in seltenen Fällen).
- Warum kann das nie $N : M$ sein?
- RaumNr ist nur innerhalb eines Gebäudes eindeutig.
- Schlüssel ist: GebNr **und** RaumNr.

Prüfungen als schwache Entität

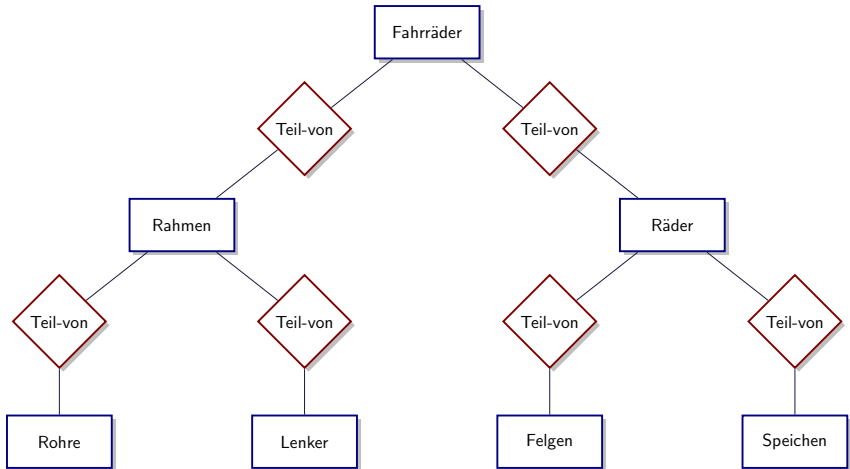


- Prüfung identifiziert durch MatrNr des Studenten plus Schlüssel/ID der Prüfung
- Mehrere Prüfer in einer Prüfung
- Mehrere Vorlesungen werden in einer Prüfung abgefragt

Generalisierung

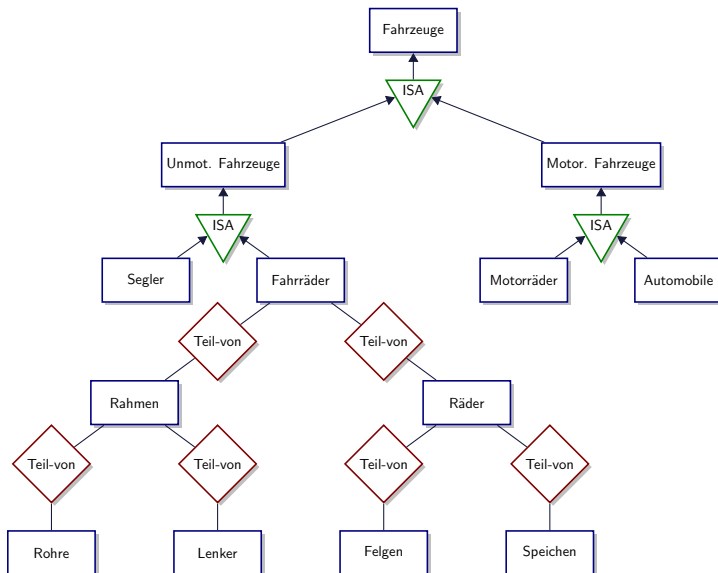


Aggregation

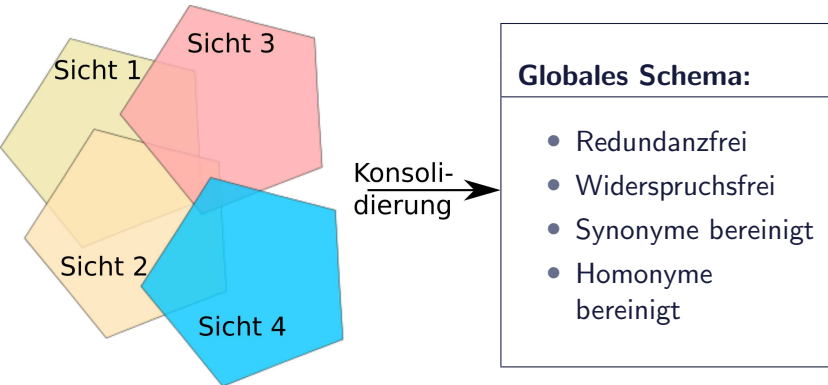


und so weiter ...

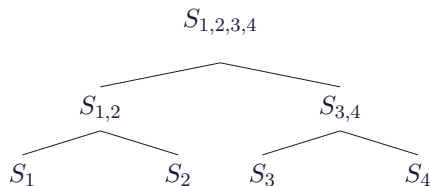
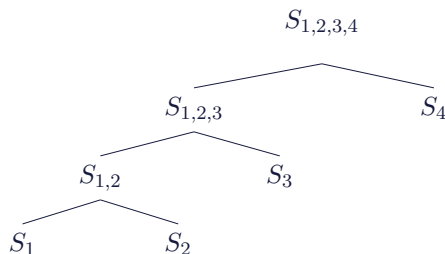
Generalisierung und Aggregation



Konsolidierung von Teilschemata oder Sichtenintegration

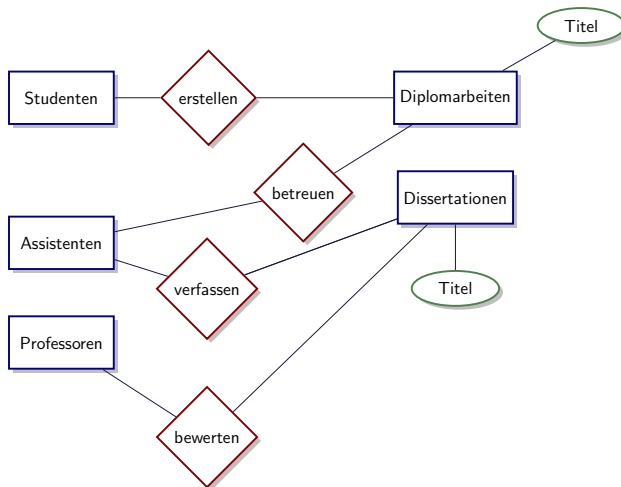


Mögliche Konsolidierungsbäume

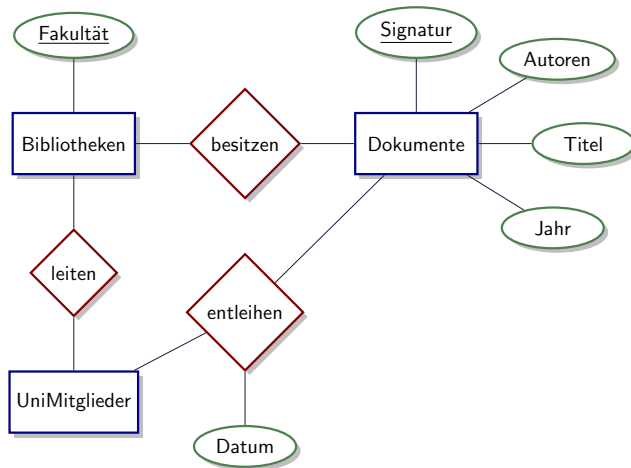


- Gegeben: Teilschemata S_1 , S_2 , S_3 und S_4 .
- Mögliche Konsolidierungsbäume zur Herleitung des globalen Schemas $S_{1,2,3,4}$
- Oben: maximal hoher Konsolidierungsbaum (links-tief).
- Unten: minimal hoher Konsolidierungsbaum (balanciert).
- Vorgehensweise hängt vom Einzelfall ab.

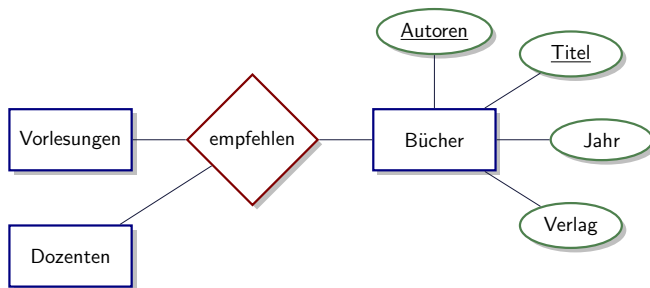
Drei Sichten einer Universitäts-Datenbank



Sicht 1: Erstellung von Dokumenten als Prüfungsleistung



Sicht 2: Bibliotheksverwaltung

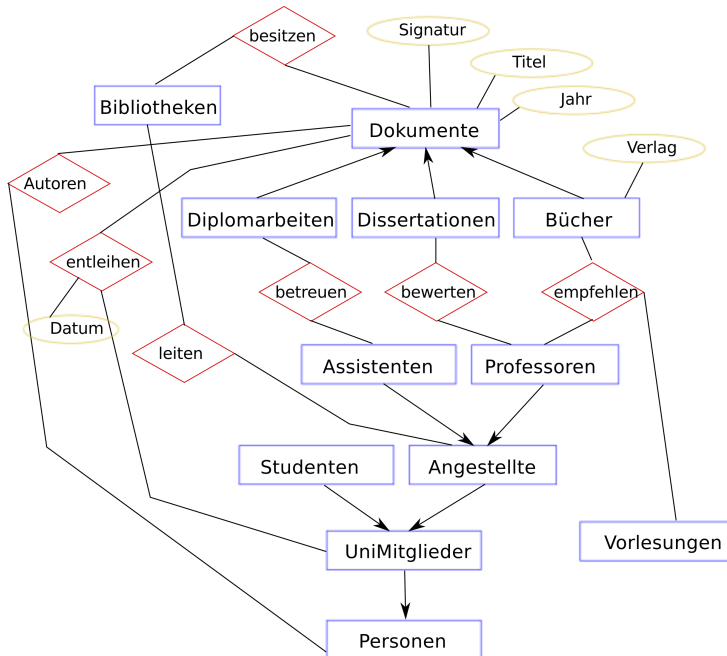


Sicht 3: Buchempfehlungen für Vorlesung

Beobachtungen für die Konsolidierung

- Die Begriffe **Dozenten** und **Professoren** sind synonym verwendet worden.
- Der Entitytyp **UniMitglieder** ist eine **Generalisierung** von **Studenten**, **Professoren** und **Assistenten**.
- Fakultätsbibliotheken werden sicherlich von Angestellten (und nicht von Studenten) geleitet. → Beziehung **leiten** in Sicht 2 muss angepasst werden. Insbesondere da im globalen Schema eine Spezialisierung von UniMitglieder in **Studenten** und **Angestellte** vorgenommen wird.
- **Dissertationen**, **Diplomarbeiten** und **Bücher** sind Spezialisierungen von **Dokumenten**, die in den Bibliotheken verwaltet werden.

- Wir können davon ausgehen, dass alle an der Universität erstellten Diplomarbeiten und Dissertationen in Bibliotheken verwaltet werden.
- Die in Sicht 1 festgelegten Beziehungen **erstellen** und **verfassen** modellieren denselben Sachverhalt wie das Attribut **Autoren** von **Bücher** in Sicht 3.
- Alle in einer Bibliothek verwalteten Dokumente werden durch die Signatur identifiziert.



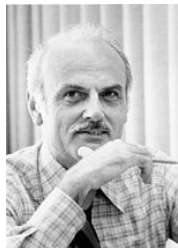
Datenmodellierung mit UML (nur ganz kurz erwähnt)

- Unified Modelling Language UML
- De-facto Standard für den objektorientierten Software- Entwurf
- Zentrales Konstrukt ist die Klasse (class) mit der gleichartige Objekte modelliert werden hinsichtlich:
- Struktur ($\approx$ Attribute)
- Verhalten ($\approx$ Operationen/Methoden)
- Assoziationen zwischen Klassen entsprechen Beziehungstypen
- Generalisierungshierarchien
- Aggregation
- für Datenbankentwurf ist strukturelle Modellierung entscheidend
- Modellierung von Verhalten weniger wichtig

Das relationale Modell

Grundlagen des relationalen Modells

- Entwickelt von Edgar. F. Codd in den Siebzigern ($\Rightarrow$ Turing-Award)
- Das am weitesten verbreitete Datenmodell
- Wird aber nach und nach um objektorientierte Konzepte erweitert



Grundlagen des relationalen Modells

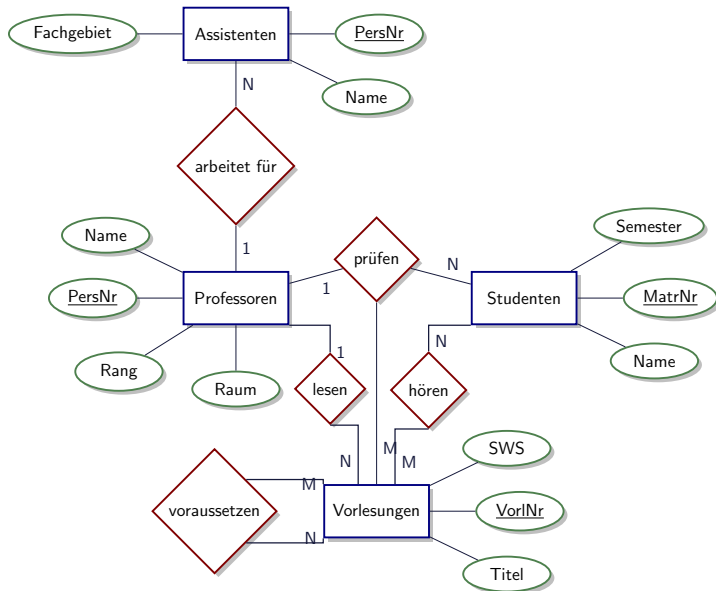
Seien $D_1, D_2, \dots, D_n$ Domänen (=Wertebereiche)

- **Relation:** $R \subseteq D_1 \times \dots \times D_n$
Beispiel: $Telefonbuch \subseteq string \times string \times integer$
- Wertebereiche dürfen identisch sein: $D_i = D_j$ für $i \neq j$
- **Relationenschema:** legt die Struktur der gespeicherten Daten fest.
Wird mit $sch(R)$ oder $\mathcal{R}$ bezeichnet.
Beispiel: Telefonbuch:
 $\{[Name : string, Adresse : string, \underline{Telefon : integer}]\}$
- **Tupel:** $t \in R$
Beispiel: $t = (\text{"Mickey Mouse"}, \text{"Main Street"}, 4711);$

Telefonbuch		
Name	Straße	<u>Telefon</u>
Mickey Mouse	Main Street	4711
Minnie Mouse	Broadway	94725
Donald Duck	Broadway	95622
...	...	...

- **Ausprägung:** der aktuelle Zustand der Datenbasis, Teilmenge des Kreuzproduktes
- **Schlüssel:** minimale Menge von Attributen, deren Werte ein Tupel eindeutig identifizieren
- **Primärschlüssel:** wird unterstrichen
 - Einer der Schlüsselkandidaten wird ausgewählt
 - Hat eine besondere Bedeutung bei der Referenzierung von Tupeln
 - später mehr dazu
- **Fremdschlüssel** verweist auf den Primärschlüssel einer anderen Relation.

Universitätsschema



Von Entitätstypen zu Relationenschemata

- **Studenten:** {[MatrNr: integer, Name: string, Semester: integer]}
- **Vorlesungen:** {[VorlNr: integer, Titel: string, SWS: integer]}
- **Professoren:** {[PersNr: integer, Name: string, Rang: string, Raum: integer]}
- **Assistenten:** {[PersNr: integer, Name: string, Fachgebiet: string]}

Im Folgenden lassen wir die Datentypen (z.B. integer) meist weg.

Umsetzung von N:M-Beziehungen

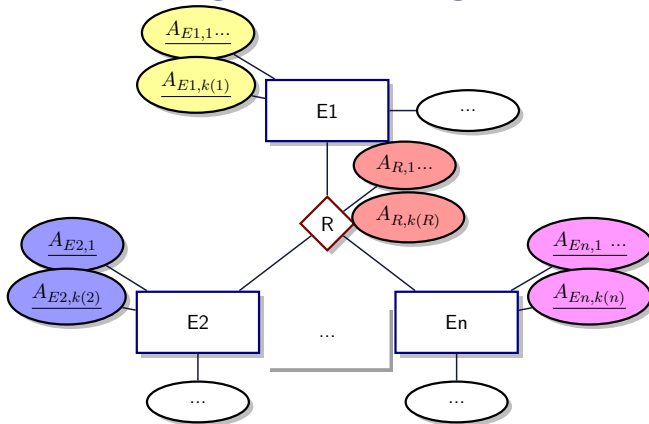
Studenten	
MatrNr	...
26120	...
27550	...
...	...

hören	
MatrNr	VorlNr
26120	5001
27550	5001
27550	4052
28106	5041
28106	5052
28106	5216
28106	5259
29120	5001
29120	5041
29120	5049

Vorlesungen	
VorlNr	...
5001	...
4052	...
...	...

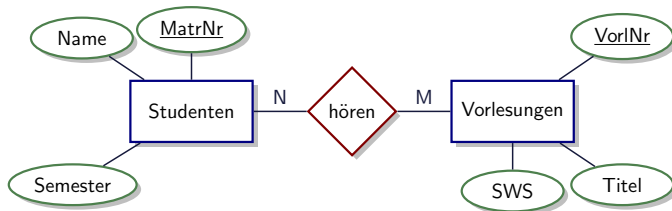


Relationale Darstellung von Beziehungen



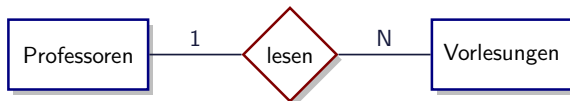
$R: \{ [\text{Schlüssel von } E_1 \mid \text{Schlüssel von } E_2 \mid \dots \mid \text{Schlüssel von } E_n \mid \text{Attribute von } R] \}$

Beziehungen unseres Beispiel-Schemas



- **hören:** { MatrNr: integer, VorlNr: integer }
- **lesen:** { [PersNr: integer, VorlNr: integer] }
- **arbeitenFür:** { [AssistentenPersNr: integer, ProfPersNr: integer] }
- **voraussetzen:** { [Vorgänger: integer, Nachfolger: integer] }
- **prüfen:** { [MatrNr: integer, VorlNr: integer, PersNr: integer, Note: decimal] }

Umsetzung von 1:N-Beziehungen



Initial-Entwurf:

- **Vorlesungen:** {[VorlNr, Titel, SWS]}
- **Professoren:** {[PersNr, Name, Rang, Raum]}
- **lesen:** {[VorlNr, PersNr]}

Umsetzung von 1:N-Beziehungen (2)

Initial-Entwurf:

- **Vorlesungen:** {[VorlNr, Titel, SWS]}
- **Professoren:** {[PersNr, Name, Rang, Raum]}
- **lesen:** {[VorlNr, PersNr]}

Verfeinerung durch Zusammenfassung:

- **Vorlesungen:** {[VorlNr, Titel, SWS, **gelesenVon**]}
- **Professoren:** {[PersNr, Name, Rang, Raum]}

Regel:

Relationen mit gleichem Schlüssel können zusammengefasst werden ...
aber nur diese und keine anderen!

Ausprägung von Professoren und Vorlesung

Professoren			
PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Vorlesungen			
VorlNr	Titel	SWS	GelesenVon
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137



Vorsicht: So geht es NICHT

Professoren				
PersNr	Name	Rang	Raum	liest
2125	Sokrates	C4	226	5041
2125	Sokrates	C4	226	5049
2125	Sokrates	C4	226	4052
...	...	...	...	...
2134	Augustinus	C3	309	5022
2136	Curie	C4	36	??
...	...	...	...	...

Vorlesungen		
VorlNr	Titel	SWS
5001	Grundzüge	4
5041	Ethik	4
5043	Erkenntnistheorie	3
5049	Mäeutik	2
4052	Logik	4
5052	Wissenschaftstheorie	3
5216	Bioethik	2
5259	Der Wiener Kreis	2
5022	Glaube und Wissen	2
4630	Die 3 Kritiken	4



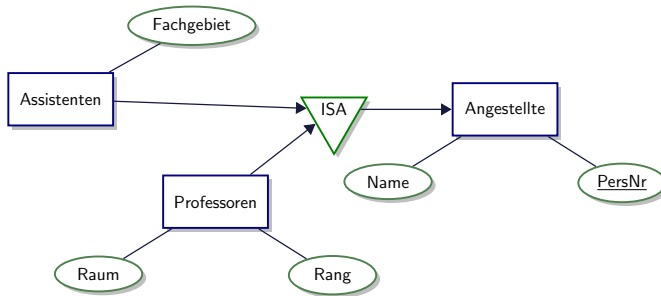
Wieso kann (wird) es Probleme geben?

Professoren				
PersNr	Name	Rang	Raum	liest
2125	Sokrates	C4	226	5041
2125	Sokrates	C4	226	5049
2125	Sokrates	C4	226	4052
...	...	...	...	...
2134	Augustinus	C3	309	5022
2136	Curie	C4	36	??
...	...	...	...	...

Vorlesungen		
VorlNr	Titel	SWS
5001	Grundzüge	4
5041	Ethik	4
5043	Erkenntnistheorie	3
5049	Mäeutik	2
4052	Logik	4
5052	Wissenschaftstheorie	3
5216	Bioethik	2
5259	Der Wiener Kreis	2
5022	Glaube und Wissen	2
4630	Die 3 Kritiken	4

- **Update-Anomalie:** Was passiert wenn Sokrates umzieht?
- **Lösch-Anomalie:** Was passiert wenn "Glaube und Wissen wegfällt?"
- **Einfüge-Anomalie:** Curie ist neu und liest noch keine Vorlesungen
- ...

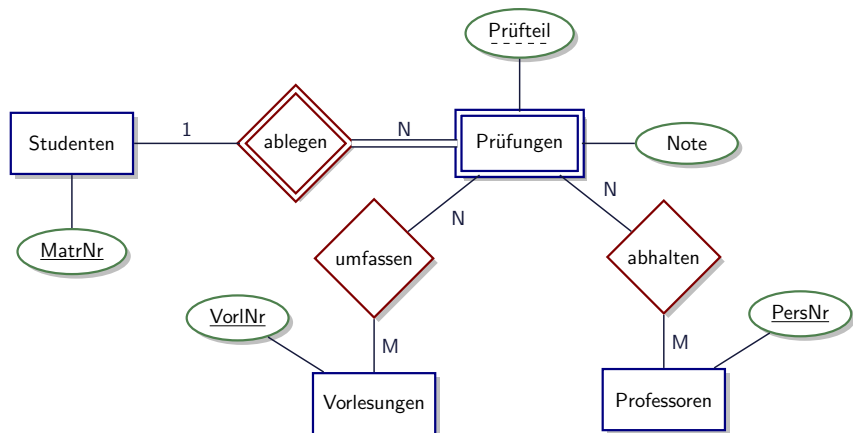
Relationale Modellierung der Generalisierung



- Angestellte: {[PersNr, Name]}
- Professoren: {[PersNr, Rang, Raum]}
- Assistenten: {[PersNr, Fachgebiet]}

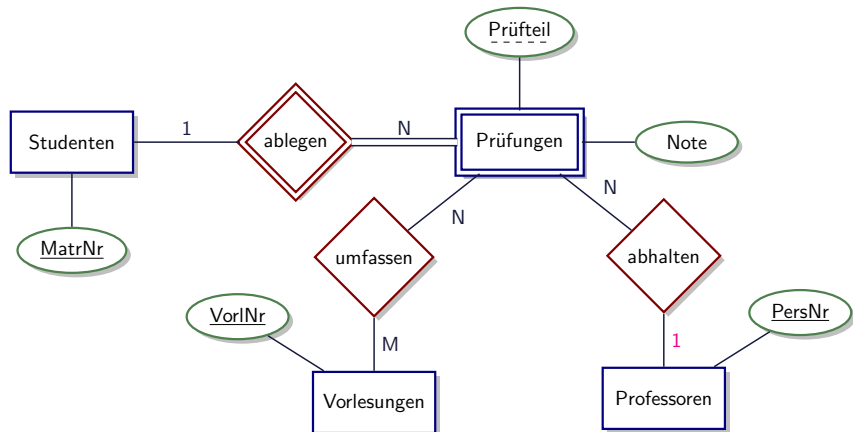
Dies ist nur eine Möglichkeit der Abbildung der Generalisierung auf Relationen. **Welche Alternativen gibt es?**

Relationale Modellierung schwacher Entitytypen



- Prüfungen: {[MatrNr: integer, Prüfteil: string, Note: integer]}
- umfassen: {[MatrNr: integer, Prüfteil: string, VorlNr: integer]}
- abhalten: {[MatrNr: integer, Prüfteil: string, PersNr: integer]}

Nur ein Prüfer je Prüfung: Was passiert?



- Prüfungen: {[MatrNr: integer, Prüfteil: string, Note: integer, **PersNr**: integer]}
- umfassen: {[MatrNr: integer, Prüfteil: string, VorlNr: integer]}

Ergebnis: Die relationale Uni-DB

Professoren			
PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Studenten		
MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Vorlesungen			
VorlNr	Titel	SWS	gelesen von
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

voraussetzen	
Vorgänger	Nachfolger
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

hören	
MatrNr	VorlNr
26120	5001
27550	5001
27550	4052
28106	5041
28106	5052
28106	5216
28106	5259
29120	5001
29120	5041
29120	5049
29555	5022
25403	5022

Assistenten			
PerslNr	Name	Fachgebiet	Boss
3002	Platon	Ideenlehre	2125
3003	Aristoteles	Syllogistik	2125
3004	Wittgenstein	Sprachtheorie	2126
3005	Rhetikus	Planetenbewegung	2127
3006	Newton	Keplersche Gesetze	2127
3007	Spinoza	Gott und Natur	2126

prüfen			
MatrNr	VorlNr	PersNr	Note
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2

Formulierung und einfache Auswertung von Anfragen

Übersicht

Inhalte der nächsten Vorlesungen:

- **Konjunktive regelbasierte Anfragen:**
 $ans(x_{na}) \leftarrow Professoren(x_{pn}, x_{na}, 'C4', x_{ra})$
- **Relationenkalküle:** $\{t.name \mid t \in Professoren \wedge t.rang = 'C4'\}$
- **Relationale Algebra:** $\pi_{name}(\sigma_{rang='C4'}(Professoren))$
- **SQL:** SELECT name FROM Professoren WHERE rang='C4'

Wir betrachten die Datenbank

CINEMA = {Movies, Location, Pariscope}

wobei die Relationen **Movies**, **Location** und **Pariscop**e die folgenden Schemata haben:

$sch(Movies) = \{Title, Director, Actor\}$

$sch(Location) = \{Theater, Address, Phone Number\}$

$sch(Pariscop)e = \{Theater, Title, Schedule\}$

Beispiele und Notationen folgen weitestgehend dem Buch “Foundations of Databases” S.Abiteboul, R. Hull und V. Vianu. PDF ist verfügbar unter <http://webdam.inria.fr/Alice/>.

Movies		
Title	Director	Actor
Immer Ärger mit Harry	Hitchcock	Gwenn
Immer Ärger mit Harry	Hitchcock	Forsythe
Immer Ärger mit Harry	Hitchcock	MacLaine
Immer Ärger mit Harry	Hitchcock	Hitchcock
....		
Schreie und Flüstern	Bergman	Andersson
Schreie und Flüstern	Bergman	Sylwan
Schreie und Flüstern	Bergman	Thulin
Schreie und Flüstern	Bergman	Ullman

Location		
Theater	Address	Phone Number
Gaumont Opéra	31 bd. des Italiens	47 42 60 33
Saint André des Arts	30 rue Saint André des Arts	43 26 48 18
Le Champo	51 rue des Ecoles	43 54 51 60
....		...
Georges V	144 av. des Champs-Élysées	45 62 41 46
Les 7 Montparnassiens	98 bd. du Montparnassiens	43 20 32 20

Pariscope		
Theater	Title	Schedule
Gaumont Opéra	Schreie und Flüstern	20:30
Saint André des Arts	Immer Ärger mit Harry	20:15
Georges V	Schreie und Flüstern	22:15
....		
Les 7 Montparnassiens	Schreie und Flüstern	20:45

Anfragen

- Wer ist der Regisseur von 'Schreie und Flüstern'?
- In welchem Theater läuft 'Schreie und Flüstern'?
- Was ist die Adresse und die Telefonnummer des Theaters 'Le Champo'?

Gib Namen und Adressen der Theater aus, die einen Bergman-Film spielen.

Zur Erinnerung:

$sch(Movies) = \{Title, Director, Actor\}$

$sch(Location) = \{Theater, Address, Phone Number\}$

$sch(Pariscope) = \{Theater, Title, Schedule\}$

Kann wie folgt berechnet werden:

Falls es jeweils Tupel r_1 , r_2 , r_3 aus den Relationen

$Movies$, $Pariscope$, $Location$ gibt, so dass

der Regisseur in r_1 'Bergman' ist

und die Titel der Tupel r_1 und r_2 gleich sind

und die Theater in Tupel r_2 und r_3 gleich sind

dann möchten wir Theater und Adresse von Tupel r_3 .

Falls es jeweils in den Relationen Movies, Pariscscope und Location Tupel $\langle x_{ti}, \text{'Bergman'}, x_{ac} \rangle$, $\langle x_{th}, x_{ti}, x_s \rangle$ und $\langle x_{th}, x_{ad}, x_p \rangle$ gibt dann nehme das Tupel $\langle \text{Theater} : x_{th}, \text{Address} : x_{ad} \rangle$ in die Antwort auf.

x_{ti} , x_{th} etc. sind Variablen.

Wir können die Anfrage umschreiben als

$$ans(x_{th}, x_{ad}) \leftarrow \text{Movies}(x_{ti}, \text{'Bergman'}, x_{ac}), \text{Pariscscope}(x_{th}, x_{ti}, x_s), \text{Location}(x_{th}, x_{ad}, x_p)$$

wobei ans (für Answer/Antwort) eine Relation über $\{\text{Theater}, \text{Address}\}$ ist

Kopf (Head) und Rumpf (Body)

Der Ausdruck links von $\leftarrow$ wird als Kopf bezeichnet. Der Ausdruck rechts von $\leftarrow$ heisst Rumpf.

$$\text{ans}(x_{th}, x_{ad}) \leftarrow \text{Movies}(x_{ti}, \text{'Bergman'}, x_{ac}), \text{Pariscope}(x_{th}, x_{ti}, x_s), \\ \text{Location}(x_{th}, x_{ad}, x_p)$$

Der obige Ausdruck kann noch vereinfacht werden als

$$\text{ans}(x_{th}, x_{ad}) \leftarrow \text{Movies}(x_{ti}, \text{'Bergman'}, _), \text{Pariscope}(x_{th}, x_{ti}, _), \\ \text{Location}(x_{th}, x_{ad}, _)$$

wobei das Zeichen $_$ benutzt wird, um alle Variablen zu ersetzen, die nur ein Mal auftreten.

Regelbasierte Konjunktive Anfrage

Sei $\mathbf{R}$ ein Datenbankschema. Eine **regelbasierte konjunktive Anfrage** über $\mathbf{R}$ hat die Form

$$ans(u) \leftarrow R_1(u_1), \dots, R_n(u_n)$$

- $u, u_1, \dots, u_n$ sind Tupel
- **Notation:** falls $v = \langle x_1, \dots, x_m \rangle$ dann schreiben wir $R(v)$ anstelle von $R(x_1, \dots, x_m)$
- u_i muss Stelligkeit (arity) passend zu R_i haben.
- Jede Variable aus u muss mindestens ein Mal in einem der $u_1, \dots, u_n$ auftreten.

Semantik von Regelbasierten Konjunktiven Anfragen

Betrachtet man eine **Instanz einer Datenbank**, also Ausprägungen von Tupeln in den einzelnen Relationen, dann kann man sich eine Regel so vorstellen, dass wenn man eine gültige Belegung des Rumpfes gefunden hat, der Kopf der Regel das Ergebnis darstellt.

Betrachten wir wieder Anfrage q als

$$ans(u) \leftarrow R_1(u_1), \dots, R_n(u_n)$$

und eine Instanz (Ausprägung) $\mathbf{I}$ der Datenbank $\mathbf{R}$. **Das Abbild von $\mathbf{I}$ unter Anfrage q ist**

$$q(\mathbf{I}) = \{\nu(u) \mid \nu \text{ ist eine Belegung der Variablen aus } q \\ \text{und } \nu(u_i) \in I(R_i), \forall i \in [1, n]\}$$

Belegung von Variablen: Beispiel

$$ans(x_{th}, x_{ad}) \leftarrow \text{Movies}(x_{ti}, \text{'Bergman'}, x_{ac}), \text{Pariscope}(x_{th}, x_{ti}, x_s), \\ \text{Location}(x_{th}, x_{ad}, x_p)$$

Betrachten wir folgende Belegung der Variablen:

 $\nu(x_{ti}) = \text{'Schreie und Flüstern'}$ $\nu(x_{ac}) = \text{'Ullman'}$ $\nu(x_{th}) = \text{'Gaumont Opera'}$ $\nu(x_s) = \text{'20:30'}$ $\nu(x_{ad}) = \text{'31 bd. des Italiens'}$ $\nu(x_p) = \text{'47 42 60 33'}$

Dies ist eine gültige Belegung, da Tupel $(\nu(x_{ti}), \text{'Bergman'}, \nu(x_{ac})) = (\text{'Schreie und Flüstern'}, \text{'Bergman'}, \text{'Ullman'}) \in I(\text{Movies})$, für unsere Beispiel Ausprägung I , analog für die beiden anderen Relationen bzw. Tupel.

Weitere Beispiele

Finde Name und Personalnummer aller C4 Professoren:

$$ans(x_{na}, x_{pn}) \leftarrow Professoren(x_{pn}, x_{na}, 'C4', x_{ra})$$

Welche Vorlesungen bietet Prof. Sokrates an?

$$ans(x_{ti}) \leftarrow Professoren(x_{pn}, 'Sokrates', x_{rg}, x_{ra}), \\ Vorlesungen(x_{vn}, x_{ti}, x_{sw}), lesen(x_{pn}, x_{vn})$$

Intensionale vs. Extensionale Relationen

Wir berechnen durch die Erstellung von Anfragen nicht nur Ergebnisse, sondern **definieren dabei implizit auch neue Relationen**, z.B. die neue Relation *C4Profs*, die alle Professoren mit Rang C4 enthält:

$$C4Profs(x_{pn}, x_{na}, x_{ra}) \leftarrow Professoren(x_{pn}, x_{na}, 'C4', x_{ra})$$

Man unterscheidet zwischen Relationen, die ursprünglich in der Datenbank vorhanden sind, den sogenannten **extensionalen** Relationen, und Relationen, die durch Regeln definiert werden, den sogenannten **intensionalen** Relationen.

Auswertung (aka. Berechnung) der Anfrage

Naive brute-force Methode zur Berechnung der Anfrageergebnisse:

- Betrachtung aller möglichen Belegung anhand der in den Relationen auftretenden Werte.
- Für jede dieser Belegungen schauen ob es eine gültige Belegung ist.

Diese Vorgehensweise ist natürlich sehr teuer. Z.B. für $Movies(x_1, 'Bergman', x_2)$ machen nur Belegungen Sinn, die für dieses Tupel mit x_1 und x_2 aus $Movies$ den Regisseur gleich 'Bergman' haben. Idealerweise kann man mit sogenannten Indexen die Auswahl die Suche nach geeigneten Tupeln beschleunigen.

Dazu kommen wir später im **Abschnitt über Anfrageverarbeitung und Indexstrukturen**.

Monotonie

- **Monotonie:** Eine Anfrage q über R ist monoton, wenn für alle Ausprägungen (Instanzen) I, J über R , gilt: Falls $I \subseteq J$ dann $q(I) \subseteq q(J)$.
- D.h. wenn neue Tupel zu den Relationen hinzukommen, fallen keine Ergebnistupel weg.

Konjunktive Anfragen sind monoton

Welche Anfragen sind somit nicht möglich?

Anfragen im konjunktiven Kalkül

Gegeben eine konjunktive regelbasierte Anfrage

$$ans(e_1, \dots, e_m) \leftarrow R_1(u_1), \dots, R_n(u_n)$$

Die dazu äquivalente Anfrage im **konjunktiven Kalkül** ist

$$\{e_1, \dots, e_m \mid \exists x_1, \dots, x_k (R_1(u_1) \wedge \dots \wedge R_n(u_n))\}$$

Die Variablen $x_1, x_2, \dots, x_k$ sind die Variablen, die im Rumpf aber nicht im Kopf der regelbasierten Anfrage auftreten.

Anfragen im konjunktiven Kalkül: Beispiel

$$\begin{aligned}ans(x_{th}, x_{ad}) \leftarrow & \textit{Movies}(x_{ti}, \text{'Bergman'}, x_{ac}), \\ & \textit{Pariscope}(x_{th}, x_{ti}, x_s), \\ & \textit{Location}(x_{th}, x_{ad}, x_p)\end{aligned}$$

$$\begin{aligned}ans := \{x_{th}, x_{ad} \mid \exists x_{ti} \exists x_{ac} \exists x_s \exists x_p (\textit{Movies}(x_{ti}, \text{'Bergman'}, x_{ac}) \wedge \\ \textit{Pariscope}(x_{th}, x_{ti}, x_s) \wedge \\ \textit{Location}(x_{th}, x_{ad}, x_p))\}\end{aligned}$$

Sei $\mathbf{R}$ ein Datenbankschema. Eine Formel über $\mathbf{R}$ im konjunktiven Kalkül ist ein Ausdruck, der eine der folgenden Formen besitzt:

- (a) ein Atom über $\mathbf{R}$ der Form $R_i(u_i)$
- (b) $(\phi \wedge \psi)$, mit Formeln ϕ und ψ über $\mathbf{R}$; oder
- (c) $\exists x\phi$, wobei x eine Variable ist und ϕ ist eine Formel über $\mathbf{R}$.

Freie Variablen:

Ein Auftreten einer Variablen x in einer Formel ϕ ist **frei** falls

- (i) ϕ ist ein Atom; oder
- (ii) $\phi = (\psi \wedge \xi)$ und das Auftreten von x ist frei in ψ oder ξ ; oder
- (iii) $\phi = \exists y\psi$, x und y sind verschiedene Variablen, und das Auftreten von x ist frei in ψ

Falls eine Variable nicht frei ist, so nennen wir sie **gebunden**.

Anfragen im konjunktiven Kalkül

Eine Anfrage im konjunktiven Kalkül über Datenbankschema R ist ein Ausdruck der Form

$$\{e_1, \dots, e_m \mid \phi\}$$

wobei ϕ eine Formel des konjunktiven Kalküls ist und die Variablen $e_1, \dots, e_m$ identisch zur Menge der freien Variablen in ϕ .

Theorem

Regelbasierte konjunktive Anfragen und Anfragen im konjunktiven Kalkül sind äquivalent.

Gleichheit bzw. Vergleichsoperatoren

$$\begin{aligned}ans(x_{th}, x_{ad}) \leftarrow & \textit{Movies}(x_{ti}, x_d, x_{ac}), x_d = \text{'Bergman'}, \\ & \textit{Pariscope}(x_{th}, x_{ti}, x_s), \\ & \textit{Location}(x_{th}, x_{ad}, x_p)\end{aligned}$$

ist identisch zu

$$\begin{aligned}ans(x_{th}, x_{ad}) \leftarrow & \textit{Movies}(x_{ti}, \text{'Bergman'}, x_{ac}), \\ & \textit{Pariscope}(x_{th}, x_{ti}, x_s), \\ & \textit{Location}(x_{th}, x_{ad}, x_p)\end{aligned}$$

Ebenfalls möglich Vergleichsoperatoren, z.B.

$$ans(x_{ma}) \leftarrow \textit{Studenten}(x_{ma}, x_{na}, x_{se}), x_{se} \geq 12$$

Anfrage-Programm

Wie erwähnt, kann konzeptionell eine Anfrage so aufgefasst werden, dass sie eine neue Relation erzeugt, die dann in nachfolgenden Anfragen benutzt werden kann.

Ein **konjunktives Anfrage-Programm** ist eine Sequenz von Regeln in der Form

$$\begin{array}{lcl} S_1(u_1) & \leftarrow & body_1 \\ S_2(u_2) & \leftarrow & body_2 \\ & \dots & \\ S_m(u_m) & \leftarrow & body_m \end{array}$$

Rekursion sei nicht zugelassen.

Beispiel

$$S_1(x, z) \leftarrow Q(x, y), R(y, z, w)$$

$$S_2(x, y, z) \leftarrow S_1(x, w), R(w, y, v), S_1(v, z)$$

$$S_3(x, z) \leftarrow S_2(x, u, v), Q(v, z)$$

Q	
1	2
2	1
2	2

R		
1	1	1
2	3	1
3	1	2
4	4	1

S_1	
1	3
2	1
2	3

S_2		
1	1	1
1	1	3
2	1	1
2	1	3

S_3	
1	2
2	2

Beispiel

$$\begin{aligned}S_1(x, z) &\leftarrow Q(x, y), R(y, z, w) \\S_2(x, y, z) &\leftarrow S_1(x, w), R(w, y, v), S_1(v, z) \\S_3(x, z) &\leftarrow S_2(x, u, v), Q(v, z)\end{aligned}$$

Wir können die ersten beiden Zeilen auch schreiben als

$$\begin{aligned}S_2(x, y, z) &\leftarrow Q(x_1, y_1), R(y_1, z_1, w_1), x = x_1, w = z_1, \\&\quad R(w, y, v), Q(x_2, y_2), R(y_2, z_2, w_2), v = x_2, z = z_2\end{aligned}$$

bzw. ohne Gleichheits-Operator

$$\begin{aligned}S_2(x, y, z) &\leftarrow Q(x, y_1), R(y_1, w, w_1), \\&\quad R(w, y, v), Q(v, y_2), R(y_2, z, w_2)\end{aligned}$$

Sichten (Englisch: Views)

$$\begin{aligned} Marilyn(x_t) &\leftarrow Movies(x_t, x_d, 'Monroe') \\ ChampoInfo(x_t, x_s, x_p) &\leftarrow Pariscope('Le Champo', x_t, x_s), \\ &\quad Location('Le Champo', x_a, x_p) \end{aligned}$$

Versuchen wir damit die folgende Anfrage auszudrücken: **“Welche Titel in Marilyn werden im Le Champo um 21:00 Uhr gespielt?”**

$$ans(x_t) \leftarrow Marilyn(x_t), ChampoInfo(x_t, '21:00', x_p)$$

Falls diese Sichten nur virtuell sind (also nicht wirklich materialisiert/ausgeprägt), so wird effektiv folgende Anfrage ausgeführt:

$$\begin{aligned} ans(x_t) &\leftarrow Movies(x_t, x_d, 'Monroe'), \\ &\quad Pariscope('Le Champo', x_t, '21:00'), \\ &\quad Location('Le Champo', x_a, x_p) \end{aligned}$$

Views (2)

Eine alternative Formulierung:

$$Marilyn := \{x_t \mid \exists x_d(Movies(x_t, x_d, 'Monroe'))\};$$
$$ChampoInfo := \{x_t, x_s, x_p \mid \exists x_a(Pariscope('Le Champo', x_t, x_s) \\ \wedge Location('Le Champo', x_a, x_p))\};$$
$$ans := \{x_t \mid Marilyn(x_t) \\ \wedge \exists x_p(ChampoInfo(x_t, '21:00', x_p))\}$$

Was ist mit Negation?

Lassen wir nun Literale L_i der Form $R(v)$ oder $\neg R(v)$ zu, d.h. wir haben Regeln

$$q : S(u) \leftarrow L_1, \dots, L_n$$

wobei R der Name einer Relation ist, v ist ein Tupel der passenden Stelligkeit und S tritt nicht im Rumpf der Regel auf (also: nicht rekursiv).

“In welchen Hitchcock Filmen hat Hitchcock nicht mitgespielt?”

$$\begin{aligned} ans(x) \leftarrow & \textit{Movies}(x, \text{'Hitchcock'}, z), \\ & \neg \textit{Movies}(x, \text{'Hitchcock'}, \text{'Hitchcock'}) \end{aligned}$$

Was muss in der Datenbank gelten, damit diese Anfrage korrekt ist?

„Gib alle Filme aus, bei denen nur Schauspieler mitgewirkt haben,
die schonmal unter Hitchcock Regie gespielt haben.“

$$Hitch-actor(z) \leftarrow Movies(x, 'Hitchcock', z)$$

$$not-ans(x) \leftarrow Movies(x, y, z), \neg Hitch-actor(z)$$

$$ans(x) \leftarrow Movies(x, y, z), \neg not-ans(x)$$

Relationenkalküle: Das Domänenkalkül

Wenn wir zum konjunktiven Kalkül die **Negation** hinzunehmen, erhalten wir ein **Relationenkalkül**, nämlich das **Domänenkalkül**. Es gibt auch noch ein weiteres Relationenkalkül, das **Tupelkalkül**, welches wir uns später ansehen.

Wir erlauben nun auch gleich noch weitere **Vergleichsoperatoren**, haben also Basis-Formeln (Atome) der Form $R(u)$ mit Relation R und Tupel u über Domänenvariablen und Konstanten, sowie Atome der Form $e = e'$, $e \neq e'$, $e \geq e'$ etc. für Domänenvariablen oder Konstanten e und e' .

Alle Atome sind Formeln. Darauf basierend haben wir **Formeln der Form**

- $(\phi \wedge \psi)$ für Formeln ψ und ϕ
- $(\phi \vee \psi)$ für Formeln ψ und ϕ
- $\neg\phi$ für Formeln ϕ
- $\exists x\phi$ für Variable x und Formel ϕ
- $\forall x\phi$ für Variable x und Formel ϕ

“In welchen Hitchcock Filmen hat Hitchcock nicht mitgespielt?”

$$\{x_t \mid \exists x_a \text{Movies}(x_t, \text{'Hitchcock'}, x_a) \\ \wedge \neg \text{Movies}(x_t, \text{'Hitchcock'}, \text{'Hitchcock'})\}$$

“Gib alle Filme aus, bei denen nur Schauspieler mitgewirkt haben, die schonmal unter Hitchcock Regie gespielt haben.”

$$\{x_t \mid \exists x_d, x_a \text{Movies}(x_t, x_d, x_a) \\ \wedge \forall y_a (\exists y_d \text{Movies}(x_t, y_d, y_a) \\ \Rightarrow \exists z_t \text{Movies}(z_t, \text{'Hitchcock'}, y_a))\}$$

Das Relationale Tupelkalkül

Das bislang betrachtete Relationen Kalkül wird auch als **relationales Domänenkalkül** bezeichnet, da dort Domänenvariablen an die Domänen der einzelnen Attribute (Spalten) der Relationen gebunden werden.

Eine Anfrage im **relationalen Tupelkalkül** hat die Form

$$\{t \mid P(t)\}$$

wobei t eine **Tupelvariable** ist und $P(t)$ ein Prädikat (eine Formel). $P(t)$ muss erfüllt sein, damit t Teil des Ergebnis ist.

$$\{s.title \mid s \in Movies (s.director = 'Hitchcock' \\ \wedge \neg \exists u \in Movies (u.title = s.title \wedge u.actor = 'Hitchcock'))\}$$

Das Relationale Tupelkalkül: Definition

Formeln des **Tupelkalküls** bestehen aus Atomen der Form $t \in R$, mit Tupelvariable t und Relation R , der Form $t.x = t.y$, $t.x > t.y$, etc. für Tupelvariablen t und s und Attributnamen x und y , sowie der Form $t.x = c$ und $t.x > c$ etc. für Konstanten c .

Alle Atome sind Formeln und dann haben wir noch **Formeln der Form**

- $(\phi \wedge \psi)$ für Formeln ψ und ϕ
- $(\phi \vee \psi)$ für Formeln ψ und ϕ
- $\neg\phi$ für Formeln ϕ
- $\exists x\phi$ für Variable x und Formel ϕ
- $\forall x\phi$ für Variable x und Formel ϕ

Beispiele

- C4-Professoren:**

$$\{p \mid p \in Professoren \wedge p.Rang = C4\}$$

- Paare von Professoren (Name) und Assistenten (PersNr):**

$$\{p.Name, a.PersNr \mid p \in Professoren \wedge a \in Assistenten \\ \wedge p.PersNr = a.Boss\}$$

- Studenten mit mindestens einer Vorlesung von Prof. Curie:**

$$\{s \mid s \in Studenten \\ \wedge \exists h \in hören (s.MatrNr = h.MatrNr \\ \wedge \exists v \in Vorlesungen (h.VorlNr = v.VorlNr \\ \wedge \exists p \in Professoren (p.PersNr = v.gelesenVon \\ \wedge p.Name = 'Curie'))))\}$$

.... in SQL ... ist es sehr ähnlich:

Studenten mit mindestens einer Vorlesung von Prof. Curie:

```
SELECT s.*  
FROM Studenten s  
WHERE EXISTS (  
    SELECT h.*  
    FROM hören h  
    WHERE h.MatrNr=s.MatrNr AND EXISTS (  
        SELECT *  
        FROM Vorlesungen v  
        WHERE v.VorlNr=h.VorlNr AND EXISTS (  
            SELECT *  
            FROM Professoren p  
            WHERE p.Name='Curie' AND p.PersNr=v.gelesenVon)))
```

Sicherheit von Anfragen

- **Einschränkung auf Anfragen mit endlichem Ergebnis.**
- Zum Beispiel ist die Anfrage

$$\{n \mid \neg(n \in Professoren)\}$$

nicht sicher.

- **Das Ergebnis ist unendlich.**
- **Bedingung: Ergebnis des Ausdrucks muss Teilmenge der Domäne der Formel sein.**
- Die **Domäne** einer Formel enthält
 - alle in der Formel vorkommenden Konstanten
 - alle Attributwerte von Relationen, die in der Formel referenziert werden

Unsichere Anfragen im Domänenkalkül

$$\{x \mid \neg \text{Movies}(\text{'Schreie und Flüstern'}, \text{'Bergman'}, x)\}$$

$$\{x_{pn}, x_{na}, x_{rg}, x_{ra} \mid \neg \text{Professoren}(x_{pn}, x_{na}, x_{rg}, x_{ra})\}$$

$$\{x, y \mid \text{Movies}(\text{'Schreie und Flüstern'}, \text{'Bergman'}, x) \\ \vee \text{Movies}(y, \text{'Bergman'}, \text{'Ullman'})\}$$

Wieso sind diese Anfragen unsicher?

Sicherheit des Domänenkalküls

- Sicherheit ist analog zum Tupelkalkül, aber etwas komplizierter, da Variablen nicht an Tupel einer Relation sondern an einzelne Domänen gebunden sind.
- Zum Beispiel ist

$$\{x_{pn}, x_{na}, x_{rg}, x_{ra} \mid \neg(\text{Professoren}(x_{pn}, x_{na}, x_{rg}, x_{ra}))\}$$

nicht sicher.

- Ein Ausdruck

$$\{x_1, x_2, \dots, x_n \mid P(x_1, x_2, \dots, x_n)\}$$

ist **sicher**, falls folgende drei Bedingungen gelten:

$\{x_1, x_2, \dots, x_n | P(x_1, x_2, \dots, x_3)\}$ sicher falls

1. Falls **Tupel** $(c_1, c_2, \dots, c_n)$ mit Konstanten c_i im Ergebnis enthalten ist, so muss jedes c_i ($1 \leq i \leq n$) in der Domäne von P enthalten sein.
2. **Für jede existenz-quantifizierte Teilformel** $\exists x(P_1(x))$ muss gelten, dass P_1 nur für Elemente aus der Domäne erfüllbar sein kann – oder evtl. für gar keine.

Das heißt: Wenn für eine Konstante c das Prädikat $P_1(c)$ erfüllt ist, so muss c in der Domäne von P_1 enthalten sein.

3. **Für jede universal-quantifizierte Teilformel** $\forall x(P_1(x))$ muss gelten, dass sie dann und nur dann erfüllt ist, wenn $P_1(x)$ für alle Werte der Domäne von P_1 erfüllt ist.

Das heißt: $P_1(d)$ muss für alle d , die nicht in der Domäne von P_1 enthalten sind, auf jeden Fall erfüllt sein.

Zusammenfassung

- **Konjunktive regelbasierte Anfragen und Kalküle**, als regel- bzw. logikbasierte Möglichkeit Anfragen zu erstellen
- Dabei geben diese Anfragen vor, **wie Ergebnistupel aussehen sollen bzw. wie sie zusammengehören, aber nicht wie das Ergebnis tatsächlich berechnet werden soll.**
- Sogenannte **deklarative Anfragen**.
- **Domänenkalkül und Tupelkalkül gleich ausdrucksstark (mächtig).**
- Wie bereits erwähnt, **ist auch SQL deklarativ und durch das relationale Tupelkalkül inspiriert.**
- Wir betrachten nun die relationale Algebra.
- Und danach SQL.

Übersicht

- Konjunktive regelbasierte Anfragen:
 $ans(x_{na}) \leftarrow Professoren(x_{pn}, x_{na}, 'C4', x_{ra})$
- Relationenkalküle: $\{t.name \mid t \in Professoren \wedge t.rang = 'C4'\}$
- **Relationale Algebra**: $\pi_{name}(\sigma_{rang='C4'}(Professoren))$
- **SQL**: SELECT name FROM Professoren WHERE rang='C4'

Die Operatoren der relationalen Algebra

- Selektion σ
- Projektion π
- Kreuzprodukt $\times$
- Join (Verbund) $\bowtie$
- Umbenennung ρ
- Differenz $\setminus$
- Division $\div$
- Vereinigung $\cup$
- Schnitt $\cap$
- Semi-Join (linker) $\ltimes$
- Semi-Join (rechter) $\rtimes$
- linker äußerer Join $\ltimes\Join$
- rechter äußerer Join $\Join\rtimes$
- äußerer Join $\Join\Join$

Dazu kommen später noch ein paar Erweiterungen.

Grundlagen der relationalen Algebra

Es gibt **Relationen** (Ausprägungen: $R \subseteq D_1 \times D_2 \times \dots \times D_n$) und **Relationenschema**: $\{[\text{attributname1: datentyp1, attributname2: datentyp2, ...}]\}$.

- Das Schema einer Relation wird auch mit $sch(R)$ oder $\mathcal{R}$ beschrieben.
- Relationen sind Mengen (keine Multimengen)
- Reihenfolge der Tupel sowie Reihenfolge der Attribute spielt keine Rolle (Attribute haben Namen).

Die Operatoren und ihre Verwendung:

- Eingabe eines Operators ist eine oder mehrere Relationen.
- **Ausgabe ist auch wieder eine Relation.**
- D.h. Operatoren sind kombinierbar (mit gewissen Regeln).

Beispielrelationen

Professoren			
PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Vorlesungen			
VorlNr	Titel	SWS	GelesenVon
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Projektion

Projektion

$\pi_{Rang}(Professoren)$

$\pi_{Rang}(Professoren)$
Rang
C4
C3

- Allgemein: $\pi_{A_1, A_2, \dots, A_k}(R)$, für Attribute A_i aus $\mathcal{R}$
- Wählt aus Attributen der Argumentrelation die angegebenen aus.

Selektion

Selektion

$\sigma_{Semester > 10}(Studenten)$

$\sigma_{Semester > 10}(Studenten)$		
MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12

- Allgemein: σ_F
- Selektionsprädikat F besteht aus
 - Operatoren: $\vee$ (oder) $\wedge$ (und) $\neg$ (nicht)
 - Arithmetischen Vergleichsoperatoren: $<$, $\leq$, $=$, $>$, $\geq$, $\neq$
 - ... und natürlich aus Attributnamen der Argumentrelation oder Konstanten als Operanden

Vereinigung, Schnitt und Mengendifferenz

Zwei Relationen mit gleichem Schema (=Attributnamen und Domänen) können zusammengefasst werden.

Falls Schema nicht gleich: evtl. Attribute umbenennen

Beispiel:

$$\pi_{PersNr, Name}(Assistenten) \cup \pi_{PersNr, Name}(Professoren)$$

=Projektion zweier Relationen auf gemeinsame Attribute, gefolgt von der Vereinigung.

Analog dazu: der Schnitt zweier Relationen.

Mengendifferenz:

$$R \setminus S$$

enthält alle Tupel der Relation R , die nicht in Relation S sind.

Kartesisches Produkt (Kreuzprodukt)

Gegeben zwei Relationen R und S :

$$R \times S$$

beinhaltet **ALLE!** $|R| * |S|$ möglichen Paare von Tupeln aus R und S .

Enthält viele (oft auch viele unsinnige) Kombinationen!

Das **resultierende Schema** $sch(R \times S) = sch(R) \cup sch(S) = \mathcal{R} \cup \mathcal{S}$.

Beim Referenzieren der Attribute des resultierenden Schemas wird $R.X$ und $S.Y$ verwendet, insbesondere bei Überlappungen der einzelnen Schemata (=keine Unklarheiten was gemeint ist). Oder Eindeutigkeit durch Umbenennung.

Umbenennung

Gegeben eine Relation R . Dann steht

$$\rho_S(R)$$

für die **Relation R unter neuem Namen S** . Oder: $\rho_{A \leftarrow B}(R)$ für Umbenennung von Attribut B in A .

Umbenennen von Attributen und/oder Relationen ist oftmals notwendig.

Beispiel: Was wird hier berechnet?

$$\pi_{V1.Vorgaenger}(\sigma_{V2.Nachfolger=5216 \wedge V1.Nachfolger=V2.Vorgaenger}(\rho_{V1}(voraussetzen) \times \rho_{V2}(voraussetzen)))$$

V1		V2	
Vorgänger	Nachfolger	Vorgänger	Nachfolger

Der natürliche Verbund (Join)

Gegeben zwei Relationen (+ Schemata) mit gemeinsamen Attributen $B_1, \dots, B_k$

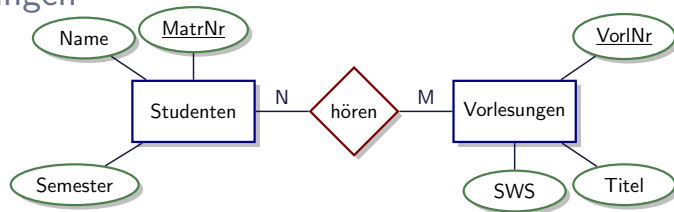
- $R(A_1, \dots, A_m, \mathbf{B}_1, \dots, \mathbf{B}_k)$
- $S(\mathbf{B}_1, \dots, \mathbf{B}_k, C_1, \dots, C_n)$

Der natürliche Verbund (Join) wird berechnet durch

$$R \bowtie S = \pi_{A_1, \dots, A_m, R.B_1, \dots, R.B_k, C_1, \dots, C_n} (\sigma_{R.B_1=S.B_1 \wedge \dots \wedge R.B_k=S.B_k} (R \times S))$$

$R \bowtie S$											
$\mathcal{R} - \mathcal{S}$				$\mathcal{R} \cap \mathcal{S}$				$\mathcal{S} - \mathcal{R}$			
A_1	A_2	...	A_m	B_1	B_2	...	B_k	C_1	C_2	...	C_n
...	...	...	...	...	...	...	...	...	...	...	...

Anwendung von Joins bzgl. Schlüssel/Fremdschlüssel Beziehungen



- **hören:** { MatrNr: integer, VorlNr: integer }

Im Beispiel hier ist das Attribut MatrNr aus hören ein Fremdschlüssel, der auf MatrNr in der Relation Studenten verweist. Analog für Attribut VorlNr, welches auf VorlNr der Relation Vorlesungen verweist.

Mittels Join können für jedes Tupel aus der Relation hören die zu MatrNr und VorlNr passenden Studenten und Vorlesungen gefunden werden.

Mehrere Joins zusammen

- Es können beliebig viele Relationen durch Join Operatoren verknüpft werden.
- Die Reihenfolge spielt dabei keine Rolle (**Joins sind kommutativ und assoziativ**).

$$(Studenten \bowtie hoeren) \bowtie Vorlesungen$$

$(Studenten \bowtie hoeren) \bowtie Vorlesungen$						
MatrNr	Name	Semester	VorlNr	Titel	SWS	gelesenVon
26120	Fichte	10	5001	Grundzüge	4	2137
27550	Jonas	12	5022	Glaube und Wissen	2	2134
28106	Carnap	3	4052	Wissenschaftstheorie	3	2126
...	...	...	...	...	...	...

Allgemeiner Join (Theta-Join)

Gegeben zwei Relationen (+ Schemata)

- $R(A_1, \dots, A_n)$
- $S(B_1, \dots, B_m)$

θ ist ein beliebiges Prädikat über den beteiligten Attributen.

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

$R \bowtie S$							
$\mathcal{R}$				$\mathcal{S}$			
A_1	A_2	...	A_n	B_1	B_2	...	B_m
...	...	...	...	...	...	...	...

Equi-Join: Join-Prädikat θ darf nur auf Gleichheit (=) prüfen.

Überlegungen zum Join

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

Der Join-Operator macht also die relationale Algebra gar nicht ausdrucksstärker, da er durch das Kreuzprodukt mit anschließender Selektion ausgedrückt werden kann, bzw. definiert ist.

Dies gibt auch bereits die erste Idee vor, **wie der Join zweier Relationen berechnet werden kann:**

- Betrachte alle möglichen Paare von Tupeln aus $R \times S$
- Wende Prädikat θ an und schaue ob es "true" liefert.
- Falls ja verknüpfe Tupel und füge resultierendes Tupel in Ergebnis ein.

Weitere Join Typen: Äußere Joins

Diese Join Typen spezifizieren wie mit Tupeln umgegangen wird, die keinen Joinpartner gefunden haben.

- **linker äußerer Join** ($\bowtie\leftarrow$): auch “partnerlose” Tupel der linken Relation bleiben erhalten
- **rechter äußerer Join** ($\rightarrow\bowtie$): auch “partnerlose” Tupel der rechten Relation bleiben erhalten
- **vollständiger äußerer Join** ($\bowtie\leftarrow\rightarrow$): die “partnerlosen” Tupel beider Relationen bleiben erhalten

Natürlicher Join und linker äußerer Join

Natürlicher Join

L				R				Resultat				
A	B	C		C	D	E		A	B	C	D	E
a_1	b_1	c_1	$\bowtie$	c_1	d_1	e_1	$=$	a_1	b_1	c_1	d_1	e_1
a_2	b_2	c_2		c_3	d_2	e_2						

Linker äußerer Join

L				R				Resultat				
A	B	C		C	D	E		A	B	C	D	E
a_1	b_1	c_1	$\bowtie_{\text{L}}$	c_1	d_1	e_1	$=$	a_1	b_1	c_1	d_1	e_1
a_2	b_2	c_2		c_3	d_2	e_2		a_2	b_2	c_2	$\perp$	$\perp$

Rechter äußerer und äußerer Join

Rechter äußerer Join

L				R				Resultat				
A	B	C		C	D	E		A	B	C	D	E
a_1	b_1	c_1	$\bowtie$	c_1	d_1	e_1	=	a_1	b_1	c_1	d_1	e_1
a_2	b_2	c_2		c_3	d_2	e_2		$\perp$	$\perp$	c_3	d_2	e_2

(Voller) Äußerer Join

L				R				Resultat				
A	B	C		C	D	E		A	B	C	D	E
a_1	b_1	c_1	$\bowtie$	c_1	d_1	e_1	=	a_1	b_1	c_1	d_1	e_1
a_2	b_2	c_2		c_3	d_2	e_2		a_2	b_2	c_2	$\perp$	$\perp$
								$\perp$	$\perp$	c_3	d_2	e_2

Weitere Join Typen: Semi Joins

Idee: Finde alle Tupel der linken Relation, die Joinpartner in der rechten Relation haben.

$$L \ltimes R = \pi_{\mathcal{L}}(L \bowtie R)$$

$L \rtimes R$ ist analog definiert.

Es gilt:

$$L \rtimes R = R \ltimes L$$

allerdings sind Semi-Joins sowie die linken und rechten äußeren Joins **nicht kommutativ!**

Semi-Joins

Semi-Join von L mit R

L				R				Resultat		
A	B	C		C	D	E		A	B	C
a_1	b_1	c_1	$\bowtie$	c_1	d_1	e_1	$=$	a_1	b_1	c_1
a_2	b_2	c_2		c_3	d_2	e_2				

Semi-Join von R und L

L				R				Resultat		
A	B	C		C	D	E		C	D	E
a_1	b_1	c_1	$\bowtie$	c_1	d_1	e_1	$=$	c_1	d_1	e_1
a_2	b_2	c_2		c_3	d_2	e_2				

Anti-Joins: Tupel im Ergebnis, falls kein Joinpartner existiert

Anti-Join von L mit R

L		
A	B	C
a_1	b_1	c_1
a_2	b_2	c_2

▷

R		
C	D	E
c_1	d_1	e_1
c_3	d_2	e_2

=

Resultat		
A	B	C
a_2	b_2	c_2

Anti-Join von R mit L

L		
A	B	C
a_1	b_1	c_1
a_2	b_2	c_2

◁

R		
C	D	E
c_1	d_1	e_1
c_3	d_2	e_2

=

Resultat		
C	D	E
c_3	d_2	e_2

$$\begin{aligned}
e_1 \times e_2 &:= \{y \circ x \mid y \in e_1 \wedge x \in e_2\} \\
e_1 \bowtie_{\theta} e_2 &:= \{y \circ x \mid y \in e_1 \wedge x \in e_2 \wedge \theta(y, x)\} \\
e_1 \ltimes_{\theta} e_2 &:= \{y \mid y \in e_1 \wedge \exists x \in e_2 \text{ s.t. } \theta(y, x)\} \\
e_1 \rhd_{\theta} e_2 &:= \{y \mid y \in e_1 \wedge \neg \exists x \in e_2 \text{ s.t. } \theta(y, x)\} \\
e_1 \Join_{\theta} e_2 &:= (e_1 \bowtie e_2) \cup ((e_1 \rhd_{\theta} e_2) \times \{\perp_{\mathcal{A}(e_2)}\}) \\
e_1 \Join_{\theta} e_2 &:= (e_1 \bowtie_{\theta} e_2) \\
&\quad \cup ((e_1 \rhd_{\theta} e_2) \times \{\perp_{\mathcal{A}(e_2)}\}) \\
&\quad \cup (\{\perp_{\mathcal{A}(e_1)}\} \times (e_2 \rhd_{\theta} e_1))
\end{aligned}$$

$\mathcal{A}(e_i)$ ist hierbei die Menge der Attribute von e_i und $\perp_{\mathcal{A}(e_i)}$ das “NULL” Tupel passend zum Schema von e_i .

Die relationale Division

Wird für allquantifizierte Anfragen eingesetzt

Ein Tupel t ist in $R \div S$ falls für jedes $v \in S$ ein $u \in R$ existiert, so dass:

$$u.S = v.S$$

$$u.(\mathcal{R} \setminus S) = t.(\mathcal{R} \setminus S)$$

Voraussetzung: $S \subseteq \mathcal{R}$

Formale Definition:

$$R \div S = \pi_{(\mathcal{R} \setminus S)}(R) \setminus \pi_{(\mathcal{R} \setminus S)}((\pi_{(\mathcal{R} \setminus S)}(R) \times S) \setminus R)$$

Die relationale Division: Beispiel

R	
M	V
m_1	v_1
m_1	v_2
m_1	v_3
m_1	v_4
m_2	v_1
m_2	v_2
m_3	v_1
m_3	v_3
m_4	v_3

S_1
V
v_1

S_2
V
v_1
v_2

S_3
V
v_1
v_2
v_3

$R \div S_1$
M
m_1
m_2
m_3

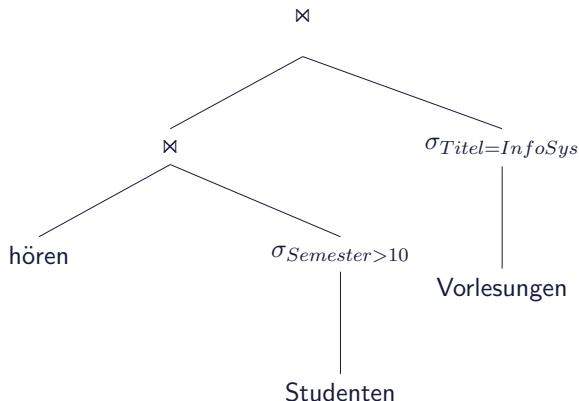
$R \div S_2$
M
m_1
m_2

$R \div S_3$
M
m_1

Operatorbaum-Darstellung

Alternative Darstellung von Ausdrücken der relationalen Algebra. Gerade für größere Ausdrücke viel übersichtlicher als “inline” Darstellung.

Beispiel:



Minimale Menge von benötigten Operatoren

Um alle Ausdrücke der relationalen Algebra ausdrücken zu können sind folgende Operatoren ausreichend:

- Projektion
- Selektion
- Kreuzprodukt
- Vereinigung
- Differenz
- Umbenennung

Wie kann z.B. der Mengendurchschnitt damit ausgedrückt werden?

Ausdrucksstärke

Theorem: Die folgenden Anfragesprachen besitzen die gleiche Ausdrucksstärke.

- Die **relationale Algebra**
- Das **relationale Tupelkalkül** eingeschränkt auf sichere Ausdrücke.
- Das **relationale Domänenkalkül** eingeschränkt auf sichere Ausdrücke.

Deklarative vs. Imperative Anfragesprachen

Im Gegensatz zur deklarativen (aka. deskriptiven) Formulierung von Anfragen in den beiden relationalen Kalkülen bestehen die Anfragen in der relationalen Algebra aus expliziter Angabe von Berechnungsschritten in Form von Operatoren.

Für eine Anfrage in einem der Kalküle ist also nicht vorgegeben wie diese berechnet werden soll, in der relationalen Algebra hingegen schon.

Wir können für die relationale Algebra Regeln angeben wie die Operatoren innerhalb eines Ausdrucks verschoben werden können, bei Beibehaltung der Semantik der Anfrage.

Wir werden dies im Kapitel über Anfrageverarbeitung und Optimierung ausnutzen, um kostengünstigere “Anfragepläne” zu finden.

Äquivalenzerhaltende Transformationsregeln

1. **Aufbrechen von Konjunktionen im Selektionsprädikat**

$$\sigma_{c_1 \wedge c_2 \wedge \dots \wedge c_n}(R) \equiv \sigma_{c_1}(\sigma_{c_2}(\dots(\sigma_{c_n}(R))\dots))$$

2. **σ ist kommutativ**

$$\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$$

3. **π -Kaskaden**

Falls $L_1 \subseteq L_2 \subseteq \dots \subseteq L_n$, dann gilt

$$\pi_{L_1}(\pi_{L_2}(\dots(\pi_{L_n}(R))\dots)) \equiv \pi_{L_1}(R)$$

Äquivalenzerhaltende Transformationsregeln

4. Vertauschen von σ und π

Falls die Selektion sich nur auf Attribute $A_1, \dots, A_n$ der Projektionsliste bezieht, können die beiden Operationen vertauscht werden:

$$\pi_{A_1, \dots, A_n}(\sigma_c(R)) \equiv \sigma_c(\pi_{A_1, \dots, A_n}(R))$$

5. $\cup, \cap$ und $\bowtie$ sind kommutativ

$$R \bowtie_c S \equiv S \bowtie_c R$$

Äquivalenzerhaltende Transformationsregeln

6. Vertauschen von σ mit $\bowtie$

Falls das Selektionsprädikat c nur auf Attribute der Relation R zugreift, kann man die beiden Operationen vertauschen:

$$\sigma_c(R \bowtie_j S) \equiv \sigma_c(R) \bowtie_j S$$

Falls das Selektionsprädikat c eine Konjunktion der Form $c_1 \wedge c_2$ ist und c_1 sich nur auf Attribute aus R und c_2 sich nur auf Attribute aus S bezieht, gilt folgende Äquivalenz:

$$\sigma_c(R \bowtie_j S) \equiv \sigma_{c_1}(R) \bowtie_j \sigma_{c_2}(S)$$

Äquivalenzerhaltende Transformationsregeln

7. Vertauschen von π mit $\bowtie$

Die Projektionsliste L sei: $L = \{A_1, \dots, A_n, B_1, \dots, B_m\}$, wobei A_i Attribute aus R und B_i Attribute aus S seien. Falls sich das Joinprädikat c nur auf Attribute aus L bezieht, gilt folgende Umformung:

$$\pi_L(R \bowtie_c S) \equiv (\pi_{A_1, \dots, A_n}(R)) \bowtie_c (\pi_{B_1, \dots, B_m}(S))$$

Äquivalenzerhaltende Transformationsregeln

8. **Die Operationen $\bowtie, \cap, \cup$ sind jeweils (einzeln betrachtet) assoziativ.** Wenn also Φ eine dieser Operationen bezeichnet, so gilt:

$$(R\Phi S)\Phi T \equiv R\Phi(S\Phi T)$$

9. **Die Operation σ ist distributiv mit $\cap, \cup, -$.** Falls Φ eine dieser Operationen bezeichnet, gilt:

$$\sigma_c(R\Phi S) \equiv (\sigma_c(R))\Phi(\sigma_c(S))$$

10. **Die Operation π ist distributiv mit $\cup$.**

$$\pi_c(R \cup S) \equiv (\pi_c(R)) \cup (\pi_c(S))$$

Äquivalenzerhaltende Transformationsregeln

11. **Die Join- und/oder Selektionsprädikate können mittels de Morgans Regeln umgeformt werden**

$$\neg(c_1 \wedge c_2) \equiv (\neg c_1) \vee (\neg c_2)$$

$$\neg(c_1 \vee c_2) \equiv (\neg c_1) \wedge (\neg c_2)$$

12. **Ein kartesisches Produkt, das von einer Selektionsoperation gefolgt wird, deren Selektionsprädikat Attribute aus beiden Operanden des kartesischen Produktes enthält, kann in eine Joinoperation umgeformt werden.**

Erweiterungen der Relationalen Algebra

Wir betrachten nun einige praxisrelevante Erweiterungen der relationalen Algebra.

- Multimengen
- Aggregation
- Verschiedene weitere Operatoren (Duplikateliminierung, Sortierung, erweiterte Projektion)

Gruppierung und Aggregation: Motivation

- Wie viele Studenten gibt es?
- Was ist die durchschnittliche Anzahl von Semestern?

Studenten		
MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

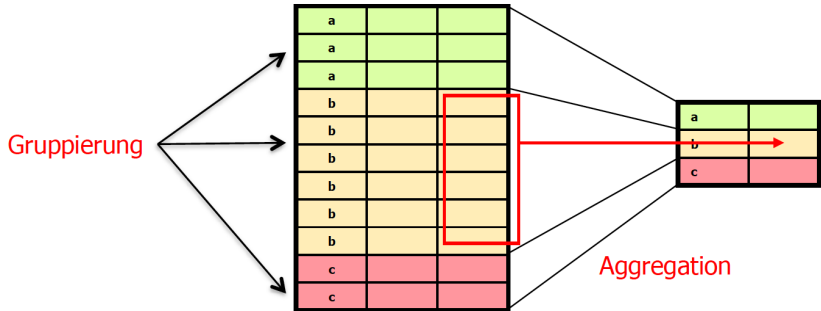
Gruppierung und Aggregation: Idee

Ermittlung von “verdichteten” Daten (Summe, Durchschnitt, ...)

Schritt 1 (optional): Bilde Gruppen von Tupeln aufgrund Wertegleichheit von Attributen, z.B. MatrNr.

Schritt 2: Wende für jede Gruppe Aggregatfunktion(en) an, z.B. AVG(Semester).

Ergebnis: Für jede Gruppe ein Tupel



Gruppierungsoperator

$$\gamma_{MatrNr, AnzahlVLs} \leftarrow COUNT(VorlNr)(hoeren)$$

Gruppierungsoperator γ ermöglicht Spezifikation von

- **Gruppierungsattributen** und **Aggregatfunktion** mit zu aggregierenden Attributen
- Gegeben Relation R mit Attributen $A = \{A_1, \dots, A_n\}$
- Verwendung von Gruppierungs/Aggregationsoperator als

$$\gamma_{GA, AF}(R) \text{ mit}$$

- Gruppierungsattributen $GA \subseteq A$
- Aggregatfunktion $AF = AF_1, \dots, AF_k$ mit

$$AF_i = AN_i \leftarrow (COUNT \mid SUM \mid AVG \mid MIN \mid MAX)(A_i)$$

- AN_i ist Name des entstehenden Attributs (optional)
- Zusätzlich $COUNT(*)$ erlaubt, zum Zählen der Tupel einer Gruppe

Gruppierungsoperator (2)

Die Attribute in GA beschreiben wie die Tupel der Relation R gruppiert werden.

D.h. eine Gruppe besteht aus allen Tupeln aus R mit gleichen GA -Werten

Ergebnisrelation:

- Enthält für jede vorkommende Wertekombination(!) von GA ein Tupel
- Dieses Tupel besitzt GA -Werte und berechnete AF -Werte für jede Gruppe

Falls $GA=\emptyset$ wird auf der gesamten Relation (eine große Gruppe) aggregiert. Ergebnis besteht in diesem Fall nur aus den berechneten AF -Werten (1 Tupel). Und $GA=\emptyset$ wird auch einfach weggelassen in der Anfrage. Beispiele: $\gamma_{COUNT(*)}(Studenten)$ oder $\gamma_{SUM(SWS)}(Vorlesungen)$

Gruppierungsoperator: Beispiel mit Zählen (Count)

hoeren	
MatrNr	VorlNr
26120	5001
27550	5001
27550	4052
28106	5041
28106	5052
28106	5216
28106	5259
29120	5001
29120	5041
29120	5049
29555	5022
25403	5022
29555	5001

$\gamma_{MatrNr, AnzahlVLs \leftarrow COUNT(VorlNr)}(hoeren)$	
MatrNr	AnzahlVLs
26120	1
25403	1
27550	2
28106	4
29120	3
29555	2

Der erweiterte Projektions-Operator

Wo in der ursprünglichen Definition der Projektion nur Attributnamen angegeben werden, also $\pi_{A_1, \dots, A_k}(R)$ **lassen wir in der Erweiterung des Projektions-Operators auch Funktionen zu, die ein oder mehrere Attribute als Argumente haben.**

Ausgabe der Funktionen können benannt werden.

Beispiel:

R

A	B	C
1	2	3
1	3	4
3	2	1
4	4	4

$\pi_{X \leftarrow A+B, Y \leftarrow A*1.19, C}(R)$

X	Y	C
3	1.19	3
4	1.19	4
5	3.57	1
8	4.76	4

Sortierung

$$\tau_L(R)$$

ist eine **Liste** von Tupeln aus Relation R , geordnet nach den Attributen, die in L genannt sind. In lexikographischer Ordnung. Optional mit Angabe der Sortierung: Absteigend oder Aufsteigend, pro Attribut. Default: Aufsteigend.

$$\tau_{A,B}(R)$$

A	B	C
1	2	3
1	3	4
3	2	1
4	4	4

$$\tau_{B,C}(R)$$

A	B	C
3	2	1
1	2	3
1	3	4
4	4	4

$$\tau_{C\downarrow}(R)$$

A	B	C
1	3	4
4	4	4
1	2	3
3	2	1

Duplikate in den Daten: Multimengen (Bags)

Die bislang betrachteten Anfragesprachen arbeiten auf Mengen, im Sinne von Relationen, die Mengen von Tupeln sind.

Es werden bei dieser Mengen-Betrachtung dabei implizit Duplikate entfernt. Insbesondere nach einer Projektion treten oft Duplikate auf.

Datenbanksysteme arbeiten in der Regel mit **Multimengen** (aka. **Bags**), d.h. die Relationen (Tabellen) dürfen auch Duplikate enthalten.

Zum Beispiel sind $\{a,b,b\}$ und $\{a,b\}$ Multimengen, wobei letztere auch eine Menge ist.

Operationen auf Multimengen

Selektion, Projektion, Join, Kreuzprodukt, Umbenennung (sowieso), Aggregation/Gruppierung, Sortierung funktionieren auch für Multimengen.

Was ist mit Mengendifferenz, Mengendurchschnitt, Mengenvereinigung für Multimengen?

Operationen auf Multimengen: Projektion

Duplikate, die durch die Projektion entstehen bleiben erhalten.

R			$\pi_C(R)$	
A	B	C	C	
1	2	3	3	
1	3	4	4	
3	2	4	4	
4	4	4	4	

Vgl. in SQL: SELECT vs. SELECT DISTINCT

Operationen auf Multimengen: Selektion

 R

A	B	C
1	2	5
3	4	6
1	2	7
1	2	7

 $\sigma_{C \geq 6}(R)$

A	B	C
3	4	6
1	2	7
1	2	7

Auf jedes Tupel wird Prädikat der Selektion angewendet. Sich qualifizierende Tupel werden in Ergebnis aufgenommen. Duplikate werden dabei nicht eliminiert.

Operationen auf Multimengen: Kreuzprodukt

Jedes Tupel aus der einen Relation wird mit allen Tupeln der anderen Relation gepaart. Egal ob Duplikat oder nicht. Tritt ein Tupel r m -mal in R auf und ein Tupel s tritt n -mal in S auf, dann gibt es da Tupel rs in $R \times S$ genau mn -mal.

 R

A	B
1	2
1	2

 S

B	C
2	3
4	5
4	5

 $R \times S$

A	R.B	S.B	C
1	2	2	3
1	2	2	3
1	2	4	5
1	2	4	5
1	2	4	5
1	2	4	5

Operationen auf Multimengen: Natürlicher Join

 R

A	B
1	2
1	2

 S

B	C
2	3
4	5
4	5

 $R \bowtie S$

A	B	C
1	2	3
1	2	3

Tupel (1,2) aus R wird mit (2,3) aus S verknüpft. Da es zwei Kopien von (1,2) in R gibt und eine Kopie von (2,3) in S , gibt es zwei Paare von joinbaren Tupeln, also **zwei Kopien von (1,2,3) im Ergebnis**.

Operationen auf Multimengen: Theta Join

 R

A	B
1	2
1	2

 S

B	C
2	3
4	5
4	5

 $R \bowtie_{R.B < S.B} S$

A	R.B	S.B	C
1	2	4	5
1	2	4	5
1	2	4	5
1	2	4	5

Operationen auf Multimengen: Kardinalität und $\subseteq$

Für ein Bag B bezeichnen wir mit $\#B(x)$ die Anzahl der Vorkommen von Element x in B (aka. Multiplizität).

Wir haben dann

- **Kardinalität** von Bag B als $|B| = \sum_x \#B(x)$
- $A \subseteq B$ als $\#A(x) \leq \#B(x) \ \forall x$

Operationen auf Multimengen: Vereinigung, Differenz, Schnittmenge

- **Vereinigung:** $\#(X \cup Y)(z) = \#X(z) + \#Y(z)$
- **Schnittmenge:** $\#(X \cap Y)(z) = \min(\#X(z), \#Y(z))$
- **Differenz:** $\#(X \setminus Y)(z) = \max(0, \#X(z) - \#Y(z))$

Anmerkung: Die Definition der Vereinigung für Bags ist nicht identisch zu der Vereinigung von Bags aus der Mathematik, welche definiert wäre als $\#(X \cup_{max} Y)(z) = \max(\#X(z), \#Y(z))$. Mit dieser Definition würden auch die Regeln für Mengen bei Bags gelten.

Die oben angegebene Definition $\#(X \cup Y)(z) = \#X(z) + \#Y(z)$ folgt jedoch genau dem was in DB-Systemen (siehe später SQL) oder Programmiersprachen benutzt wird, im Sinne der “**Konkatenation**” von Bags. $X \cup_{max} Y \equiv (X \setminus Y) \cup Y$

Beispiel: Verletzte Regel für Bags

$R \cap (S \cup T) \equiv (R \cap S) \cup (R \cap T)$ gilt für Mengen, aber nicht für Bags

Seien R , S und T jeweils das Bag $\{1\}$

Dann ist die linke Seite: $S \cup T = \{1,1\}$; $R \cap (S \cup T) = \{1\}$.

Und die rechte Seite: $R \cap S = R \cap T = \{1\}$

$(R \cap S) \cup (R \cap T) = \{1\} \cup \{1\} = \{1,1\} \neq \{1\}$

Operator für Duplikateliminierung

$$\delta(R)$$

ergibt eine Relation, in der jedes Tupel einmal auftritt für Tupel die in R einmal oder mehrfach auftreten.

 R

A	B
1	2
3	4
1	2
1	2

 $\delta(R)$

A	B
1	2
3	4

δ als Spezialfall von γ

Technisch gesehen ist der Duplikatseliminierungsoperator δ redundant. Für Relation $R(A_1, A_2, \dots, A_n)$ ist die Relation $\delta(R)$ äquivalent zu $\gamma_{A_1, A_2, \dots, A_n}(R)$.

- D.h. um Duplikate zu eliminieren gruppieren wir nach allen Attributen, **ohne Anwendung einer Aggregatfunktion.**
- Also jede Gruppe besteht aus einem Tupel, welches einmal oder mehrfach in R aufgetreten ist.

Für Mengen (keine Multimengen/Bags) ist γ auch eine Erweiterung der Projektionsoperators. D.h. $\gamma_{A_1, A_2}(R)$ ist identisch zu $\pi_{A_1, A_2}(R)$ falls R eine Menge ist.

Falls aber R eine Multimenge ist so eliminiert γ Duplikate und π nicht.

SQL

SQL

- **Deklarative** Anfragesprache (“was” nicht “wie”)
- **Inspiziert durch das relationale Tupelkalkül**
- Setzte sich aus drei Sprachen zusammen:
- **Datendefinitions (DDL)-Sprache:**
 - erstellt/ändert das Schema
 - create, alter, drop
- **Datenmanipulations (DML)-Sprache**
 - ändert Ausprägungen
 - insert, update, delete
- **Anfragesprache:**
 - berechnet Anfragen auf den Ausprägungen
 - select * from where ...

Relationen vs. Tabellen

- In SQL betrachten wir aber Tabellen
 - Tabellen können Duplikate enthalten
 - Tabellen können eine Ordnung haben
 - Tabellen haben nicht unbedingt einen Schlüssel

Es war einmal die Geschichte von SQL

- 1986 erste SQL-Norm durch ANSI verabschiedet
- SQL-92 (SQL 2)
- SQL-99 (SQL 3): rekursive Anfragen, Trigger, OO
- SQL-2003: XML, Fensteranfragen
- SQL-2006: XML, XQuery
- SQL-2008: Verschiedenes
- Standards von verschiedenen DBMS nicht vollständig implementiert
- DBMS haben oft zusätzlich eigene SQL-Erweiterungen

Die Datenbanksysteme (nicht vollständig)

- 1970: Relationales Modell, Codd
- seit 70er Jahren System R, IBM (SEQUEL → SQL)
- 1979: Oracle 2 (erste Version von Oracle)
- 1982: IBM DB2
- 1984: Teradata
- 1985: Informix
- 1987: Sybase
- 1989: Postgres
- 1989: Microsoft SQL Server
- 1996: MySQL
- 2004: Apache Derby
- 2004: MonetDB
- 2007: H-Store
- 2010: HyPerDB
- 2012: VoltDB ...

In dieser Vorlesung wird PostgreSQL verwendet

... in Beispielen in den Folien sowie in den Übungen

Postgresql

- “The world’s most advanced open source database”
- Einfach zu Installieren
- Gute SQL Unterstützung
- Transaktionen
- Gute Dokumentation (es gibt auch Bücher dazu)



<http://www.postgresql.org/>

PGAdmin User Interface

Download unter <http://www.pgadmin.org/>

Einfache Datendefinitionen in SQL

Datentypen

- **character**(n), **char** (n)
- **character varying** (n), **varchar** (n)
- **numeric**(p,s), **integer**
- **blob** oder **raw** für sehr große binäre Daten
- **clob** für sehr große String-Attribute
- **date** für Datumsangaben
- ...

<http://www.postgresql.org/docs/current/static/datatype.html>

<http://www.postgresql.org/docs/current/static/sql-syntax.html>

Datendefinitions (DDL) Sprache

Tabellen erstellen:

```
create table Professoren(  
    PersNr    integer,  
    Name      varchar (10)  
    Rang      character (2));
```

Tabellen verändern:

```
alter table Professoren  
    add Raum integer;
```

```
alter table Professoren  
    alter column Name type varchar(30);
```

<http://www.postgresql.org/docs/current/static/ddl-alter.html>

Datendefinitions (DDL) Sprache

Tabellen löschen:

```
drop table Professoren;
```

Datenmanipulationssprache: insert

```
insert into Studenten (MatrNr, Name)  
  values (28121, 'Archimedes');
```

```
insert into Studenten (Name)  
  values ('Meier');
```

Ohne Angabe der Spaltennamen ist die Reihenfolge der Attribute relevant!

```
insert into hören  
  select MatrNr, VorlNr  
  from Studenten, Vorlesungen  
  where Titel = 'Logik';
```

<http://www.postgresql.org/docs/current/static/dml.html>

Anfragesprache

<http://www.postgresql.org/docs/current/static/queries.html>

```
select <Liste von Spalten>  
      from <Liste von Tabellen>  
      where <Bedingung>;
```

select * wählt alle verfügbaren Spalten aus!

Relationale Algebra $\rightarrow$ SQL

- Projektion $\pi \rightarrow$ select
- Kreuzprodukt $\times \rightarrow$ from
- Selektion $\sigma \rightarrow$ where

Select from where ...

select PersNr, Name
from Professoren
where Rang = 'C4';

Professoren	
PersNr	Name
2125	Sokrates
2126	Russel
2136	Curie
2137	Kant

Professoren			
PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Anmerkung:

SQL legt nicht fest in welcher Reihenfolge Selektion, Projektion oder Join ausgeführt werden.

Values Statement

- Erzeugt konstante Tabelle
- **values** (1, 'eins'), (2, 'zwei'), (3, 'drei');
- Dann z.B. `select * from (values (1,'eins')) s;`
- Achtung: in Postgres braucht values in FROM einen Alias (hier s)
- Values Statement ist äquivalent zu:

```
select 1 as column1, 'eins' AS column2
union all
select 2, 'zwei'
union all
select 3, 'drei';
```

Verwendung z.B. in Select Statement (auch in Joins) als normale Tabelle

```
select * from
(values (1,'eins'), (2,'zwei') ) as table1 (nummer, wert);
```

Prädikate in der **where** Klausel

Beschreibt Bedingungen, die für Ergebnistupel gelten müssen.

- Bedingungen in der where-Klausel können logisch kombiniert werden mit: **and, or, not**
- Vergleichsoperatoren sind u.a.: $=, <, \leq, >, \geq, like, between$

Between und Like

```
select Name, MatrNr  
from Studenten  
where semester between 1 and 4
```

Könnte auch mit Vergleichsoperatoren ausgedrückt werden ...

```
select Name, Matr  
from Studenten  
where Name like 'M%'
```

% steht für eine beliebige Zeichenkette (auch Länge 0).

Es gibt auch _

für EIN beliebiges Zeichen.

Sortierung (Order By) und Top Ergebnisse (Limit)

```
select    PersNr, Name, Rang  
from      Professoren  
order by  Rang desc, Name asc;
```

- **asc**: aufsteigend (Englisch: ascending)
- **desc**: absteigend (Englisch: descending)

Möchte man nur die ersten k (hier =5) Ergebnisse haben:
Limit Anweisung

```
select    Name, Semester  
from      Studenten  
order by  Semester desc  
limit     5
```

Limit hat leicht andere Syntax in versch. DBS, z.B. Oracle, SQL Server.

Duplikateliminierung

```
select Rang  
from Professoren;
```

Rang
C4
C3
C3
C4
C4
C3
C4

```
select distinct Rang  
from Professoren
```

Rang
C4
C3

Anfragen über mehrere Relationen

Welcher Professor liest “Mäeutik”?

```
select Name, Titel  
from Professoren, Vorlesungen  
where PersNr = gelesenVon and Titel = 'Mäeutik'
```

In der relationalen Algebra sieht das wie folgt aus:

$$\pi_{Name, Titel}(\sigma_{PersNr=gelesenVon \wedge Titel='Mäeutik'}(Professoren \times Vorlesungen))$$

Und im Tupelkalkül....?

Anfragen über mehrere Relationen

Professoren			
PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
...	...	...	...
2137	Kant	C4	7

Vorlesungen			
VorlNr	Titel	SWS	gelesenVon
5001	Grundzüge	4	2137
5041	Ethik	4	2125
...	...	...	...
5049	Mäeutik	2	2125
...	...	...	...
4630	Die 3 Kritiken	4	2137



Verknüpfung durch Kreuzprodukt

×

Ergebnis des Kreuzprodukts:

PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	gelesenVon
2125	Sokrates	C4	226	5001	Grundzüge	4	2137
1225	Sokrates	C4	226	5041	Ethik	4	2125
...	...	...	...	...	...	...	...
2125	Sokrates	C4	226	5049	Mäeutik	2	2125
...	...	...	...	...	...	...	...
2126	Russel	C4	232	5001	Grundzüge	4	2137
2126	Russel	C4	232	5041	Ethik	4	2125
...	...	...	...	...	...	...	...
2317	Kant	C4	7	4630	Die 3 Kritiken	4	2137

Was fällt auf?

Kreuzprodukt erzeugt VIELE unnötige Kombinationen!

Nun: für die Auswahl der interessanten Tupel brauchen wir die Selektion →

Nach Anwenden der Selektion “PersNr=gelesenVon” und “Titel=‘Mäeutik’” erhalten wir:

PersNr	Name	Rang	Raum	VorlNr	Titel	SWS	gelesenVon
2125	Sokrates	C4	226	5049	Mäeutik	2	2125

Weiter mit der Projektion auf Name und Titel:

Name	Titel
Sokrates	Mäeutik

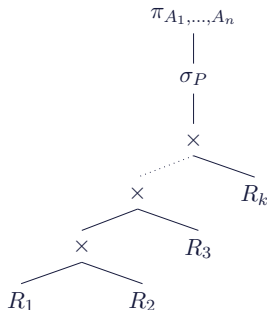
Übersetzung von SQL in die relationale Algebra

Allgemeine Form einer
(ungeschachtelten) SQL-Anfrage:

select $A_1, \dots, A_n$
from $R_1, \dots, R_k$
where P ;

Übersetzung in die relationale Algebra:

$$\pi_{A_1, \dots, A_n}(\sigma_P(R_1 \times \dots \times R_k))$$



Abfragen über mehrere Relationen

Welche Studenten hören welche Vorlesungen?

```
select Name, Titel  
from Studenten, hören, Vorlesungen  
where Studenten.MatrNr=hören.MatrNr and  
        hören.VorlNr=Vorlesungen.VorlNr;
```

Alternativ:

```
select s.Name, v.Titel  
from Studenten s, hören h, Vorlesungen v  
where s.MatrNr=h.MatrNr and  
        h.VorlNr=v.VorlNr;
```

Aggregatfunktion und Gruppierung

Aggregatfunktionen **avg**, **max**, **min**, **count**, **sum**.

```
select avg (Semester)
from Studenten;
```

Gruppierung:

```
select gelesenVon, sum(SWS)
from Vorlesungen
group by gelesenVon;
```

Bedeutung:

1. alle Tupel, die den gleichen Wert für Attribut gelesenVon haben, werden zu einer Gruppe zusammengefasst
2. für jede dieser Gruppen wird dann die Summe gebildet

Was es bei der Aggregation zu Beachten gilt

- SQL erzeugt pro Gruppe ein Ergebnistupel
- Also **müssen** alle in der **select**-Klausel aufgeführten Attribute – außer den aggregierten – auch in der **group by**-Klausel aufgeführt werden.

So geht es also nicht:

```
select Rang, count(PersNr), Name
from Professoren
group by Rang
```

Welcher Name sollte hier auch für einen bestimmten Rang zurückgegeben werden?

Es muss sichergestellt werden, dass sich das Attribut innerhalb einer Gruppe nicht ändert.

Was es bei der Aggregation zu Beachten gilt

- Aber: **umgekehrt** muss dies **nicht** notwendigerweise gelten.
- D.h. wenn ein Attribut im group by steht, muss es nicht unbedingt im select aufgeführt werden!

Beispiel:

```
select Name  
from Professoren  
group by Name, Rang
```

Beispiel: Was ist OK und was erzeugt Fehler.

	select			
group by		rang, name	rang	name
	rang, name	OK	OK	OK
	rang	ERROR	OK	ERROR
	name	ERROR	ERROR	OK

select from Professoren ... group by;

- Alle im select aufgeführten Attribute, über die nicht aggregiert wird, müssen im group by aufgeführt werden.
- ABER: Nicht alle im group by aufgeführten Attribute müssen im select aufgeführt sein!

Weitere Beispiele zur Gruppierung

- **select** count(*)
from Vorlesungen
group by gelesenVon
- **OK**: Ergebnis = Anzahl der Elemente pro Gruppe
- **select** gelesenVon, count(*)
from Vorlesungen
group by SWS
- **ERROR**: gelesenVon muss ins group by!
- **select** gelesenVon, count(*)
from Vorlesungen
group by SWS, gelesenVon
- **OK**.

Aggregatfunktion und Gruppierung

```
select gelesenVon, Name, sum(SWS)
from Vorlesungen, Professoren
where gelesenVon=PersNr and Rang = 'C4'
group by gelesenVon, Name
having avg (SWS) >= 3;
```

1. Gruppieren nach **gelesenVon, Name**
2. Welche Gruppen erfüllen die Bedingung in **having**
3. Im select wird dann das Aggregat pro Gruppe berechnet, hier: **die Summe der SWS**.

Ausführen einer Anfrage mit group by

Vorlesung × Professoren							
VorlNr	Titel	SWS	gelesenVon	PersNr	Name	Rang	Raum
5001	Grundzüge	4	2137	2125	Sokrates	C4	226
5041	Ethik	4	2125	2125	Sokrates	C4	226
...	...	...	...	...	...	...	...
4630	Die 3 Kritiken	4	2137	2137	Kant	C4	7



where

Vorlesung × Professoren							
VorlNr	Titel	SWS	gelesenVon	PersNr	Name	Rang	Raum
5001	Grundzüge	4	2137	2137	Kant	C4	7
5041	Ethik	4	2125	2125	Sokrates	C4	226
5043	Erkenntnisth.	3	2126	2126	Russel	C4	232
5049	Mäeutik	2	2125	2125	Sokrates	C4	226
4052	Logik	4	2125	2125	Sokrates	C4	226
5052	Wissenschaftsth.	3	2126	2126	Russel	C4	232
5216	Bioethik	2	2126	2126	Russel	C4	232
4630	Die 3 Kritiken	4	2137	2137	Kant	C4	7



group by

Vorlesung × Professoren							
VorlNr	Titel	SWS	gelesenVon	PersNr	Name	Rang	Raum
5041	Ethik	4	2125	2125	Sokrates	C4	226
5049	Mäeutik	2	2125	2125	Sokrates	C4	226
4052	Logik	4	2125	2125	Sokrates	C4	226
5043	Erkenntnisth.	3	2126	2126	Russel	C4	232
5052	Wissenschaftsth.	3	2126	2126	Russel	C4	232
5216	Bioethik	2	2126	2126	Russel	C4	232
5001	Grundzüge	4	2137	2137	Kant	C4	7
4630	Die 3 Kritiken	4	2137	2137	Kant	C4	7



having-Bedingung

Vorlesung × Professoren							
VorlNr	Titel	SWS	gelesenVon	PersNr	Name	Rang	Raum
5041	Ethik	4	2125	2125	Sokrates	C4	226
5049	Mäeutik	2	2125	2125	Sokrates	C4	226
4052	Logik	4	2125	2125	Sokrates	C4	226
5001	Grundzüge	4	2137	2137	Kant	C4	7
4630	Die 3 Kritiken	4	2137	2137	Kant	C4	7



Aggregation **sum** und Projektion

gelesenVon	Name	sum(SWS)
2125	Sokrates	10
2137	Kant	8

Mengenoperationen: union

```
(select Name  
from Assistenten)  
union  
(select Name  
from Professoren);
```

```
(select Name  
from Assistenten)  
union  
(select Name  
from Assistenten);
```

```
(select Name  
from Assistenten)  
union all  
(select Name  
from Assistenten);
```

union eliminiert Duplikate, union all hingegen nicht.

Mengenoperationen: intersection und minus bzw. except

A
x
1
2
3

B
x
2
3
4

(select * from A)
intersect
(select * from B);

A
x
2
3

(select * from A)
except
(select * from B);

A
x
1

intersect all, except all

Wie bei union muss man bei intersect und except ein **all** dazuschreiben, wenn man Multimengensemantik möchte.

Zum Beispiel ergibt

```
(values (1),(1),(1),(1))  
  except  
(values (1),(1))
```

eine leere Tabelle, aber mit **except all** kommt eine Tabelle mit zwei Tupeln der Form $\langle 1 \rangle$ heraus

where in: Beispiel Unkorrelierte Unteranfragen

```
select Name  
from Professoren  
where PersNr in (select gelesenVon  
                  from Vorlesungen);
```

Vergleich von Wert mit Menge von Werten

where exists: Beispiel Korrelierte Unteranfragen

```
select Name
from Professoren
where exists (select *
              from Vorlesungen v
              where v.gelesenVon = p.PersNr);
```

Was passiert, wenn man das **select** der Unteranfrage ändert?

where exists: Beispiel Korrelierte Unteranfragen (2)

```
select Name  
from Professoren p  
where exists (select 42  
               from Vorlesungen v  
               where v.gelesenVon = p.PersNr);
```

Ist das Ergebnis ein anderes?

Ist die innere Menge leer?

where any

Vergleich von Wert mit Menge von Werten

```
select Name, Semester  
from Studenten  
where Semester > any (select Semester  
                        from Studenten);
```

where all

Vergleich von Wert mit Menge von Werten

```
select Name  
from Studenten  
where Semester >= all (select Semester  
                        from Studenten);
```

Was passiert bei Vergleich mit > anstelle von >=?

Allquantifizierung

SQL hat keinen Allquantor

- **Allquantifizierung muss durch eine äquivalente Anfragen mit Existenzquantifizierung ausgedrückt werden.**
- Tupel-Kalkül-Formulierung der Anfrage: Wer hat alle vierstündigen Vorlesungen gehört?

$$\{s | s \in \text{Studenten} \wedge \forall v \in \text{Vorlesungen} (v.SWS = 4 \Rightarrow \exists h \in \text{hören} \\ (h.VorlNr = v.VorlNr \wedge h.MatrNr = s.MatrNr))\}$$

Eliminierung durch $\Rightarrow$ und $\forall$ durch Äquivalenzen

- $\forall t \in R(P(t)) = \neg(\exists t \in R(\neg P(t)))$
- $X \Rightarrow Y = \neg X \vee Y$

Umformung des Kalkül-Ausdrucks ...

Elimination $\forall$:

$$\{s | s \in \text{Studenten} \wedge \neg(\exists v \in \text{Vorlesungen} \neg(v.SWS = 4 \Rightarrow \exists h \in \text{hören} \\ (h.VorlNr = v.VorlNr \wedge h.MatrNr = s.MatrNr))))\}$$

Elimination $\Rightarrow$:

$$\{s | s \in \text{Studenten} \wedge \neg(\exists v \in \text{Vorlesungen} \neg(\neg(v.SWS = 4) \vee \exists h \in \text{hören} \\ (h.VorlNr = v.VorlNr \wedge h.MatrNr = s.MatrNr))))\}$$

Durch Anwendung von DeMorgan erhalten wird:

$$\{s | s \in \text{Studenten} \wedge \neg(\exists v \in \text{Vorlesungen} (v.SWS = 4 \wedge \neg(\exists h \in \text{hören} \\ (h.VorlNr = v.VorlNr \wedge h.MatrNr = s.MatrNr))))))\}$$

Umsetzung in SQL:

$$\{s | s \in \textit{Studenten} \wedge \neg(\exists v \in \textit{Vorlesungen}(v.\textit{SWS} = 4 \wedge \neg(\exists h \in \textit{ hoeren} \\ (h.\textit{VorlNr} = v.\textit{VorlNr} \wedge h.\textit{MatrNr} = s.\textit{MatrNr}))))))\}$$

```
select s.*  
from Studenten s  
where not exists  
  (select *  
   from Vorlesungen v  
   where v.SWS=4 and not exists (  
     select *  
     from hoeren h  
     where h.VorlNr=v.VorlNr and h.MatrNr=s.MatrNr ));
```

Allquantifizierung durch count-Aggregation

Allquantifizierung kann auch immer durch eine count-Aggregation ausgedrückt werden:

```
select h.MatrNr
from hoeren h, vorlesungen v
where h.vorlNr = v.vorlNr and v.sws = 4
group by h.MatrNr
having count(*) = (select count(*)
                    from Vorlesungen
                    where sws=4);
```

Unteranfragen in where-Klausel

```
select *  
from pruefen  
where Note < (select avg(Note)  
               from pruefen);
```

Vergleich von Wert mit Wert

Unteranfragen in select-Klausel

- **Für jedes Ergebnistupel wird die Unteranfrage ausgeführt**
- Unteranfrage ist korreliert (greift auf Attribute der umschließenden Anfrage zu)

```
select PersNr, Name, (select sum(SWS)
                      from Vorlesungen
                      where gelesenVon = PersNr) as Lehrbelastung
from Professoren;
```

Pro äußerer Zeile wird ein Wert berechnet

Unkorrelierte vs. korrelierte Unteranfragen

Korrelierte Formulierung

```
select s.*  
from Studenten s  
where exists  
    (select p.*  
     from Professoren p  
     where p.GebDatum > s.GebDatum);
```

Unkorrelierte vs. korrelierte Unteranfragen

Äquivalente unkorrelierte Formulierung

```
select s.*  
from Studenten s  
where s.GebDatum <  
      (select max(p.GebDatum)  
       from Professoren p);
```

- **Vorteil:** Ergebnis der Unteranfrage kann materialisiert werden
- Unteranfrage braucht nur einmal ausgewertet zu werden

Entschachtelung korrelierter Unteranfragen

```
select a.*  
from Assistenten a  
where exists  
    (select p.*  
    from Professoren p  
    where a.Boss = p.PersNr and  
        p.GebDatum > a.GebDatum);
```

Entschachtelung durch Join

```
select a.*  
from Assistenten a, Professoren p  
where a.Boss=p.PersNr and p.GebDatum > a.GebDatum);
```

WITH statements

```
with AnzahlStudentenJeVL as (  
    select VorlNr, count(*) as AnzProVorl  
    from hoeren  
    group by VorlNr  
)  
GesamtanzahlStudenten as (  
    select count (*) as GesamtAnz  
    from Studenten  
)  
select h.VorlNr, h.AnzProVorl, g.GesamtAnz,  
h.AnzProVorl/cast(g.GesamtAnz as float) as Marktanteil  
from AnzahlStudentenJeVL h,  
GesamtanzahlStudenten g;
```

Erzeugt temporäre Tabelle innerhalb der Anfrage

Nullwerte

- **null entspricht “unbekannter Wert”, “nicht definiert”**
- **Nullwerte können auch im Zuge der Abfrageauswertung entstehen (z.B. äußere Joins)**
- Manchmal sehr überraschende Abfrageergebnisse, wenn Nullwerte vorkommen. **Beispiel:**

```
select count (*)
```

```
from Studenten where Semester < 13 or Semester >= 13;
```

- Wenn es Studenten gibt, deren Semester-Attribut den Wert null hat, werden diese nicht mitgezählt.
- Der Grund liegt in folgenden Regeln für den Umgang mit Nullwerten begründet.

COUNT und Nullwerte

Bei `count(spaltenname)` werden Nullwerte nicht mitgezählt, ebenso wie bei `count(distinct spaltenname)`.

Bei `count(*)` werden hingegen auch Tupel, die nur aus Nullwerten bestehen mitgezählt.

“`select count(*) from (select null as x) as y`” liefert also 1 und

“`select count(x) from (select null as x) as y`” liefert 0

Auswertung bei Nullwerten

- In arithmetischen Ausdrücken werden Nullwerte propagiert, d.h. sobald ein Operand null ist, wird auch das Ergebnis null. Dementsprechend wird z.B. $\text{null} + 1$ zu null ausgewertet-aber auch $\text{null} * 0$ wird zu null ausgewertet.
- **SQL hat eine dreiwertige Logik**, die nicht nur **true** und **false** kennt, sondern auch einen dritten Wert **unknown**. Diesen Wert liefern Vergleichsoperationen zurück, wenn mindestens eines ihrer Argumente null ist. Beispielsweise wertet SQL das Prädikat $(\text{PersNr}=\dots)$ immer zu unknown aus, wenn die PersNr des betreffenden Tupels den Wert null hat.
- Logische Ausdrücke werden nach den folgenden Tabellen berechnet:

Auswertung bei Nullwerten

not	
true	false
unknown	unknown
false	true

and	true	unknown	false
true	true	unknown	false
unknown	unknown	unknown	false
false	false	false	false

or	true	unknown	false
true	true	true	true
unknown	true	unknown	unknown
false	true	unknown	false

Das case-Konstrukt

```
select MatrNr, ( case when Note < 1.5 then 'sehr gut'  
                  when Note < 2.5 then 'gut'  
                  when Note < 3.5 then 'befriedigend'  
                  when Note <= 4.0 then 'ausreichend'  
                  else 'nicht bestanden' end)  
from pruefen;
```

Die erste qualifizierende when-Klausel wird ausgeführt.

Joins

- **cross join**: Kreuzprodukt
- **natural join**: natürlicher Join
- **join** oder **inner join**: Theta-Join
- **left**, **right** oder **full outer join**: äußerer Join

```
select *  
from  $R_1, R_2$   
where  $R_1.A = R_2.B$ ;
```

kann wie folgt geschrieben werden:

```
select *  
from  $R_1$  join  $R_2$  on  $R_1.A = R_2.B$ ;
```

Right Outer Join

```
select *
from pruefen f right outer join Studenten s on
      f.MatrNr=s.MatrNr;
```

	matrn timer	vorlnr integer	persnr integer	note numeric(2,1)	matrn integer	name character varying(30)	semester integer
1					24002	Xenokrates	18
2	25403	5041	2125	2.0	25403	Jonas	12
3					26120	Fichte	10
4					26830	Aristoxenos	8
5	27550	4630	2137	2.0	27550	Schopenhauer	6
6	28106	5001	2126	1.0	28106	Carnap	3
7					29120	Theophrastos	2
8					29555	Feuerbach	2
9					42	mueller	

Dreiwege Right Outer Join

```

select p.PersNr, p.Name, f.PersNr, f.Note, f.MatrNr,
s.MatrNr, s.Name
from Professoren p right outer join
      (pruefen f right outer join Studenten s on
      f.MatrNr=s.MatrNr)
on p.PersNr=f.PersNr;
  
```

	persnr integer	name character varying(30)	persnr integer	note numeric(2,1)	matrn integer	matrn integer	name character varying(30)
1						24002	Xenokrates
2	2125	Sokrates	2125	2.0	25403	25403	Jonas
3						26120	Fichte
4						26830	Aristoxenos
5	2137	Kant	2137	2.0	27550	27550	Schopenhauer
6	2126	Russel	2126	1.0	28106	28106	Carnap
7						29120	Theophrastos
8						29555	Feuerbach
9						42	mueller

Dreiwege Left Outer Join

```

select p.PersNr, p.Name, f.PersNr, f.Note, f.MatrNr,
s.MatrNr, s.Name
from Professoren p left outer join
    (pruefen f left outer join Studenten s on f.MatrNr=
    s.MatrNr)
on p.PersNr=f.PersNr;
  
```

	persnr integer	name character varying(30)	persnr integer	note numeric(2,1)	matrn integer	matrn integer	name character varying(30)
1	2125	Sokrates	2125	2.0	25403	25403	Jonas
2	2126	Russel	2126	1.0	28106	28106	Carnap
3	2127	Kopernikus					
4	2133	Popper					
5	2134	Augustinus					
6	2136	Curie					
7	2137	Kant	2137	2.0	27550	27550	Schopenhauer

Full Outer Join

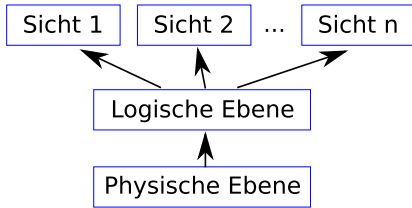
```

select p.PersNr, p.Name, f.PersNr, f.Note, f.MatrNr,
s.MatrNr, s.Name
from Professoren p full outer join
(pruefen f full outer join Studenten s on
f.MatrNr= s.MatrNr)
on p.PersNr=f.PersNr;

```

	persnr integer	name character varying(30)	persnr integer	note numeric(2,1)	matrn integer	matrn integer	name character varying(30)
1	2125	Sokrates	2125	2.0	25403	25403	Jonas
2	2126	Russel	2126	1.0	28106	28106	Carnap
3	2127	Kopernikus					
4	2133	Popper					
5	2134	Augustinus					
6	2136	Curie					
7	2137	Kant	2137	2.0	27550	27550	Schopenhauer
8						26830	Aristoxenos
9						29120	Theophrastos
10						42	mueller
11						29555	Feuerbach
12						24002	Xenokrates
13		Prof. Dr.-Ing. S. Michel		RPTU	26120	26120	Fischbe

Abstraktionsebenen eines Datenbanksystems



Teilmenge des Datenbankschemas, z.B., verschiedene Rollen

Datenbankschema, z.B. Menge von Tabellen

Geräte, Dateien, Dateiformate, z.B., Dateien, Festplatten

Datenunabhängigkeit:

- physische Unabhängigkeit
- logische Datenunabhängigkeit → **Sichten (Views)**

Beispiele für Sichten

```
create view ProfsUndIhreVorlesungen as  
  select v.titel, p.name  
  from Professoren p, Vorlesungen v  
  where p.persnr=v.gelesenvon;
```

```
select * from ProfsUndIhreVorlesungen;
```

```
create view AnzahlSWSPProProf as  
  select p.name, sum(v.sws)  
  from professoren p, vorlesungen v  
  where p.persnr=v.gelesenvon  
  group by p.name;
```

```
select sum(sum) from AnzahlSWSPProProf;
```

Beispiele für Sichten

create or replace view

```
    ProfsUndIhreVorlesungen as  
select v.titel, p.name  
from Professoren p, Vorlesungen v  
where p.persnr=v.gelesenvon;
```

```
drop view AnzahlSWSPProProf;  
create view AnzahlSWSPProProf as  
    select p.name, p.persnr, sum(v.sws)  
    from professoren p, vorlesungen v  
    where p.persnr=v.gelesenvon  
    group by p.name, p.persnr;
```

Achtung:

replace view erwartet dieselben Spalten in derselben Reihenfolge mit denselben Typen.

Alternative:

erst löschen, dann neu erzeugen. oder: Postgres SQL-Erweiterung **alter view**

Beispiele für Sichten

Relation **pruefen**: ([MatrNr, VorlNr, PersNr, Note])

```
create view pruefenSicht as  
  select MatrNr, VorlNr, PersNr  
  from pruefen;
```

Was ist der Unterschied zu **pruefen**?

```
create view pruefenSicht2 (M,V,P) as  
  select MatrNr, VorlNr, PersNr  
  from pruefen;
```

Was ist der Unterschied zu **pruefenSicht**?

```
create view pruefenSicht3 (M,V) as  
  select MatrNr, VorlNr, PersNr  
  from pruefen;
```

Was ist der Unterschied zu **pruefenSicht2**?

Sichten vs. Materialisierte Sichten

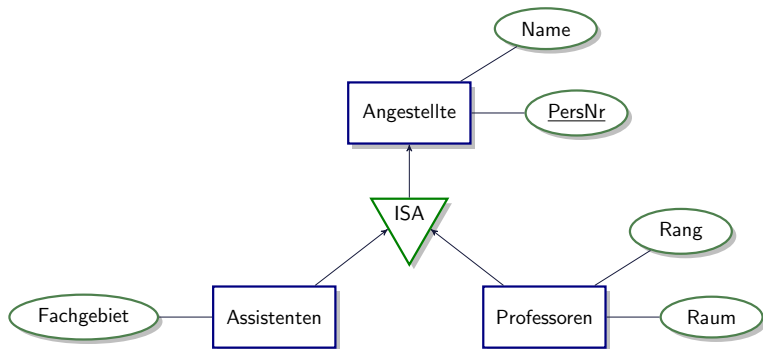
(Dynamische) Sicht

- =Makro für Query
- Ergebnis der Anfrage wird nicht vorberechnet
- Rechenaufwand zum Zeitpunkt der Anfrage (=query time)
- erst wenn die Sicht benutzt wird, wird auch das Ergebnis der Sicht berechnet

Materialisierte Sicht

- Ergebnis der Sicht wird vorberechnet
- wenn eine Anfrage die Sicht benutzt, ist das Ergebnis der Sicht bereits vollständig berechnet
- Rechenaufwand vorher (=index time)
- Problem bei Updates:
Tabellen, die zur Vorbereitung der Sicht benutzt werden, ändern sich ⇒ Materialisierte Sicht muss (oft) angepaßt werden

Wiederholung: Generalisierung



Sichten zur Modellierung von Generalisierung

create table Angestellte

(PersNr **integer not null,**
Name **varchar (30) not null);**

create table ProfDaten

(PersNr **integer not null,**
Rang **character (2),**
Raum **integer);**

create table AssistentenDaten

(PersNr **integer not null,**
Fachgebiet **varchar (30),**
Boss **integer);**

```
create view Professoren as  
  select *  
  from Angestellte A join ProfDaten D  
  on A.PersNr=D.PersNr;
```

```
create view Assistenten as  
  select *  
  from Angestellte A join AssistentenDaten D  
  on A.PersNr=D.PersNr;
```

Untertypen als Sicht

- Also Join von Untertyp und Obertyp
- nur bei Zugriff auf Professoren oder Assistenten muss View ausgeführt werden
- D.h. Zugriff auf Obertyp ist bevorzugt (günstig).

Alternative:

create table Professoren

(PersNr **integer not null,**
Name **varchar (30) not null,**
Rang **character (2)**
Raum **integer);**

create table Assistenten

(PersNr **integer not null,**
Name **varchar (30) not null,**
Fachgebiet **varchar (30)**
Boss **integer);**

create table Andere Angestellte

(PersNr **integer not null,**
Name **varchar (30) not null);**

```
create view Angestellte as  
    (select PersNr, Name  
    from Professoren)  
union  
(select PersNr, Name  
    from Assistenten)  
union  
    (select *  
    from AndereAngestellte);
```

Obertypen als Sicht

- Union von Ober- und Untertypen
- Also: bevorzugt Zugriff auf Untertypen
- Nur bei Zugriff auf Obertyp muss die View ausgeführt werden

Updates in SQL

```
update studenten  
  set semester=semester+1
```

kombiniert mit where:

```
update professoren  
  set rang='C4'  
where name='Kopernikus'
```

Änderbarkeit von Sichten

Beispiel für nicht veränderbare Sichten

```
create view WieHartAlsPruefer as  
  select PersNr, avg(Note)  
  from pruefen  
  group by PersNr;  
  
update WieHartAlsPruefer  
  set Durchschnittsnote=1.0  
  where PersNr=(select PersNr  
                from Professoren  
                where Name='Sokrates');
```

Änderbarkeit von Sichten

Beispiel für nicht veränderbare Sichten

```
create view VorlesungenSicht as  
  select Titel, SWS, Name  
  from Vorlesungen, Professoren  
  where gelesenVon=PersNr;  
  
insert into VorlesungenSicht  
  values ('Nihilismus', '2', 'Nobody');
```

Wie soll das DBMS dieses Tupel den Tabellen Professoren und Vorlesungen zuordnen?

Änderbarkeit von Sichten in SQL

Daten in einer View sind veränderbar (update/insert/delete), wenn ...

in SQL-92

- nur eine Tabelle
- nur Selektion und Projektion
- keine Aggregatfunktionen, Gruppierung und Duplikateliminierung

in SQL-99

- wie SQL-92
- oder: mehrere Tabellen UND Felder, auf die sich das Update bezieht, können mit Hilfe des Primärschlüssels eindeutig zugeordnet werden.
- D.h. auch bei einem Join kann man manchmal ändern.

Änderbarkeit von Sichten in Postgres

Bis zur Version 9.2 waren View in Postgresql generell nicht aktualisierbar!

Ab Version 9.3 im Sinne von SQL-92.

Also in Version ≤ 9.2 hätten auch z.B. inserts auf diese triviale View nicht funktioniert.

```
create view Studis as
```

```
  select *
```

```
  from studenten;
```

```
insert into studis values (42, 'mueller', 11);
```

ERROR: cannot insert into a view.

HINT: You need an unconditional ON INSERT DO INSTEAD rule.

Postgresqls Regel (Rule) System

- Erlaubt Handhabung von INSERT/UPDATE/DELETE auf Views durch spezielle Regeln

Rules (Nicht relevant für Klausur)

```
create [ or replace ] rule name  
as on event to table [where condition]  
do [ also | instead ] action
```

- event: update/insert/delete
- action:
 - nothing: mache nichts
 - command: eine Aktion
 - command1; command2; ... : mache mehrere Aktionen
- also
- instead

<http://www.postgresql.org/docs/9.3/static/rules-update.html>

View update mit rule **instead** (Nicht relevant für Klausur)

```
create rule StudilInsert  
as on insert to studis  
do instead  
    insert into studenten  
        values (NEW.matrn timer, NEW.name, NEW.semester);
```

Jetzt OK:

```
insert into studis values (42,'mueller',11);
```

View update mit rule **also** (Nicht relevant für Klausur)

```
create or replace rule StudilInsert
as on insert to studis
do also
    insert into studenten
        values (NEW.matrnr, NEW.name, NEW.semester);

insert into studis values (42,'mueller',11);
```

ERROR: cannot insert into a view.

HINT: You need an unconditional ON INSERT DO INSTEAD rule.

Endlosrekursion (Nicht relevant für Klausur)

```
create or replace rule StudentenInsert
as on insert to studenten
do also
    insert into studenten
        values (NEW.matrnr, NEW.name, NEW.semester);
```

```
insert into studenten values (777,'mueller',11);
```

ERROR: infinite recursion detected in rules for relation "studenten"

Ohne Rekursion (Nicht relevant für Klausur)

```
create or replace rule StudentenInsert
as on insert to studenten
do also
    insert into hoeren
    values (New.matrn timer, 5001);

select count(*) from studenten;

insert into studenten values (777, 'mueller', 11);

select count(*) from studenten;

select * from hoeren where matrn timer=777;
```

Ausblick auf kommende Vorlesungen

- **Weiterführende SQL Konzepte**
 - Fensteranfragen
 - Rekursion in SQL
- **Anwendungsprogrammierung mit JDBC**
- **PL/pgSQL**
 - Sprache/Konzept um prozeduralen Code zu schreiben und direkt in der Datenbank (nicht im Client mit Java/JDBC) auszuführen.
- **Trigger**
 - Beschreibe welche Anweisungen ausgeführt werden sollen, wenn bestimmte Bedingung erfüllt ist. Z.B. füge Tupel in Relation Bestellungen ein, wenn ein Produkt im Lager weniger als 10 Mal vorrätig ist.

Zur Erinnerung: Die relationale Uni-DB

Professoren			
PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

Studenten		
MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Vorlesungen			
VorlNr	Titel	SWS	gelesen von
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

voraussetzen	
Vorgänger	Nachfolger
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

hören	
MatrNr	VorlNr
26120	5001
27550	5001
27550	4052
28106	5041
28106	5052
28106	5216
28106	5259
29120	5001
29120	5041
29120	5049
29555	5022
25403	5022

Assistenten			
PersNr	Name	Fachgebiet	Boss
3002	Platon	Ideenlehre	2125
3003	Aristoteles	Syllogistik	2125
3004	Wittgenstein	Sprachtheorie	2126
3005	Rhetikus	Planetenbewegung	2127
3006	Newton	Keplersche Gesetze	2127
3007	Spinoza	Gott und Natur	2126

prüfen			
MatrNr	VorlNr	PersNr	Note
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2

Weiterführende SQL Konzepte

Ranking, Partitionierung und Fensteranfragen

- SQL:2003 Standard
- Ermöglicht das Gruppieren/Partitionieren von Tupeln der Ergebnismenge.
- Und Anwendung einer Aggregat-Funktion auf diese Partitionen
- Z.B. **select** , count(*) **over** (**partition by** MatrNr) as vlcount **from** ...

Rekursion

- Beispiel: Gegeben eine Relation setztVoraus, in der für eine Vorlesung jeweils die direkten Vorgänger (welche vorausgesetzt werden) angegeben sind
- Wie können dann rekursiv (beliebig tief) Vorlesungen gefunden werden, auf denen eine Vorlesung aufbaut?

```

select s.name, count(*) over
(partition by s.matrnr) as vlcount,
h.vorlNr
from hoeren h, studenten s
where h.matrnr=s.matrnr
order by s.name asc;

```

Im Vergleich dazu:

```

select s.name, count(*) as vlcount,
h.vorlNr
from hoeren h, studenten s
where h.matrnr=s.matrnr
group by s.name, h.vorlNr
order by s.name asc;

```

	name character varying(30)	vlcount bigint	vorlNr integer
1	Carnap	4	5259
2	Carnap	4	5216
3	Carnap	4	5052
4	Carnap	4	5041
5	Feuerbach	2	5001
6	Feuerbach	2	5022
7	Fichte	1	5001
8	Jonas	1	5022
9	Schopenhauer	2	5001
10	Schopenhauer	2	4052
11	Theophrastos	3	5049
12	Theophrastos	3	5041
13	Theophrastos	3	5001

	name character varying(30)	vlcount bigint	vorlNr integer
1	Carnap	1	5041
2	Carnap	1	5052
3	Carnap	1	5216
4	Carnap	1	5259
5	Feuerbach	1	5001
6	Feuerbach	1	5022
7	Fichte	1	5001
8	Jonas	1	5022
9	Schopenhauer	1	4052
10	Schopenhauer	1	5001
11	Theophrastos	1	5001
12	Theophrastos	1	5041
13	Theophrastos	1	5001

Im Vergleich dazu:

```
select s.name, count(*) as vlcount
from hoeren h, studenten s
where h.matrnr=s.matrnr
group by s.name
order by s.name asc;
```

	name character varying(30)	vlcount bigint
1	Carnap	4
2	Feuerbach	2
3	Fichte	1
4	Jonas	1
5	Schopenhauer	2
6	Theophrastos	3

.... mit mehreren Partitionen

```

select s.name, count(*) over (partition by s.matrnr) as vlcount,
count(*) over (partition by h.vorlNr) as studentcount, h.vorlNr
from hoeren h, studenten s
where h.matrnr=s.matrnr
order by s.name asc;

```

- **vlcount:** Anzahl Tupel mit demselben Namen
- **studentcount:** Anzahl Tupel mit derselben VorlNr

	name character varying(30)	vlcount bigint	studentcount bigint	vorlNr integer
1	Carnap	4	1	5052
2	Carnap	4	2	5041
3	Carnap	4	1	5216
4	Carnap	4	1	5259
5	Feuerbach	2	2	5022
6	Feuerbach	2	4	5001
7	Fichte	1	4	5001
8	Jonas	1	2	5022
9	Schopenhauer	2	4	5001
10	Schopenhauer	2	1	4052
11	Theophrastos	3	1	5049
12	Theophrastos	3	2	5041
13	Theophrastos	3	4	5001

rank()

```
select s.name, count(*) over (partition by s.matnr as vlcount,
rank() over (partition by s.name order by h.vorlNr) as rank, h.vorlNr
from hoeren h, studenten s
where h.matnr=s.matnr
order by s.name asc;
```

- **rank:** Position in Gruppe wenn partitioniert nach s.name

	name character varying(30)	vlcount bigint	rank bigint	vorlNr integer
1	Carnap	4	1	5041
2	Carnap	4	2	5052
3	Carnap	4	3	5216
4	Carnap	4	4	5259
5	Feuerbach	2	1	5001
6	Feuerbach	2	2	5022
7	Fichte	1	1	5001
8	Jonas	1	1	5022
9	Schopenhauer	2	1	4052
10	Schopenhauer	2	2	5001
11	Theophrastos	3	1	5001
12	Theophrastos	3	2	5041
13	Theophrastos	3	3	5049

rank() vs. dense_rank()

Berechne den Rang von Studenten basierend auf GPA.

```
select ID, rank() over (order by GPA desc) as student_rank  
from student_grades  
order by student_rank;
```

- Was passiert, wenn zwei Studenten den gleichen GPA haben?
- Z.B. wenn bei Studenten den höchsten GPA haben bekommen beide Rang 1
- die nächsten Studenten bekommen dann Rang 3, etc.
- Wenn es drei Studenten geben würde mit dem zweithöchsten GPA, dann würde der nächste zu vergebene Rang 6 sein.
- Bei Funktion **dense_rank** gibt es keine fehlenden Ränge, d.h. die Tupel mit dem zweithöchsten Wert würden Rang 2 bekommen, etc.

Beispiel ohne rank() berechnet

Die Anfrage von zuvor kann auch wie folgt ausgedrückt werden:

```
select ID, (1+(select count(*)  
               from student_grades B  
               where B.GPA > A.GPA)) as student_rank  
from student_grades A  
order by student_rank;
```

- **Die naive Ausführung dieser Anfrage hat quadratischen Aufwand:** Für jeden Studenten wird der Rang berechnet durch **linearen Aufwand** (laufen über alle anderen Studenten).
- **In der vorhergehenden Notation mit rank() kann das Datenbanksystem geschickter vorgehen:** Relation sortieren und den Rang einfach berechnen.

window frames

select h.matrnr, count(h.matrnr) over ()
from hoeren h;

- keine Partitionierung
- alle Werte gleich

	matrnr integer	count bigint
1	26120	13
2	27550	13
3	27550	13
4	28106	13
5	28106	13
6	28106	13
7	28106	13
8	29120	13
9	29120	13
10	29120	13
11	29555	13
12	25403	13
13	29555	13

select h.matrnr, count(h.matrnr) over (order by
 h.matrnr)
from hoeren h;

- keine Partitionierung
- ABER: **laufendes Fenster**
- Fenster enthält
 - alle Werte bis hierhin
 - einschließlich derjenigen mit **derselben** matrnr

	matrnr integer	count bigint
1	25403	1
2	26120	2
3	27550	4
4	27550	4
5	28106	8
6	28106	8
7	28106	8
8	28106	8
9	29120	11
10	29120	11
11	29120	11
12	29555	13
13	29555	13

Regeln für Fensterfunktion

- **Nur im SELECT und ORDER BY erlaubt**
- Nicht in **group by**, **having** und **where**
- Warum? Ausführung der Fensterfunktionen passiert logisch nach diesen Statements
- **kann mit normaler Aggregation kombiniert werden:**

```
select h.matrnr, count(*) as countstar, count(*) over (order by h.matrnr)
as countpartition
from hoeren h
group by h.matrnr;
```

- **countpartition:** laufendes Fenster über Ergebnis der Gruppierung!

	matrnr integer	countstar bigint	countpartition bigint
1	25403	1	1
2	26120	1	2
3	27550	2	3
4	28106	4	4
5	29120	3	5
6	29555	2	6

```
select h.matrnr, count(*) as countstar, count(*) over () as countpartition  
from hoeren h  
group by h.matrnr;
```

- keine Partitionierung!
- kein laufendes Fenster!

Auswertungsreihenfolge:

- erst **where**
- dann **group by**
- dann **having**
- dann **Fenster**

Angenommen wir haben eine Tabelle `tot_credits`, in der die Summe aller von Studenten erreichten Credit Points stehen, pro Jahr, also ein Eintrag pro Jahr.

```
select year, avg(num_credits)
      over (order by year rows 3 preceding)
      as avg_total_credits
from tot_credits;
```

- Diese Anfrage berechnet die avg. Anzahl von Credits über dem Tupel selbst und 3 vorherigen Tupeln, in der spezifizierten Reihenfolge.
- Wir haben also z.B. für das Jahr 2009 einen Durchschnitt, der über die Jahre 2006, 2007, 2008 und 2009 berechnet wurde.
- Ähnlich mit Angaben von following:

```
select year, avg(num_credits)
      over (order by year rows
            between 3 preceding and 2 following)
      as avg_total_credits
from tot_credits;
```

Beispiel

Wetter	
Zeit	Temperatur
1	15
2	16
3	17
4	17
5	16
6	18
7	20
8	21
9	22
10	21
11	20
12	23

select zeit,
avg(temperatur) over
(order by zeit rows
between 1 preceding
and 1 following) as
avg_temp from wetter
order by zeit

Zeit	avg_temp
1	15.5
2	16
3	16.66
4	16.66
5	17
6	18
...	...

Beispiel (2)

Mit der Aggregationsfunktion `array_agg` können wir zur Veranschaulichung schauen welche Werte dieses laufende Fenster umfasst:

select zeit, array_agg(temperatur) **over** (**order by** zeit **rows between 1 preceding and 1 following**) **as** info from wetter **order by** zeit

Zeit	info
1	{15,16}
2	{15,16,17}
3	{16,17,17}
4	{17,17,16}
5	{17,16,18}
6	{16,18,20}
...	...

Bereichsbasierte Fenster-Spezifikation

- Die Fenster zuvor waren spezifiziert durch Anzahl der Tupel (z.B. 3 preceding and 2 following)
- Nun: Fenster-Spezifikation berücksichtigt Bereich (Range) von Daten

```
select year, avg(num_credits)
      over (order by year range between year-4 and year)
      as avg_total_credits
from tot_credits;
```

Bereichsbasierte Fenster-Spezifikation

- Angenommen wir haben nun eine Relation `tot_credits_depts(dept_name, year, num_credits)`
- Also wie zuvor bloß per Department
- Dann können wir nun noch anhand von Department partitionieren:

```
select dept_name,year, avg(num_credits)
      over (partition by dept_name
      order by year range between year-4 and year)
      as avg_total_credits
from tot_credits_dept;
```

Hinweis: Einige Beispiele sind zum Teil direkt übernommen aus dem Buch "Database System Concepts" von Silberschatz, Korth, Sudarshan.

Übersicht Syntax (in Postgresql)

<http://www.postgresql.org/docs/current/static/sql-expressions.html#SYNTAX-WINDOW-FUNCTIONS>

```
function_name ([expression [, expression ... ]]) OVER window_name
function_name ([expression [, expression ... ]]) OVER ( window_definition )
function_name ( * ) OVER window_name
function_name ( * ) OVER ( window_definition )
```

where window_definition has the syntax

```
[ existing_window_name ]
[ PARTITION BY expression [, ...] ]
[ ORDER BY expression [ ASC | DESC | USING operator ] [ NULLS
                                                                    { FIRST | LAST } ] [, ...] ]

[ frame_clause ]
```

and the optional frame_clause can be one of

```
{ RANGE | ROWS } frame_start
{ RANGE | ROWS } BETWEEN frame_start AND frame_end
```

where frame_start and frame_end can be one of

```
UNBOUNDED PRECEDING
value PRECEDING
CURRENT ROW
value FOLLOWING
UNBOUNDED FOLLOWING
```


Voraussetzen	
vorgaenger	nachfolger
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

Vorlesungen	
vorlnr	titel
5001	Grundzuege
5041	Ethik
5043	Erkenntnistheorie
5049	Maeeutik
4052	Logik
5052	Wissenschaftstheorie
5216	Bioethik
5259	Der Wiener Kreis
5022	Glaube und Wissen
4630	Die 3 Kritiken

Welche Vorlesungen müssen besucht werden, um die Vorlesung “Der Wiener Kreis” verstehen zu können?

Wie kann dies in SQL berechnet werden?

Rekursion

**Welche Vorlesungen müssen besucht werden, um die Vorlesung
“Der Wiener Kreis” verstehen zu können?**

```
select Vorgaenger  
from voraussetzen, Vorlesungen  
where Nachfolger=VorINr and  
      Titel = 'Der Wiener Kreis';
```

Rekursion

Welche Vorlesungen müssen besucht werden, um die Vorlesung “Der Wiener Kreis” verstehen zu können?

```
select Vorgaenger  
from voraussetzen, Vorlesungen  
where Nachfolger=VorINr and  
      Titel = 'Der Wiener Kreis';
```

Hier werden allerdings nur die direkten Vorgänger berechnet. Ergebnis: Vorlesung mit VorINr=5052.

Der Wiener Kreis 5259

Wissenschaftstheorie 5052

Bioethik 5216

Erkenntnistheorie 5043

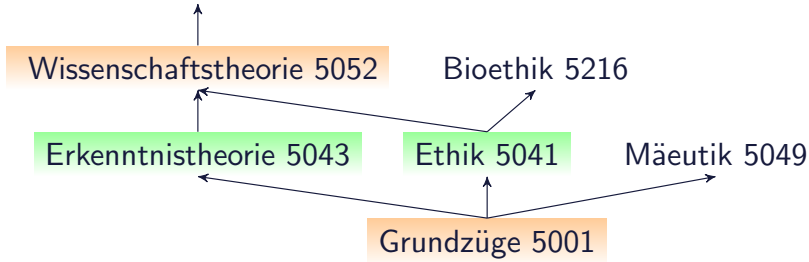
Ethik 5041

Mäeutik 5049

Grundzüge 5001

```
select v1.Vorgaenger  
from voraussetzen v1, voraussetzen v2, Vorlesungen v  
where v1.Nachfolger= v2.Vorgaenger and  
v2.Nachfolger= v.VorlNr and  
v.Titel='Der Wiener Kreis';
```

Der Wiener Kreis 5259



Vorgänger der Tiefe n

```
select v1.Vorgaenger
from voraussetzen v1
    ...
    voraussetzen vn_minus_1
    voraussetzen vn,
    Vorlesungen v
where v1.Nachfolger= v2.Vorgaenger and
    ...
    vn_minus_1.Nachfolger= vn.Vorgaenger and
    vn.Nachfolger = v.VorlNr and
    v.Titel='Der Wiener Kreis';
```

Wie kann man alle Vorgänger finden?

Tiefe 1 union Tiefe 2 union Tiefe 3 union ...

Transitive Hülle

Was hier sehr hilfreich wäre: **Berechnung der transitiven Hülle.**

$$trans_{A,B}(R) = \{(a,b) | \exists k \in \mathbb{N} (\exists \tau_1, \tau_2, \dots, \tau_k \in R($$

$$\tau_1.A = a \wedge$$

$$\tau_1.B = \tau_2.A \wedge$$

$$\tau_2.B = \tau_3.A \wedge$$

...

$$\tau_{k-1}.B = \tau_k.A \wedge$$

$$\tau_k.B = b))\}$$

Enthält alle Paare, für die ein “Pfad” beliebiger Länge k in R existiert.

Values Statement

- Erzeugt konstante Tabelle
- **values** (1, 'eins'), (2, 'zwei'), (3, 'drei');

ist äquivalent zu:

```
select 1 as column1, 'eins' AS column2
union all
select 2, 'zwei'
union all
select 3, 'drei';
```

Verwendung z.B. in Select Statement (auch in Joins) als normale Tabelle

```
select * from
(values (1,'eins'), (2,'zwei')) as table1 (nummer, wert);
```


Vergleiche hierzu:

```
with mytable(mycolumn) as(  
    values (1)  
)  
select mycolumn+1  
from mytable;
```

Rekursionen in SQL

Definieren eine rekursive "View", basierend auf Union (all) von Zwei Anfragen: Einer nicht-rekursiven Anfrage und einer rekursiven.

```
with recursive mytable(mycolumn) as (  
    values(1)                                nicht rekursiver Teil  
    union all                                union all  
        select mycolumn+1  
        from mytable                        rekursiver Teil:  
        where mycolumn < 100                nur dieser darf mytable referenzieren  
)  
select sum(mycolumn)                        eigentlicher Aufruf  
from mytable;
```

Ergebnis: 5050

Achtung: Darauf achten, dass die Rekursion terminiert!

Auswertung von rekursiven Anfragen

Schritt 1

- Evaluiere den nicht-rekursiven Teil. Für UNION (nicht aber für UNION ALL), entferne Duplikate. Alle verbliebenen Tupel werden in Ergebnis hinzugefügt und auch in eine temporäre Tabelle kopiert.

Schritt 2: Solange temporäre Tabelle nicht leer ist wiederhole:

- (a) Evaluiere den rekursiven Teil, wobei die Eigenreferenz durch die temporäre Tabelle ersetzt wird. Für UNION (aber nicht UNION ALL), entferne Duplikate und auch Duplikate zu vorherigen Ergebnissen. Füge verbliebene Tupel in Ergebnis hinzu und ebenso in eine temporäre Zwischenergebnis-Tabelle.
- (b) Ersetzt Inhalt der temporären Tabelle mit dem Inhalt der Zwischenergebnis-Tabelle, leere die temporäre Zwischenergebnis-Tabelle.

Rekursion: Anfrage für Voraussetzungen

```
with recursive TransitivVorl (Vorg, Nachf)
as (
    select Vorgaenger, Nachfolger
    from voraussetzen
union all
    select distinct t.Vorg, v.Nachfolger
    from TransitivVorl t, voraussetzen v
    where t.Nachf=v.Vorgaenger
)
select *
from TransitivVorl
order by (vorg,nachf) asc;
```

Rekursion: Anfrage für Voraussetzungen

```
with recursive TransitivVorl (Vorg, Nachf)
as (
    select Vorgaenger, Nachfolger
    from voraussetzen
union all
    select distinct t.Vorg, v.Nachfolger
    from TransitivVorl t, voraussetzen v
    where t.Nachf=v.Vorgaenger
)
select v2.titel
from TransitivVorl tv, vorlesungen v1, vorlesungen v2
where tv.nachf=v1.vorlnr and v1.titel='Der Wiener Kreis'
    and vorg=v2.vorlnr;
```

with recursive TransitivVorl (Vorg, Nachf, iteration)
as (

select Vorgaenger, Nachfolger, **1 as iteration**
from voraussetzen

union all

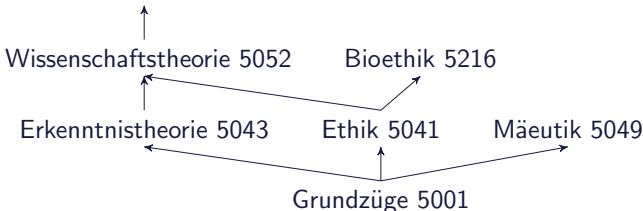
select distinct t.Vorg, v.Nachfolger, **1+ t.iteration**
from TransitivVorl t, voraussetzen v
where t.Nachf=v.Vorgaenger

)

select * from TransitivVorl **order by** iteration;

	vorg integer	nachf integer	iteration integer
1	5001	5041	1
2	5001	5043	1
3	5001	5049	1
4	5041	5216	1
5	5043	5052	1
6	5041	5052	1
7	5052	5259	1
8	5043	5259	2
9	5041	5259	2
10	5001	5052	2
11	5001	5216	2
12	5001	5259	3

Der Wiener Kreis 5259



Nur zur Info: User-defined Function (UDF), die für eine bestimmte Vorlesung alle Vorgänger berechnet. UDFs betrachten wir später noch.

```
CREATE OR REPLACE FUNCTION alleVorgaenger(id integer)
RETURNS TABLE (VorlNr integer) AS $$
BEGIN
    -- temporaere Tabellen
    CREATE TEMP TABLE IF NOT EXISTS vorgaengerTabelle(VorlNr integer);
    CREATE TEMP TABLE IF NOT EXISTS neu_vorgaengerTabelle(VorlNr integer);
    CREATE TEMP TABLE IF NOT EXISTS temp_Tabelle(VorlNr integer);
    DELETE FROM vorgaengerTabelle;
    DELETE FROM temp_Tabelle;
    DELETE FROM neu_vorgaengerTabelle;

    --los geht es mit Suche der direkten Vorgaenger
    INSERT INTO neu_vorgaengerTabelle (select v.vorgaenger from voraussetzen v where v.nachfolger = id);
    LOOP
        INSERT INTO vorgaengerTabelle (select r.vorlNr from neu_vorgaengerTabelle r);
        INSERT INTO temp_Tabelle --suche neue Vorgaenger
            (SELECT v.vorgaenger
             FROM neu_vorgaengerTabelle r, voraussetzen v
             WHERE v.nachfolger = r.vorlNr
             ) --aber nur Neue
            EXCEPT (SELECT r.vorlNr FROM vorgaengerTabelle r);
        DELETE from neu_vorgaengerTabelle;
        INSERT into neu_vorgaengerTabelle (select * from temp_Tabelle);
        DELETE FROM temp_Tabelle;
        IF NOT EXISTS (SELECT * FROM neu_vorgaengerTabelle) THEN
            EXIT; --exit aus Schleife, falls keine weiteren Vorgaenger gefunden
        END IF;
    END LOOP;
    RETURN QUERY SELECT  r.vorlNr FROM vorgaengerTabelle r;
END;
$$ LANGUAGE plpgsql;
```

Rekursion in SQL

- Iteration wird so lange ausgeführt bis keine neuen Tupel hinzugefügt werden.
- Also ein Fixpunkt erreicht ist.

Voraussetzungen

- Die rekursive Anfrage muss **monoton** sein, d.h. ausgeführt auf einer Instanz V_1 der rekursiven View muss Ergebnis eine Obermenge von den Ergebnissen auf Instanz V_2 sein, falls V_1 eine Obermenge von V_2 ist.
- D.h. wenn mehr Tupel zur View hinzugefügt werden muss die rekursive Anfrage mindestens die gleichen Tupel wie zuvor liefern.
- z.B. sollte die rekursive Anfrage also kein NOT EXISTS enthalten.

Datenbank-Zugriff via JDBC

Java Database Connectivity

- Bietet Schnittstelle für den Zugriff auf ein DBMS aus Java-Anwendungen

JDBC: Connect und einfache Anfrage

```
1 //registriere geeigneten Treiber (hier fuer Postgresql)
2 Class.forName("org.postgresql.Driver");
3 //erzeuge Verbindung zur Datenbank
4 Connection conn = DriverManager.getConnection(
5     "jdbc:postgresql://localhost/university",
6     "username", "password");
7
8 //erzeuge ein einfaches Statement Objekt
9 Statement stmt = conn.createStatement();
10
11     //mit execute Query koennen nun darauf Anfragen
    ausgefuehrt werden
12     //Ergebnisse in Form eines ResultSet Objekts
13     ResultSet rset = stmt.executeQuery("SELECT p.
    persnr from professoren p");
```

JDBC: Connect und einfache Anfrage

```

14 //dieses besitzt Metadaten
15 ResultSetMetaData metadata = rset.getMetaData();
16
17 //welche Attribute (Spalten) besitzen die
Ergebnis-Tupel?
18 int column_count = metadata.getColumnCount();
19
20 for (int index=1; index<=column_count; index++) {
21     System.out.println("Spalte "+index+" heisst
" +
22         metadata洗getColumnName(index));
23 }
24
25 //iteriere nun ueber Ergebnisse
26 while (rset.next()) {
27     System.out.println(rset.getString(1));
28 }

```

JDBC Treiber für Postgresql

<http://jdbc.postgresql.org/>

Siehe insbesondere Dokumentation dazu (mit Beispielen), sowie ganz allgemein die **Dokumentation zum Paket java.sql**:

<https://docs.oracle.com/javase/8/docs/api/index.html?java/sql/package-summary.html>

JDBC - wichtige Funktionalitäten

Laden des Treibers

- Kann auf verschiedene Weise erfolgen, z.B. durch explizites Laden mit dem Klassenlader:

```
Class.forName(DriverClassName);
```

Aufbau einer Verbindung

- Connection-Objekt repräsentiert die Verbindung zum DB-Server
- Beim Aufbau werden URL der DB, Benutzername und Passwort aus Strings übergeben (teilweise optional).

```
Connection conn =  
    DriverManager.getConnection(url ,  
                                login , password);
```

JDBC - wichtige Funktionalitäten (2)

Anweisungen

- Mit dem Connection-Objekt können u.a. Metadaten der DB erfragt und Statement-Objekte zum Absetzen von SQL-Anweisungen erzeugt werden
- Erzeugen einer SQL-Anweisung zur direkten (einmaligen) Ausführung

```
Statement stmt = conn.createStatement();
```

- PreparedStatement-Objekt erlaubt das Erzeugen und Vorbereiten von (parametrisierten) SQL-Anweisungen zur wiederholten Ausführung

```
PreparedStatement pstmt = conn.prepareStatement(  
    "select * from personal where gehalt >= ?");
```

Schließen von Verbindungen, Statements, usw.

```
stmt.close();  
conn.close();
```

JDBC - Anweisungen

Anweisungen (Statements)

- Werden in einem Schritt vorbereitet und ausgeführt

Die Methode `executeQuery`

- führt die Anfrage aus und liefert Ergebnis zurück

```
Statement stmt = conn.createStatement();  
ResultSet rset = stmt.executeQuery(  
    "select pnr, name, gehalt  
    from personal where gehalt >= 40000");  
//wir sehen gleich, wie man mit diesem ResultSet  
arbeitet
```

JDBC - Anweisungen

Die Methode executeUpdate

- werden zur direkten Ausführung von UPDATE-, INSERT-, DELETE- und DDL-Anweisungen benutzt

```
Statement stmt = conn.createStatement();  
int n = stmt.executeUpdate(  
    "update personal  
    set gehalt = gehalt * 1.10  
    where gehalt < 20000");
```

//n enthaelt die Anzahl der aktualisierten Zeilen

JDBC - Prepared Anweisungen

PreparedStatement-Objekt

```
PreparedStatement pstmt;  
double gehalt = 50000.00;  
pstmt = conn.prepareStatement(  
    "select * from personal where gehalt >= ?");
```

- Symbol ? markiert hier freie Parameter
- Vor der Ausführung sind dann die Parameter einzusetzen.
- Durch Methoden entsprechend Datentyp, z.B.

```
pstmt.setDouble(1, gehalt);
```

<https://docs.oracle.com/javase/8/docs/api/java/sql/PreparedStatement.html>

JDBC - Prepared Anweisungen (2)

Ausführen einer Prepared-Anweisung als Anfrage

```
PreparedStatement pstmt;  
double gehalt = 50000.00;  
pstmt = conn.prepareStatement(  
    "select * from personal where gehalt >= ?");
```

Vorbereitung und Ausführung

```
pstmt = conn.prepareStatement(  
    "delete from personal where name = ?");  
pstmt.setString(1, "Maier");
```

```
int n = pstmt.executeUpdate();
```

//Methoden der Prepared-Anweisungen haben keine Argumente

JDBC - Ergebnismengen und Cursor

Select-Anfragen und Ergebnisübergabe

- Jede JDBC-Methode, mit der man Anfragen an das DBMS stellen kann, liefert ResultSet-Objekte als Rückgabewert

```
ResultSet rset = stmt.executeQuery(  
    "select pnr, name, gehalt  
    from personal where gehalt >= " + gehalt);
```

- Cursor-Zugriff und Konvertierung der DBMS-Datentypen in passende Java-Datentypen erforderlich
- JDBC-Cursor ist durch die Methode **next()** der Klasse ResultSet implementiert

<https://docs.oracle.com/javase/8/docs/api/java/sql/ResultSet.html>

JDBC - Ergebnismengen und Cursor (2)

Cursor → `getInt("pnr")` `getString("name")` `getDouble("gehalt")`

↓ `next()`

↓	123	Maier	23352.00
	456	Schulze	34553.00

Zugriff aus Java-Programm

```
while ( rset.next() ) {  
    System.out.print(res.getInt(" pnr")+ "\t" );  
    System.out.print(res.getString(" name")+ "\t" );  
    System.out.println(res.getDouble(" gehalt" ));  
}
```

JDBC - Versch. Typen von ResultSets

TYPE_FORWARD_ONLY

- nur Aufruf von **next()** möglich

TYPE_SCROLL_INSENSITIVE

- Scroll-Operationen sind möglich, aber Aktualisierungen der Datenbank verändern ResultSet nach seiner Erstellung nicht

JDBC - Versch. Typen von ResultSets (2)

TYPE_SCROLL_SENSITIVE

- Scroll-Operationen möglich und Änderungen in der Datenbank werden berücksichtigt

ResultSet lässt Änderungen zu oder nicht:

- CONCUR_UPDATABLE
- CONCUR_READ_ONLY

```
Statement stmt = conn.createStatement(  
    ResultSet.TYPE_SCROLL_SENSITIVE,  
    ResultSet.CONCUR_UPDATABLE);  
ResultSet rset = stmt.executeQuery(...);  
rset.updateString("name", "Schmitt");  
rset.updateRow();
```

JDBC - Zugriff auf Metadaten

Allgemeine Metadaten

- Klasse **DatabaseMetaData** zum Abfragen von DB-Informationen

Informationen über ResultSets

- JDBC bietet die Klasse **ResultSetMetaData**

```
ResultSet rset = stmt.executeQuery("select ...");  
ResultSetMetaData rsmd = rset.getMetaData();
```

- Abfragen von Spaltenanzahl, Spaltennamen und deren Typen

```
int anzahlSpalten = rsmd.getColumnCount();  
String spaltenName = rsmd.getColumnName(1);  
String typeName = rsmd.getColumnTypeName(1);
```

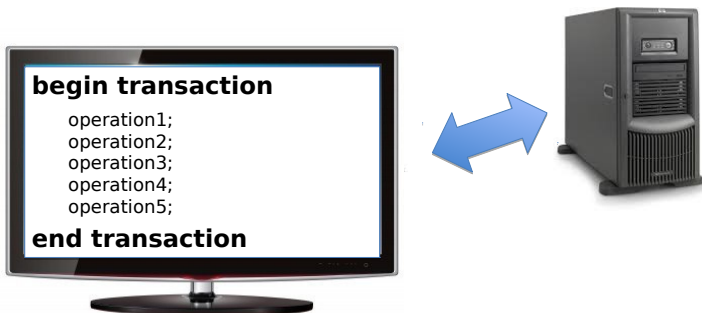
JDBC - Fehlerbehandlung

- Spezifikation der Ausnahmen, die eine Methode werfen kann, steht bei ihrer Deklaration (throws Exception)
- Wird Code in einem try-Block ausgeführt, werden im catch-Block Ausnahmen abgefangen.

```
try {  
    //code .....  
}  
catch (SQLException e) {  
    System.out.println("Es ist ein Fehler aufgetreten  
:");  
    System.out.println("Msg: " + e.getMessage());  
    System.out.println("SQLState: " + e.getSQLState());  
;  
    System.out.println("ErrorCode: " + e.getErrorCode  
());  
    //und zum Debuggen noch gleich dazu  
    e.printStackTrace();  
}
```


Anwendungsprogrammierung: Transaktionen

- Haben nun nicht nur eine einzelne SQL-Anweisung, sondern ganze Folge davon, je nach Anwendung.
- **Eine oder mehrere Anweisungen werden als Transaktion zusammengefasst bzw. betrachtet.** Z.B. Abheben von Geld am Geldautomat.
- Wird im Detail im **Kapitel über Transaktionen** betrachtet.



Anmerkung: SQL Injections

SQL Anfragen wird in Anwendung erstellt, wobei id eine Benutzereingabe ist

```
.... "SELECT author, subject, text " +  
      "FROM artikel WHERE ID=" + id
```

Aufruf z.B. durch Webserver `http://webserver/cgi-bin/find.cgi?ID=42`

SQL Injection zum Ausspähen von Daten

`http://webserver/cgi-bin/find.cgi?ID=42+UNION+SELECT+
login,+password,+'x'+FROM+user`

Führt zur SQL Anweisung:

```
select author, subject, text from artikel  
where ID=42 union select login, password, 'x' from user;
```

Und andere Fälle bis hin zum Einschleusen von beliebigem Code auf Rechner + öffnen einer Shell (abh. von DBMS)

Hilfe u.a. durch Benutzen von PreparedStatements

Übersicht unter: <http://de.wikipedia.org/wiki/SQL-Injection>

Weitere Call-Level-Interfaces (CLIs) für PostgreSQL

Für C++: libpqxx

- <http://pqxx.org/>

Für Ruby: pg

- <https://rubygems.org/gems/pg>

```
require 'pg'
conn = PG::Connection.open(:host => 'localhost',
                           :dbname => 'university', :user => 'username',
                           :password => 'my password')
res = conn.exec("select name from studenten")
res.each do |row|
  puts row['name']
end
```

Call-level-Interface (CLI) vs. Embedded SQL

- Unter Verwendung einer Bibliothek werden aus dem Anwendungsprogramm (Wirtssprache) Funktionen aufgerufen, die mit dem DBMS kommunizieren.
- **JDBC ist ein Beispiel für ein CLI**
- **Im embedded SQL werden hingegen SQL Anweisungen direkt in der Wirtssprache benutzt.**
- (Dennoch werden diese letztendlich durch Aufrufe von DBMS-spezifischen Bibliotheken realisiert)
- Hier nur kurz erwähnt. Syntax von Embedded SQL (SQLJ) nicht klausurrelevant.

Embedded SQL (ESQL)

Idee

- Benutze SQL-Anweisungen direkt im Programmcode
- Syntax in Java:

```
#sql { <sql-statement> };
```

- Syntax in C oder C++

```
EXEC SQL <sql-statement>;
```

Zum Beispiel:

```
1 EXEC SQL
2 SELECT vorname, nachname
3 INTO :vorname, :nachname
4 FROM mitarbeitertabelle
5 WHERE pnr = :pnr;
```

SQLJ: Embedded SQL für Java

- Einbettung von SQL in Java
- Anweisungen der Form

```
#sql { <sql-statement> };
```

Zum Beispiel:

```
#sql {INSERT INTO emp (ename, sal)  
      VALUES ( 'Joe', 43000) };
```

SQLJ: Embedded SQL für Java

Idee

- SQL Anweisungen werden direkt im Java Code benutzt (embedded)
- Ein Precompiler übersetzt diesen gemischten Code (in *.sqlj Dateien) in normalen Java Code (in *.java Dateien).
- Java Code benutzt dann JDBC oder Implementierung ähnlich zum JDBC Konzept.

SQLJ: Vor- und Nachteile im Vergleich zu JDBC

Vorteile

- Einfacher (kompakter) Code
- Verwendung der gleichen Variablen in SQL und in Java

Nachteile

- Erfordert extra Übersetzung in “normales” Java.

Umsetzung/Unterstützung

- Wird von Oracle angeboten (im eigenen DBMS)
- Sonst kaum (nicht) anzutreffen

Stored Procedures / User-Defined Functions

Stored Procedures/UDFs

Bislang: Kommunikation mit DBMS via Anwendungsprogrammen:
Einzelne Statements, Verarbeitung in Wirtssprache.

Manchmal ist es aber sinnvoll, Teile der Anwendung direkt im DBMS auszuführen und nicht via einzelnen SQL Statements.

Vorteile

- Daten müssen nicht erst auf dem DBMS zur Anwendung gebracht werden (und umgekehrt)
- Höhere Performance
- Code kann wiederverwendet werden (zwischen Anwendungen)

Nachteile

- Etwas aufwendiger zu erstellen.
- Debugging schwieriger.

SQL vs. SQL/PSM vs. PL/SQL bzw. PL/pgSQL

SQL

- Standard Query Language für Datenbanksysteme. Deklarativ.
- SQL Anweisungen können via Anwendungsprogrammierung (JDBC oder Embedded SQL) an DB geschickt werden.

PSM

- Persistent, Stored Modules (PSM), bzw. Sprache diese zu realisieren.
- Im SQL:2003 Standard definiert. Erlaubt es prozeduralen Code direkt innerhalb der DB zu schreiben.

PL/SQL bzw. PL/pgSQL

- Procedural Language/(PostgreSQL) Structured Query Language
- Prozedurale Sprache, benutzt in Oracle bzw. Postgresql
- Inspiriert von bzw. implementiert PSM.
- PL/SQL bzw. PL/pgSQL erlauben diese Anwendungslogik als Prozedur innerhalb des DBMS zu definieren.

Vorteile von Stored Procedures / User Defined Functions

- **Ausführungspläne können vor-übersetzt werden**, sind **wiederverwendbar**
- **Anzahl der Zugriffe** des Anwendungsprogramms auf das DBMS werden reduziert, ebenso wie Menge an Daten, die zwischen Anwendung und DBMS hin und her geschickt wird.
- Prozeduren sind als **gemeinsamer Code** für verschiedene Anwendungsprogramme nutzbar
- Es wird ein **höherer Isolationsgrad** der Anwendung von dem DBMS erreicht.

Beispiel

```
CREATE FUNCTION wieVieleVL (_matnr int)
                        RETURNS int AS $$
```

```
DECLARE
```

```
    qty int;
```

```
BEGIN
```

```
    SELECT COUNT(*) INTO qty
    FROM hoeren h
    WHERE h.matnr = _matnr;
```

```
    RETURN qty;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
select *
from wieVieleVL(28106);
```

Liefert Ergebnis: 4

Unterschied Stored Procedures und UDFs

Generell

- UDF = user-defined function
- Stored procedure muss explizit mit CALL aufgerufen werden (SQL EXEC CALL name)
- UDF kann direkt in SQL ohne CALL benutzt werden

In Postgresql

- In Postgres gibt es allerdings keinen Unterschied zwischen stored procedures und UDFs
- EXEC CALL gibt es in Postgres nicht.
- UDF wird aufgerufen in SELECT statements, z.B.
select from myFunction(44234234);

UDFs in Postgresql

Verschiedene Arten von UDFs

- Query language Funktionen (SQL)
- Procedural language Funktionen (PL/pgSQL, Perl, ...)
- Interne Funktionen
- C Funktionen
- PL/Java erlaubt auch die Nutzung von Java
(<http://pgfoundry.org/projects/pljava/>)

<https://www.postgresql.org/docs/current/static/xfunc.html>

Query Language (SQL) Funktionen

Mit Hilfe des Schlüsselworts **LANGUAGE** wird angegeben welche Sprache zur Definition dieser Funktion benutzt wurde, hier SQL.

- Diese Funktion hat den Rückgabewert `void`
- Sie ist definiert als einfaches SQL delete Statement und besitzt auch keine Eingabeparameter.

```
CREATE FUNCTION clean_emp() RETURNS  
void AS $$  
    DELETE FROM emp  
    WHERE salary < 0;  
$$ LANGUAGE SQL;
```


Query Language Funktionen (2)

- Diese Funktion hat als Parameter ein Tupel der Relation **emp**, die neben Name des Mitarbeiters, dessen Gehalt (Salary), Alter und Raum (Als Point-Objekt) enthält.
- Der Rückgabewert ist Typ `numeric`

```
INSERT INTO emp VALUES ( 'Bill', 4200, 45, '(2,1)' );
```

```
CREATE FUNCTION double_salary(emp)  
  RETURNS numeric AS $$  
    SELECT $1.salary * 2 AS salary;  
$$ LANGUAGE SQL;
```

Query Language Funktionen (3)

```
CREATE FUNCTION addtoroom (professoren)  
RETURNS int AS $$  
select $1.raum+1 ;  
$$ LANGUAGE SQL
```

Anwendung/Aufruf:

```
select name, addtoroom(professoren.*) from professoren;
```

Query Language Funktionen (4)

Hier wird eine Funktion definiert, die ein Dummy-Tupel für einen neuen Professor erzeugt (gemäß der Relation `professoren`):

```
CREATE FUNCTION neuerProf()  
RETURNS professors  
AS $$  
    SELECT 1 as persnr, text 'Unbekannt' AS name,  
           text 'C3' as rang, 123 as raum;  
$$ LANGUAGE SQL;
```

Anwendung zum Beispiel:

```
insert into professors (select * from neuerProf());
```

Query Language Funktionen (5)

Diese Funktion hat mehrere Eingaben und mehrere Ausgaben:

```
CREATE FUNCTION sum_n_product
    (x int , y int , OUT sum int , OUT product int )
AS $$ SELECT $1 + $2 , $1 * $2
$$ LANGUAGE SQL;
```

Beispielaufruf:

```
SELECT * FROM sum_n_product(11,42);
sum | product
-----+-----
 53 |    462
(1 row)
```

Query Language Funktionen (6)

Rückgabewerte: Einzelne Zeilen vs. Tabellen

```
CREATE FUNCTION alleProfs()  
RETURNS professoren  
as $$  
select * from professoren;  
$$ LANGUAGE SQL;
```

Beispielaufruf:

```
select * from alleProfs();
```

persnr	name	rang	raum
2125	Sokrates	C4	226

(1 row)

Was macht `select alleProfs();` ?

Tabellen als Rückgabewerte: Table Functions

```
CREATE FUNCTION getProfs(int)
RETURNS TABLE(persnr int) AS $$
  SELECT persnr from professoren p
  WHERE p.persnr < $1 ;
$$ LANGUAGE SQL;
```

```
select * from getProfs(2130);
```

```
persnr
-----
  2125
  2126
  2127
(3 rows)
```

PL/pgSQL

Anstelle von SQL in den vorherigen Beispielen wird nun PL/pgSQL betrachtet.

```
[ <<label>> ]  
[ DECLARE  
    declarations ]  
BEGIN  
    statements  
END [ label ];
```

PL/pgSQL

```
CREATE FUNCTION sales_tax(subtotal real)
RETURNS real AS $$
BEGIN
    RETURN subtotal * 0.06;
END;
$$ LANGUAGE plpgsql;
```


PL/pgSQL

```
CREATE FUNCTION
```

```
    concat_selected_fields(in_t sometablename)
```

```
RETURNS text AS $$
```

```
BEGIN
```

```
    RETURN in_t.f1 || in_t.f3 || in_t.f5 || in_t.f7;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

PL/pgSQL

Funktion mit zwei Eingabeparametern und zwei Ausgabeparametern:

```
CREATE FUNCTION
```

```
    sum_n_product(x int , y int ,  
                  OUT sum int , OUT prod int )
```

```
AS $$
```

```
BEGIN
```

```
    sum := x + y;
```

```
    prod := x * y;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

PL/pgSQL: SELECT INTO

SQL Anfragen, die nur eine Zeile zurückliefern, können direkt in Variablen eingelesen werden, z.B.

```
SELECT * INTO myrec FROM emp WHERE empname = myname;
```

Falls mehrere Ergebnisse geliefert werden, wird die erste Zeile benutzt.
Durch die Angabe von STRICT, also

```
SELECT * INTO STRICT myrec FROM emp  
WHERE empname = myname;
```

wird darauf geachtet, dass es nur genau ein Ergebnis gibt (ansonsten wird eine Exception geworfen).

Siehe **EXECUTE** für dynamische Anfragen und **PERFORM** für Anfragen ohne Ergebnis: [https:](https://www.postgresql.org/docs/current/static/plpgsql-statements.html)

[//www.postgresql.org/docs/current/static/plpgsql-statements.html](https://www.postgresql.org/docs/current/static/plpgsql-statements.html)

PL/pgSQL: Kontrollstrukturen

PL/pgSQL bietet die übliche Auswahl und Kontrollstrukturen wie IF-Statements und Schleifen (LOOP WHILE, FOR), EXIT (=break), CONTINUE

LOOP

```
    IF count > 0 THEN
```

```
        EXIT;
```

```
    END IF;
```

```
END LOOP;
```

PL/pgSQL: Kontrollstrukturen und SQL Anfragen

Über Anfrageergebnisse iterieren

```
CREATE OR REPLACE FUNCTION testit() RETURNS int as
$$
DECLARE
    myprofs RECORD;
    myint int = 0;
BEGIN
    FOR myprofs in SELECT * FROM professoren
                    WHERE persnr < 2130
    LOOP
        myint = myprofs.persnr + myint;
    END LOOP;
    return myint;
END;
$$ LANGUAGE plpgsql;
```

PL/pgSQL: IF Statement

```
IF number = 0 THEN
    result := 'zero';
ELSIF number > 0 THEN
    result := 'positive';
ELSIF number < 0 THEN
    result := 'negative';
ELSE
    — dann muss die Zahl wohl NULL sein ...
    result := 'NULL';
END IF;
```

Siehe auch **CASE** statements.

PL/pgSQL: Weitere Befehle - RAISE NOTICE

Zum Ausgeben von Meldungen/Warnungen oder einfach zur zum Debuggen Ihrer PL/pgSQL Codes:

```
RAISE NOTICE 'Parameter x = % und y = %', x, y
```

PL/pgSQL: Exception Handling

Syntax

```
....  
EXCEPTION  
    WHEN condition [ OR condition ... ] THEN  
    ...
```

Beispiel

```
BEGIN  
    x := x + 1;  
    y := x / 0;  
EXCEPTION  
    WHEN division_by_zero THEN  
        RAISE NOTICE 'caught division_by_zero';  
        RETURN x;  
    — bzw. äquivalent: WHEN SQLSTATE '22012' THEN  
END;
```


Zusammenfassung UDFs bzw. PL/pgSQL

- Die Implementierung von Teilen der Anwendungslogik direkt im Datenbanksystem kann einige Vorteile haben. Z.B. Wiederverwendbarkeit von Code und dass Daten nicht bewegt werden müssen.
- Realisiert im DBMS durch Stored Procedures bzw. User Defined Functions (UDFs)
- Basierend auf prozeduraler Sprache (PSM=Persistent Stored Modules)
- Haben uns PL/pgSQL als Beispiel genauer angeschaut

Ausblick

- Integritätskontrolle im DBMS durch Trigger: Dort werden in PL/pgSQL Prozeduren geschrieben, die beim Eintreffen eines Ereignisses ausgeführt werden.

Integrität / Trigger

Integritätskontrolle: Motivation

Idee/Beobachtung

- Abbildung der “Miniwelt” in relationales Modell dreht sich nicht nur um Speichern von Daten in Tabellen
- Wichtiger Aspekt ist die Kontrolle/Überwachung der Daten in den Relationen (und auch relationenübergreifend) zur Identifikation illegaler Zustände.

Realisierung

- Integritätsbedingungen (Constraints) spezifizieren akzeptable DB-Zustände
- Änderungen an der Datenbank werden zurückgewiesen, wenn sie entsprechend der Integritätsbedingungen als falsch bzw. ungültig erkannt werden.

Integritätskontrolle durch die Datenbank

DBS-basierte Integritätskontrolle

- größere Sicherheit
- vereinfachte Anwendungserstellung
- Unterstützung von interaktiven sowie programmierten DB-Änderungen
- leichtere Änderbarkeit von Integritätsbedingungen
- ggf. Leistungsvorteile

Arten von Integritätsbedingungen

Integritätsbedingungen abhängig vom Relationenmodell

- Primärschlüsseigenschaft
- Referentielle Integrität für Fremdschlüssel
- Definitionsbereiche (Domains) für Attribute

Reichweite der Bedingung

- Attributwert-Bedingungen (z.B. Geburtsjahr > 1900)
- Satzbedingungen (z.B. Geburtsdatum $<$ Einstellungsdatum)
- Satztyp-Bedingungen (z.B. Eindeutigkeit von Attributwerten)
- Satztypübergreifende Bedingungen (z.B. referentielle Integrität zwischen verschiedenen Tabellen)

Klar, je geringer die Reichweite, desto einfacher lassen sich Bedingungen überprüfen.

Arten von Integritätsbedingungen (2)

Statische vs. dynamische Bedingungen

- Statische Bedingungen (Zustandsbedingungen): beschränken zulässige DB-Zustände (z.B. Gehalt < 500000)
- Dynamische Integritätsbedingungen (Übergangsbedingungen): zulässige Zustandsübergänge (z.B. Gehalt darf nicht kleiner werden)
- Variante dynamischer Integritätsbedingungen: temporale IBs für längerfristig Bedingungen

Zeitpunkt der Überprüfbarkeit: unverzüglich vs. verzögert

- Verzögerte Bedingungen lassen sich nur durch eine Folge von Änderungen erfüllen (typisch: mehrere Sätze, mehrere Tabellen) und
- Benötigen Transaktionsschutz (als zusammengehörige Änderungssequenzen)

Integritätsbedingungen

- =zusätzliches Sicherheitssystem
- Ziel: Dateninkonsistenzen vermeiden
- Strategie: versuche zu verhindern, dass inkonsistente Daten in die DB eingefügt werden
- Bedingungen an die möglichen Ausprägungen der Datenbank

Was wir bereits in der VL behandelt haben:

- keine zwei Tupel dürfen denselben Wert bezüglich ihres Schlüssels haben
- Kardinalitäten/Stelligkeiten von Beziehungen
- Generalisierung: jede Entität aus dem Untertyp muss auch im Obertyp enthalten sein
- Domänen (d.h. zulässige Werte) von Attributen
- Bedingungen an Zustandsübergänge der Datenbank

Einfache statische Integritätsbedingungen

nicht NULL

... persnr integer **NOT NULL** ...

Einschränkungen des Wertebereichs

... check Semester **between 1 and 20**

Aufzählungstypen

... check Rang **in** ('C2','C3','C4') ...

Referentielle Integrität

Fremdschlüssel

- verweisen auf Tupel einer Relation
- z.B. gelesenVon in **Vorlesungen** verweist auf Tupel in **Professoren**

Referentielle Integrität bedeutet:

- Fremdschlüssel müssen auf existierende Tupel verweisen **oder** einen Nullwert enthalten.

Gegenbeispiel:

insert into Vorlesungen

values (5100, 'Agentenwesen', 4, 007);

Wie kann dieses “insert” verhindert werden?

Festlegung von Schlüsseln

unique:

- für alle Kandidatenschlüssel
- minimale Teilmenge von Attributen, die Tupel eindeutig definiert
- mehrere Kandidatenschlüssel für eine Relation sind möglich
- darf NULL sein

primary key:

- Primärschlüssel
- ein Schlüssel aus der Menge der Kandidatenschlüssel
- nur ein Primärschlüssel pro Relation
- impliziert not NULL

foreign key:

- Fremdschlüssel
- impliziert, dass NULL möglich ist, kann aber als not NULL definiert werden **unique foreign key**: modelliert 1:1-Beziehung

Referentielle Integrität in SQL

- Kandidatenschlüssel: **unique**
- Primärschlüssel: **primary key**
- Fremdschlüssel: **[foreign key] references**

Beispiel:

```
create table R  
  ( $\alpha$  integer primary key,  
   ... );
```

```
create table S  
  (...,  
    $\kappa$  integer references R( $\alpha$ ),  
   ... );
```

Zusammengesetzte Schlüssel

create table R

(α ,
 β ,
...,
primary key(α, β),
...);

create table S

(...,
 κ_1 ,
 κ_2 ,
...,
foreign key(κ_1, κ_2) **references** R(α, β)
...);

Kandidatenschlüssel

```
create table R  
  ( $\alpha$ ,  
    $\beta$ ,  
    $\gamma$ ,  
   ...,  
   unique( $\beta, \gamma$ )  
  ... );
```

Beispiel: Schlüssel in Postgres

```
CREATE TABLE assistenten
```

```
(
```

```
    persnr integer NOT NULL,
```

```
    "name" character varying(30) NOT NULL,
```

```
    fachgebiet character varying(30),
```

```
    boss integer,
```

```
    CONSTRAINT assistenten_pkey PRIMARY KEY (persnr),
```

```
    CONSTRAINT assistenten_boss_fkey FOREIGN KEY (boss)
```

```
        REFERENCES professoren (persnr) MATCH SIMPLE
```

```
        ON UPDATE NO ACTION ON DELETE NO ACTION
```

```
)
```

MATCH SIMPLE erlaubt NULL-Werte in einem Teil des Schlüssels

MATCH FULL verhindert dies

Verhalten bei Änderungsoperationen

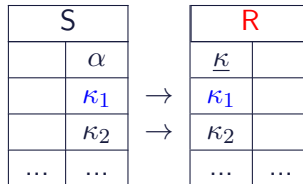
Bei Änderung von referenzierten Daten haben wir mindestens 3 Optionen:

1. **Zurückweisen** der Änderungsoperation (=Default)
2. **Propagieren** der Änderungen: **cascade**
3. Verweise auf **<unbekannt>** setzen: **set null**

Desweiteren in Postgres:

4. auf Defaultwert setzen: **set default**

Beispiel: Änderungsoperation auf **R**



update **R**

set $\kappa = \kappa'_1$
where $\kappa = \kappa_1$

delete from **R**

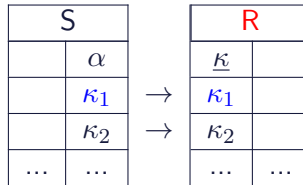
where $\kappa = \kappa_1$

Wie soll damit umgegangen werden?

Option 1: Zurückweisen der Änderung

create table S

(...,
 α **integer references** $R(\kappa)$);

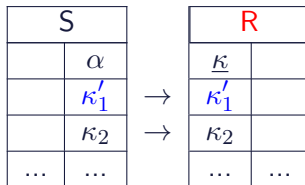


Ergebnis der Änderungsoperation: DB bleibt unverändert

Option 2: Kaskadieren der Änderung

create table S

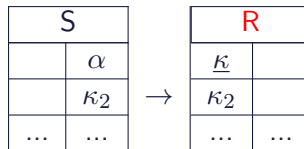
(...,
 α integer references $R(\kappa)$
 on update **cascade**);



Ergebnis der Änderungsoperationen:
 Schlüssel in R **und** S geändert.

create table S

(...,
 α integer references $R(\kappa)$
 on delete **cascade**);

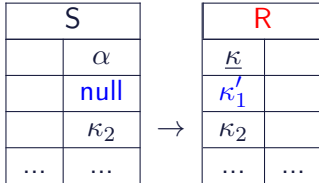


Ergebnis der Änderungsoperationen:
 Schlüssel in R **und** S gelöscht.

Option 3: Ändern und auf NULL setzen

create table S

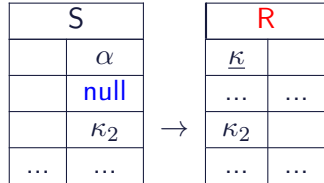
(...,
 α **integer references** $R(\kappa)$
on update set null);



Ergebnis der Änderungsoperationen:
 Tupel aus R auf κ'_1 , in S auf null
 geändert.

create table S

(...,
 α **integer references** $R(\kappa)$
on delete set null);



Ergebnis der Änderungsoperationen:
 Tupel aus R gelöscht, Schlüssel in S
 auf null geändert.

Kaskadierendes Löschen

```
delete from Professoren where Name = 'Sokrates';
```

```
create table Vorlesungen
```

```
(...,
```

```
    gelesenVon integer
```

```
        references Professoren(PersNr)
```

```
        on delete cascade);
```

```
create table hoeren
```

```
(...,
```

```
    VorlNr integer
```

```
        references Vorlesungen(VorlNr)
```

```
        on delete cascade);
```

Das Univeritätsschema mit Integritätsbedingungen

```
create table Studenten
```

```
(MatrNr  integer primary key,  
Name    varchar(30) not null,  
Semester integer check Semester between 1 and 13);
```

```
create table Professoren
```

```
( PersNr  integer primary key,  
Name     varchar(30) not null,  
Rang     character(2) check (Rang in ('C2', 'C3', 'C4')),  
Raum     integer unique );
```

create table Assistenten

```
( PersNr  integer primary key,  
  Name    varchar(30) not null,  
  Fachgebietvarchar(30),  
  Boss    integer,  
  foreign  key (Boss) references Professoren(PersNr)  
          on delete set null);
```

create table Vorlesungen

```
( VorlNr  integer primary key,  
  Titel   varchar(30),  
  SWS     integer,  
  gelesenVoninteger references Professoren(PersNr)  
          on delete set null);
```

create table hoeren

```
( MatrNr      integer references Studenten(MatNr)
                        on delete cascade,
  VorlNr      integer references Vorlesungen(VorlNr)
                        on delete cascade,
  primary key(MatNr, VorlNr));
```

create table voraussetzen

```
(Vorgaenger integer references Vorlesungen(VorlNr)
                        on delete cascade,
  Nachfolger integer references Vorlesungen(VorlNr)
                        on delete cascade,
  primary key (Vorgaenger, Nachfolger));
```

create table pruefen

(MatrNr integer references Studenten(MatNr)

on delete cascade,

VorlNr integer references Vorlesungen(VorlNr),

PersNr integer references Professoren(PersNr)

on delete set null,

Note numeric (2,1) check (Note between 0.7 and 5.0),

primary key (MatrNr, VorlNr));

Anmerkungen

Es gibt noch weitere interessante Möglichkeiten bei der Erstellung von Tabellen. z.B.

- Automatische Zuweisung von IDs: `create table studenten (matrnr serial, .....)`. Achtung: Syntax `serial` vs. `autoincrement` vs. hängt vom Datenbanksystem ab.
- **Default** Werte: `create table studenten (matrnr int, vertiefung varchar(20) default 'Informationssysteme', ...)`
- Verbindung von default Werten und UDFs: `vertiefung varchar(20) default getRandVertiefung(), .....`

Beispiel:

```
create table mytable(id serial primary key, vertiefung float default random(), name char(10))
```

Dann einfach bei insert: `insert into mytable(name) values ('Meyer')`

Komplexere Konsistenzbedingungen?

```
CREATE TABLE pruefen
(MatrNr integer REFERENCES Studenten on DELETE CASCADE,
VorlNr integer REFERENCES Vorlesungen,
Note numeric(2,1) CHECK (Note BETWEEN 0.7 and 5.0),
PRIMARY KEY (MatrNr, VorlNr),
CONSTRAINT VorherHoeren
CHECK ( EXISTS ( SELECT *
FROM hoeren h
WHERE h.VorlNr = pruefen.VorlNr and
h.MatrNr = pruefen.MatrNr))
);
```

- Bei jeder Änderung und Einfügung wird die check-Klausel ausgewertet.
- Operation wird nur durchgeführt, wenn der check true ergibt.
- Geht leider nicht in aktuellem Postgres 10: **ERROR: cannot use subquery in check constraint**
- nur einfache boolsche Bedingungen im check in Postgres z.B. $a < 42$

In SQL-92

Primary-Key- und Foreign-Key-Klauseln

- Referentielle Integrität: deklarative Spezifikation unterschiedlicher Reaktionsmöglichkeiten für Wegfall (Löschung, Änderung) eines referenzierten Satzes bzw. Primärschlüssels (CASCADE, ...)

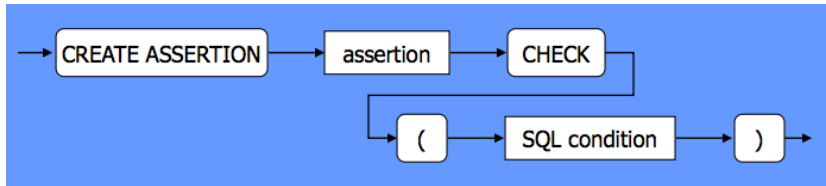
Festlegung von Wertebereichen für Attribute durch Angabe eines Datentyps bzw. Domains

- Optional: Angabe von Default-Werten, Eindeutigkeit (UNIQUE), Nullwerte-Ausschluss (NOT NULL)
- Allgemeine Wertebereichsbeschränkungen über CHECK-Klausel

In SQL-92: Assertions

Assertions: Spezifikation allgemeiner, z.B. tabellenübergreifender Bedingungen

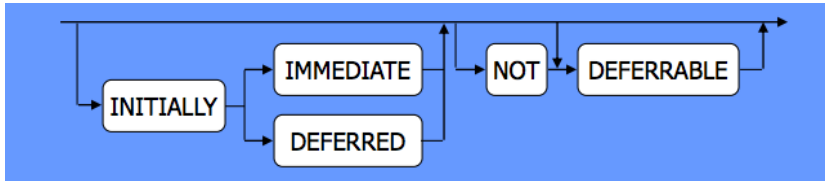
- Beziehen sich typischerweise auf mehrere Relationen
- Lassen sich als eigenständige DB-Objekte definieren
- Erlauben die Verschiebung ihres Überprüfungszeitpunktes
- Syntax der Assertion-Anweisung



In SQL-92: Assertions - IMMEDIATE vs. DEFERRED

Direkte (IMMEDIATE) oder verzögerte (DEFERRED) Überwachung

- **IMMEDIATE**: am Ende der Änderungsoperation (Default)
- **DEFERRED**: am Transaktionsende (COMMIT)



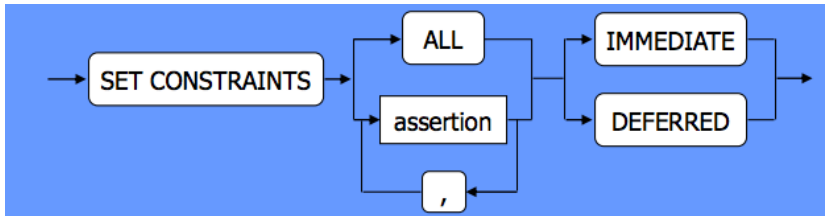
In SQL-92: Assertions - Beispiel

```
CREATE TABLE Pers
(Pnr    int PRIMARY KEY,
Gehalt int CHECK(Value < 500000),
Anr    int NOT NULL
    FOREIGN KEY REFERENCES Abt
    ON DELETE CASCADE
    ...);
```

```
CREATE ASSERTION A1
CHECK (NOT EXISTS
    (SELECT * FROM Abt A
    WHERE A.Anzahl_Angest <
    (SELECT COUNT(*)
    FROM Pers P
    WHERE P.Anr = A.Anr)))
INITIALLY DEFERRED;
```

In SQL-92: IMMEDIATE oder DEFERRED in Transaktion

- Überprüfung kann innerhalb einer Transaktion gesteuert werden:



- Entweder für alle Constraints/Assertions oder, für benannte Constraints/Assertions selektiv.
- Unter Berücksichtigung von NOT DEFERRABLE
- Wird Postgresql teilweise(!) unterstützt, aber nur für einfache check constraints.

CHECK OPTION CONSTRAINT in Views

Gegeben folgende Sicht auf die Tabelle professoren

```
CREATE VIEW profsView AS  
SELECT *  
FROM professoren  
WHERE persnr < 2126;
```

Kann das folgende INSERT-Statement ausgeführt werden?

```
INSERT INTO profsView VALUES(2777, 'Michel', 'C4', 444);
```


CHECK OPTION CONSTRAINT in Views (2)

Antwort auf vorherige Frage ist: Ja, es kann ausgeführt werden.

- Das Tupel wird also in die Tabelle professoren eingefügt
- Aber es ist nicht in der View enthalten, da nicht gilt $2777 < 2126$

Um zu vermeiden, dass Tupel eingefügt werden, die nicht in der View enthalten sind, kann die Definition der View wie folgt angepasst werden:

```
CREATE VIEW profsView AS
SELECT *
FROM professoren
WHERE persnr < 2126
WITH CHECK OPTION;
```

CHECK OPTION CONSTRAINT in Views (3)

Alternativ mit **CASCADDED CHECK OPTION**, dann werden auch Bedingungen der evtl. Zugrunde liegenden Views überprüft.

CHECK OPTION bzw. CASCADDED CHECK OPTION ist von Postgres seit Version 9.4 unterstützt.

Beispiel CASCADED CHECK OPTION CONSTRAINT

```
CREATE VIEW V1 AS SELECT COL1  
FROM T1 WHERE COL1 > 10
```

Es gibt in V1 kein constraint, also funktioniert das folgende Statement:

```
INSERT INTO V1 VALUES (5)
```

View V2 benutzt V1 und hat eine CASCADED CHECK OPTION:

```
CREATE VIEW V2 AS SELECT COL1  
FROM V1 WITH CASCADED CHECK OPTION
```

Also klappt das folgende Insert nicht, da ein CHECK auf V1 forciert wird:

```
INSERT INTO V2 VALUES (5)
```

(Fortsetzung auf nächster Folie)

Fortsetzung des Beispiels

Eine weitere View V3, die von V2 (und somit auch von V1) abhängt:

```
CREATE VIEW V3 AS SELECT COL1  
FROM V2 WHERE COL1 < 100
```

Das folgende Statement funktioniert nicht, da der Wert 5 ein Problem mit V2 darstellt, da V2 ja die CASCADED CHECK OPTION hat (und daher den Check auf V1 forciert).

```
INSERT INTO V3 VALUES (5)
```

Das folgende Statement funktioniert allerdings:

```
INSERT INTO V3 VALUES (200)
```

Wieso? Es macht keine Probleme bzgl. View V2 (bzgl. V1). Es würde zwar nicht in der View V3 auftauchen, aber das ist kein Problem, weil V3 kein CHECK besitzt.

Database Trigger: Idee

- Assertions/Constraints legen fest was erlaubt ist.
- **Trigger ermöglichen auf bestimmte Ereignisse zu reagieren!**
- Z.B wenn neue Zeilen in eine Tabelle bzw. Sicht eingefügt wurden bzw. werden sollen
- Tritt ein bestimmtes Ereignis auf so wird ein Trigger “gefeuert”, d.h. eine vorher festgelegte Aktion (Prozedur bzw. Funktion) ausgeführt.
- Somit können z.B. komplexe Constraints überprüft werden oder Tabellen automatisch angepasst werden

Allgemeines Prinzip: ECA – Event, Condition, Action

Beispiel für Einsatz von Triggern

- Automatisches Nachbestellen, wenn Lagerbestände leerlaufen
- Automatisches Anschreiben von Studenten, wenn creditPoints < vordefinierte Schranke
- Automatische Mahnung für Rechnungen/ablaufende Abos/etc.
- Bibliothek: Anschreiben von Nutzern wenn Ausleihfrist überschritten
- Überwachung von Kontobeständen
 - Obergrenzen für Überweisungen
 - Untergrenzen für Kontostände
- etc.

Ein Teil der Anwendungslogik ist dann in den Triggern definiert.

Beispiel Trigger

- Idee: Jeder neue Student wird automatisch durch einen Eintrag in der Tabelle hören für die Vorlesung Logik registriert.
- Also: **AFTER INSERT ON** studenten
- Und für jede neue Zeile einzeln, also **FOR EACH ROW**

```
CREATE TRIGGER pflichtvorlesungLogikTrigger
AFTER INSERT ON  studenten
FOR EACH ROW
EXECUTE PROCEDURE
    pflichtvorlesungLogik_function();
```

Dies ist “nur” der Trigger. Die eigentliche Funktion/Prozedur die beim Eintreffen des Trigger-Ereignisses aufgerufen wird heisst **pflichtvorlesungLogik_function()**.

Beispiel Trigger

```
1 CREATE OR REPLACE
2     FUNCTION pflichtvorlesungLogik_function()
3 RETURNS TRIGGER AS
4 $$
5 BEGIN
6     — kurze Meldung ausgeben
7     RAISE NOTICE 'Ach, sieh mal an, Student/in %', NEW.name;
8     — fuege Student/in in Tabelle hoeren ein
9     INSERT INTO hoeren VALUES (New.matrn timer, 4052);
10    RETURN NULL;
11 END
12 $$
13 LANGUAGE plpgsql;
```


Database Trigger: Überlegungen

Wann soll ein Trigger ausgelöst werden?

- Zeitpunkt: BEFORE / AFTER / INSTEAD OF
- Auslösende Operation: INSERT / DELETE / UPDATE

Wie spezifiziert man Aktionen?

- Bezug auf verschiedene DB-Zustände erforderlich
- OLD/NEW erlaubt Referenz von alten/neuen Werten

Ist die Trigger-Ausführung vom Zustand der DB abhängig?

- WHEN Bedingung (optional)

Database Trigger: Überlegungen (2)

Wann soll wie verändert werden?

- Pro Tupel (Row) oder pro DB-Operation (Statement):
Trigger-Granulat?
- mit einer SQL-Anweisung oder mit einer Prozedur aus PL/pgSQL etc?

Existiert das Problem der Terminierung und der Auswertungsreihenfolge?

- Mehrere Trigger-Definitionen pro Tabelle sowie
- mehrere Trigger-Auslösungen pro Ereignis möglich

Ausführungsreihenfolge

- Ein SQL-Änderungs-Statement XYZ wurde abgesetzt
- Frage: **Wann wird welche Art von Trigger ausgeführt?**

FOR EACH STATEMENT:

- Trigger wird ausgeführt für jedes matchende Event
- **einfache** Ausführung des Triggers
- unabhängig von der Anzahl der geänderten Zeilen
- BEFORE Statement: vor allen row-level Triggern
- AFTER Statement: nach allen row-level Triggern

D.h. Reihenfolge ist:

BEFORE Statement → BEFORE Row → AFTER Row → AFTER Statement

Sichtbarkeit von Änderungen

- Ein SQL-Änderungs-Statement XYZ wurde abgesetzt
- Frage: **Welcher Trigger sieht welche Änderung?**

FOR EACH STATEMENT:

- BEFORE: sieht nichts von XYZ
- AFTER: sieht alles

Sichtbarkeit von Änderungen

- Ein SQL-Änderungs-Statement XYZ wurde abgesetzt
- Frage: **Welcher Trigger sieht welche Änderung?**

FOR EACH ROW:

- BEFORE: sieht nichts von XYZ
- AFTER: sieht alles
- Aber: BEFORE sieht u.U. Änderungen anderer BEFORE Trigger
- Reihenfolge der ROW BEFORE Trigger ist aber nicht festgelegt

Beispiel Trigger

- Idee: Jeder neue Student wird automatisch durch einen Eintrag in der Tabelle hören für die Vorlesung Logik registriert.
- Also: **AFTER INSERT ON** studenten
- Und für jede neue Zeile einzeln, also **FOR EACH ROW**

```
CREATE TRIGGER pflichtvorlesungLogikTrigger  
AFTER INSERT ON  studenten  
FOR EACH ROW  
EXECUTE PROCEDURE  
    pflichtvorlesungLogik_function();
```

Dies ist “nur” der Trigger. Die eigentliche Funktion/Prozedur die beim Eintreffen des Trigger-Ereignisses aufgerufen wird heisst **pflichtvorlesungLogik_function()**.

Beispiel Trigger

```
1 CREATE OR REPLACE
2     FUNCTION pflichtvorlesungLogik_function()
3 RETURNS TRIGGER AS
4 $$
5 BEGIN
6     — kurze Meldung ausgeben
7     RAISE NOTICE 'Ach, sieh mal an, Student/in %', NEW.name;
8     — fuege Student/in in Tabelle hoeren ein
9     INSERT INTO hoeren VALUES (NEW.matrnr, 4052);
10    RETURN NULL;
11 END
12 $$
13 LANGUAGE plpgsql;
```

Trigger: Syntax

```
CREATE [ CONSTRAINT ] TRIGGER name { BEFORE | AFTER | INSTEAD OF }  
    { event [ OR ... ] }  
ON table  
[ FROM referenced_table_name ]  
[ NOT DEFERRABLE | [ DEFERRABLE ] { INITIALLY IMMEDIATE |  
INITIALLY DEFERRED } ]  
[ FOR [ EACH ] { ROW | STATEMENT } ]  
[ WHEN ( condition ) ]  
EXECUTE PROCEDURE function_name ( arguments )
```

where event can be one of:

```
INSERT  
UPDATE [ OF column_name [, ... ] ]  
DELETE  
TRUNCATE
```


Trigger: Parameterübergabe an Funktion

- Es macht natürlich wenig Sinn für jede Pflichtvorlesung eine eigene Funktion zu schreiben.
- Besser ist es schonmal die gleiche Funktion zu benutzen und die Vorlesung um die es geht als Parameter zu übergeben, z.B. wie hier für die Vorlesung mit Vorlnr 5001:

```
CREATE TRIGGER pflichtvorlesungTrigger5001  
AFTER INSERT ON Studenten  
FOR EACH ROW  
EXECUTE PROCEDURE pflichtvorlesung_function(5001);
```

In Postgresql: Trigger-Funktionen haben KEINE Parameter in ihrer Definition. Aber, es können Parameter übergeben werden an die Trigger-Funktion. Dies geschieht über die Variable TG_ARGV

Beispiel Trigger: Parameterübergabe via TG_ARGV

```
1 CREATE OR REPLACE FUNCTION pflichtvorlesung_function()  
2 RETURNS TRIGGER AS  
3 $$  
4 DECLARE  
5     vorlnr_param int;  
6     titel_param varchar;  
7 BEGIN  
8     — PflichtVL wird per TG_ARGV uebergeben  
9     vorlnr_param := TG_ARGV[0];  
10    — Fuer NOTICE ist es huebscher Titel der VL zu haben  
11    SELECT titel into titel_param FROM vorlesungen where  
12           vorlnr=vorlnr_param;  
13    RAISE NOTICE 'Ach, Student/in %', NEW.name ;  
14    RAISE NOTICE 'sollte auf jeden Fall die VL % (%) hoeren  
15           ', titel_param , vorlnr_param;  
16    INSERT INTO hoeren VALUES (New.matnr, vorlnr_param);  
17    RETURN NULL;  
18 END $$ LANGUAGE plpgsql;
```

Beispiel Trigger: Aufruf

```
INSERT INTO studenten VALUES (29000, 'Michel', '1');
```

NOTICE: Ach, Student/in Michel

NOTICE: sollte auf jeden Fall die VL Grundzuege (5001) hoeren

Hinweis zum “Herumspielen/Debuggen”

Damit die Tabelle, in die eingefügt wird, nicht zugemüllt wird bietet es sich an hier eine Transaktion mit rollback() zu benutzen:

```
BEGIN TRANSACTION;
```

```
INSERT INTO studenten VALUES (29000, 'Michel', '1');
```

```
ROLLBACK;
```

Weiteres Beispiel

Tabelle mit Informationen über Angestellte:

```
CREATE TABLE emp (  
    empname text ,  
    salary integer ,  
    last_date timestamp ,  
    last_user text  
);
```

Der Eintrag `last_date` soll den Zeitpunkt der letzten Änderung an der jeweiligen Zeile angeben. `last_user` soll entsprechend den Namen des Benutzers enthalten, der diese Zeile eingefügt bzw. aktualisiert hat.

```
CREATE TRIGGER emp_stamp BEFORE INSERT OR UPDATE ON emp  
    FOR EACH ROW EXECUTE PROCEDURE emp_stamp();
```

Aus: <https://www.postgresql.org/docs/current/static/plpgsql-trigger.html>

Weiteres Beispiel (2)

<https://www.postgresql.org/docs/current/static/plpgsql-trigger.html>

```
1 CREATE FUNCTION emp_stamp() RETURNS trigger AS $$
2 BEGIN
3   — Name und Gehalt sollten nicht NULL sein
4   IF NEW.empname IS NULL THEN
5     RAISE EXCEPTION 'empname cannot be null';
6   END IF;
7   IF NEW.salary IS NULL THEN
8     RAISE EXCEPTION '% cannot have null salary', NEW.
      empname;
9   END IF;
10
11  — Vermutlich wird niemand fuer ein negatives Gehalt
      Arbeiten wollen
12  IF NEW.salary < 0 THEN
13    RAISE EXCEPTION '% cannot have a negative salary',
      NEW.empname;
14  END IF;
```

Weiteres Beispiel (2)

```
15  — Merken wer diesen Eintrag geändert hat und wann
16  NEW.last_date := current_timestamp;
17  NEW.last_user := current_user;
18  RETURN NEW;
19  END;
20  $$ LANGUAGE plpgsql;
21
```

Rückgabewerte der Trigger-Funktion: In Postgresql

In Postgresql gibt eine Triggerfunktion entweder NULL zurück oder ein Tupel, das dem Schema der Relation auf die der Trigger definiert ist entspricht.

- Return value von AFTER ROW Triggern sowie für BEFORE oder AFTER STATEMENT Triggern werden immer ignoriert; der Wert kann also auch NULL sein.
- Für BEFORE ROW Trigger wird der Rückgabewert NULL so gedeutet, dass keine nachfolgenden Trigger (für diese Zeile) mehr ausgeführt werden. Wird ein nicht-NULL Wert zurückgegeben, so arbeiten Trigger, die danach aufgerufen werden mit diesem Wert.
- Vorsicht bei DELETE Triggern bei return NULL (besser RETURN OLD; wird sowieso ignoriert)

Ausführliche Beschreibung unter: <https://www.postgresql.org/docs/current/static/plpgsql-trigger.html>

Row-Trigger: Referenzen auf alte bzw. neue Versionen der Zeilen

- Für Trigger, die pro Zeile (Row) ausgeführt werden, sogenannte Row-Trigger kann direkt auf die betroffene Zeile zugegriffen werden.
- Bei INSERT natürlich nur auf die neue Zeile. Schlüsselwort NEW.
- Bei UPDATE auf NEW sowie OLD.
- Bei DELETE nur auf OLD

Referenzen Definieren

- Die jeweiligen alten bzw. neuen Versionen können direkt referenziert werden oder
- es können via REFERENCES Alias eingeführt werden (Im SQL Standard, nicht in PG)

Für Row-Trigger sind die o.g. Referenzen gültig unabhängig davon ob der Trigger BEFORE oder AFTER der Operation ausgeführt wird.

Trigger in Postgresql

- Nicht komplett kompatibel zum SQL Standard
- Z.B. kann man in PG keinen Alias OLD bzw. NEW einführen
- und generell für STATEMENT Trigger nicht auf OLD_TABLE bzw. NEW_TABLE zugreifen
- SQL Standard sagt Trigger sollten in Reihenfolge der Erzeugung ausgeführt werden, Postgresql aber sortiert nach Namen
- CREATE CONSTRAINT TRIGGER ist eine Erweiterung in Postgres
 - In Zusammenhang mit **create table** verwendbar
 - Schließt den Kreis zu komplexeren check constraints

<https://www.postgresql.org/docs/current/static/sql-createtrigger.html>

Gültige Kombinationen

Granulat	Aktivierungszeit	Triggernde Operation	Übergangsvariablen erlaubt	Übergangstabellen erlaubt
ROW	BEFORE	INSERT	NEW	NONE
ROW	BEFORE	UPDATE	OLD, NEW	NONE
ROW	BEFORE	DELETE	OLD	NONE
ROW	AFTER	INSERT	NEW	NEW_TABLE
ROW	AFTER	UPDATE	OLD, NEW	OLD_TABLE, NEW_TABLE
ROW	AFTER	DELETE	OLD	OLD_TABLE
STATEMENT	BEFORE	INSERT	NONE	NONE
STATEMENT	BEFORE	UPDATE	NONE	NONE
STATEMENT	BEFORE	DELETE	NONE	NONE
STATEMENT	AFTER	INSERT	NONE	NEW_TABLE
STATEMENT	AFTER	UPDATE	NONE	OLD_TABLE, NEW_TABLE
STATEMENT	AFTER	DELETE	NONE	OLD_TABLE

Probleme mit Triggern

Mögliche rekursive Aufrufe von Triggern

- Änderung auf Tabelle A feuert Trigger Tr1
- Tr1 ändert Tabelle B
- Änderung auf Tabelle B feuert Trigger Tr2
- Tr2 ändert Tabelle A ...

Mögliches verwirrendes Geflecht von Triggern in der Praxis

- Was passiert bei einer Änderungsoperation wirklich?
- Terminiert die Rekursion der Trigger?
- Führen die Trigger Inkonsistenzen in die DB ein?

Deshalb: Vorsicht beim Einsatz von Triggern! Insbesondere bei Triggern, die selbst Daten einfügen.

Wird in Postgresql nicht überprüft:

“It is the trigger programmer’s responsibility to avoid infinite recursion in such scenarios.” (aus der Postgres Dokumentation über Trigger).

Übersicht

Kommende ~2 Vorlesungen

- Was ist ein guter relationaler Entwurf und wie kann ein schlechter Entwurf verbessert werden?

Danach betrachten wir, wie ein Datenbanksystem aufgebaut ist bzw. im Kern funktioniert, zuerst bzgl. Performance.

- Wie werden Daten auf Festplatte abgelegt?
- Wie werden (SQL) Anfragen ausgeführt und welche Kosten fallen dabei an?
- Wie kann effizient auf Daten zugegriffen werden?
- Wie kann die Ausführung einer Anfrage optimiert werden?

Danach kommen weitere Aspekte, wie Transaktionsverwaltung (=Fehlerbehandlung sowie Mehrbenutzersynchronisation).

Relationale Entwurfstheorie

ProfVorl						
PersNr	Name	Rang	Raum	VorlNr	Titel	SWS
2125	Sokrates	C4	226	5041	Ethik	4
2125	Sokrates	C4	226	5049	Mäeutik	2
2125	Sokrates	C4	226	4052	Logik	4
...	...	...	...	...	...	...
2132	Popper	C3	52	5259	Der Wiener Kreis	2
2137	Kant	C4	7	4630	Die 3 Kritiken	4

Wieso ist dies ein schlechtes Schema?

ProfVorl						
PersNr	Name	Rang	Raum	VorlNr	Titel	SWS
2125	Sokrates	C4	226	5041	Ethik	4
2125	Sokrates	C4	226	5049	Mäeutik	2
2125	Sokrates	C4	226	4052	Logik	4
...	...	...	...	...	...	...
2132	Popper	C3	52	5259	Der Wiener Kreis	2
2137	Kant	C4	7	4630	Die 3 Kritiken	4

Wieso ist dies ein schlechtes Schema?

- **Änderungsanomalien:** Sokrates zieht um, von Raum 226 in Raum 338. Was passiert?
- **Einfügeanomalien:** Neuer Professor ohne Vorlesungen?
- **Löschanomalien:** Letzte Vorlesung eines Profs wird gelöscht. Was passiert?

Funktionale Abhängigkeiten

Functional Dependency (FD)

- Schema $\mathcal{R} = \{A, B, C, D\}$
- Ausprägung R
- Seien $\alpha \subseteq \mathcal{R}$ und $\beta \subseteq \mathcal{R}$
- $\alpha \rightarrow \beta$ genau dann, wenn $\forall r, s \in R$ gilt: $r.\alpha = s.\alpha \Rightarrow r.\beta = s.\beta$
- D.h. **die α -Werte bestimmen die β -Werte funktional** (=eindeutig)

R			
A	B	C	D
a4	b2	c4	d3
a1	b1	c1	d1
a1	b1	c1	d2
a2	b2	c3	d2
a3	b2	c4	d3

$$\{A\} \rightarrow \{B\}$$

$$\{A\} \rightarrow \{C\}$$

$$\{C, D\} \rightarrow \{B\}$$

Aber nicht $\{B\} \rightarrow \{C\}$

Notation. auch ohne $\{$ bzw $\}$:
 $C, D \rightarrow B$ oder auch $CD \rightarrow B$

Einhaltung funktionaler Abhängigkeiten

Alternative Formulierung

- Die FD $\alpha \rightarrow \beta$ ist in R erfüllt, wenn für jede mögliche Ausprägung von α gilt:

$$|\pi_{\beta}(\sigma_{\alpha=c}(R))| \leq 1$$

- “die Menge aller Tupel von R mit $\alpha = c$ (für beliebiges c) projiziert auf β enthält keinen oder genau einen Wert”
- ansonsten wäre es auch keine “Funktion”

Daraus folgt ein einfacher Algorithmus:

```
boolean giltFD(Relation  $R$ , FD  $\alpha \rightarrow \beta$ ) {  
    sortiere  $R$  nach  $\alpha$  Werten  
    für jede Gruppe  $G_i \subseteq R$  von Tupeln mit Wert  $\alpha_i$  aus  $\alpha$ :  
        falls nicht alle  $\beta_i$  aus  $\beta$  identisch sind:  
            return falsch;  
    return wahr;  
}
```

Schlüssel

- $\alpha \subseteq \mathcal{R}$ ist ein Superschlüssel, falls gilt: $\alpha \rightarrow \mathcal{R}$
- trivial: es gilt $\mathcal{R} \rightarrow \mathcal{R}$
- Allerdings sind Superschlüssel nicht notwendigerweise minimal!

Volle funktionale Abhängigkeit:

- β ist voll funktional abhängig von α (Notation: $\overset{\bullet}{\rightarrow}$) genau dann wenn gilt:
 - $\alpha \rightarrow \beta$
 - α kann nicht mehr verkleinert (=linksreduziert) werden, d.h.
 - $\forall A \in \alpha$ folgt, dass $(\alpha - \{A\}) \rightarrow \beta$ nicht gilt, bzw. alternativ:
 - $\forall A \in \alpha : \neg((\alpha - \{A\}) \rightarrow \beta)$
- $\alpha \subseteq \mathcal{R}$ ist ein **Kandidatenschlüssel**, falls gilt: $\alpha \overset{\bullet}{\rightarrow} \mathcal{R}$

Ein Kandidatenschlüssel wird als **Primärschlüssel** ausgewählt!

Beispiel zur Schlüsselbestimmung

Städte			
Name	BLand	Vorwahl	EW
Frankfurt	Hessen	69	650000
Frankfurt	Brandenburg	335	84000
München	Bayern	89	1200000
Passau	Bayern	851	50000
Saarbrücken	Saarland	681	175000
Kaiserslautern	Rheinland-Pfalz	631	100000
...	...	...	...

Kandidatenschlüssel von Städte:

- {Name, BLand}
- {Name, Vorwahl}

Bestimmung funktionaler Abhängigkeiten

- Tabelle **Professoren**: {[PersNr, Name, Rang, Raum, Ort, Straße, PLZ, Vorwahl, BLand, EW, Landesregierung]}
- {PersNr} $\rightarrow$ {PersNr, Name, Rang, Raum, Ort, Straße, PLZ, Vorwahl, BLand, EW, Landesregierung}
- {Ort,BLand} $\rightarrow$ {EW, Vorwahl}
- {PLZ} $\rightarrow$ {BLand, Ort, EW}
- {BLand, Ort, Straße} $\rightarrow$ {PLZ}
- {BLand} $\rightarrow$ {Landesregierung}
- {Raum} $\rightarrow$ {PersNr}

Außerdem kann abgeleitet werden:

- {Raum} $\rightarrow$ {PersNr, Name, Rang, Raum, Ort, Straße, PLZ, Vorwahl, BLand, EW, Landesregierung}
- {PLZ} $\rightarrow$ {Landesregierung}

Herleitung weiterer FDs

- Aus einer Menge $\mathcal{F}$ von FDs sind weitere FDs herleitbar
- $\mathcal{F}^+$ wird **Hülle** (**closure**) von $\mathcal{F}$ genannt
- $\mathcal{F}^+$ besteht aus allen aus $\mathcal{F}$ herleitbaren FDs
- Inferenzregeln, die **Armstrong Axiome**, beschreiben die Herleitung

Die Armstrong-Axiome

$\alpha, \beta, \gamma, \delta$ sind Teilmengen der Attribute aus $\mathcal{R}$

- **Reflexivität:** Falls β eine Teilmenge von α ist ($\beta \subseteq \alpha$) dann gilt immer $\alpha \rightarrow \beta$.
Insbesondere gilt also immer $\alpha \rightarrow \alpha$.
- **Verstärkung:** Falls $\alpha \rightarrow \beta$ gilt, dann gilt auch $\alpha\gamma \rightarrow \beta\gamma$. Hierbei stehe z.B. $\alpha\gamma$ für $\alpha \cup \gamma$
- **Transitivität:** Falls $\alpha \rightarrow \beta$ und $\beta \rightarrow \gamma$ gilt, dann gilt auch $\alpha \rightarrow \gamma$.

Die Armstrong-Axiome sind korrekt und vollständig.

D.h. mit Hilfe dieser Axiome können alle gültigen FDs hergeleitet werden.

Die Armstrong-Axiome (2)

Nicht notwendig, aber oftmals komfortabel für Herleitungen:

- **Vereinigungsregel:**

Wenn $\alpha \rightarrow \beta$ und $\alpha \rightarrow \gamma$ gelten, dann auch $\alpha \rightarrow \beta\gamma$.

- **Dekompositionsregel:**

Wenn $\alpha \rightarrow \beta\gamma$ gilt, dann gelten auch $\alpha \rightarrow \beta$ und $\alpha \rightarrow \gamma$.

- **Pseudotransitivitätsregel:**

Wenn $\alpha \rightarrow \beta$ und $\gamma\beta \rightarrow \delta$, dann gilt auch $\alpha\gamma \rightarrow \delta$.

Attributhülle

Die Attributhülle $\text{AttrHülle}(\mathcal{F}, \alpha)$ einer Attributmenge α bzgl. FDs $\mathcal{F}$ ist die Menge von Attributen α^+ , für die gilt $\alpha \rightarrow \alpha^+$.

Gegeben:

- eine Menge $\mathcal{F}$ von FDs
- eine Menge von Attributen $\alpha \subseteq \mathcal{R}$

Algorithmus:

```
Erg :=  $\alpha$ ;  
while (Änderungen an  $Erg$ ) do  
  for each FD  $\beta \rightarrow \gamma$  in  $\mathcal{F}$  do  
    if  $\beta \subseteq Erg$  then  $Erg := Erg \cup \gamma$ ;  
 $\alpha^+ := Erg$ ;
```


Beispielanwendung

Ziel:

Bestimme, ob κ einen Superschlüssel einer Relation $\mathcal{R}$ bzgl. der FDs in $\mathcal{F}$ bildet.

Lösung:

Durch Aufruf $\text{AttrHülle}(\mathcal{F}, \kappa)$ erhalten wir κ^+ .

Falls $\kappa^+ = \mathcal{R}$: κ ist Superschlüssel von $\mathcal{R}$.

Äquivalente FD-Mengen

- Im Allgemeinen: **Es gibt viele unterschiedliche äquivalente Mengen von funktionalen Abhängigkeiten.**
- **Zwei Mengen $\mathcal{F}$ und $\mathcal{G}$ heißen äquivalent** (Notation: $\mathcal{F} \equiv \mathcal{G}$), **wenn ihre Hüllen gleich sind**, d.h. $\mathcal{F}^+ = \mathcal{G}^+$.
- **Bedeutung: die gleichen Mengen von FDs müssen herleitbar sein.**

Beobachtung:

- $\mathcal{F}^+$ kann sehr groß sein
- viele redundante Abhängigkeiten
- in der Praxis unübersichtlich

Ziel: **kleinstmögliche** Menge $\mathcal{F}_c$ finden, so dass immer noch gilt:
 $\mathcal{F}_c^+ = \mathcal{F}^+$.

Kanonische Überdeckung $\mathcal{F}_c$

Folgende Eigenschaften müssen erfüllt sein:

1. $\mathcal{F}_c \equiv \mathcal{F}$, d.h., $\mathcal{F}_c^+ = \mathcal{F}^+$
2. In $\mathcal{F}_c$ existieren keine FDs $\alpha \rightarrow \beta$, bei denen α oder β überflüssige Attribute enthalten. D.h. es muss gelten:
 - (a) $\forall A \in \alpha : (\mathcal{F}_c - (\alpha \rightarrow \beta) \cup ((\alpha - A) \rightarrow \beta)) \not\equiv \mathcal{F}_c$
 - (b) $\forall B \in \beta : (\mathcal{F}_c - (\alpha \rightarrow \beta) \cup (\alpha \rightarrow (\beta - B))) \not\equiv \mathcal{F}_c$
3. Jede linke Seite einer funktionalen Abhängigkeit in $\mathcal{F}_c$ ist einzigartig. Dies kann durch sukzessive Anwendung der Vereinigungsregel auf FDs der Art $\alpha \rightarrow \beta$ und $\alpha \rightarrow \gamma$ erzielt werden, so dass die beiden FDs durch $\alpha \rightarrow \beta\gamma$ ersetzt werden.

Algorithmus zur Bestimmung von $\mathcal{F}_c$

1. **Führe für jede FD $\alpha \rightarrow \beta \in \mathcal{F}$ die Linksreduktion durch**, also:
 - Überprüfe für alle $A \in \alpha$, ob A überflüssig ist, d.h. ob $\beta \subseteq \text{AttrHülle}(\mathcal{F}, \alpha - A)$
2. **Führe für jede (verbliebene) FD die Rechtsreduktion durch**:
 - Überprüfe für alle $B \in \beta$, ob $B \in \text{AttrHülle}(\mathcal{F} - (\alpha \rightarrow \beta) \cup (\alpha \rightarrow (\beta - B)), \alpha)$ gilt. In diesem Fall ist B auf der rechten Seite überflüssig und kann eliminiert werden, d.h. $\alpha \rightarrow \beta$ wird durch $\alpha \rightarrow (\beta - B)$ ersetzt.
3. **Entferne die FDs der Form $\alpha \rightarrow \emptyset$, die im 2. Schritt entstanden sind**.
4. **Fasse mittels Vereinigungsregel FDs der Form $\alpha \rightarrow \beta_1, \dots, \alpha \rightarrow \beta_n$ zusammen**, so dass $\alpha \rightarrow (\beta_1 \cup \dots \beta_n)$ verbleibt.

Beispiel für Herleitung einer kanonischen Überdeckung

- $\mathcal{F} = \{A \rightarrow B, B \rightarrow C, AB \rightarrow C\}$
- **Schritt 1:** Linksreduktion ersetzt $AB \rightarrow C$ durch $A \rightarrow C$
- **Schritt 2:** Rechtsreduktion ersetzt $A \rightarrow C$ durch $A \rightarrow \emptyset$
- **Schritt 3:** eliminiere $A \rightarrow \emptyset$
- **Schritt 4:** keine Zusammenfassungen notwendig
- Damit ist $\mathcal{F}_c = \{A \rightarrow B, B \rightarrow C\}$

Normalisierung von Relationen

Um Qualitätsprobleme im ursprünglichen Entwurf zu beheben, wird das bestehende Relationenschema $\mathcal{R}$ in mehrere Relationenschemata $\mathcal{R}_1, \dots, \mathcal{R}_n$ zerlegt, die dann “besser” sind.

- Die Güte einer Zerlegung wird mit **Normalformen** beschrieben.
- Normalformen: 1NF, 2NF, 3NF, BCNF, 4NF, ...

Korrektheitskriterien für Zerlegung:

- **Verlustlosigkeit:** Die in der ursprünglichen Relationenausprägung R des Schemas $\mathcal{R}$ enthaltenen Daten müssen aus den Ausprägungen $R_1, \dots, R_n$ der neuen Relationenschemata $\mathcal{R}_1, \dots, \mathcal{R}_n$ rekonstruierbar sein.
- **Abhängigkeitsbewahrung:** Alle FDs in $\mathcal{F}_{\mathcal{R}}$ sollten in den $\mathcal{F}_{\mathcal{R}_1}, \mathcal{F}_{\mathcal{R}_2}, \dots, \mathcal{F}_{\mathcal{R}_n}$ bewahrt bleiben.

Verlustlosigkeit

- Zerlegung ist gültig, wenn: $\mathcal{R} = \mathcal{R}_1 \cup \mathcal{R}_2$
- D.h. alle Attribute aus $\mathcal{R}$ bleiben in der Zerlegung erhalten
- Wir definieren:
 - $R_1 := \pi_{\mathcal{R}_1}(R)$
 - $R_2 := \pi_{\mathcal{R}_2}(R)$
- Die Zerlegung von $\mathcal{R}$ in $\mathcal{R}_1$ und $\mathcal{R}_2$ ist **verlustlos**, falls für jede mögliche (gültige) Ausprägung R von $\mathcal{R}$ gilt:

$$R = R_1 \bowtie R_2$$

Die Zerlegung muss also durch einen natürlichen Verbund (Join) rekonstruierbar sein.

D.h. $\mathcal{R}_1 \cap \mathcal{R}_2 \neq \emptyset$

“sinnvoller” Schlüssel existiert

Formale Charakterisierung Verlustloser Zerlegungen

- Gegeben Zerlegung von $\mathcal{R}$ in $\mathcal{R}_1$ und $\mathcal{R}_2$
- $\mathcal{F}_{\mathcal{R}}$ ist die Menge der FDs in $\mathcal{R}$
- **Zerlegung ist verlustlos, wenn mindestens eine der folgenden FDs herleitbar ist:**
 - $(\mathcal{R}_1 \cap \mathcal{R}_2) \rightarrow \mathcal{R}_1 \in \mathcal{F}_{\mathcal{R}}^+$ **oder** d.h. Schlüssel bestimmt $\mathcal{R}_1$
 - $(\mathcal{R}_1 \cap \mathcal{R}_2) \rightarrow \mathcal{R}_2 \in \mathcal{F}_{\mathcal{R}}^+$ d.h. Schlüssel bestimmt $\mathcal{R}_2$

Beispiel:

- Seien α, β und γ paarweise disjunkte Attributmengen
- $\mathcal{R} = \alpha \cup \beta \cup \gamma$, $\mathcal{R}_1 = \alpha \cup \beta$, $\mathcal{R}_2 = \alpha \cup \gamma$, $\mathcal{R}_1 \cap \mathcal{R}_2 = \alpha$
- Dann muss eine der Bedingungen gelten:
 - $\beta \subseteq \text{AttrHülle}(\mathcal{F}_{\mathcal{R}}, \alpha)$ **oder**
 - $\gamma \subseteq \text{AttrHülle}(\mathcal{F}_{\mathcal{R}}, \alpha)$

D.h. die gemeinsamen Joinattribute α müssen $\mathcal{R}_1$ oder $\mathcal{R}_2$ bestimmen.

Dies ist eine hinreichende aber nicht notwendige Bedingung!

Beispiel für Verlust

Biertrinker		
Kneipe	Gast	Bier
Kowalski	Kemper	Pils
Kowalski	Eickler	Hefeweizen
Innsteg	Kemper	Hefeweizen

wird zerlegt in ...

$\pi_{Kneipe, Gast}$

Besucht	
Kneipe	Gast
Kowalski	Kemper
Kowalski	Eickler
Innsteg	Kemper

$\pi_{Gast, Bier}$

Trinkt	
Gast	Bier
Kemper	Pils
Eickler	Hefeweizen
Kemper	Hefeweizen

Beispiel für Verlust: Wiederherstellung

 $\pi_{Kneipe, Gast}$

Besucht	
Kneipe	Gast
Kowalski	Kemper
Kowalski	Eickler
Innsteg	Kemper

 $\pi_{Gast, Bier}$

Trinkt	
Gast	Bier
Kemper	Pils
Eickler	Hefeweizen
Kemper	Hefeweizen

Mit Join verbunden gibt ..

Biertrinker		
Kneipe	Gast	Bier
Kowalski	Kemper	Pils
Kowalski	Kemper	Hefeweizen
Kowalski	Eickler	Hefeweizen
Innsteg	Kemper	Pils
Innsteg	Kemper	Hefeweizen

Erläuterung des Biertrinker-Beispiels

- Unser Biertrinker-Beispiel war **keine verlustlose Zerlegung**.
- **Es gibt in Biertrinker nämlich nur die folgende nicht-triviale funktionale Abhängigkeit:**

$$\{Kneipe, Gast\} \rightarrow \{Bier\}$$

- $\mathcal{R} = \{Kneipe\} \cup \{Gast\} \cup \{Bier\}$
- $\mathcal{R}_1 = \{Kneipe\} \cup \{Gast\}$, $\mathcal{R}_2 = \{Gast\} \cup \{Bier\}$,
 $\mathcal{R}_1 \cap \mathcal{R}_2 = \{Gast\}$.
- Dann muss eine der Bedingungen gelten:
 - $\{Kneipe\} \subseteq AttrHülle(\{Kneipe, Gast\} \rightarrow \{Bier\}, \{Gast\})$ **oder**
 - $\{Bier\} \subseteq AttrHülle(\{Kneipe, Gast\} \rightarrow \{Bier\}, \{Gast\})$
- Das ist nicht der Fall!
- Keine der zwei möglichen, die Verlustlosigkeit garantierenden FDs gilt:

$$\begin{aligned}\{Gast\} &\rightarrow \{Bier\} \\ \{Gast\} &\rightarrow \{Kneipe\}\end{aligned}$$

Beispiel für Verlustfreie Zerlegung

Eltern		
Vater	Mutter	Kind
Johann	Martha	Else
Johann	Maria	Theo
Heinz	Martha	Cleo

wird zerlegt in ...

$\pi_{Vater, Kind}$

Väter	
Vater	Kind
Johann	Else
Johann	Theo
Heinz	Cleo

$\pi_{Mutter, Kind}$

Mütter	
Mutter	Kind
Martha	Else
Maria	Theo
Martha	Cleo

Erläuterung der Zerlegung der Eltern-Relation

- $\mathcal{R} = \{Vater, Mutter, Kind\}$
- $\mathcal{F}_{\mathcal{R}} = \{\{Kind\} \rightarrow \{Mutter\}, \{Kind\} \rightarrow \{Vater\}\}$
- $\mathcal{R}_1 = \{Vater, Kind\}, \mathcal{R}_2 = \{Mutter, Kind\}, \mathcal{R}_1 \cap \mathcal{R}_2 = \{Kind\}$
- Dann muss **eine** der Bedingungen gelten:
 - $\{Vater\} \subseteq AttrH\ddot{u}lle(\mathcal{F}_{\mathcal{R}}, \{Kind\})$ **oder**
 - $\{Mutter\} \subseteq AttrH\ddot{u}lle(\mathcal{F}_{\mathcal{R}}, \{Kind\})$
- Hier gelten sogar beide Bedingungen.
- **Also ist die Zerlegung verlustlos.**

Abhängigkeitsbewahrung:

- ... ist 2. Korrektheitskriterium für eine Zerlegung
- $\mathcal{R}$ ist zerlegt in $\mathcal{R}_1, \dots, \mathcal{R}_n$
- Um bei neu eingefügten Daten zu überprüfen, ob es neue Abhängigkeiten gibt, könnte man zur Sicherheit jedes mal den Join $R_1 \bowtie \dots \bowtie R_n$ berechnen und auf Abhängigkeiten überprüfen
- Das ist allerdings **sehr** teuer!

Idee: die Abhängigkeiten sollten lokal überprüfbar sein!

- D.h. Überprüfung kann lokal auf Relationen $\mathcal{R}_1, \dots, \mathcal{R}_n$ gemacht werden
- Dafür muss folgende Bedingung gelten:
 $\mathcal{F}_{\mathcal{R}} \equiv (\mathcal{F}_{\mathcal{R}_1} \cup \dots \cup \mathcal{F}_{\mathcal{R}_n})$ bzw. $\mathcal{F}_{\mathcal{R}}^+ = (\mathcal{F}_{\mathcal{R}_1} \cup \dots \cup \mathcal{F}_{\mathcal{R}_n})^+$
- eine solche Zerlegung heißt auch **hüllengetreue Dekomposition!**

Gegenbeispiel

- **PLZverzeichnis:** {[Straße, Ort, Bland, PLZ]}
- **Annahmen:**
 - Orte werden durch ihren Namen (Ort) und das Bundesland (Bland) eindeutig identifiziert.
 - Innerhalb einer Straße ändert sich die Postleitzahl nicht
 - PLZ Gebiete gehen nicht über Ortsgrenzen und Orte nicht über Bundeslandgrenzen hinweg.

Daraus ergeben sich die folgenden FDs:

- $\{PLZ\} \rightarrow \{Ort, Bland\}$
- $\{Straße, Ort, Bland\} \rightarrow \{PLZ\}$

Betrachten wir die folgende Zerlegung von PLZverzeichnis:

- Straßen: {[PLZ, Straße]}
- Orte: {[PLZ, Ort, Bland]}

Zerlegung der Relation PLZverzeichnis

PLZVerzeichnis			
Ort	Bland	Straße	PLZ
Frankfurt	Hessen	Goethestraße	60313
Frankfurt	Hessen	Galgenstraße	60437
Frankfurt	Brandenburg	Goethestraße	15234

 $\pi_{PLZ, Straße}$
 $\pi_{Ort, Bland, PLZ}$

Straßen	
PLZ	Straße
15234	Goethestraße
60313	Goethestraße
60437	Galgenstraße

Orte		
Ort	Bland	PLZ
Frankfurt	Hessen	60313
Frankfurt	Hessen	60437
Frankfurt	Brandenburg	15234

- Diese Zerlegung ist **verlustlos** da $\{PLZ\} \rightarrow \{Orte, Bland\}$ und PLZ das einzige gemeinsame Attribut.
- die FD $\{Straße, Ort, Bland\} \rightarrow \{PLZ\}$ ist im zerlegten Schema **nicht** mehr enthalten: Also: Zerlegung ist **nicht abhängigkeiterhaltend**.

$\pi_{PLZ, Straße}$

Straßen	
PLZ	Straße
15234	Goethestraße
60313	Goethestraße
60437	Galgenstraße
15235	Goethestraße

 $\pi_{Ort, Bland, PLZ}$

Orte		
Ort	BLand	PLZ
Frankfurt	Hessen	60313
Frankfurt	Hessen	60437
Frankfurt	Brandenburg	15234
Frankfurt	Brandenburg	15235

Einfügen der **roten** Tupel in Zerlegung ist OK ..

Zerlegung der Relation PLZverzeichnis (2)

Durch einen Join der Zerlegung erhalten wir dann ...

PLZVerzeichnis			
Ort	Bland	Straße	PLZ
Frankfurt	Hessen	Goethestraße	60313
Frankfurt	Hessen	Galgenstraße	60437
Frankfurt	Brandenburg	Goethestraße	15234
Frankfurt	Brandenburg	Goethestraße	15235

Was fällt auf?

- Join erzeugt zusätzliches **rotes** Tupel, das die FD {Straße, Ort, Bland} $\rightarrow$ {PLZ} in **PLZverzeichnis** verletzt!
- Aber: nur durch die Ausführung des Joins ist diese Verletzung der FD aufzudecken.

Zusammenfassung

- Damit eine Zerlegung **korrekt** ist, muss sie verlustlos und abhängigkeiterhaltend sein.
- **Verlustlos** = keine Daten gehen verloren oder werden zusätzlich erzeugt bei einem Join der zerlegten Relationen.
- **abhängigkeitserhaltend** = FDs existieren nur lokal aber nicht Relationen-übergreifend

Bisher:

- Korrektheit der Zerlegung

Jetzt:

- Bewertung der Güte von Relationenschemata
- Zerlegung von Relationenschemata, um eine höhere Güte zu erreichen
- Güte = 1NF (NF=Normalform), 2NF, 3NF, BCNF, 4NF, ...

Erste Normalform (1NF)

Intuition: keine mengenwertigen Attributwerte

Eine Relation $\mathcal{R}$ ist in 1NF, wenn sie keine zusammengesetzten, mengenwertigen oder relationenwertigen Domänen hat.

Gegenbeispiel:

Eltern		
Vater	Mutter	Kind
Johann	Martha	{Else, Lucie}
Johann	Maria	{Theo, Josef}
Heinz	Martha	{Cleo}

1NF:

Eltern		
Vater	Mutter	Kind
Johann	Martha	Else
Johann	Martha	Lucie
Johann	Maria	Theo
Johann	Maria	Josef
Heinz	Martha	Cleo

Exkurs: NF^2 Relationen

- Non-First Normal-Form-Relationen
- = geschachtelte Relationen

Eltern			
Vater	Mutter	Kinder	
Johan	Martha	Else	5
		Lucie	3
Johann	Maria	Theo	3
		Josef	1
Heinz	Martha	Cleo	9

- Vorteil: keine unnötige Wiederholung (=Redundanz) von Vater und Mutter
- NF^2 ist eng verwandt mit hierarchischen Datenmodellen (XML)
- **im Folgenden setzen wir immer 1NF voraus**

Zweite Normalform

- Intuition: nicht mehr als ein Konzept pro Relation modellieren!

Eine Relation $\mathcal{R}$ mit zugehörigen FDs $\mathcal{F}_{\mathcal{R}}$ ist in 2NF, falls jedes Nichtschlüssel-Attribut $A \in \mathcal{R}$ voll funktional abhängig ist von jedem Kandidatenschlüssel der Relation.

- Seien $\kappa_1, \dots, \kappa_i$ die Kandidatenschlüssel von $\mathcal{R}$
- Sei $A \in \mathcal{R} - (\kappa_1 \cup \dots \cup \kappa_i)$
- Ein solches Attribut A wird als **nicht-prim** (Nichtschlüssel-Attribut) bezeichnet.
- Gegensatz: Schlüsselattribute bezeichnet man als **prim**.
- Dann muss für **alle** Kandidatenschlüssel κ_j ($1 \leq j \leq i$) gelten:

$$\kappa_j \xrightarrow{\bullet} A \in \mathcal{F}^+$$

Also mit anderen Worten:

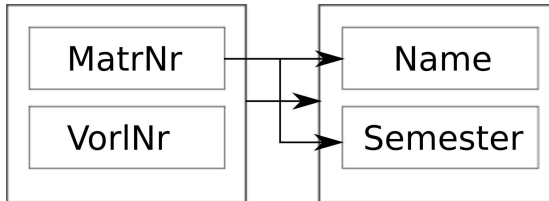
- A ist **voll funktional abhängig** von **jedem** κ_j
- κ_j muss bereits **linksreduziert** sein!
- 2NF **verhindert partielle** Abhängigkeiten

Gegenbeispiel

StudentenBelegung			
MatrNr	VorlNr	Name	Semester
26120	5001	Fichte	10
27550	5001	Schopenhauer	6
27550	4052	Schopenhauer	6
28106	5041	Carnap	3
28106	5052	Carnap	3
28106	5216	Carnap	3
28106	5259	Carnap	3
...	...	...	...

- **Kandidatenschlüssel** {MatrNr, VorlNr}
- **prim:** {MatrNr, VorlNr}
- **nicht-prim:** {Name, Semester}
- StudentenBelegung ist **nicht in 2NF**. Wieso?
 - $\{\text{MatrNr}\} \rightarrow \{\text{Name}\}$, damit **nicht voll** funktional abhängig von {MatrNr, VorlNr}
 - $\{\text{MatrNr}\} \rightarrow \{\text{Semester}\}$, damit **nicht voll** funktional abhängig von {MatrNr, VorlNr}
- **Abhängigkeit zu Teilschlüssel!**

Änderungsanomalien im Gegenbeispiel



- **Einfügeanomalie:** Was macht man mit Studenten, die keine Vorlesungen hören?
- **Updateanomalien:** Wenn z.B. Carnap ins vierte Semester kommt, muss man sicherstellen, dass alle vier Tupel geändert werden.
- **Löschanomalie:** Was passiert wenn Fichte seine einzige Vorlesung absagt?
- **Zerlegung in zwei Relationen:**
 - hoeren: $\{[MatrNr, VorlNr]\}$
 - Studenten: $\{[MatrNr, Name, Semester]\}$
- **Beide Relationen sind in 2NF.**

Dritte Normalform

- **Intuition:** Nicht-Schlüssel Attribut darf kein anderes Nicht-Schlüssel Attribut bestimmen.

Ein Relationenschema $\mathcal{R}$ ist in dritter Normalform, wenn für jede für $\mathcal{R}$ geltende FD der Form $\alpha \rightarrow B$ mit Attribut $B \in \mathcal{R}$ mindestens eine von drei Bedingungen gilt:

1. $B \in \alpha$, d.h. die FD ist **trivial**
2. α ist **Superschlüssel** von R
3. B ist **prim**

Eigenschaften:

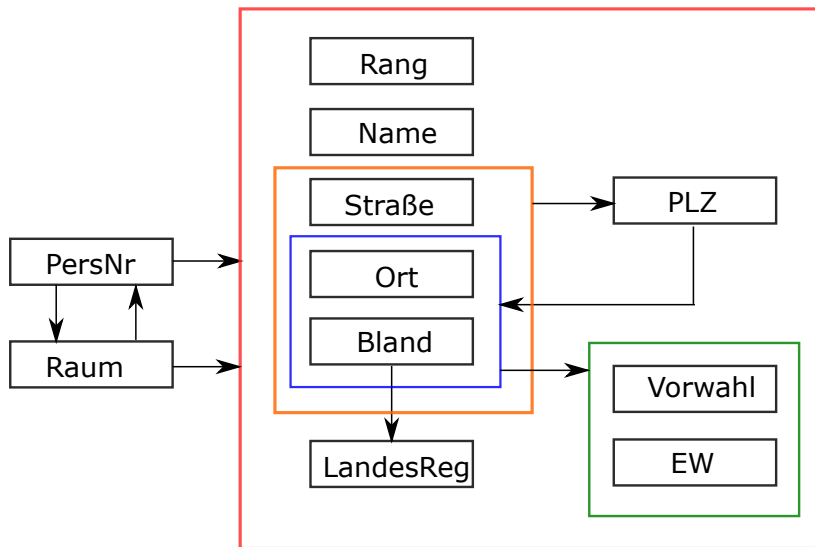
- **3NF verhindert partielle und transitive Abhängigkeiten**
- $3NF \Rightarrow 2NF$

Gegenbeispiel: Nicht 2NF

StudentenBelegung			
MatrNr	VorlNr	Name	Semester
26120	5001	Fichte	10
27550	5001	Schopenhauer	6
27550	4052	Schopenhauer	6
28106	5041	Carnap	3
28106	5052	Carnap	3
28106	5216	Carnap	3
28106	5259	Carnap	3
...	...	...	...

- Kandidatenschlüssel {MatrNr, VorlNr}
- {MatrNr} $\rightarrow$ {Name}
 - nicht trivial
 - MatrNr kein Superschlüssel
 - Name ist nicht prim
 - Also: **nicht in 3NF**
- analog für {MatrNr} $\rightarrow$ {Semester}
- Abhängigkeit zu **Teilschlüssel durch Superschlüssel-Bedingung in 3NF-Definition abgefragt!**

Gegenbeispiel: 2NF aber nicht 3NF



Gegenbeispiel: 2NF aber nicht 3NF

- ProfessorenAdr: {[PersNr, Name, Rang, Raum, Ort, Straße, PLZ, Vorwahl, Bland, EW, Landesregierung]}
- Kandidatenschlüssel: PersNr und Raum
- FDs:
 1. {PersNr} $\rightarrow$ {Name, Rang, Raum, Ort, Straße, Bland}
 2. {Raum} $\rightarrow$ {PersNr}
 3. {Straße, Bland, Ort} $\rightarrow$ {PLZ}
 4. {Ort, Bland} $\rightarrow$ {EW, Vorwahl}
 5. {Bland} $\rightarrow$ {Landesregierung}
 6. {PLZ} $\rightarrow$ {Bland, Ort}

Warum ist diese Relation in 2NF?

Warum ist diese Relation nicht in 3NF?

Gegenbeispiel

- ProfessorenAdr: {[PersNr, Name, Rang, Raum, Ort, Straße, PLZ, Vorwahl, Bland, EW, Landesregierung]}
- Kandidatenschlüssel: PersNr und Raum
- FDs:
 1. {PersNr} $\rightarrow$ {Name, Rang, Raum, Ort, Straße, Bland}
 2. {Raum} $\rightarrow$ {PersNr}
 3. {Straße, Bland, Ort} $\rightarrow$ {PLZ}
 4. {Ort, Bland} $\rightarrow$ {EW, Vorwahl}
 5. {Bland} $\rightarrow$ {Landesregierung}
 6. {PLZ} $\rightarrow$ {Bland, Ort}
- FD1 ist OK: PersNr ist Superschlüssel, Raum prim
- FD2 ist OK: Raum ist Superschlüssel, PersNr prim
- **FD3-6 nicht OK: weder trivial, noch Superschlüssel, noch prim.**

Synthesealgorithmus

Algorithmus ermittelt zu einem gegebenen Relationenschema $\mathcal{R}$ mit funktionalen Abhängigkeiten $\mathcal{F}$ eine Zerlegung in $\mathcal{R}_1, \dots, \mathcal{R}_n$ die alle drei folgenden Kriterien erfüllt:

1. $\mathcal{R}_1, \dots, \mathcal{R}_n$ ist eine verlustlose Zerlegung von $\mathcal{R}$.
2. Die Zerlegung $\mathcal{R}_1, \dots, \mathcal{R}_n$ ist abhängigkeiterhaltend.
3. Alle $\mathcal{R}_1, \dots, \mathcal{R}_n$ sind in 3NF.

Synthesealgorithmus

1. **Bestimme die kanonische Überdeckung $\mathcal{F}_c$ zu $\mathcal{F}$.** Wiederholung:
 - (i) Linksreduktion
 - (ii) Rechtsreduktion
 - (iii) Entfernung von FDs der Form $\alpha \rightarrow \emptyset$
 - (iv) Zusammenfassung gleicher linker Seiten
2. **Für jede funktionale Abhängigkeit $\alpha \rightarrow \beta \in \mathcal{F}_c$:**
 - Kreiere ein Relationenschema $\mathcal{R}_\alpha := \alpha \cup \beta$
 - Ordne $\mathcal{R}_\alpha$ die FDs $\mathcal{F}_\alpha := \{\alpha' \rightarrow \beta' \in \mathcal{F}_c \mid \alpha' \cup \beta' \subseteq \mathcal{R}_\alpha\}$ zu.
3. **Falls eines der in Schritt 2 erzeugten Schemata einen Kandidatenschlüssel von $\mathcal{R}$ bzgl. $\mathcal{F}_c$ enthält, sind wir fertig.**
Sonst wähle einen Kandidatenschlüssel $\kappa \subseteq \mathcal{R}$ aus und definiere folgendes Schema:
 - $\mathcal{R}_\kappa := \kappa$
 - $F_\kappa := \emptyset$
4. **Eliminiere diejenigen Schemata $\mathcal{R}_\alpha$, die in einem anderen Relationenschema $\mathcal{R}_{\alpha'}$ enthalten sind, d.h., $\mathcal{R}_\alpha \subseteq \mathcal{R}_{\alpha'}$**

Anwendung: Schritt 1

ProfessorenAdr: {[PersNr, Name, Rang, Raum, ort, Straße, PLZ, Vorwahl, Bland, EW, Landesregierung]}

Schritt 1 (kanonische Überdeckung) enthält die FDs:

1. {PersNr} $\rightarrow$ {Name, Rang, Raum, Ort, Straße, Bland}
2. {Raum} $\rightarrow$ {PersNr}
3. {Straße, Bland, Ort} $\rightarrow$ {PLZ}
4. {Ort, Bland} $\rightarrow$ {EW, Vorwahl}
5. {Bland} $\rightarrow$ {Landesregierung}
6. {PLZ} $\rightarrow$ {Bland, Ort}

Anwendung: Schritt 2

Schritt 2: Aus den FDs Relationen erzeugen

1. $\{\text{PersNr}\} \rightarrow \{\text{Name, Rang, Raum, Ort, Straße, Bland}\}$
Professoren: $\{[\text{PersNr, Name, Rang, Raum, Ort, Straße, Bland}]\}$
2. $\{\text{Raum}\} \rightarrow \{\text{PersNr}\}$
ProfessorenRäume: $\{[\text{Raum, PersNr}]\}$
3. $\{\text{Straße, Bland, Ort}\} \rightarrow \{\text{PLZ}\}$
PLZverzeichnis: $\{[\text{Straße, Bland, Ort, PLZ}]\}$
4. $\{\text{Ort, Bland}\} \rightarrow \{\text{EW, Vorwahl}\}$
StädteVerzeichnis: $\{[\text{Ort, Bland, EW, Vorwahl}]\}$
5. $\{\text{Bland}\} \rightarrow \{\text{Landesregierung}\}$
Regierung: $\{[\text{Bland, Landesregierung}]\}$
6. $\{\text{PLZ}\} \rightarrow \{\text{Bland, Ort}\}$
PLZverzeichnis2: $\{[\text{PLZ, Bland, Ort}]\}$

Anwendung: Schritt 3

Schritt 3: Neue Relation falls keine Rel. mit Kand.-Schlüssel:

1. $\{\text{PersNr}\} \rightarrow \{\text{Name, Rang, Raum, Ort, Straße, Bland}\}$
Professoren: $\{[\textbf{PersNr}, \text{Name, Rang, Raum, Ort, Straße, Bland}]\}$
2. $\{\text{Raum}\} \rightarrow \{\text{PersNr}\}$
ProfessorenRäume: $\{[\textbf{Raum, PersNr}]\}$
3. $\{\text{Straße, Bland, Ort}\} \rightarrow \{\text{PLZ}\}$
PLZverzeichnis: $\{[\text{Straße, Bland, Ort, PLZ}]\}$
4. $\{\text{Ort, Bland}\} \rightarrow \{\text{EW, Vorwahl}\}$
StädteVerzeichnis: $\{[\text{Ort, Bland, EW, Vorwahl}]\}$
5. $\{\text{Bland}\} \rightarrow \{\text{Landesregierung}\}$
Regierung: $\{[\text{Bland, Landesregierung}]\}$
6. $\{\text{PLZ}\} \rightarrow \{\text{Bland, Ort}\}$
PLZverzeichnis2: $\{[\text{PLZ, Bland, Ort}]\}$
 - **Raum** und **PersNr** sind beide Kandidatenschlüssel.
 - Schritt 3 erzeugt für dieses Beispiel keine neue Relation.

Anwendung: Schritt 4

1. Professoren: {[**PersNr**, Name, Rang, **Raum**, Ort, Straße, Bland]}
2. ProfessorenRäume: {[**Raum**, **PersNr**]} $\subseteq$ **Professoren**
3. PLZverzeichnis: {[Straße, Bland, Ort, PLZ]}
4. StädteVerzeichnis: {[Ort, Bland, EW, Vorwahl]}
5. Regierung: {[Bland, Landesregierung]}
6. PLZverzeichnis2: {[PLZ, Bland, Ort]} $\subseteq$ **PLZverzeichnis**

Fertig!

Anderes Beispiel für Schritt 3

- **StudentenBelegung**(**MatrNr**, **VorlNr**, Name, Semester)
- Kanonische Überdeckung hierfür:

$$\{\text{MatrNr}\} \rightarrow \{\text{Name}, \text{Semester}\}$$

- Daraus folgt die Relation:

Student(**MatrNr** , Name, Semester)

- **Student** enthält keinen Kandidatenschlüssel.
- Deswegen Schritt 3: **hören**(**MatrNr**, **VorlNr**)
- Ansonsten würde die Information wer welche VL hört wegfallen.

Wiederholung: Dritte Normalform

- Intuition: Nicht-Schlüssel Attribut darf kein anderes Nicht-Schlüssel Attribut bestimmen.

Ein Relationenschema $\mathcal{R}$ ist in dritter Normalform, wenn für jede für $\mathcal{R}$ geltende FD der Form $\alpha \rightarrow B$ mit Attribut $B \in \mathcal{R}$ mindestens eine von drei Bedingungen gilt:

1. $B \in \alpha$, d.h. die FD ist **trivial**
2. α ist **Superschlüssel** von R
3. B ist **prim**

Eigenschaften:

- 3NF verhindert **partielle und transitive** Abhängigkeiten
- 3NF $\Rightarrow$ 2NF

Boyce-Codd-Normalform

Die Boyce-Codd-Normalform (BCNF) ist nochmals eine **Verschärfung der 3NF**. **Intuition:** Jedes Attribut darf **nur** den gesamten Schlüssel beschreiben und nichts anderes.

Ein Relationenschema $\mathcal{R}$ mit FDs $\mathcal{F}$ ist in BCNF, wenn für jede für $\mathcal{R}$ geltende funktionale Abhängigkeit der Form $\alpha \rightarrow B$ mit Attribut $B \in \mathcal{R}$ mindestens eine von zwei Bedingungen gilt:

1. $B \in \alpha$, d.h. die FD ist **trivial**
 2. α ist Superschlüssel von $\mathcal{R}$
- Unterschied zu 3NF: 3. Bedingung fällt weg (B ist prim).
 - **Man kann jede Relation verlustlos in BCNF-Relationen zerlegen**
 - **Aber: manchmal lässt sich dabei die Abhängigkeitserhaltung nicht erzielen!**

Städte ist in 3NF, aber nicht in BCNF

- Städte: {[Ort, Bland, Ministerpräsident/in, EW]}
- Geltende FDs:
 1. {Ort, Bland} $\rightarrow$ {EW}
 2. {Bland} $\rightarrow$ {Ministerpräsident/in}
 3. {Ministerpräsident/in} $\rightarrow$ {Bland}
- Kandidatenschlüssel:
 - {Ort, Bland}
 - {Ort, Ministerpräsident/in}
- Linke Seite von FD1 ist Superschlüssel (Bedingung 2)
- Rechte Seiten von FD2 und FD3 sind prim (Bedingung 3)
- Also: **3NF**
- Linke Seiten von FD2 und FD3 sind aber **kein Superschlüssel**
- Also: **nicht in BCNF**
- **D.h.: BCNF verhindert zusätzlich zu 3NF transitive Abhängigkeiten zu Schlüssel-Attributen**
- Wer welches Bundesland regiert wird hier also mehrfach abgespeichert

Dekomposition

Man kann grundsätzlich jedes Relationenschema $\mathcal{R}$ mit funktionalen Abhängigkeiten $\mathcal{F}$ so in $\mathcal{R}_1, \dots, \mathcal{R}_n$ zerlegen, dass gilt

- $\mathcal{R}_1, \dots, \mathcal{R}_n$ ist eine verlustlose Zerlegung von $\mathcal{R}$
- Alle $\mathcal{R}_1, \dots, \mathcal{R}_n$ sind in BCNF.

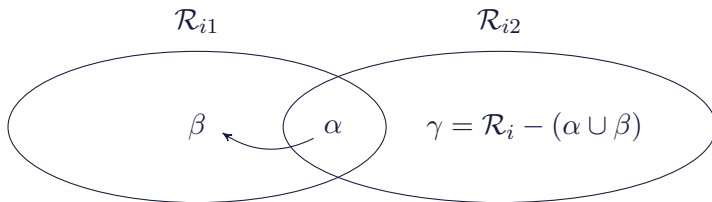
Es kann allerdings nicht immer erreicht werden, dass die Zerlegung $\mathcal{R}_1, \dots, \mathcal{R}_n$ abhängigkeiterhaltend ist

Dies ist in der Praxis selten.

Dekompositions-Algorithmus für BCNF

- Starte mit $Z = \{\mathcal{R}\}$
- Solange es noch ein Relationenschema $\mathcal{R}_i \in Z$ gibt, das nicht in BCNF ist, mache folgendes:
 1. Es gibt also eine für $\mathcal{R}_i$ geltende nicht-triviale funktionale Abhängigkeit $\alpha \rightarrow \beta$ mit:
 - $\alpha \cap \beta = \emptyset$ (α und β sind disjunkt)
 - $\neg(\alpha \rightarrow \mathcal{R}_i)$ (α kein Superschlüssel von $\mathcal{R}_i$)
 2. Finde eine solche FD:
Man sollte sie so wählen, dass β alle von α funktionalen abhängigen Attribute $B \in (\mathcal{R}_i - \alpha)$ enthält, damit der Dekompositionsalgorithmus möglichst schnell terminiert.
 3. Zerlege $\mathcal{R}_i$ in $\mathcal{R}_{i1} := \alpha \cup \beta$ und $\mathcal{R}_{i2} := \mathcal{R}_i - \beta$
 4. Entferne $\mathcal{R}_i$ aus Z und füge $\mathcal{R}_{i1}$ und $\mathcal{R}_{i2}$ ein:
 $Z := (Z - \mathcal{R}_i) \cup \mathcal{R}_{i1} \cup \mathcal{R}_{i2}$

Veranschaulichung eines Dekompositionsschrittes



Dekomposition der Relation Städte in BCNF-Relationen

- Städte: {[Ort, Bland, Ministerpräsident/in, EW]}
- Geltende FDs:
 1. {Ort, Bland} $\rightarrow$ {EW}
 2. {Bland} $\rightarrow$ {Ministerpräsident/in}
 3. {Ministerpräsident/in} $\rightarrow$ {Bland}

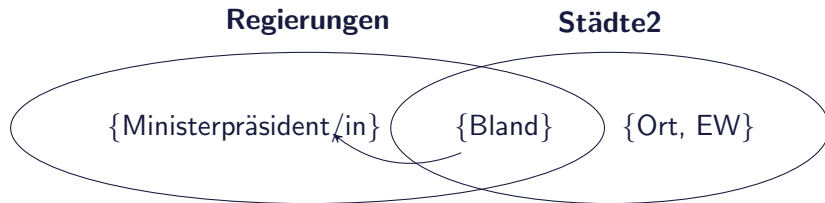
$\mathcal{R}_{i1}$

- **Regierungen:** {[Bland, Ministerpräsident/in]}

$\mathcal{R}_{i2}$

- **Städte:** {[Ort, Bland, EW]}

Veranschaulichung



- Zerlegung ist verlustlos und auch abhängigkeiterhaltend.
- **warum?**

Weiteres Beispiel: Dekomposition des PLZVerzeichnis in BCNF-Relationen

PLZVerzeichnis: {[Straße, Ort, Bland, PLZ]}

1. {Straße, Ort, Bland} → {PLZ} OK
2. {PLZ} → {Ort, Bland} verletzt BCNF warum?

Betrachte nun die Zerlegung:

- Orte: {[PLZ, Ort, Bland]}
- Straßen: {[PLZ, Straße]}

Diese Zerlegung

- ist verlustlos, **aber**
- **nicht abhängigkeiterhaltend, warum?**

Mehrwertige Abhängigkeiten

Bislang:

Funktionale Abhängigkeiten der Form

- Ausprägung R
- Seien $\alpha \subseteq \mathcal{R}$ und $\beta \subseteq \mathcal{R}$
- $\alpha \rightarrow \beta$ genau dann, wenn $\forall r, s \in R$ gilt: $r.\alpha = s.\alpha \Rightarrow r.\beta = s.\beta$
- D.h. die α -Werte bestimmen die β -Werte funktional (=eindeutig)

Nun: Mehrwertige Abhängigkeiten (multivalued dependencies)

Notation: $\alpha \twoheadrightarrow \beta$

Falls einem Attributwert α eine **Menge** von β -Werten zugeordnet werden.

Genaue Definition folgt (gleich).

Mehrwertige Abhängigkeiten: Beispiel

Fähigkeiten		
PersNr	Sprache	ProgSprache
3002	griechisch	C
3002	lateinisch	Pascal
3002	griechisch	Pascal
3002	lateinisch	C
3005	deutsch	Ada

Mehrwertige Abhängigkeiten dieser Relation:

- $\{\text{PersNr}\} \twoheadrightarrow \{\text{Sprache}\}$ und
- $\{\text{PersNr}\} \twoheadrightarrow \{\text{ProgSprache}\}$

MVDs führen zu Redundanz und Anomalien

Mehrwertige Abhängigkeiten

R		
A	B	C
a	b	c
a	bb	cc
a	b	cc
a	bb	c

- $A \twoheadrightarrow B$
- $A \twoheadrightarrow C$

Bei zwei Tupeln mit gleichen α -Werten kann man die β -Werte vertauschen und die resultierenden Tupel müssen auch in der Relation sein.

Mehrwertige Abhängigkeiten: Definition 1

	R		
	$\overbrace{A_1, \dots, A_i}^{\alpha}$	$\overbrace{A_{i+1}, \dots, A_j}^{\beta}$	$\overbrace{A_{j+1}, \dots, A_n}^{\gamma}$
t_1	$a_1, \dots, a_i$	$a_{i+1}, \dots, a_j$	$a_{j+1}, \dots, a_n$
t_2	$a_1, \dots, a_i$	$b_{i+1}, \dots, b_j$	$b_{j+1}, \dots, b_n$
t_3	$a_1, \dots, a_i$	$a_{i+1}, \dots, a_j$	$b_{j+1}, \dots, b_n$
t_4	$a_1, \dots, a_i$	$b_{i+1}, \dots, b_j$	$a_{j+1}, \dots, a_n$

$\alpha \twoheadrightarrow \beta$ gilt genau dann, wenn für jede Ausprägung von R gilt:

- wenn es zwei Tupel t_1 und t_2 mit gleichen α -Werten gibt, dann muss es auch zwei Tupel t_3 und t_4 geben mit

- $t_1.\alpha = t_2.\alpha = t_3.\alpha = t_4.\alpha$

(alle α -Werte gleich)

- $t_3.\beta = t_1.\beta, t_4.\beta = t_2.\beta$

(β -Paare gleich)

- $t_3.\gamma = t_2.\gamma, t_4.\gamma = t_1.\gamma$

(γ -Paare **vertauscht**)

Veranschaulichung: Spezialfall

Veranschaulichung für MVD $\alpha \twoheadrightarrow \beta$:

Wenn α, β, γ jeweils nur aus einem Attribut A, B , und C bestehen:

Wenn $\{b_1, \dots, b_i\}$ und $\{c_1, \dots, c_j\}$ die B bzw. C -Werte für einen bestimmten A -Wert a sind, dann muss die Relation auch die folgenden $(i * j)$ Tupel enthalten:

$$\{a\} \times \{b_1, \dots, b_i\} \times \{c_1, \dots, c_j\}$$

Mehrwertige Abhängigkeiten: Definition 2

- Eine mehrwertige Abhängigkeit (multivalued dependency, MVD) $\alpha \twoheadrightarrow \beta$ besagt, dass einem Attribut α in $\mathcal{R}$ eine **Menge** von β -Werten zugeordnet werden.
- Wenn die MVD $\alpha \twoheadrightarrow \beta$ in R erfüllt, dann kann es als Erweiterung zu FDs α, β, c geben mit

$$|\pi_{\beta}(\sigma_{\alpha=c}(R))| > 1$$

- Diese Zuordnung ist **unabhängig** von den restlichen Attributen in $\mathcal{R}$

Mit anderen Worten:

- α bestimmt **nicht nur** einen **einzelnen** Wert (ein singleton)
- genau das ist ja bei einer normalen FD $\alpha \rightarrow \beta$ der Fall!
- **sondern** eine Menge von Werten
- diese Wertemenge ist unabhängig von den anderen Attributen in $\gamma = \mathcal{R} - \alpha - \beta$

Natürlich: Jede FD ist auch eine MVD (aber nicht umgekehrt)

Beispiel

Fähigkeiten		
PersNr	Sprache	ProgSprache
3002	griechisch	C
3002	lateinisch	Pascal
3002	griechisch	Pascal
3002	lateinisch	C
3005	deutsch	Ada

 $\pi_{PersNr, Sprache}$

Sprachen	
PersNr	Sprache
3002	griechisch
3002	lateinisch
3005	deutsch

 $\pi_{PersNr, ProgSprache}$

ProgSprachen	
PersNr	ProgSprache
3002	C
3002	Pascal
3005	Ada

Verlustlose Zerlegung bei MVDs: hinreichende + notwendige Bedingung

Die Zerlegung von $\mathcal{R}$ in $\mathcal{R}_1$ und $\mathcal{R}_2$ ist verlustlos genau dann, wenn

- $\mathcal{R} = \mathcal{R}_1 \cup \mathcal{R}_2$
- **und** mindestens eine von zwei MVDs gilt:
 - $\mathcal{R}_1 \cap \mathcal{R}_2 \twoheadrightarrow \mathcal{R}_1$ **oder**
 - $\mathcal{R}_1 \cap \mathcal{R}_2 \twoheadrightarrow \mathcal{R}_2$

Für unser Beispiel gilt:

- $\{\text{PersNr}, \text{Sprache}, \text{ProgrSprache}\} = \{\text{PersNr}, \text{Sprache}\} \cup \{\text{PersNr}, \text{ProgSprache}\}$
- $\{\text{PersNr}\} \twoheadrightarrow \{\text{Sprache}\}$
- $\{\text{PersNr}\} \twoheadrightarrow \{\text{ProgSprache}\}$

D.h. es gelten sogar beide MVDs!

MVDs in Paaren

- **Es gilt zusätzlich:** wenn $\alpha \twoheadrightarrow \beta$, dann gilt immer
 - $\alpha \twoheadrightarrow \gamma$
 - mit $\gamma = \mathcal{R} - \alpha - \beta$
- **D.h. MVDs treten immer als Paare auf**
- Wir könnten MVDs deshalb auch so notieren: $\alpha \twoheadrightarrow \beta | \gamma$

Triviale MVDs ...

- sind solche, die von jeder Relationenausprägung R von $\mathcal{R}$ erfüllt werden.
- $\alpha \cup \beta \subseteq \mathcal{R}$
- Eine MVD $\alpha \twoheadrightarrow \beta$ ist trivial genau dann, wenn
 1. $\beta \subseteq \alpha$ oder
 2. $\beta = \mathcal{R} - \alpha$
- **Nur** die Bedingung 1 galt auch für normale FDs.
- Beispiel für Bedingung 2:
 - $\mathcal{R} = \{PersNr, Sprache\}$
 - $\alpha = \{PersNr\}$
 - $\beta = \{Sprache\}$
 - $\mathcal{R} - \alpha = \{PersNr, Sprache\} - \{PersNr\} = \{Sprache\} = \beta \Rightarrow$
MVD ist trivial!

Vierte Normalform

Eine Relation $\mathcal{R}$ ist in 4NF, wenn für jede MVD $\alpha \twoheadrightarrow \beta$ eine der folgenden Bedingungen gilt:

- Die MVD ist trivial **oder**
- α ist Superschlüssel von $\mathcal{R}$
- D.h. 4NF ist sehr ähnlich zu BCNF!
- Unterschied:
 - MVDs statt FDs
 - Definition von “trivial” wurde erweitert.
- 4NF erfüllt $\Rightarrow$ BCNF erfüllt, da jede FD eine MVD ist.

Dekomposition in 4NF

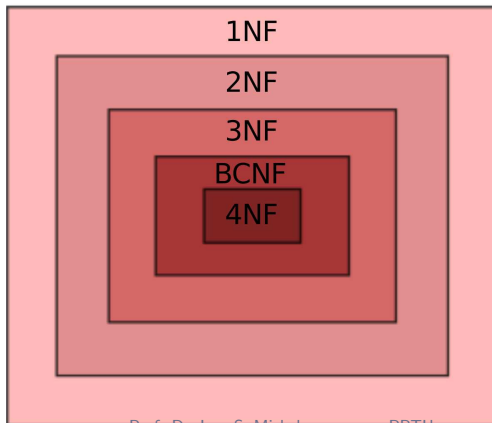
Starte mit der Menge $Z := \{\mathcal{R}\}$

Solange es noch ein Relationenschema $\mathcal{R}_i$ in Z gibt, das nicht in 4NF ist, mache folgendes:

- Es gibt also eine für $\mathcal{R}_i$ geltende nicht-triviale MVD $(\alpha \twoheadrightarrow \beta)$, für die gilt:
 - $\alpha \cap \beta = \emptyset$
 - $\neg(\alpha \rightarrow \mathcal{R}_i)$
- Finde eine solche MVD
- Zerlege $\mathcal{R}_i$ in $\mathcal{R}_{i1} := \alpha \cup \beta$ und $\mathcal{R}_{i2} := \mathcal{R}_i - \beta$
- Entferne $\mathcal{R}_i$ aus Z und füge $\mathcal{R}_{i1}$ und $\mathcal{R}_{i2}$ ein, also $Z := (Z - \mathcal{R}_i) \cup \{\mathcal{R}_{i1}\} \cup \{\mathcal{R}_{i2}\}$

Alle Normalformen auf einen Blick

- Die Verlustlosigkeit ist für alle Zerlegungsalgorithmen in alle Normalformen garantiert.
- Die Abhängigkeitserhaltung kann nur bis zur dritten Normalform garantiert werden.



abhängigkeitserhaltende
Zerlegung



verlustlose
Zerlegung



Zusammenfassung Entwurfstheorie

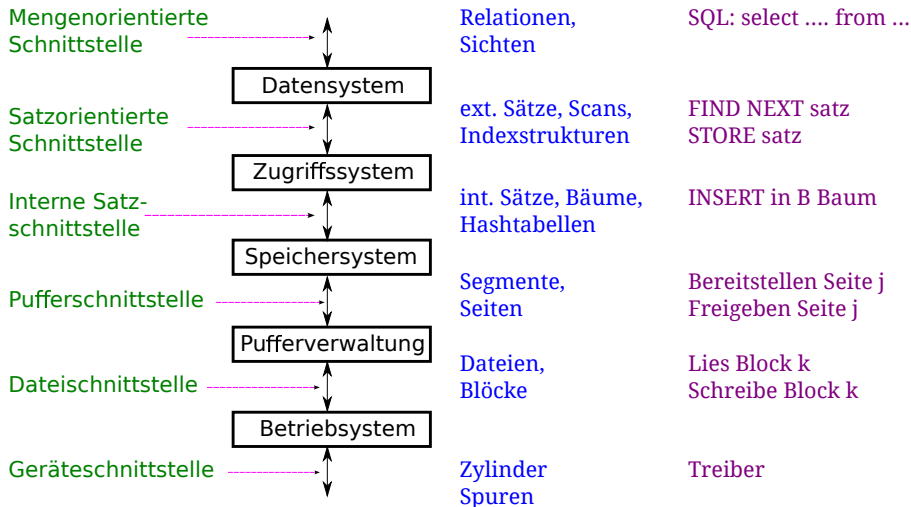
- **Ziel: Bewertung der Güte einer Relation; Vermeidung von Anomalien durch Zerlegung in “bessere” Relationen**
- **Betrachtung von funktionalen Abhängigkeiten zur Identifikation von Redundanz**
- **Korrektheit von Zerlegung** definiert als verlustlos und abhängigkeitsbewahrend
- **1NF, 2NF, 3NF, BCNF und 4NF betrachtet**
- **Algorithmen zur Zerlegung/Synthese**

Übersicht und Ausblick

Ausblick auf kommende Vorlesungen

- **Zugriffskosten**
- **Physische Datenorganisation**
- **Pufferverwaltung**
- **Indexstrukturen**
- **Regelbasierte Anfrageoptimierung**

5 Schichtenmodell einer Datenbank

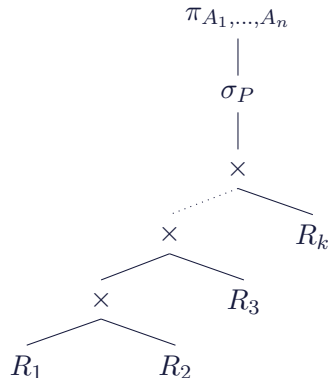


Übersicht Anfrageverarbeitung

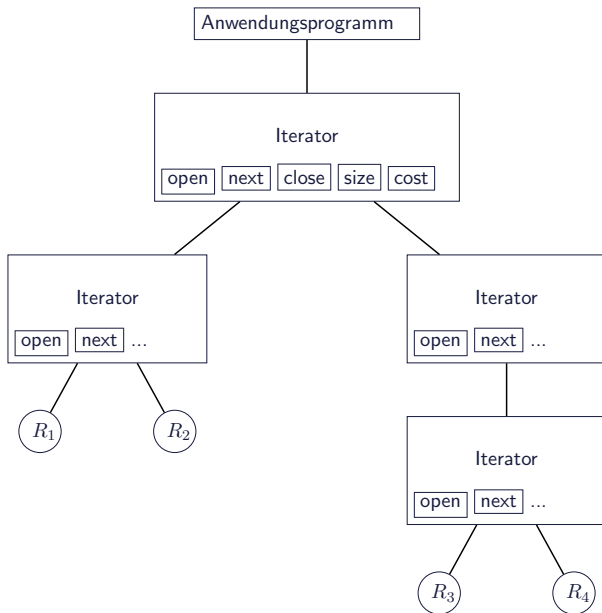
- **Eingabe:** Anfrage als Text (String)
- Kompilierzeitsystem (compile time system) übersetzt und optimiert die Anfrage
- Zwischenprodukt: **Anfrage als Anfrageplan** (query plan)
- Laufzeitsystem (runtime system) führt die Anfrage aus
- **Ausgabe:** Ergebnisse der Anfrage

Übersetzung von SQL in Baum aus Operatoren

select $A_1, \dots, A_n$
from $R_1, \dots, R_k$
where P ;



Was sind Operatoren, wie läuft Verarbeitung ab und wie werden Operatoren Implementiert?



Iterator

- **Open:** Konstruktor, initialisiert, öffnet die Eingabe
- **Next:** Liefert das nächste Ergebnis
- **Close:** Schließt die Eingabe
- **Cost und Size:** Geben Informationen über die geschätzten (!) Kosten

Empfehlenswert: Übersichtsartikel von G. Graefe: Query Evaluation Techniques for Large Databases, ACM Computing Surveys, 1993, volume 25, number 2, pages 73-170.

Pull-basierte Verarbeitung

- **Operatoren werden in Form von Iteratoren implementiert**
- **Datenfluss hierbei ist “von unten nach oben”**
- **Konsument-Produzent Verhältnis**
- Der konsumierende Iterator bezieht Tupel von seinen Eingaben, die ebenfalls Iteratoren sind, durch deren `next()` Schnittstelle
- **Im Allgemeinen werden Zwischenergebnisse nicht explizit materialisiert.**

Anmerkung: Push-basierte Verarbeitung

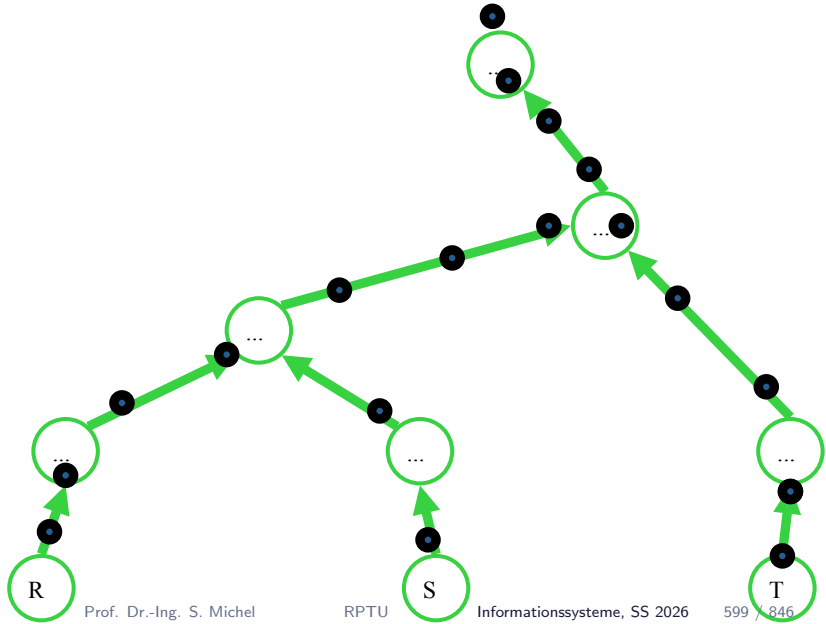
Es gibt insbesondere bei Systemen zur Datenstromverarbeitung die sogenannte Push-basierte Verarbeitung. Dort registrieren sich “Konsumenten” an Datenquellen oder anderen Operatoren. Falls diese neue Daten haben, werden die Daten an die Konsumenten weiter gereicht (aktiv).

Blockierende Operatoren

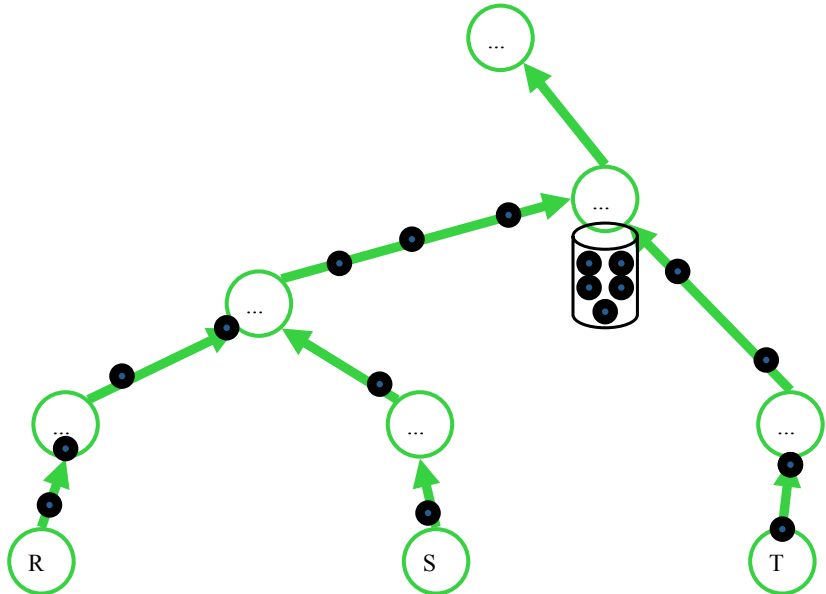
Idealerweise

- **Operatoren blockieren den Datenfluss nicht**
- D.h. beim Aufruf von `next()` werden darunter liegende Operatoren angefragt via `next` und das Ergebnis direkt weiter geleitet. Nur wenige Tupel werden dabei gelesen.
- **Erlaubt Pipelining**
- Im Gegensatz dazu: Operatoren, die den Datenfluss “blockieren”, sogenannte Pipeline-Breaker

Pipelining vs. Pipeline-Breaker



Pipelining vs. Pipeline-Breaker



Pipeline-Breaker

- Sortieren
- Duplikate eliminieren (unique, distinct)
- Aggregation: min, max, avg, ...
- Joins (je nach Implementierung)
- Union (je nach Implementierung)

Implementierung von Operatoren

- **Bei der Erzeugung eines physischen Anfrageplans muss entschieden werden wie genau die Anfrage ausgeführt werden soll**
- Welche Implementierungen gibt es für die diversen Operatoren?
- Welche Implementierung ist effizienter?
- Und wie kann dies überhaupt berechnet/vorhergesagt werden? (Kostenmodelle)
- Gibt es Indexstrukturen, die ausgenutzt werden können?
- Falls ja, macht es auch Sinn einen Index zu benutzen?
- ...

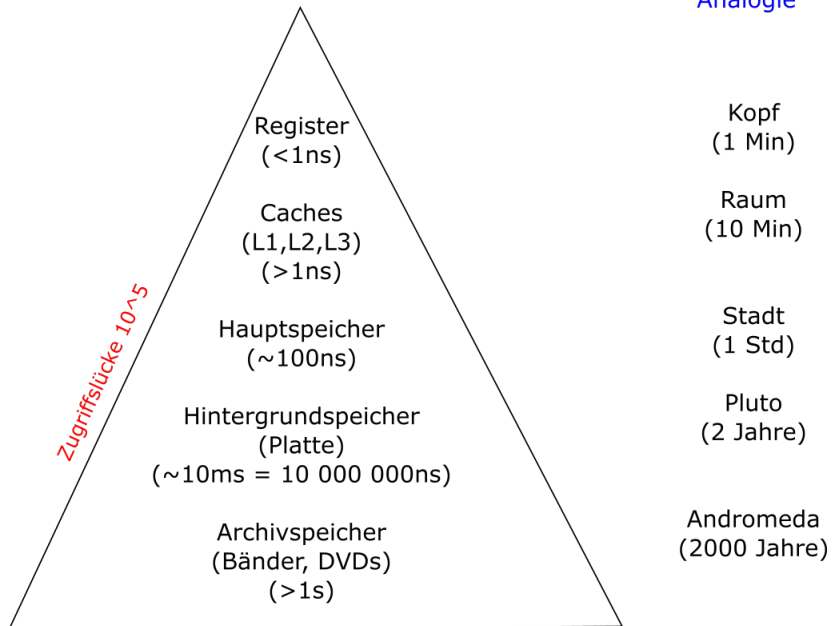
Implementierung von Operatoren wird in der **VL Datenbanksysteme** besprochen.

Was kostet wieviel?

- L1 cache reference 0,5 ns
- L2 cache reference 7 ns
- Main memory reference 100 ns
- Compress 1K bytes with Zippy 10 000 ns
- Send 2K bytes over 1 Gbps network 20 000 ns
- Read 1 MB sequentially from memory 250 000 ns
- Round trip within same datacenter 500 000 ns
- Disk seek 10 000 000 ns
- Read 1 MB sequentially from network 10,000,000 ns
- Read 1 MB sequentially from disk 30 000 000 ns
- Send packet CA → Netherlands → CA 150 000 000 ns

Numbers by Jeff Dean (Google)

Analogie



Problemstellung

Langfristige Speicherung und Organisation der Daten im Hauptspeicher?

- Speicher ist (noch) flüchtig!
- Speicher ist (noch) zu teuer und zu klein.
- Kostenverhältnis Festplatte vs. DRAM

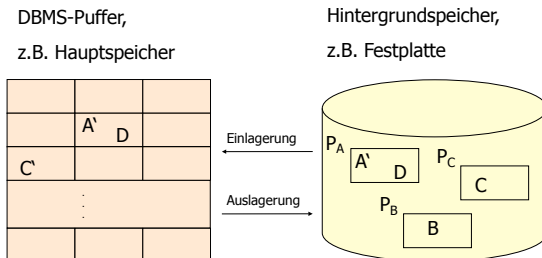
Mindestens zweistufige Organisation der Daten erforderlich

- Langfristige Speicherung auf großen, billigen und nicht-flüchtigen Speichern (Externspeicher)
- Verarbeitung erfordert Transport zum/vom Hauptspeicher

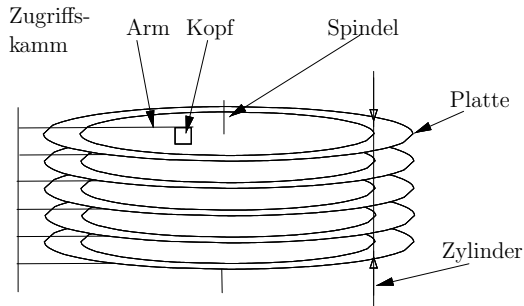
Zweistufige Speicherhierarchie

Vereinfachte E/A-Architektur:

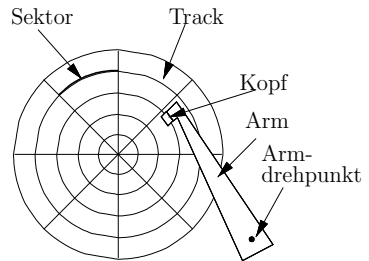
- Modell für Externspeicheranbindung
- Vor Bearbeitung sind die Seiten in den Datenbank-Puffer zu kopieren
- Geänderte Seiten müssen zurückgeschrieben werden



Aufbau einer (klassischen) Festplatte



a. Seitenansicht



b. Draufsicht

Aufbau Festplatte (Tracks, Sektoren, Zonen)

- **Track:** Teil eines Zylinders auf einer Platte
- **Sektor:** Teil eines Tracks. Anzahl Sektoren pro Track ursprünglich gleich für alle Tracks
- Aber, auf Zylinder/Tracks weiter “außen” passen mehr Sektoren. Daher, aktuelle Hardware, variable Anzahl von Tracks: äußere Tracks haben mehr Sektoren als innere Tracks; Zylinder sind in Zonen unterteilt.

Blöcke

- Aka. Sektoren, Aka. physical Record. Kleinste Transfereinheit bei Block-Storage-Devices. Seit wenigen Jahren typischerweise 4KB oder sonst (historisch) 512 Byte groß.
- **Achtung**, es gibt auch Unterschiede zu Blöcken bzw. Seiten des Dateisystems, dort kann ein Block auch mehrere Festplattenblöcke umfassen.

Beispiel

Die Festplatte SAMSUNG HD103SJ, die in meinem Desktop PC im Büro eingebaut ist hat laut (Linux tool) `hdparm -i` (oder `hdparm -I` für mehr Details):

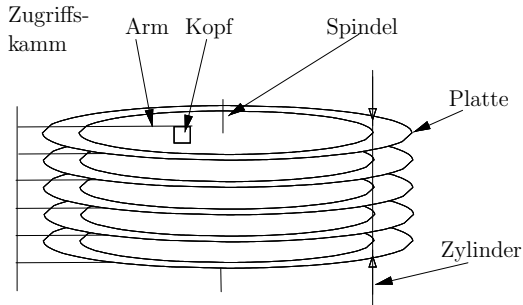
- 1953525168 Sektoren
- a 512 Bytes.

Das macht: 1 TB

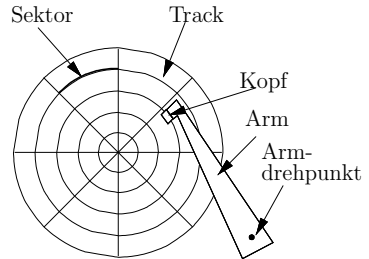
Ausgabe (Auszug) von `hdparm -I /dev/sda`

```
CHS current addressable sectors:    16514064
LBA   user addressable sectors:    268435455
LBA48 user addressable sectors:    1953525168
Logical Sector size:                512 bytes
Physical Sector size:               512 bytes
device size with M = 1024*1024:     953869 MBytes
device size with M = 1000*1000:     1000204 MBytes (1000 GB)
```

Aufbau einer (klassischen) Festplatte

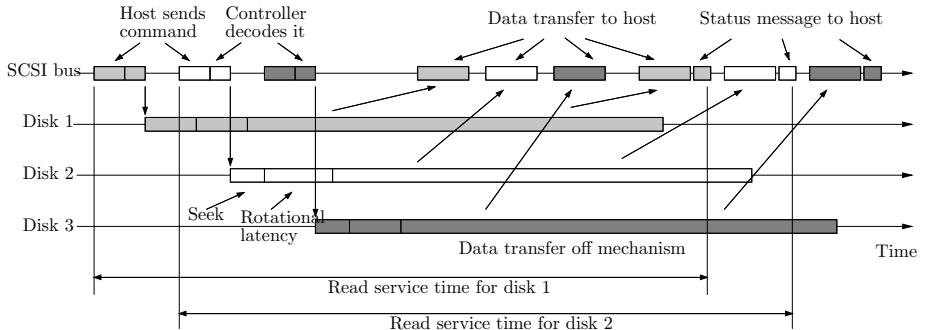


a. Seitenansicht



b. Draufsicht

Reading/Writing a Block



Schwierig die Kosten genau zu berechnen

Kosten eines Festplattenzugriffs hängen ab von:

- der aktuellen **Position des Lesekopfs**
- der **Position (Drehung/Winkel) der Platte**

Diese Informationen sind zur Zeit der Übersetzung der Anfrage **nicht bekannt**.

Daher: Kosten über mehrere Zugriffe (Mittel), einfaches Modell.

Einfaches Kostenmodell

Parameter des Kostenmodells:

- Durchschnittliche **Latenzzeit** (average latency time):
Durchschnittliche Zeit für Positionierung (seek+rotational delay)
 - Durchschnittliche Zugriffszeit für einen einzelnen Zugriff
- **Lese- / Schreibrate** (sustained read/write rate):
 - Nach Positionierung: Datentransferrate bei sequentiellm Zugriff

Beispiel: Performance Parameter für Beispiel-Festplatte

Modell aus 2004		
Parameter	Wert	Abkürzung
Kapazität (capacity)	180 GB	D_{cap}
Latenz (average latency time)	5 ms	D_{lat}
Leserate (sustained read rate)	100 MB/s	D_{srr}
Schreibrate (sustained write rate)	100 MB/s	D_{swr}

Dann: **Zeit um n Bytes zu lesen** ist geschätzt als $D_{\text{lat}} + n/D_{\text{srr}}$.

Sequenzielle und Wahlfreie Zugriffe (Sequential vs. Random I/O)

Es wird unterschieden zwischen zwei verschiedenen Arten von Zugriffen (I/O):

- **Sequenzielle (sequential)** I/O und
- **Wahlfreie (random)** I/O.

In unserem einfachen Kostenmodell:

- für sequenzielle Zugriffe: es gibt eine Positionierung des Lesekopfes, danach wird mit Leserate gelesen.
- für wahlfreie Zugriffe: es gibt eine Positionierung pro gelesener Einheit – typischerweise einer Seite von z.B. 8 KB

Beispielanwendung des einfachen Kostenmodells

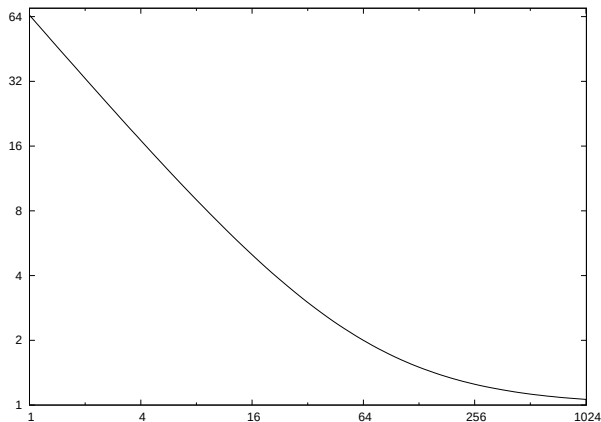
Lese 100 MB

- Sequenzielles Lesen: $5 \text{ ms} + 1 \text{ s}$
- Lesen durch wahlfreie Zugriffe (Seitengröße 8KB): 65 s

Time to Read 100 MB

x-Achse: Größe der zu lesenden Chunks in 8 KB

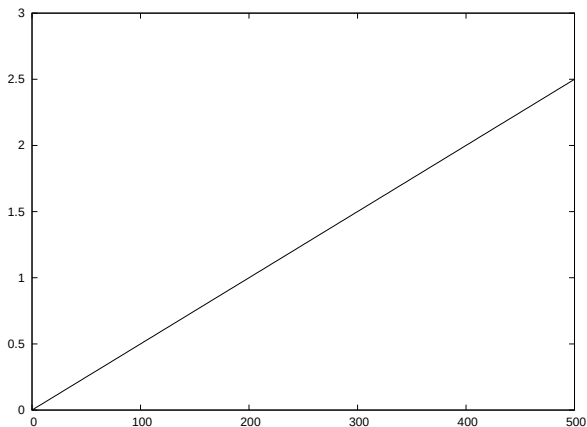
y-Achse: Sekunden



Time to Read n Random Pages

x-Achse: n

y-Achse: Sekunden



Beispiel: Berechnung Zugriffskosten

- Lesen einer Datei von 100 MB Größe, gespeichert in 12800 8 KB Seiten.
- In unserem einfachen Modell kostet das wahlfreie (random access) Lesen von 200 Seiten ungefähr genauso lange wie das Lesen der gesamten 100 MB im sequenziellen Modus.

Das heißt, das Lesen von 1/64 einer 100 MB Datei im wahlfreien Zugriff dauert genauso lang wie das Lesen der gesamten Datei im sequenziellen Modus.

Break-Even-Point im einfachen Kostenmodell

Sei a die Positionierungs-Zeit, s die Leserate, p die Seitengröße und d die Anzahl an fortlaufend abgelegten Bytes. Dann ist der **Break-Even-Point** gegeben durch

$$\begin{aligned}n * (a + p/s) &= a + d/s \\n &= (a + d/s)/(a + p/s) \\&= (as + d)/(as + p)\end{aligned}$$

a und s sind Parameter, die durch die Festplatte gegeben sind (also unveränderlich). Für gegebenes d hängt der Break-Even-Point nur von der Seitengröße ab.

Lessons Learned

- **Sequenzielles Lesen ist sehr viel schneller als wahlfreies Lesen.**
- **Das Datenbanksystem sollte dies idealerweise ausnutzen.**

Möglichkeiten

- Sorgfältig ausgewähltes physisches Layout auf der Festplatte (z.B. Zylinder- oder Track-Aligned, Clustering)
 - Festplatten Scheduling, multi-page Requests
 - Prefetching
 - Puffer
- und nicht zu vergessen:
- Effiziente und robuste Algorithmen (Implementierungen) der algebraischen Operatoren

Neuere Entwicklungen: SSDs

- Was ändert sich bei SSDs (Solid State Disks) gegenüber traditionellen mechanischen Festplatten?
- **Sehr viel mehr IOPs** (Input/Output Operations Per Second) möglich, Unterschied in Größenordnungen: Z.B. 1000 Leseoperationen a 4KB Block pro Sekunde einer SSD gegenüber 100 pro Sekunde einer normalen Festplatte.
- Dennoch sequenzielles Lesen günstiger als wahlfreies Lesen.
- **Was ändert dies für die Anfrageoptimierung?** Siehe Papier unten.

	Disk	Flash
Model	WD VelociRaptor 10Krpm	OCZ RevoDrive
Capacity	300gb	120gb
Price	\$164	\$300
Random Read	10ms	90µs
Seq. Read	120mb/s	190mb/s

Pelley et al. Do Query Optimizers Need to be SSD-aware?, ADMS Workshop, 2011.

Neuere Entwicklungen: In-Memory Datenbanken

In-Memory Datenbanken

- Verfügbarer RAM oft ausreichend groß, um gesamte Datenbank zu halten.
- Oft betrachtet in Zusammenhang mit Mehrkernsystemen und NUMA (**Non Uniform Memory Access**) Architekturen.
- Wo ist der neue **Flaschenhals für die Performance?**

Physische Organisation einer Datenbank

Das Datenbanksystem organisiert den physikalischen Speicher in verschiedene Schichten.

- **Datenbank**: Menge von Dateien
- **Datei**: Sequenz von Blöcken.
- **Segmente**: Organisationseinheit im DBMS (bzgl. Sperren, Rechten, etc.)

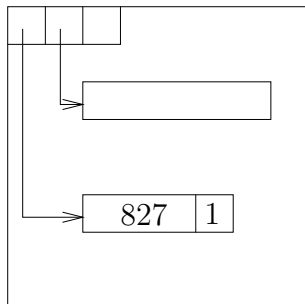
Zugriffseinheiten

- **Segmente**
- **Seiten** werden in Segmenten gespeichert
- **Seite** enthält Sätze
- **Satz**: In einer Seite gespeicherte Sequenz von Bytes. Menge von echten Daten, verschiedene Felder.
- Bzw. man redet auch von **Tupeln** im DB (Relationen) Kontext

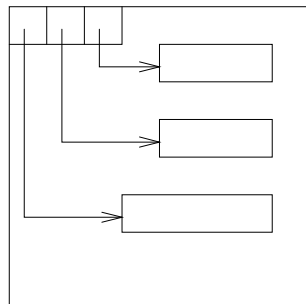
Seite

273	2
-----	---

273



827



- Seite (Page) ist organisiert in Anzahl von Bereiche (Slots)
- Slots zeigen auf Daten
- ... oder auch auf andere Seiten

Tuple Identifier (TID)

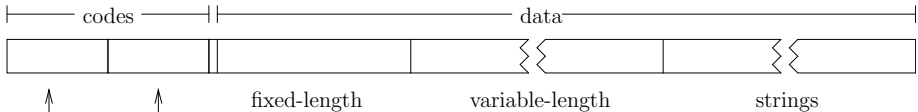
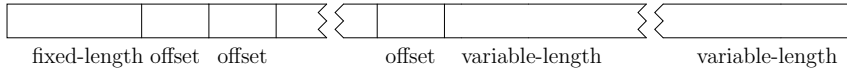
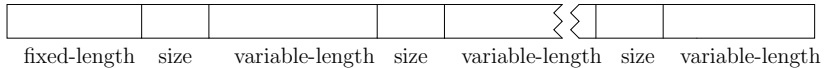
Ein TID ist eine Paar bestehend aus

- **Seiten ID** (z.B. Datei/Segment Nummer plus Nummer der Seite)
- **Slot Nummer**

TID wird manchmal auch Row Identifier (RID) genannt

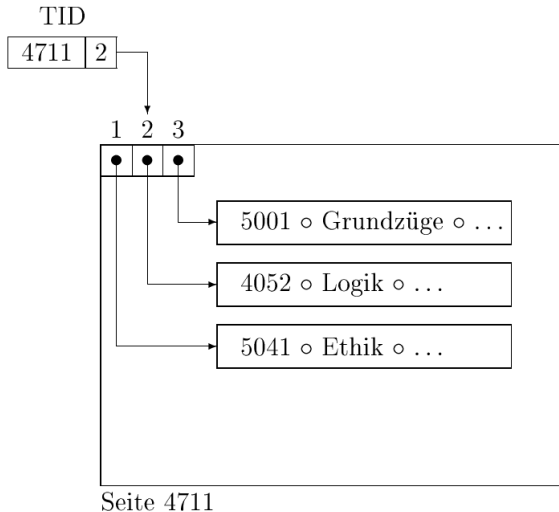
Satz Layout

Verschiedene mögliche Layouts:

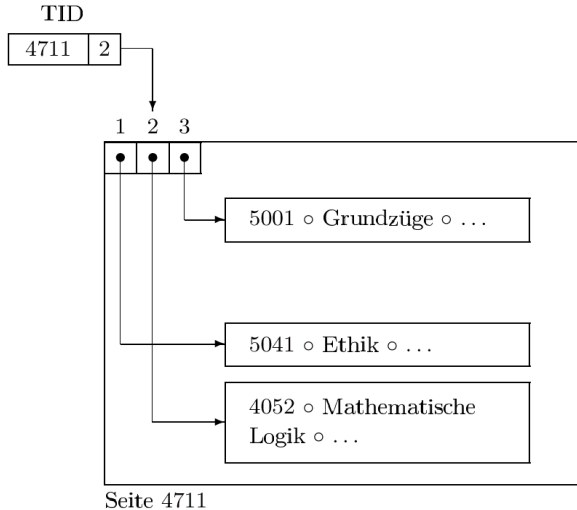


↑
length and offset encoding
encoding for dictionary-based compression

Speicherung von Tupeln auf Seiten



Verschieben von Tupeln innerhalb einer Seite



Verdrängen von Tupeln auf andere Seiten

- Falls eine Seite zu klein wird.
- Verschiebe Tupel in eine andere (z.B. neue, leere) Seite
- Füge in ursprünglicher Seite eine TID hinzu die auf den neuen Ort verweise

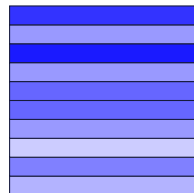
Was passiert bei mehrfachem Verschieben?

- Verweis in der ursprünglichen Seite wird angepasst.
- ⇒ Länge der Verweiskette auf zwei beschränkt

Organisation der Dateien

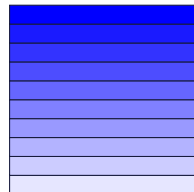
Haufen

- Einfachster Weg eine Datei zu organisieren ist neu ankommende Sätze in die Seite (den Block) am Ende der Datei einzufügen.
- Einfügen ist sehr effizient.
- Suche ist teuer.



Sortierte Dateien

- Gegeben ein Schlüssel anhand dessen Sätze geordnet werden können
- Effizientere Suche (binär) - Obwohl so nicht realisiert (siehe z.B. B+ Baum)



Beispiel: Sortierte Datei (Hier nach Name)

Name	MatrNr	Semester
------	--------	----------

Block 1

Aaron	443421	10
Adam	233499	1
...		
Acosta	561921	1

Block 2

Allen	581722	9
Anderson	339163	8
...		
Archer	965492	1

Block 3

Arnold	672961	3
Arnold	759311	1
...		
Atkins	173522	8

⋮

Block n

Wright	672961	4
Wyatt	197646	7
...		
Zimmer	524145	12

Suche in Dateien

*“If you don’t find it in the index,
look very carefully through the entire catalog”
— Sears, Roebuck, and Co., Consumers’ Guide, 1897*

(aus dem Buch von Ramakrishnan & Gehrke)

Lineare Suche

Binäre Suche in sortierten Dateien

Suche via Indexen

Grundidee eines Index

Abbildung: Schlüssel $\rightarrow$ Menge von Einträgen

Beispiele:

- MatrikelNummer $\rightarrow$ persönliche Daten des Studenten
- PLZ $\rightarrow$ Name und andere Informationen einer Stadt
- Term $\rightarrow$ alle Dokumente in denen diesesr Term enthalten ist

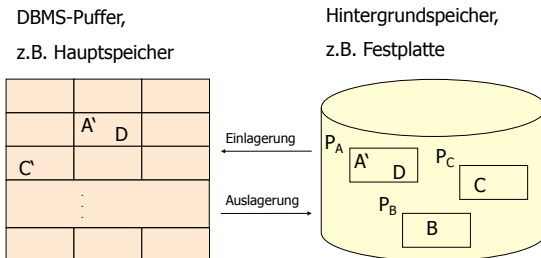
Natürlich möchte man bei diesen häufig auftretenden Anfragen die Antwortzeit gering halten.

Dafür wird ein Index angelegt: Ein Index materialisiert diese Abbildung!

Betrachten wir im nächsten Kapitel.

Datenbank-Pufferverwaltung

- **Motivation:** Bereits erwähnte Zugriffslücke zwischen Hauptspeicher und Festplatte (Externspeicher)
- **Idee:** Halte Seiten, auf die zugegriffen wurde, in einem Puffer im Hauptspeicher
- Dieser Puffer kann durchaus sehr groß sein (hunderte Megabyte oder viele Gigabytes). Trotzdem viel kleiner als DB selbst.



Datenbank-Pufferverwaltung (2)

5 Minuten Regel

“Pages referenced every five minutes should be memory resident.”

Siehe Papier von Jim Gray & Franco Putzolu:

<http://www.hpl.hp.com/techreports/tandem/TR-86.1.pdf>

Empfehlung zum Thema Datenbank-Pufferverwaltung: Das Buch von Härder und Rahm: “Datenbanksysteme - Konzept und Techniken der Implementierung” ist hier sehr ausführlich.

Ersetzungsstrategien: Konzept und Klassifizierung

Wenn eine Seite nicht im Puffer auffindbar ist wird sie in Puffer eingetragen. Falls dieser bereits voll ist, muss eine andere Seite weichen, aber welche?

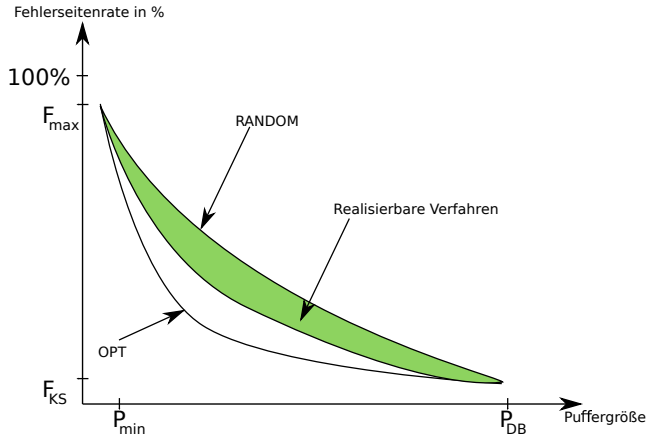
Klassifizierung von **Strategien** anhand ...

- ob das **Alter seit Einlagerung**, seit letztem Zugriff oder überhaupt nicht, und
- ob alle **Referenzen**, die letzte Referenz oder keine

bei der Auswahlentscheidung (welche Seite ersetzt werden soll) zum Tragen kommt.

Erste (triviale) Idee: Zufälliges Ersetzen von Seiten (RANDOM Strategie).

Optimales vs. Zufälliges vs. Realisierbare Verfahren



$P_{\min}$ = minimale Größe des DB-Puffers

P_{DB} = Datenbankgröße

F_{KS} = Fehlerseitenrate bei Kaltstart

FIFO

First-In, First-Out

- Ersetzt diejenige Seite, die am längsten im Puffer ist

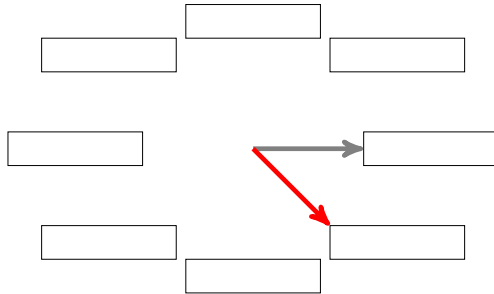


Illustration: Kreisförmige Anordnung. Zeiger verweist auf den ältesten Eintrag. Grauer Zeiger= alt. Roter Zeiger=neu.

Least-Frequently-Used (LFU)

- **Ersetzt Seite mit der geringsten Referenz- (Zugriffs-) Häufigkeit**
- Für jede Seite wird ein **Zugriffszähler** (aka. Referenzzähler RZ) geführt.
- **Die Seite mit dem kleinsten Zählerstand wird ersetzt.**

Seiten, auf die kurzzeitig sehr sehr oft zugegriffen wurde bleiben sehr lange im Puffer, auch wenn nicht mehr wirklich benutzt.

Alter des Zugriffs (bzw. der letzten Zugriffe) wird nicht berücksichtigt.

- “Problem” kann durch periodisches Herabsetzen der Referenzzähler adressiert werden.

Least-Recently-Used (LRU)

- **Ersetzung basierend auf Zeit seit dem letzten Zugriff auf Seite.**
- Halte Seiten in Form eines Stacks:
 - Eine Seite kommt bei jeder Referenz auf oberste Position
 - Seite auf der untersten Position des Stacks wird bei Bedarf ersetzt

Beispiel:

Zugriff (in dieser Reihenfolge) auf Seiten:

A, B, C, D, A, C, A, B, B, B, C, D, A. Puffer hat Platz für 3 Seiten. Seite E soll eingelagert werden. Welche Seite muss weichen?

Beobachtung:

Was passiert, wenn in einer Leseoperation von der Festplatte viele neue Seiten gelesen werden?

Wieso ist das problematisch?

Theoretische Sicht

Wir betrachten einen **String von Referenzen auf Seiten**

$$r_1, r_2, \dots, r_t, \dots$$

wobei $r_t = p$ bedeutet, dass auf Seite p zugegriffen wurde.

Zu einem Zeitpunkt t nehmen wir an, dass jede **Seite eine gewisse Wahrscheinlichkeit b_p besitzt als nächstes aufgerufen zu werden**, d.h.

$$Pr(r_{t+1} = p) = b_p.$$

Interarrival Time

- **Wie viele Zugriffe liegen zwischen den Zugriffen auf eine Seite p ?** Genau b_p^{-1} .

Interarrival Time Annäherung durch LRU

- Interarrival Time: Zwischen zwei Zugriffen auf Seite p liegen b_p^{-1} Zugriffe.
- **Diejenigen Seiten mit kleinster Interarrival Time bzw. größter Wahrscheinlichkeit b_p sollten im Puffer gehalten werden.**
- Dies wird **bei LRU approximiert** durch die Seiten mit dem Zeitpunkt des letzten Zugriffs auf Seite.

Weitere Ersetzungsstrategien werden in der VL Datenbanksysteme im Wintersemester vorgestellt.

Indexe

Überlegungen zu Performance

Beobachtung

- Je schneller die Anfrage berechnet wird, desto besser.
- Daten passen nicht in den Hauptspeicher.
- Es liegen Größenordnungen zwischen Performanz des Hauptspeichers und der Festplatte.
- Zugriffe sind unglaublich teuer!
- Insbesondere die wahlfreien (random access).

Was muss getan werden?

- Effiziente Speicherung auf Festplatte
- Wie findet man die gesuchten Daten möglichst schnell?
- Welche Garantien können bzgl. "Laufzeit" gegeben werden?

Grundidee eines Index

Abbildung: Schlüssel $\rightarrow$ Menge von Einträgen

Beispiele:

- MatrikelNummer $\rightarrow$ persönliche Daten des Studenten
- PLZ $\rightarrow$ Name und andere Informationen einer Stadt
- Term $\rightarrow$ alle Dokumente in denen dieser Term enthalten ist

Natürlich möchte man bei diesen häufig auftretenden Anfragen die Antwortzeit gering halten.

Dafür wird ein Index angelegt: Ein Index materialisiert diese Abbildung!

Was ist mit der binären Suche?

- Idee: Sortiertes Array, dann binär suchen
 - $O(n \log n)$ Kosten für das Sortieren
 - was passiert, wenn ein neuer Datensatz eingefügt werden muss?
- Binärer Suchbaum, am besten balanciert (AVL oder rot-schwarz Baum)

Aber....

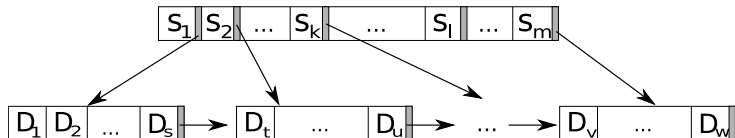
- binäre Suchbäume sind für DBMS ungeeignet
 - zu hoher Speicherverbrauch
 - schwer abzubilden auf Externspeicher
 - zu viele Cache-Misses (warum?)

⇒ viel zu langsam!

ISAM

Index-Sequential Access Method (ISAM)

Besteht aus Schlüsseln S_j und Datensätzen D_i .



- Sowohl Schlüssel als auch Daten werden geordnet abgespeichert.
- **Suche:**
 - Binäre Suche in Schlüsseln zur gewünschten Position
 - Sequentielles Lesen in Datenseiten
- **Einfügen:**
 - Auffinden der Einfügeposition (wie bei Suche)
 - Was passiert wenn die Seite, in die eingefügt werden soll, voll ist?
Nicht gut ...
- **Löschen:**
 - Auffinden der Löschposition (wie bei Suche und Einfügen)
 - Löschen des Datensatzes. Was passiert wenn die Seite leer wird?

Bäume

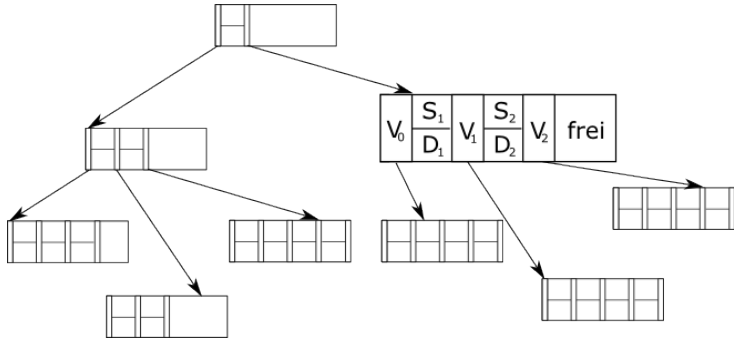
Beobachtung

- Binäre Bäume nicht optimal für Festplatten
- Wichtig: Anpassung der Kapazität der Knoten an Größe der Seiten.
- Warum?

Fanout

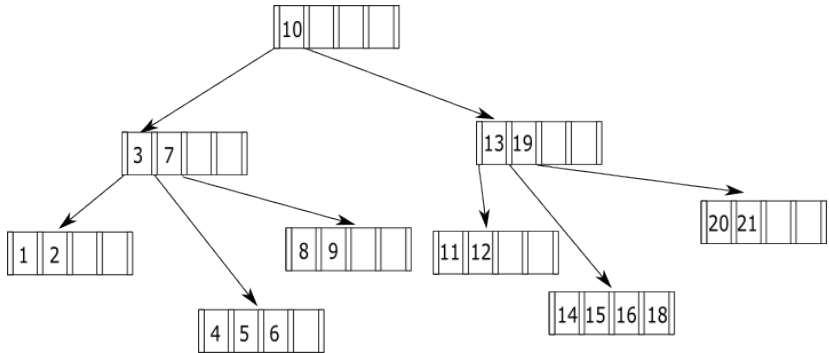
- Je breiter der Baum desto weniger Tief.
 - Je flacher desto weniger “Sprünge” zwischen Knoten
- ⇒ Weniger Zugriffe auf Seiten auf der Festplatte.

B Baum



- Verweise V_j
- Schlüssel S_i
- Datensätze D_k

B Baum Beispiel



Schlüssel in den inneren Knoten zwei Funktionen haben. Sie identifizieren Daten(sätze), und sie dienen als Wegweiser in der Baumstruktur.

Einfügen von Schlüsseln

1. **Führe eine Suche nach dem Schlüssel durch**; diese endet (scheitert) an der Einfügestelle
2. **Füge den Schlüssel dort ein.**
3. **Ist der Knoten überfüllt, teile ihn:**
 - Erzeuge einen neuen Knoten und belege ihn mit den Einträgen des überfüllten Knotens, deren Schlüssel größer ist als der des mittleren Eintrags.
 - Füge den mittleren Eintrag im Vaterknoten des überfüllten Knotens ein.
 - Verbinden den Verweis rechts des neuen Eintrags im Vaterknoten mit dem neuen Knoten
4. **Ist der Vaterknoten jetzt überfüllt?**
 - Handelt es sich um die Wurzel, so lege eine neue Wurzel an
 - Wiederhole Schritt 3 mit dem Vaterknoten

B Baum: Eigenschaften

- Balanciert

Verbesserung?

- Wie kann der Fanout erhöht werden?
- Platz in den Knoten einsparen.
- Also: Keine Daten in den Knoten (nur in der Wurzel), nur Verweise

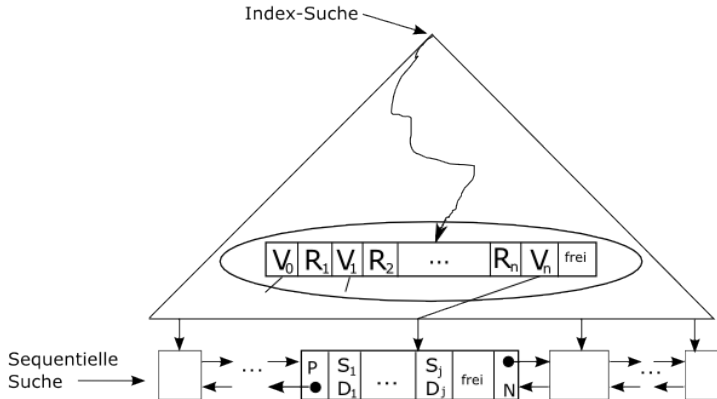
⇒ B+ Baum

Schöne Beschreibung von Prof. Härder zu B und B+ Bäumen:

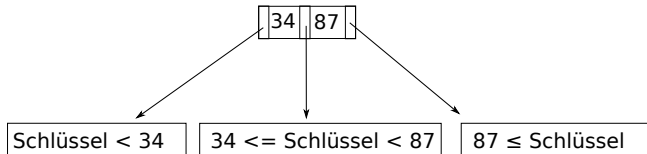
<http://www.lgis.informatik.uni-kl.de/cms/fileadmin/courses/ss2007/Informationssysteme/addons/mehrwegbaeume.full.pdf>

B+ Baum

- “Hohler” Baum: **Daten nur in den Blättern**
- Suche muss also immer bis zu den Blättern laufen
- Aufbau: Referenzschlüssel R_j , Schlüssel S_k , Daten D_i , Zeiger V_m
- **Blattknoten sequentiell verbunden!**

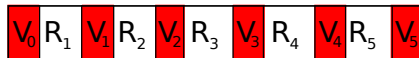


B+ Baum: Verweise



B+ Baum: Knotenstruktur

Die Struktur eines inneren Knotens:



In diesem Beispiel hat der Knoten 6 Verweise auf Kindknoten, d.h. der sogenannte **Fanout** ist 6.

B+ Baum: Eigenschaften

Ein B+ Baum vom Typ (k, k^*) hat folgende Eigenschaften:

1. Jeder Weg von der Wurzel zu einem Blatt hat die gleiche Länge
2. Jeder Knoten - außer Wurzeln und Blättern- hat mindestens k und höchstens $2k$ Einträge. Blätter haben mindestens k^* und höchstens $2k^*$ Einträge. Wurzel hat entweder maximal $2k$ oder sie ist ein Blatt mit maximal $2k^*$ Einträgen.
3. Jeder Knoten mit n Einträgen, außer den Blättern, hat $n + 1$ Kinder.

B+ Baum: Eigenschaften

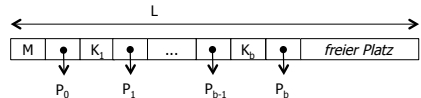
4. Seien $R_1, \dots, R_n$ die Referenzschlüssel eines inneren Knotens (d.h. auch der Wurzel) mit $n + 1$ Kindern. Seien $V_0, V_1, \dots, V_n$ die Verweise auf diese Kinder.
- (a) V_0 verweist auf den Teilbaum der Schlüssel kleiner R_1
 - (b) V_i ($i = 1, \dots, n - 1$) verweist auf einen Teilbaum, dessen Schlüssel zwischen R_i und R_{i+1} liegen (einschließlich R_i).
 - (c) V_n verweist auf den Teilbaum mit Schlüsseln größer gleich R_n .

B+ Baum: Knotenformate

M : Kennung des Seitentyps + Anzahl Einträge; feste Länge L

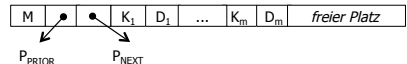
Innere Knoten

- $k \leq b \leq 2k$



Blattknoten

- $k^* \leq m \leq 2k^*$
- Zeiger P_{prior} und P_{next} dienen dazu die Blattknoten linear durchsuchbar zu machen.



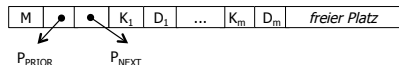
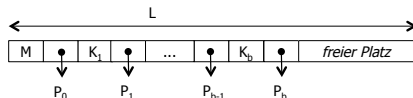
B+ Baum: Knotenformate (2)

M : Kennung des Seitentyps + Anzahl Einträge; feste Länge L , Einträge für Schlüssel (K), Daten (D) und Verweise (p)

Innere Knoten

- $k \leq b \leq 2k$
- $L = l_M + l_p + 2k(l_K + l_p)$

$$k = \left\lfloor \frac{L - l_M - l_p}{2 \times (l_K + l_p)} \right\rfloor$$



Blattknoten

- $k^* \leq m \leq 2k^*$
- $L = l_M + 2l_p + 2k^*(l_K + l_D)$

$$k^* = \left\lfloor \frac{L - l_M - 2l_p}{2 \times (l_K + l_D)} \right\rfloor$$

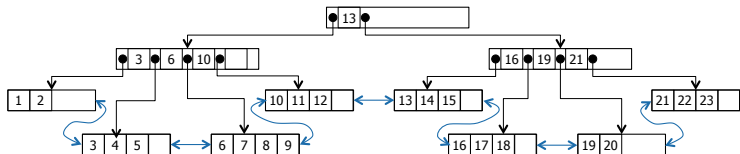
Typische Größen (in Byte):

$L = 8192$, $l_M = 4$, $l_p = 4$, $l_K = 8$,
 $l_D = 88$

Innere Knoten: $k = 341$;

Blatt: $k^* = 42$

B+ Baum

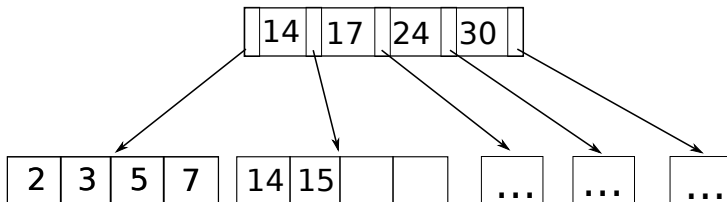


- Jeder Knoten hat die Größe eines I/O Blocks.
- Ein Knoten ist zu mindestens 50% gefüllt.
- Das Lesen eines Knotens verursacht typischerweise einen wahlfreien Zugriff auf Festplatte
- Ein B+ Baum ist i.d.R. sehr flach, d.h. Suche verursacht nur wenige wahlfreie Zugriffe.
- Zudem, die ersten 1-2 Ebenen des Baums sind i.d.R. im Hauptspeicher "gecached".

B+ Baum: Komplexeres Einfügebeispiel

Einfügen von 16 und 8

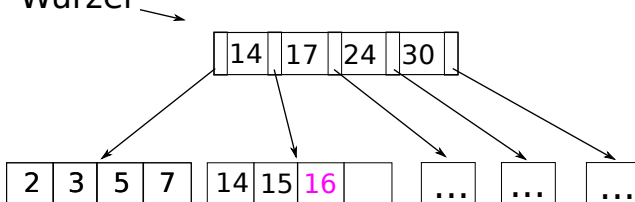
Wurzel



B+ Baum: Komplexeres Einfügebeispiel

Einfügen von 16 und 8

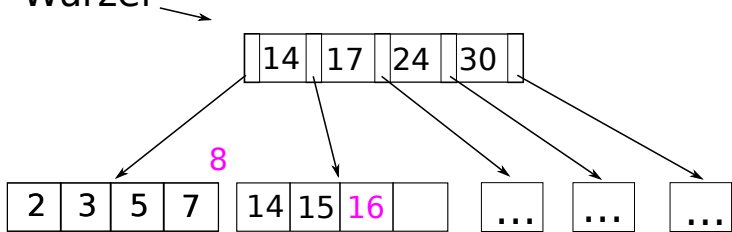
Wurzel



B+ Baum: Komplexeres Einfügebeispiel

Einfügen von 16 und 8

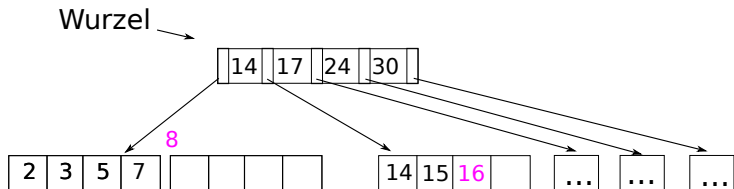
Wurzel



Zum Einfügen der 8 in den Blattknoten ist kein Platz vorhanden.

B+ Baum: Komplexeres Einfügebeispiel

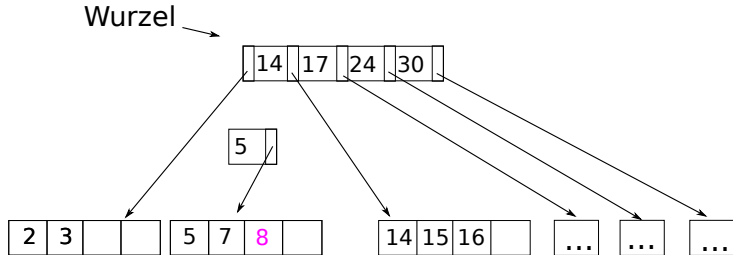
Einfügen von 16 und 8



Es wird also ein neuer Blattknoten erzeugt.

B+ Baum: Komplexeres Einfügebeispiel

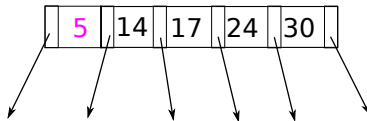
Einfügen von 16 und 8



... und der mittlere Wert nach oben kopiert.

B+ Baum - Komplexeres Einfügebeispiel

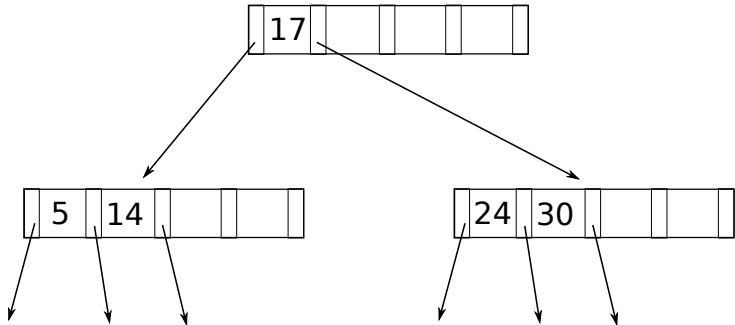
Einfügen von 16 und 8



Dieser innere Knoten ist nun voll und muss ebenfalls gesplittet werden. Dabei wird der mittlere Wert (17) in einen neuen inneren Knoten eingefügt.

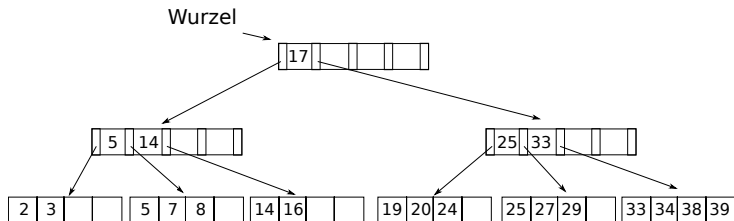
B+ Baum: Komplexeres Einfügebeispiel

Einfügen von 16 und 8



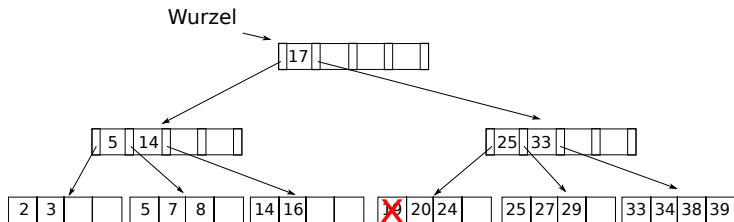
B+ Baum: Löschen Beispiel

Löschen von 19 und 20



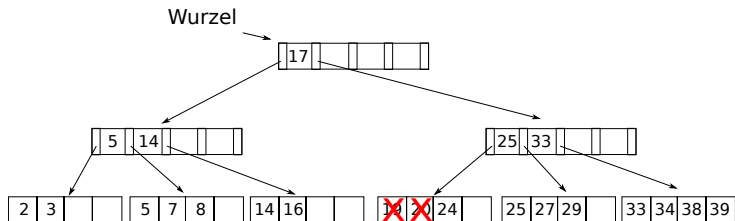
B+ Baum: Löschen Beispiel

Löschen von 19 und 20



B+ Baum: Löschen Beispiel

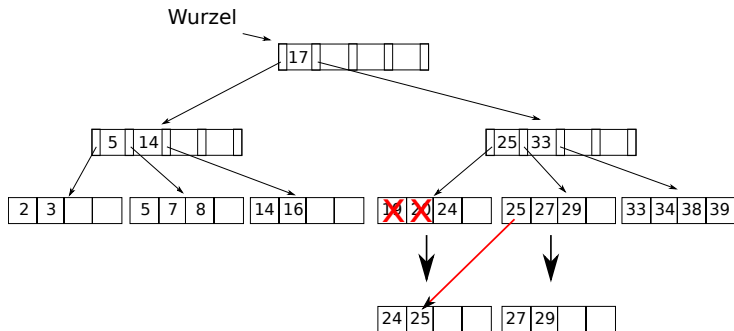
Löschen von 19 und 20



Verletzung von Bedingung, dass jedes Blatt mindestens $k^* = 2$ Einträge haben muss. Also muss ausgeglichen werden.

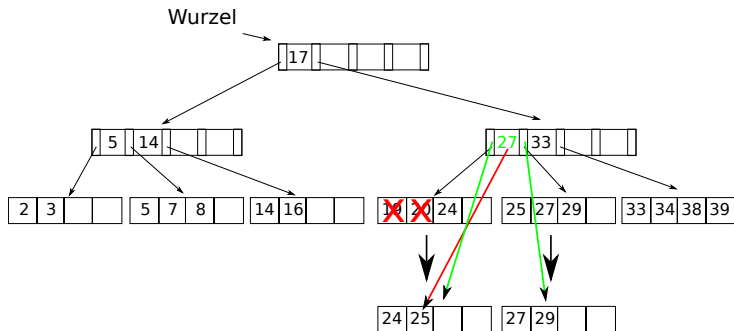
B+ Baum: Löschen Beispiel

Löschen von 19 und 20



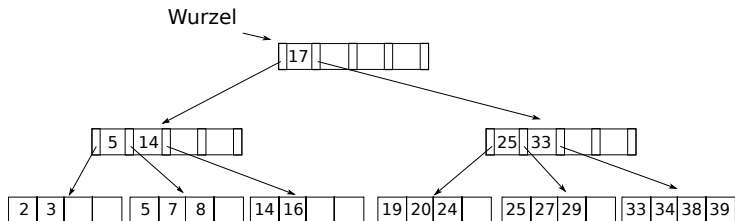
B+ Baum: Löschen Beispiel

Löschen von 19 und 20



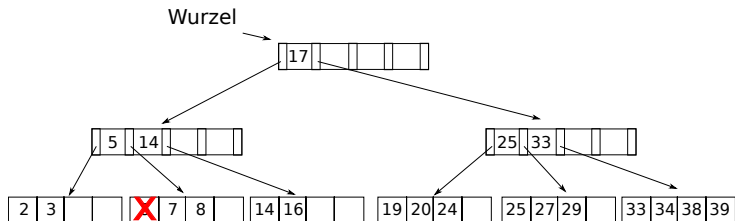
B+ Baum: Löschen Beispiel 2

Löschen von 5 und 7



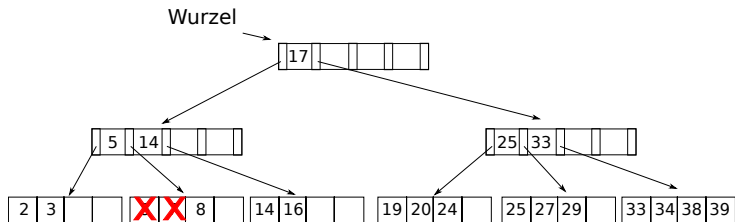
B+ Baum: Löschen Beispiel 2

Löschen von 5 und 7



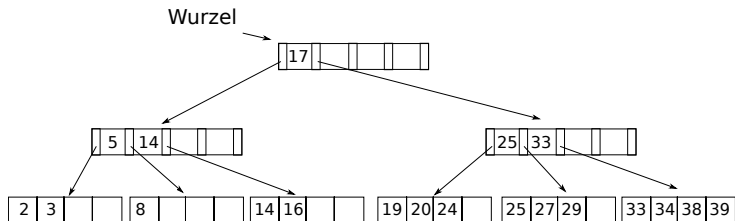
B+ Baum: Löschen Beispiel 2

Löschen von 5 und 7



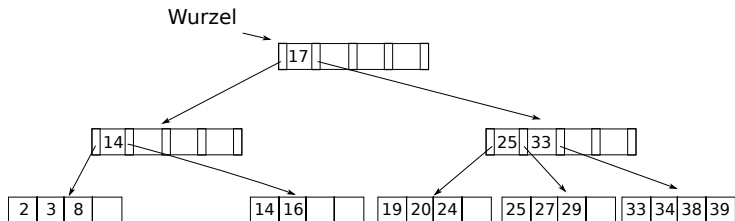
B+ Baum: Löschen Beispiel 2

Löschen von 5 und 7



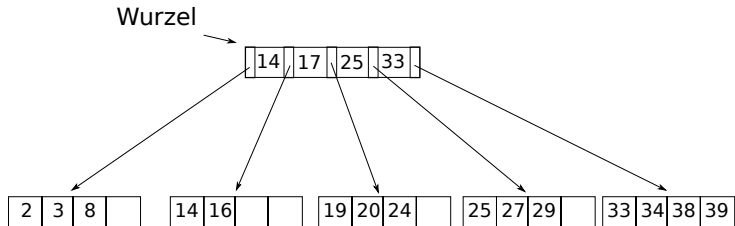
B+ Baum: Löschen Beispiel 2

Löschen von 5 und 7



B+ Baum: Löschen Beispiel 2

Löschen von 5 und 7



B* Baum

Statt wie im B+ Baum eine volle Seite auf zwei Seiten aufzuteilen, werden im B* Baum die Datensätze aus m Geschwisterknoten gleichmässig auf $(m + 1)$ Seiten aufgeteilt.

Dies **verbessert die Speicherplatzausnutzung (SPAN)**:

SPAN	$m = 1$	$m = 2$	m
worst case:	$\frac{1}{1 + 1}$	$\frac{2}{2 + 1}$	$\frac{m}{m + 1}$
avg. case:	$\ln 2 (= 69\%)$	81%	$m * \ln\left(\frac{m + 1}{m}\right)$

Allerdings werden die Kosten für das Einfügen erhöht, so dass in der Praxis $m \leq 3$.

Präfix B-Baum

Beobachtung

- Die Einträge in den inneren Knoten eines B+ Baums werden nur zur Navigation während der Suche benutzt.
- Einträge bestehen aus Verweisen (Pointern) und aus Schlüsseln.
- Pointer sind nur ein paar Bytes groß, aber Schlüssel können sehr lang sein (z.B. Donaudampfschiffahrtsgesellschaftskapitän)
- Wenn Platz gespart sind, passen mehr Einträge in einen Knoten. D.h. Baum wird flacher!

Idee

- Verwende nicht die gesamten Schlüssel, wie sie im Fall eines B+ Baums anfallen würden, sondern geeignete Präfixe.
- Ein Präfix B Baum ist ein B+ Baum, bei dem der Index-Teil des B+ Baums durch einen Index-Teil Präfix B-Baum ersetzt wurde.

Literatur: R. Bayer und K. Unterauer. Prefix B-trees. TODS, 1977.

Beispiel Split und Separator

Gegeben folgender volle Blattknoten eines B+ Baums:

Bigbird, Burt, Cookiemonster, Ernie, Snuffleopogus

Um den Schlüssel “Grouch” in diese Seite einzufügen, muss sie wie folgt in zwei Seiten aufgespalten werden:

Bigbird, Burt, Cookiemonster

Ernie, Grouch, Snuffleopogus

Statt nun “Ernie” als Schlüssel im Index zu benutzen, würde es auch ausreichen “D” oder “E” für den gleichen Zweck zu benutzen.

Separator

Im Allgemeinen kann man jeden String s benutzen, falls er die Eigenschaft hat, dass

$$\text{Cookiemonster} < s \leq \text{Ernie}$$

s kann dann im Index gespeichert werden, um die obigen beiden Seiten zu trennen.

Idee

Wenn man den möglichst kürzesten Separator benutzt, geht nur ein Minimum an Platz innerhalb einer Seite verloren. Der Baum wird flacher (weil höherer “Fan-Out”)

Präfix-Eigenschaft

- Sind die Schlüssel Worte über einem Alphabet und ist die Ordnung der Schlüssel in alphabetischer (lexikographischer) Ordnung, dann gilt folgende Eigenschaft:

Präfix-Eigenschaft

Seien x und y zwei Schlüssel, so dass $x < y$. Dann gibt es einen eindeutigen Präfix $\hat{y}$ von y mit:

- (a) $\hat{y}$ ist ein Separator zwischen x und y und
- (b) kein anderer Separator zwischen x und y ist kürzer als $\hat{y}$.

Es kann auch Sinn machen Index-Knoten nicht genau in der “Mitte” zu splitten, sondern leichte Abweichung davon zuzulassen. Wieso? Was ist ein guter Separator zwischen Donaudampfschiffahrtsgesellschaft und Donaudampfschiffahrtsgesellschaft?

Bulkloading

Problemstellung

- Gegeben eine große Menge an Datensätzen
- Wie kann ein B+ Baum darüber aufgebaut werden?
- Das Problem ist natürlich allgemeiner und tritt auch bei anderen Bäumen/Indexstrukturen auf.

Naive Möglichkeit

- Datensätze einzeln nach und nach in B+ Baum einfügen
- D.h. es wird immer von der Wurzel ab nach der passenden Einfügestelle gesucht
- Nicht sehr effizient (auch wenn die oberen Ebenen in einem DB-Puffer sind)

Bulkloading

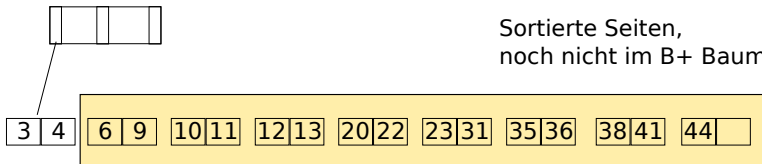
Idee

- Sortiere Daten vor
- Dann baue Baum auf, indem Datensätze der Reihe nach hinzugefügt werden

Ablauf

Der B+ Baum kann 2 Einträge pro Seite speichern. Wir fügen je ganze Seiten ein, hier zuerst die Seite mit Einträgen 3 und 4.

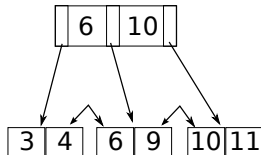
Wurzel



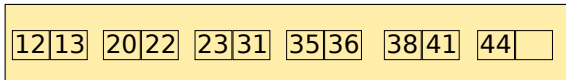
Bulkloading - Beispiel (2)

- Nachdem die beiden folgenden Seiten eingefügt worden sind ist die Wurzel nun voll.
- Bei dem Einfügen von (12,13) muss die Wurzel also aufgeteilt werden.

Wurzel

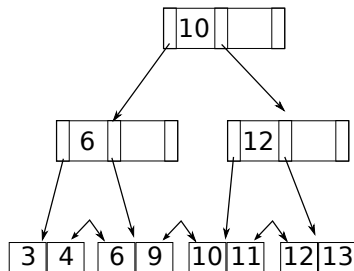


Sortierte Seiten,
noch nicht im B+ Baum

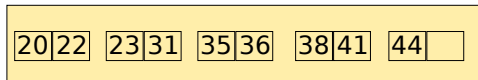


Bulkloading - Beispiel (3)

Wurzel



Sortierte Seiten,
noch nicht im B+ Baum

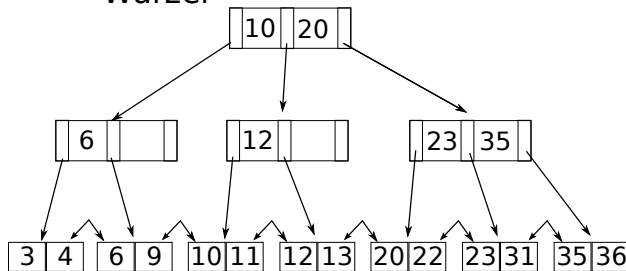


- Beim Aufteilen der Seiten wurden diese hier gleichmäßig auf die neuen Seiten unter der Wurzel aufgeteilt
- Im Allgemeinen hätte man auch anders aufteilen können.
- Z.B. nach einem bestimmten Füllgrad (z.B. 80%) oder man hätte auch alle alten Einträge in der linken Seite belassen können.

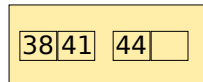
Bulkloading - Beispiel (4)

- Index-Einträge für die Blätter werden immer in den am weitesten rechts stehenden Index-Knoten direkt über der Blatt-Ebene eingefügt.
- Sollte dieser voll sein, so muss aufgeteilt werden.

Wurzel

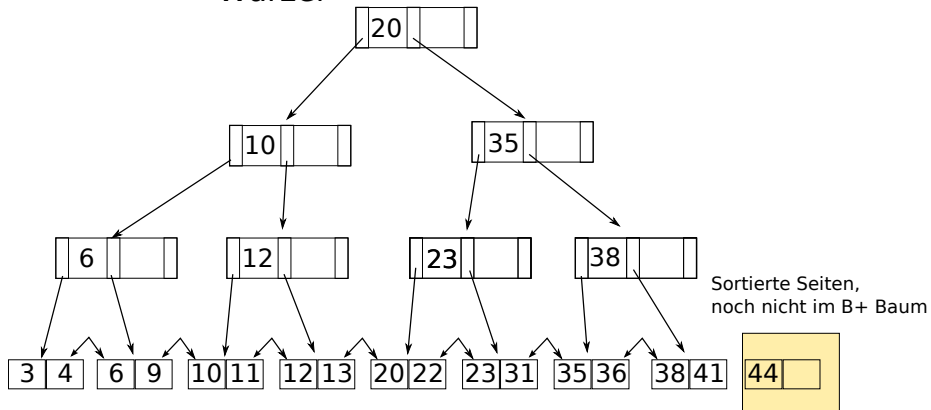


Sortierte Seiten,
noch nicht im B+ Baum



Bulkloading - Beispiel (5)

Wurzel



Hashverfahren

- Gegeben: Domäne der Schlüssel $S = \{0, \dots, M - 1\}$
- M kann sehr groß sein
- Anfragen der Form: welche Datensätze haben den Schlüssel x ?
- Sogenannte **Punktanfragen**

Grundidee

- **Abbildung auf Adressbereich** A , $|A| \ll |S|$
- $h : S \rightarrow A$ ordnet jedem Schlüssel einen Wert in A zu
- Gewünschte Eigenschaften:
 - Verteile Schlüssel gleichmäßig über Adressbereich
 - **Vermeide Kollisionen**
 - Kollision: verschiedene Werte in S werden auf den gleichen Wert in A abgebildet

Einheiten im Adressbereich werden auch Eimer (Englisch: **Buckets**) genannt.

Hashverfahren (2)

Man nimmt an, dass die Zahl der benutzten Schlüssel K viel kleiner ist als die Domäne S .

Abbildung h muss also nur die benutzten Schlüssel in K ordentlich abbilden.

Gefahr von Kollisionen

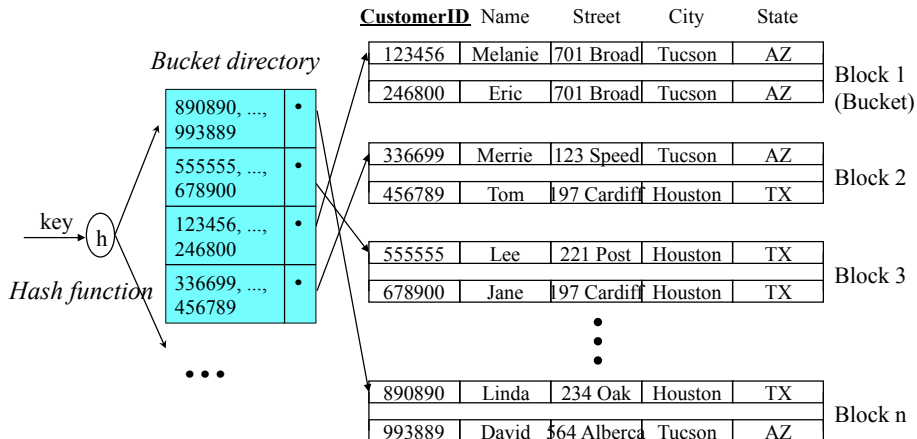
Möglichkeiten die Hashfunktion zu wählen

- $h(k) = k \bmod q$, q bestimmt Größe des Adressbereichs
- $h(k) = k^2$ oder $h(k) = c * k$, dann Auswahl gewisser Bitpositionen, um gültigen Adressbereich zu erhalten.

Fortgeschrittene Hashmethoden ($\Rightarrow$ DBS VL im Winter)

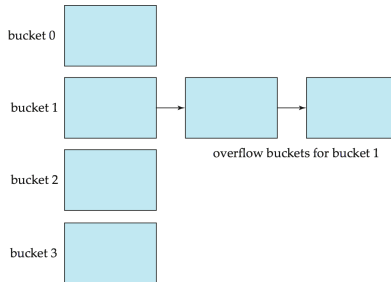
- Diese Verfahren sind sogenannte statische Verfahren
- Es gibt aber auch dynamische Verfahren, die die Hashfunktion an den Bedarf anpassen (Erweiterbares Hashing).

Statischer Hash Index: Beispiel



Statischer Hash Index

- Suche (Lookup)
 - Ein Zugriff auf das Verzeichnis (directory)
 - Ein Zugriff auf die eigentliche Datei
- Performance hängt von Wahl der Hash-Funktion ab
- Überlauf von Buckets
 - Zu viele verschiedene Schlüssel-Werte werden auf gleichen Bucket abgebildet
 - Lösung: **Überlaufbehandlung** mittels Verkettung von Überlauf-Buckets



Indexe in Postgres: B+ Baum

```
create table MeineTabelle (  
    ID int,  
    major int,  
    minor int,  
    name varchar  
);
```

B+ Baum mit Schlüssel ID

```
create index MeineTable_i1 on MeineTabelle ID;
```

B+ Baum mit Schlüssel (major, minor)

```
create index MeineTable_i2 on MeineTabelle (major, minor);
```

Indexe über mehrere Spalten

Sogenannte **Composite-Key Indexe**.

Angabe einer Reihe (**Achtung! Reihenfolge ist wichtig!**) von Spalten.

```
create index indexName on MeineTabelle (att1 , att2, att3);
```

Tupel werden dann im Index sortiert anhand dieser Attribute,
“Lexikographische Ordnung”: att1 ist primäres Kriterium, gefolgt von
att2, etc.

Optional mit Ordnung ASC oder DESC der einzelnen Attribute. Default
ist ASC. Zum Beispiel:

```
create index indexName on MeineTabelle (att1 DESC , att2, att3);
```

Indexe in Postgres: Hash

- **create index** name **on** table **using** hash (column);

Ein Hash-Index macht Sinn, falls:

- **Keine Bereichsanfragen** benötigt werden
- Order by (Schlüssel) nicht benötigt wird
- Keine oder nur kleine Joins über den Schlüssel notwendig sind

Beispiel Datenbank



- **customer** (customerID, name, street, city, state)
- **reserved** (customerID, filmID, resDate)
- **film** (filmID, title, kind, rentalPrice)

Verschiedene Selektionen

- Primärschlüssel, Punktanfrage

$$\sigma_{filmID=2}(film)$$

- Punktanfrage

$$\sigma_{title='Terminator'}(film)$$

- Bereichsanfrage

$$\sigma_{1 < rentalPrice < 4}(film)$$

- Konjunktion (d.h. logisches und)

$$\sigma_{kind='F' \wedge rentalPrice=4}(film)$$

- Disjunktion (d.h. logisches oder)

$$\sigma_{rentalPrice < 2 \vee kind='D'}(film)$$

Ziel

Ersetze die Blätter des Anfrageplans durch spezifische Zugriffs-Methoden, d.h. kann/soll ich einen Index benutzen oder besser einen Sequenziellen-Scan der Datei?

Strategien für konjunktive Anfragen

```
SELECT *  
FROM customer  
WHERE name = 'Jensen' AND street = 'Elm'  
      AND state = 'Arizona'
```

- Können die Indexe auf (name) und (street) benutzt werden?
- Kann der Index auf (name, street, state) benutzt werden?
- Kann der Index auf (name, street) benutzt werden?
- Kann der Index auf (name, street, city) benutzt werden?
- Kann der Index auf (city, name, street) benutzt werden?

Strategien für konjunktive Anfragen

```
SELECT *  
FROM customer  
WHERE name = 'Jensen' AND street = 'Elm'  
      AND state = 'Arizona'
```

- Können die Indexe auf (name) und (street) benutzt werden? Ja
- Kann der Index auf (name, street, state) benutzt werden? Ja
- Kann der Index auf (name, street) benutzt werden? Ja
- Kann der Index auf (name, street, city) benutzt werden? Ja
- Kann der Index auf (city, name, street) benutzt? Nein

Mehrdimensionale Indexstrukturen

Bislang: Eindimensionale Schlüssel.

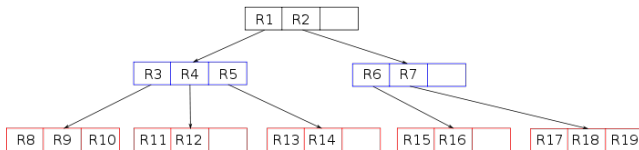
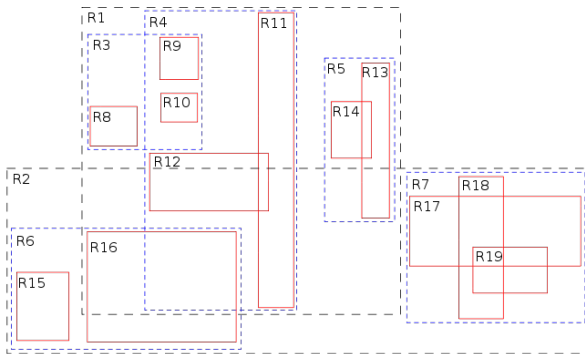
In vielen Anwendungen ist die Anzahl der Dimensionen höher, was tun?

R Baum

- R steht für Rechteck/Rectangle
- Knoten definieren minimale Rechtecke, die die enthaltenen Rechtecke umschließen
- Balanciert. Aufbau (Algorithmus) ähnlich zum B+ Baum
- Überlappung der MBRs (=Minimum Bounding Rectangles)
- Überlappung führt zu Ineffizienz während der Anfrage.

R Baum: Beispiel

Quelle: wikipedia



Nachteile von Indexen

- Platzverbrauch
- Jedes insert/delete/update muss in den Indexen nachgepflegt werden
- Jede Änderungsoperation kostet etwas: CPU und/oder I/O
- Menge der Änderungsoperation kann soviel kosten, dass diese Kosten den Nutzen des Index überwiegen

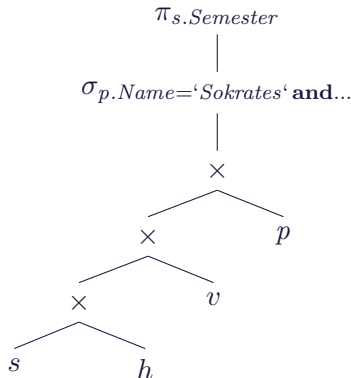
Grundregel: je mehr Leseoperationen desto mehr lohnen sich Indexe

Genauere Regeln zum “Index Tuning”, Anwendbarkeit und weitere Details zur physischen Organisation von Tupeln bzw. Referenzen auf Tupel in Indexen werden in der VL Datenbanksysteme vorgestellt.

Anfrageoptimierung

Beispiel: SQL Anfrage $\rightarrow$ Anfrageplan

select distinct s.Semester
from Studenten s, hoeren h
 Vorlesungen v, Professoren p
where p.Name='Sokrates' **and**
 v.gelesenVon = p.PersNr **and** $\Rightarrow$
 v.VorlNr = h.VorlNr **and**
 h.MatrNr = s.MatrNr;



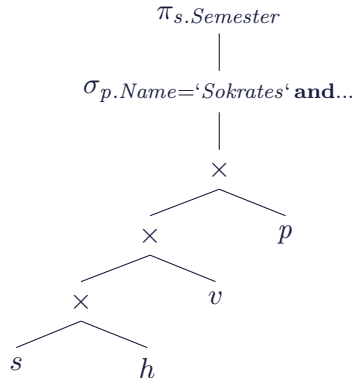
Regelbasierte Optimierung: Vorgehensweise

1. Aufbrechen von Selektionen
2. Verschieben der Selektionen soweit wie möglich nach unten im Operatorbaum (englisch: pushing selections)
3. Zusammenfassen von Selektionen und Kreuzprodukten zu Joins
4. Bestimmung der Reihenfolge der Joins in der Form, dass möglichst kleine Zwischenergebnisse entstehen
5. Unter Umständen Einfügen von Projektionen (keine Duplikateeliminierung)
6. Verschieben der Projektionen soweit wie möglich nach unten im Operatorbaum

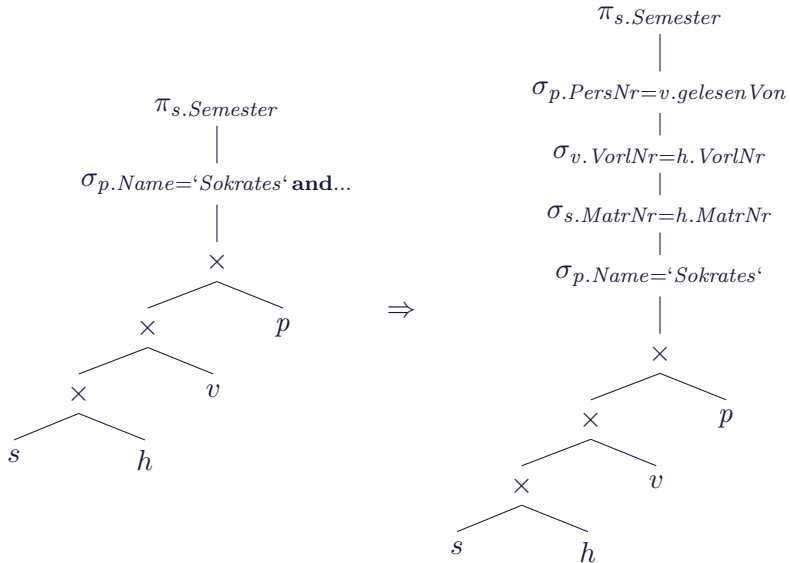
Hier kommen die Äquivalenzregeln der relationalen Algebra zum Einsatz!

Beispiel

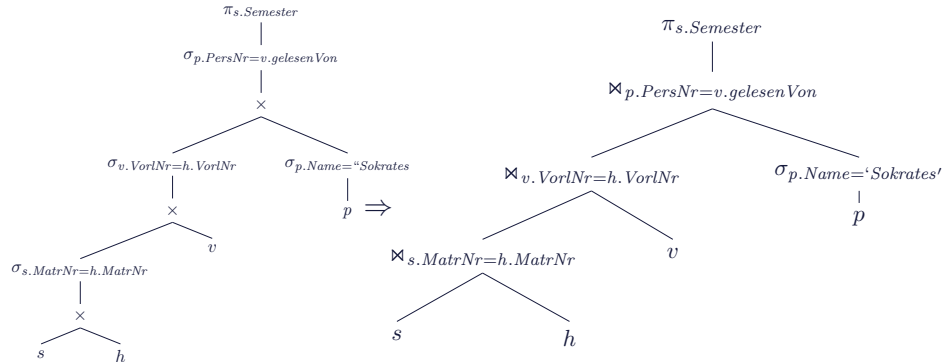
select distinct s.Semester
from Studenten s, hoeren h
 Vorlesungen v, Professoren p
where p.Name='Sokrates' **and**
 v.gelesenVon = p.PersNr **and** $\Rightarrow$
 v.VorlNr = h.VorlNr **and**
 h.MatrNr = s.MatrNr;



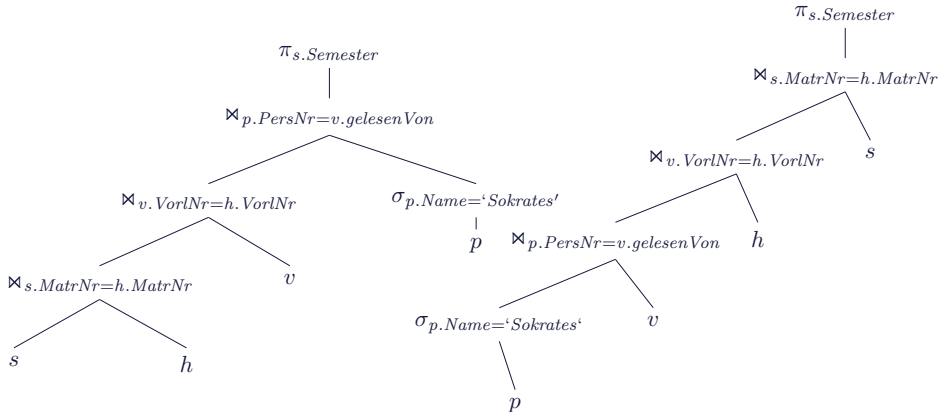
Aufspalten der Selektionsprädikate



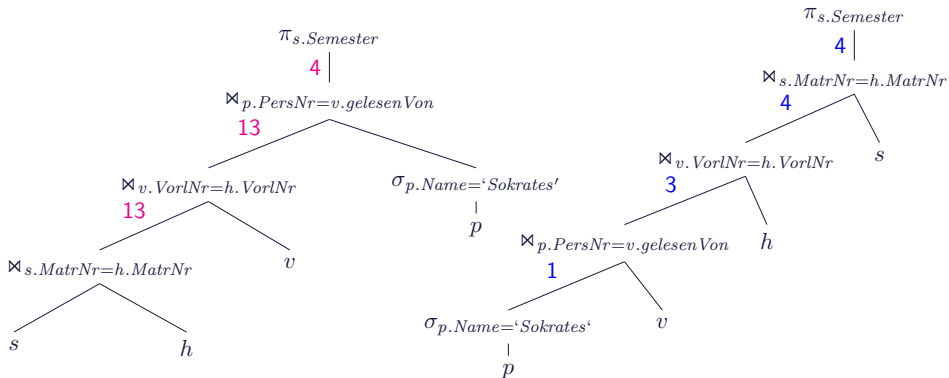
Zusammenfassung von Selektionen und Kreuzprodukten zu Joins



Optimierung der Joinreihenfolge

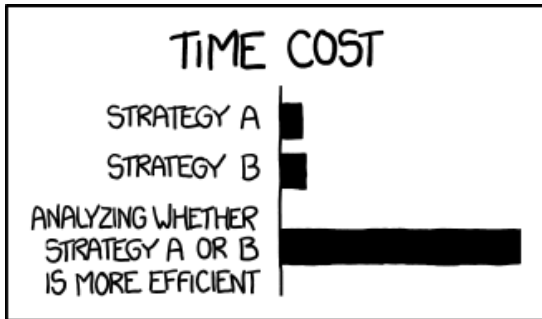


Effekt: Reduzierung der Zwischenergebnisse



Diese Zwischenkosten müssen natürlich geschätzt werden. Der Optimierer kann dann den günstigsten Plan bzgl. dieser geschätzten Kosten auswählen .

Kostenschätzung



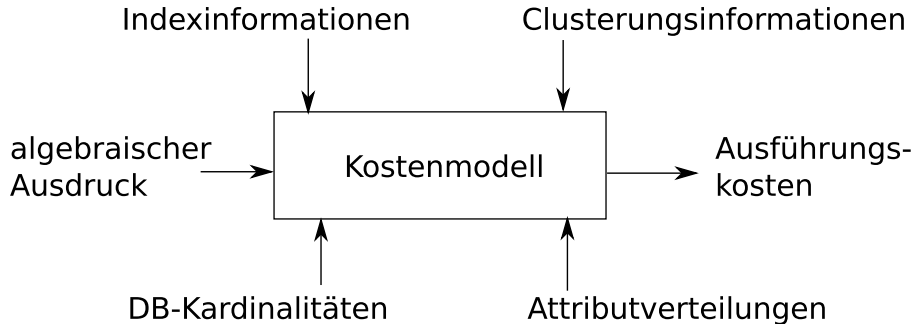
THE REASON I AM SO INEFFICIENT

(c) xkcd.com

Übersicht: Kostenbasierte Optimierung

- Generiere **alle** denkbaren Anfrageauswertungspläne:
= Enumeration/Aufzählung möglicher Pläne
- Schätze deren Kosten ab:
 - Kostenmodell
 - Statistiken
 - Histogramme
 - Kalibrierung gemäß verwendetem Rechner
 - Abhängig vom verfügbaren Speicher
 - Aufwands-Kostenmodell
 - Durchsatz-maximierend
 - versus Antwortzeit-minimierend
- Behalte den Plan mit den geringsten geschätzten Kosten

Übersicht: Kostenmodelle



In dieser Vorlesung betrachten wir nur einfache Kosten in Form von Anzahl Zwischenergebnisse. Genauere Kosten hängen auch von Wahl der Implementierung der Operatoren, von verfügbaren Indexen, Performance der Hardware, etc. ab.

Selektivitätsschätzung

Selektivitätsschätzung: Idee

- Gegeben: Anfrage und Relationen
- Wie viele Tupel sind als Ergebnis zu erwarten?
- Wie viele Tupel fallen als Zwischenergebnisse an?

Selektivitätsschätzung: Werkzeuge

- Kenntnisse/Statistiken über zugrunde liegende Daten
- Generische Annahmen über Selektivität benutzter Prädikate
- Schätzung der Selektivität von o.g. Operatoren (z.B., Join)

Selektivitätsschätzung: Grundlagen

- $T(R)$ ist die **Anzahl der Tupel** in Relation R
- $V(R,A)$ ist die **Anzahl der verschiedenen Attributsausprägungen** für Attribut A .
- Dementsprechend für mehrere Attribute: $V(R,[A_1, A_2, \dots, A_n])$

Aufgabe der Kostenschätzung

- Gegeben eine Anfrage, wie groß ist das Ergebnis und wie viele Tupel fallen als Zwischenergebnisse an?
- z.B. wie viele Tupel sind in $\sigma_{A=13434}(R)$

Selektivität: Anteil der Tupel der Eingaberelation, die nicht herausgefiltert werden. Also: Größe der Ausgabe geteilt durch Größe der Eingabe.

Schätzungen für Selektion

Gegeben eine Selektion $S = \sigma_{A=c}(R)$.

Wie viele Tupel sind in S ?

Schätzung:

$$T(S) = \frac{T(R)}{V(R,A)}$$

Gilt falls die Werte für A zufällig aus allen möglichen Werten gezogen wurden.

Ungleichheit

Was ist bei $S = \sigma_{A < c}(R)$?

Im Allgemeinen, ohne weitere Annahmen: $T(S) = T(R)/2$

Aber, Intuition: man wählt mit solchen Bedingungen meist weniger Tupel aus.

Besser: $T(S) = T(R)/3$

Schätzung für “not-equals”

Gegeben eine Selektion $S = \sigma_{A \neq c}(R)$.

Wie viele Tupel sind in S ?

Einfache Schätzung

- “Mehr oder weniger” alle Tupel erfüllen die Bedingung (naja, bis auf ein paar, aber egal)
- Also: $T(S) = T(R)$

Leicht verbessert

- Die Tupel mit $A = c$ erfüllen die Bedingung nicht.
- Also: $T(S) = T(R) \frac{V(R,A) - 1}{V(R,A)}$

Was passiert mit Kaskaden von Selektionen?

Selektivität ist Produkt der einzelnen Selektivitäten!

Selektion mit ODER-Bedingungen

Gegeben:

$$S = \sigma_{C_1 \vee C_2}(R)$$

Annahme: die Bedingungen werden nie gemeinsam erfüllt

- Also: **entweder** gilt C_1 **oder** C_2
- Schätzung: Summe der beiden einzelnen Selektivitäten.
- Beobachtung: Überschätzt oft. Was kann dann passieren?

Besser: Annahme die Bedingungen sind unabhängig

- Annahme: m_1 Tupel erfüllen C_1 und m_2 Tupel erfüllen C_2
- Dann $T(S) = T(R) * (1 - (1 - \frac{m_1}{T(R)})(1 - \frac{m_2}{T(R)}))$

Selektion mit ODER-Bedingungen: Erläuterung

Wieso $T(S) = T(R) * (1 - (1 - \frac{m_1}{T(R)})(1 - \frac{m_2}{T(R)}))$?

$1 - \frac{m_1}{T(R)}$ ist der Anteil der Tupel die C_1 **nicht** erfüllen.

analog für C_2 . Dann ist

$(1 - \frac{m_1}{T(R)})(1 - \frac{m_2}{T(R)})$ ist der Anteil der Tupel die C_1 **und** C_2 **nicht erfüllen**.

$(1 - ***)$ der Anteil der Tupel die C_1 **oder** C_2 erfüllen.

Klar: Multipliziert mit $T(R)$ ergibt $T(S)$

Andere Schätzer

Projektion

- Ändert die Kardinalität nicht (unter Bag Semantik)
- Sehr wohl aber die Größe der Tupel!
- Bei Mengensemantik: $T(S) = V(R, A)$ mit $S = \pi_A(R)$

Kartesisches Produkt

- Einfach: Produkt der beteiligten Kardinalitäten

Jetzt wird es **unklar was zu tun ist**:

Union

- Obere Schranke: Summe der beiden Kardinalitäten
- Untere Schranke: Größere der beiden Kardinalitäten
- Empfehlung aus Literatur: Irgendwas dazwischen, z.B. größere plus die halbe kleinere Kardinalität

Andere Schätzer

Schnitt

- Obere Schranke: Kleinste der beiden Kardinalitäten
- Untere Schranke: 0
- **Empfehlung aus Literatur:** Durchschnitt dieser beiden Werte.

Differenz $R - S$

- Obere Schranke: $T(R)$
- Untere Schranke: $T(R) - T(S)$
- **Empfehlung aus Literatur:** $T(R) - \frac{T(S)}{2}$

Schätzung für Joins: Natürlicher Join

Zwei Relationen: $R = (X,Y)$ und $S = (Y,Z)$

Annahme: Y ist ein einfaches Attribut, keine Menge von Attributen. X und Z dürfen Mengen sein.

Was können wir dann sagen? Leider nur sehr wenig, da z.B. folgende Fälle auftreten können:

- **Die beiden Relationen haben disjunkte Mengen für Y -Werte.**
Also ist der Join leer, d.h. $T(R \bowtie S) = 0$.
- **Y ist Schlüssel von S und Fremdschlüssel in R .** Dann findet jedes Tupel in R einen Joinpartner in S , also $T(R \bowtie S) = T(R)$
- **Fast alle Tupel aus R und S haben den gleichen Wert für Y ,**
also $T(R \bowtie S) = T(R) * T(S)$

Schätzung für Joins: Natürlicher Join: Annahmen

Häufig auftretende Fälle. Weitere Annahmen:

- Es gibt Y -Werte $y_1, y_2, y_3, \dots$
- Relationen benutzen diese Werte in dieser Reihenfolge.
- Dann: Falls $V(R, Y) \leq V(S, Y)$ so gilt auch, dass jeder Y -Wert in R auch ein Y -Wert in S ist.

Erhaltung der Wertemengen

- Falls A kein Join-Attribut ist, gilt

$$V(R \bowtie S, A) = V(R, A)$$

- D.h. Attribut A verliert durch den Join keine möglichen Werte.

Schätzung für Joins: Natürlicher Join

Gesucht: Größe des Joins $R(X,Y) \bowtie S(Y,Z)$

- Gegeben, zwei Tupel: $r \in R$ und $s \in S$
- Was ist die Wahrscheinlichkeit, dass $r.Y = s.Y$?
- Annahme: $V(R,Y) \geq V(S,Y)$, also gibt es den Y -Wert von s in R .
- Also: Wahrscheinlichkeit, dass $r.Y = s.Y$ ist $1/V(R,Y)$
- Umgekehrt: falls $V(R,Y) < V(S,Y)$ analog.
- **Insgesamt:** Übereinstimmung in Y mit Wahrscheinlichkeit $1/\max(V(R,Y), V(S,Y))$

$$T(R \bowtie S) = \frac{T(R) * T(S)}{\max(V(R,Y), V(S,Y))}$$

“Essentially, all models are wrong, but some are useful.”
(George E. P. Box)

Wie können Größen abgeschätzt werden?

Wichtig: Statistiken über Werte von Attributen, Größen von Relationen.
Aber wie kann man diese berechnen und repräsentieren?

Durch Scannen der gesamten Mengen

- Kann hin und wieder berechnet werden, natürlich besser nicht zur Anfragezeit.
- Moderne DMBS haben bestimmte Befehle dafür.

Ausprägungen für Attribut A

- Falls $V(R, A)$ nicht zu groß ist: speichere einen Zähler für jeden Wert von A . Z.B. es gibt 50 Tupel in Relation Professoren mit Rang='C4'.
- Sonst: Gruppierung. Evtl: Exaktes Speichern für eine Teilmenge der Werte (z.B. der häufigsten).

⇒ Histogramme oder parametrisierte Verteilungen!

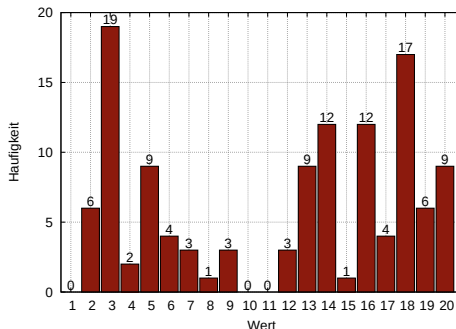
Beispieldaten

{2, 5, 3, 20, 18, 7, 16, 18, 18, 2, 2, 17, 14, 3, 20, 3, 16, 6, 7, 3, 16, 16, 15, 5, 20, 13, 16, 20, 12, 14, 13, 3, 14, 18, 14, 14, 16, 18, 19, 3, 5, 2, 5, 14, 20, 17, 3, 17, 16, 3, 2, 19, 3, 9, 13, 4, 3, 16, 14, 13, 13, 16, 20, 14, 4, 2, 3, 18, 7, 3, 5, 3, 6, 9, 18, 3, 16, 18, 20, 18, 5, 18, 5, 18, 13, 14, 19, 13, 14, 3, 14, 18, 14, 18, 18, 16, 19, 5, 3, 17, 18, 3, 19, 3, 20, 9, 16, 12, 20, 8, 12, 13, 13, 19, 18, 6, 3, 5, 18, 6}

Verteilung der Daten

Wert → Häufigkeit.

Dargestellt im "Histogramm-Stil":



Wir sehen: Es liegen 20 Werte im Wertebereich. Es gibt 120 Datenpunkte.
Alternativ: nicht die absolute Häufigkeit sondern normalisiert in $[0,1]$, also
"Wahrscheinlichkeit" bei zufälligem Zugriff auf Daten einen bestimmten
Datenpunkt zu treffen.

Zusammenfassen/Beschreiben großer Datenmengen

Beschreibung durch parametrisierte Verteilung (Funktion)

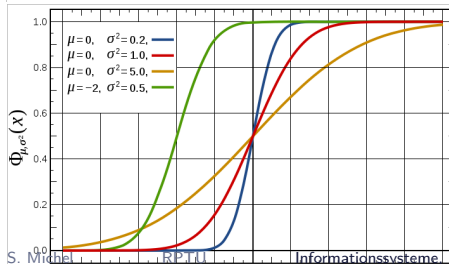
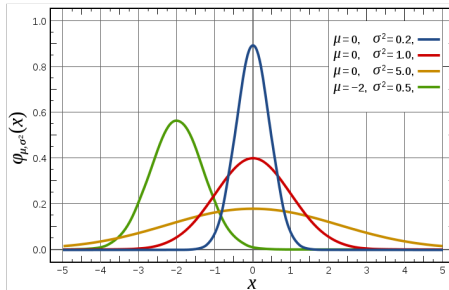
- Z.B. die Daten folgen einer Normalverteilung mit Erwartungswert μ und Varianz σ^2 .

Zusammenfassen von Werten in Zellen (aka. Eimer oder Buckets)

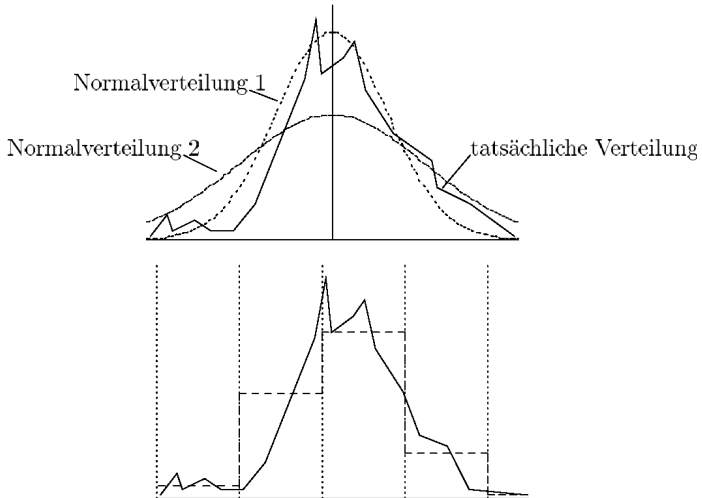
- Wie viele Werte fallen in $[0,5[$, wie viele Werte fallen in $[5,10[$, etc.

Parametrisierte Verteilungen

Dichtefunktion und Verteilungsfunktion der Normalverteilung:

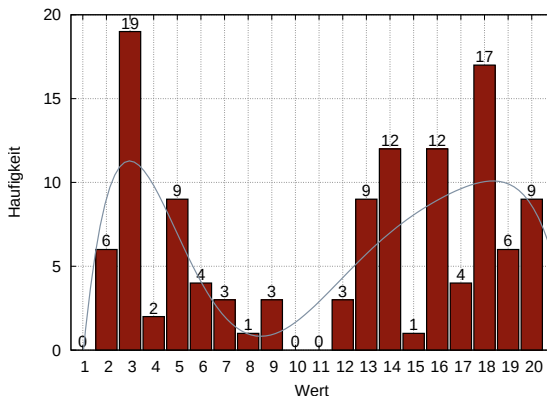


Parametrisierte Verteilungen und Histogramme



Oft ist es schwierig eine tatsächliche Verteilung durch eine parametrisierte Verteilung auszudrücken. Histogramme sind flexibler.

Parametrisierte Verteilungen

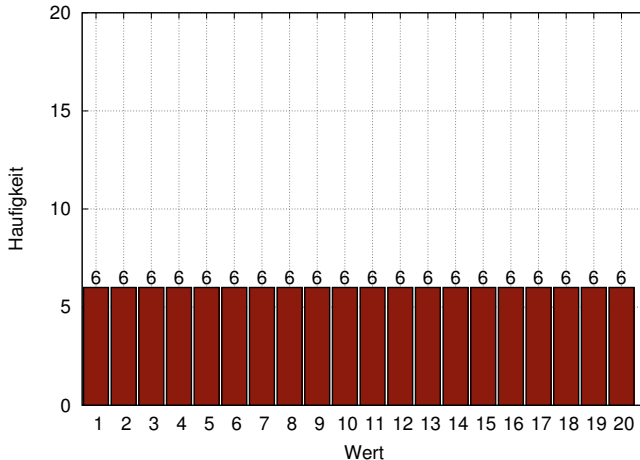


Hier, fit durch Polynom 6. Grades (mit Hilfe des Tools xmgrace):

$$f(x) := -21.884 + 29.533 * x - 9.2268 * x^2 + 1.2529 * x^3 - 0.085019 * x^4 \\ + 0.0028667 * x^5 - 3.8485 * 10^{-5} * x^6$$

Annahme Gleichverteilung

Es liegen 20 Werte im Wertebereich. Es gibt 120 Datenpunkte. Jeder Wert kommt, unter Annahme einer Gleichverteilung, also 6 Mal vor.



Oft nicht sehr realistische Darstellung (aber äußerst kompakt).

Histogramme

Histogram teilt den Wertebereich in Zellen oder Intervalle (Englisch: buckets oder cells). Für jede dieser Zellen wird die Anzahl der Elemente gespeichert, die in die Zelle fallen.

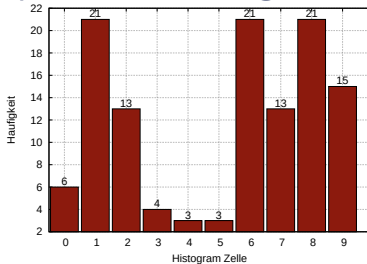
Equi-Width-Histogramme:

- Zellen haben immer die gleiche Breite im Wertebereich

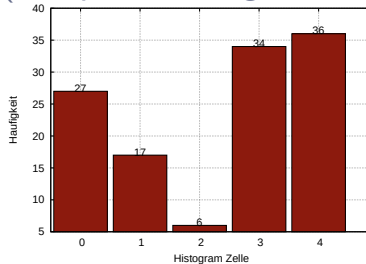
Equi-Depth (oder equi-height genannt) Histogramme:

- Zellen haben die gleiche "Höhe"
- Um dies zu erreichen: Breite der Zellen wird angepasst

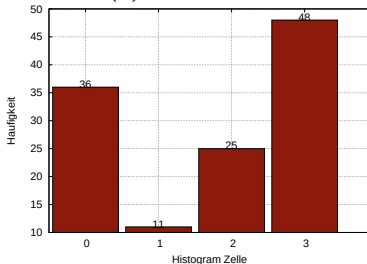
Equi-Width-Histogramme (Beispiele an o.g. Daten)



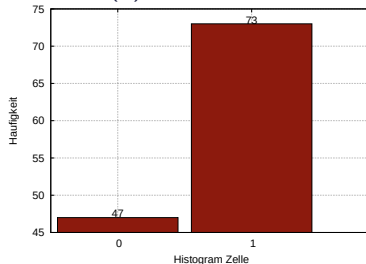
(a) Width = 2



(b) Width = 4

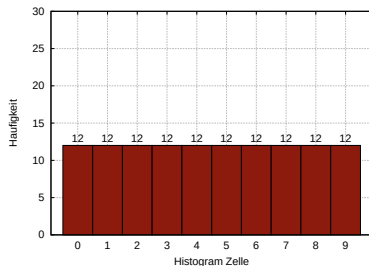


(a) Width = 5

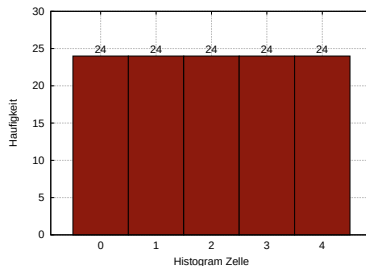


(b) Width = 10

Equi-Depth-Histogramme (Beispiele an o.g. Daten)



(a) Depth = 10%



(b) Depth = 20%

Grenzen der Zellen (je rechter Punkt, d.h. Ende)

- (a) 3,3,5,12,14,15,16,18,19,20
- (b) 3,12,15,18,20

Punktanfragen und Bereichsanfragen auf Histogrammen

Punktanfragen

- Wie viele Tupel haben den Wert $A = 10$?
- Nachschauen: Zelle in die der Wert 10 fällt.
- Annahme hier: Gleichverteilung innerhalb der Zelle.
- Resultat: Anzahl der Tupel geteilt durch Breite der Zelle.
- Bzw. wenn bereits “normalisiert” dann nur Wert der Zelle.

Bereichsanfragen

- Wie viele Tupel haben einen Wert $A > 10$?
- Nachschauen: In welche Zelle fällt der Wert 10? (Achtung insbesondere bei Equi-Depth Histogrammen)
- Resultat: Summe der Größen der darüber liegenden Zellen und anteilmäßig diese Zelle (wie oben).
- Oder: Histogramm für kumulative Verteilung betrachten

Schätzung mit Histogrammen

- Obwohl man für diskrete Daten auch Punktanfragen bearbeiten kann, ist dies im kontinuierlichen Fall nicht möglich.
- Es liegen unendlich viele Werte in jeder Histogrammzelle (mit Häufigkeit 0)
- D.h. es kann im kontinuierlichen Fall “nur” nach Häufigkeiten für Intervalle gefragt werden.

Fehlermaße

- Ist für einen exakten Wert x eine Schätzung (Näherungswert) $\hat{x}$ gegeben,
 - so heisst $|\hat{x} - x|$ absoluter Fehler und
 - $\frac{|\hat{x} - x|}{x}$ im Fall $x \neq 0$ relativer Fehler
- Für mehrere solcher Beobachtungen: Sum Squared Error (SSE), Mean Absolute Error (MAE), Mean Squared Error (MSE)

Deskriptive Statistik

- Minimaler und maximaler Wert eines Attributs
- Durchschnittlicher Wert und Median
- Weitere Aussagen über Verteilungen
- Beispielwerte

Anwendungen

- Produkt-Manager: “In 99,9% aller Fälle liegt die Antwortzeit unseres Systems XYZ unter 10ms.”
- Professor: “75% aller Studenten, die die Klausur mitgeschrieben haben, haben mindestens 80 Punkte erreicht.”

Quantile einer Verteilung

Definition

Gegeben eine Zufallsvariable X mit Verteilungsfunktion F . Für ein $p \in [0,1]$, definieren wir $F^{-1}(p)$ als kleinsten Wert x , so dass $F(x) \geq p$.

Dieser Wert $F^{-1}(x)$ wird das p Quantil von X genannt. Die Funktion F^{-1} wird **Quantilfunktion** genannt.

- Das p Quantil wird auch $100p$ **Perzentil** genannt.
- Das $1/2$ Quantil, bzw. das 50. Perzentil einer Verteilung wird auch **Median** genannt.
- Das $1/4$ Quantil, bzw. das 25. Perzentil, wird auch **erstes Quartil** genannt,
- das $3/4$ Quantil, bzw. das 75. Perzentil, wird auch **drittes Quartil** genannt.

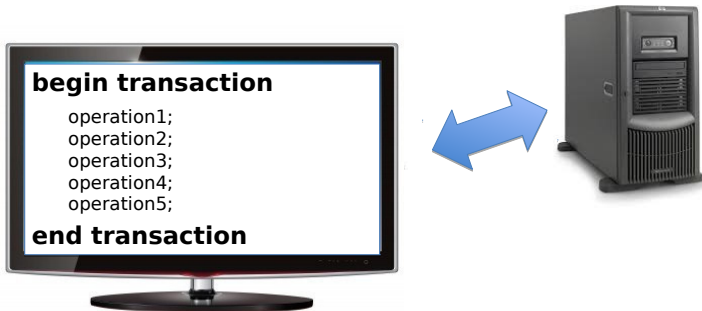
Zusammenfassung Anfrageoptimierung

- Deklarative Anfrage in Form von SQL wird vom Datenbanksystem in konkreten **Anfrageplan** überführt.
- Reihenfolge/Ordnung der Operatoren (der rel. Algebra) des Anfrageplans können durch Regeln verändert werden.
- Haben hier einfache **regelbasierte Anfrageoptimierung** betrachtet: Richtlinien wie Operatoren im Baum generell angeordnet sein sollten.
- Ebenfalls betrachtet: **Einfaches Kostenmodell**, mit dessen Hilfe Kosten (=Anzahl Zwischenergebnisse) abgeschätzt werden können.
- Bessere Abschätzung anhand konkreter Verteilungsmodelle.
- **Ausblick Vorlesung Datenbanksysteme:** Mehr zu Kostenschätzung, Optimierung der Join-Reihenfolge, verschiedene Histogramme, Abschätzung von Anzahl Unique Values durch “Probabilistic Counting”, Index-Tuning, Schema Denormalisierung.

Transaktionsverwaltung

Anwendungsprogrammierung: Transaktionen

- Nicht nur eine einzelne SQL-Anweisung, sondern ganze Folge davon, je nach Anwendung.
- Eine oder mehrere Anweisungen werden als Transaktion zusammengefasst bzw. betrachtet. Z.B. Abheben von Geld am Geldautomat.



Einführung

Bei einer typischen Transaktion in einer Bankanwendung:

1. Lese den Kontostand von A in die Variable a : $read(A,a);$
2. Reduziere den Kontostand um 50 Euro: $a := a - 50;$
3. Schreibe den neuen Kontostand in die Datenbasis: $write(A,a);$
4. Lese den Kontostand von B in die Variable b : $read(B,b);$
5. Erhöhe den Kontostand um 50 Euro: $b := b + 50;$
6. Schreibe den neuen Kontostand in die Datenbasis: $write(B,b);$

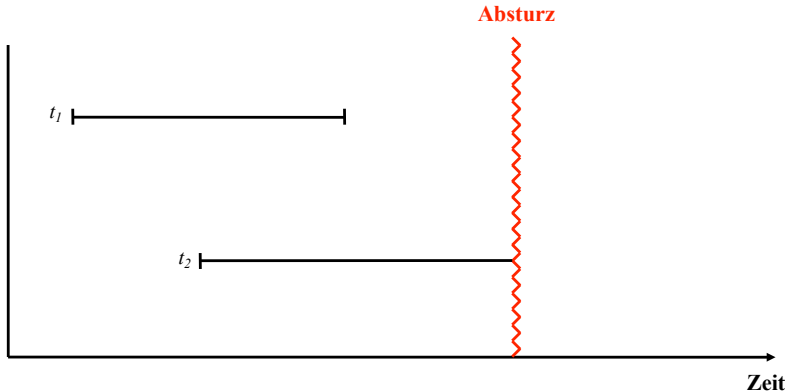
Eigenschaften von Transaktionen: ACID

- **Atomicity (Atomarität):**
Alles oder nichts.
- **Consistency:**
Konsistenter Zustand der DB → konsistenter Zustand.
- **Isolation:**
Jede Transaktion hat die DB “für sich allein”.
- **Durability (Dauerhaftigkeit):**
Änderungen erfolgreicher Transaktionen dürfen nie verloren gehen.

Operationen auf Transaktions-Ebene

- **begin of transaction (BOT):** Mit diesem Befehl wird der Beginn einer Transaktion darstellende Befehlsfolge gekennzeichnet.
- **commit:** Hierdurch wird die Beendigung der Transaktion eingeleitet. Alle Änderungen der Datenbasis werden durch diesen Befehl festgeschrieben, d.h. sie werden dauerhaft in die Datenbank eingebaut.
- **abort:** Dieser Befehl führt zu einem Selbstabbruch der Transaktion. Das Datenbanksystem muss sicherstellen, dass die Datenbasis wieder in den Zustand zurückgesetzt wird, der vor Beginn der Transaktionsausführung existierte.

Transaktionen bei System-Crash



Wie verhält sich solch ein Szenario mit ACID? Was muss beachtet werden?
Transaktionen wie t_1 nennt man auch **Gewinner**, Transaktionen wie t_2 dementsprechend **Verlierer**.

Abschluss einer Transaktion

Für den Abschluss einer Transaktion (TA) gibt es drei Möglichkeiten:

1. Den **erfolgreichen** Abschluss durch **commit**.
2. Den **erfolglosen** Abschluss durch ein **abort**.
3. Den **erfolglosen** Abschluss durch einen **Fehler**.

Transaktionsverwaltung in SQL

Beispielsequenz auf Basis des Universitätsschemas:

```
insert into Vorlesungen  
            values (5275, 'Kernphysik', 3, 2141);  
insert into Professoren  
            values (2141, 'Meitner', 'C4', 205);  
commit;
```

Transaktionsverwaltung in SQL

- **commit [work]:** Die in der Transaktion vollzogenen Änderungen werden – falls keine Konsistenzverletzung oder andere Probleme aufgedeckt werden – festgeschrieben. Das Schlüsselwort work ist optional, d.h. das Transaktionsende kann auch einfach mit commit “befohlen” werden.
- **rollback [work]:** Alle Änderungen sollen zurückgesetzt werden. Anders als der commit-Befehl muss das DBMS die “erfolgreiche” Ausführung eines rollback-Befehls immer garantieren können.

Sicherungspunkte

- **Sicherungspunkt:**
Punkt innerhalb einer TA, auf den sich aktive TA zurücksetzen lässt
- **savepoint <name>:**
definiert den Sicherungspunkt
- **rollback [work] to <name>:** setzt aktive TA zurück bis zum Sicherungspunkt <name>

Beispiel

```
begin;  
insert into tab values ...  
savepoint A;  
insert into tab values ...  
savepoint B;  
SELECT * FROM tab;  
rollback to A;  
SELECT * FROM tab;  
...
```

JDBC - Transaktionen

Transaktionen

- Bei der Erzeugung eines Connection-Objekts ist (in der Regel) als Default der Modus **autocommit** eingestellt. D.h. nach jeder Aktion wird ein Commit ausgeführt.
- Um Transaktionen als Folgen von Anweisungen abwickeln zu können, ist dieser Modus auszuschalten.

```
conn.setAutoCommit( false );
```

- Für eine Transaktion können sogenannte Konsistenzstufen (isolation levels) wie TRANSACTION_SERIALIZEABLE, TRANSACTION_REPEATABLE_READ usw. eingestellt werden.

```
conn.setTransactionIsolation(  
    Connection.TRANSACTION_SERIALIZABLE  
);
```

JDBC - Transaktionen (2)

Beendigung oder Zurücksetzung

```
conn.commit();
```

bzw.

```
conn.rollback(); //oder: conn.rollback(savepoint)
```

Sicherungspunkte (Savepoints)

```
Savepoint sp = conn.setSavepoint(); //bzw. mit Namen  
Savepoint namedSp = conn.setSavepoint("mySavePoint");
```

Programm kann mit mehreren DBMS verbunden sein

- Selektives Beenden/Zurücksetzen von Transaktionen pro DBMS
- Kein global atomares Commit möglich

JDBC - Transaktionen: Beispiel

<http://www.tutorialspoint.com/jdbc/jdbc-transactions.htm>

```
try{
    //Assume a valid connection object conn
    conn.setAutoCommit(false);
    Statement stmt = conn.createStatement();
    String SQL = "INSERT INTO Employees " +
        "VALUES (106, 20, 'Rita ', 'Tez ')" ;
    stmt.executeUpdate(SQL);
    //Submit a malformed SQL statement that breaks
    String SQL = "INSERTED IN Employees " +
        "VALUES (107, 22, 'Sita ', 'Singh ')" ;

    stmt.executeUpdate(SQL);
    // If there is no error.
    conn.commit();
}catch(SQLException se){
    // If there is any error.
    conn.rollback();
}
```

Wie unterstützt das DMBS Transaktionen?

Mehrbenutzersynchronisation (Isolation)

- Semantische Korrektheit bei Nebenläufigkeit
- Serialisierbarkeit
- Schwächere Isolationsstufen (Isolation Levels)

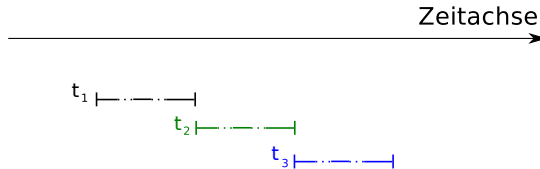
Recovery (Atomicity and Durability)

- Zurücksetzen teilweise ausgeführter TA
- Vervollständigen der Aktionen von TA
- Sicherstellen der Persistenz von TA

Warum Mehrbenutzerbetrieb?

Ausführung der drei Transaktionen t_1 , t_2 und t_3 :

(a) im Einzelbetrieb



(b) im (verzahnten) Mehrbenutzerbetrieb



Man möchte, dass die verzahnte Ausführung “semantisch” äquivalent zu einer seriellen Ausführung (d.h. alle Transaktionen schön getrennt nacheinander, eine nach der anderen, z.B. t_1 t_3 t_2) ist.

Das lost-update Problem

t_1	Time	t_2
	$/*\ x = 100\ */$	
$r(x)$	1	
	2	$r(x)$
$/*\text{update } x := x + 30\ */$	3	
	4	$/*\ \text{update } x := x + 20\ */$
$w(x)$	5	
	$/*\ x = 130\ */$	
	6	$w(x)$
	$/*\ x = 120\ */$	

Das dirty-read Problem

t_1	Time	t_2
$r(x)$	1	
$/* x := x + 100 */$	2	
$w(x)$	3	
	4	$r(x)$
	5	$/* x := x - 100 */$
failure & rollback	6	
	7	$w(x)$

Wir betrachten nun ganz kurz Recovery, danach etwas ausführlicher Mehrbenutzersynchronisation.

Welche Aspekte von

ACID

werden durch Recovery adressiert?

Implikationen von ACID auf Anforderungen zu Recovery

Durability

- Änderungen an der Datenbank, die durch erfolgreich(!) abgeschlossene (d.h. committed) Transaktionen verursacht wurden, müssen dauerhaft gespeichert sein.
- D.h. bei einem Crash der DB muss nach Wiederanlauf geschaut werden, ob dies tatsächlich der Fall ist.

Atomicity

- Falls eine Transaktion noch nicht erfolgreich abgeschlossen wurde und ein DB-Crash auftritt, muss beim Wiederanlauf darauf geachtet werden, dass etwaige Änderungen in der Datenbasis rückgängig gemacht werden.

Fehlerklassifikation

Lokaler Fehler in einer noch nicht festgeschriebenen (committed) Transaktion:

- Wirkung der TA muss zurückgesetzt werden

Fehler mit Hauptspeicherverlust:

- Abgeschlossene TAs müssen erhalten bleiben
- Noch nicht abgeschlossene TAs müssen zurückgesetzt werden

Fehler mit Hintergrundspeicherverlust:

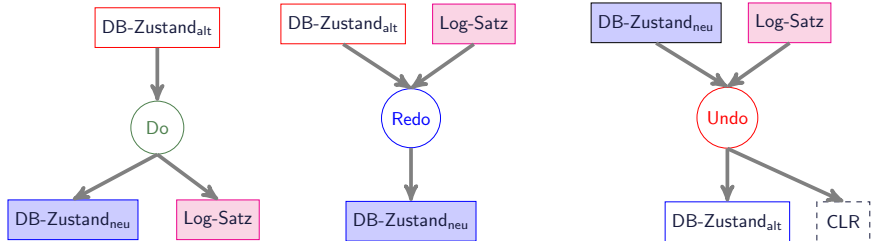
- Archiv einspielen

Vorsorge für den Fehlerfall

Logging

- Sammlung redundanter Daten bei Änderungen im Normalbetrieb, als Voraussetzung für Recovery
- Einsatz im Fehlerfall (Undo-, Redo-Recovery)

Do-Redo-Undo-Prinzip



Recovery-Oriented Computing

Systemverfügbarkeit **A** (availability)

- MTTF: Mean Time To Failure
- MTTR: Mean Time To Repair

$$A = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

Warum Recovery-Oriented Computing?

- Hardwarefehler, Softwarefehler passieren (sind nicht vermeidbar) und müssen adressiert werden.
- Lange Systemausfälle sind sehr sichtbar (z.B. Amazon, Facebook, Ebay!)

Number of Nines & Entwicklungsziele

Availability wird manchmal beschrieben in Anzahl von Neunen (Nines), wie in “five nines”, oder 99.999%. 99.99% entspricht ungefähr 1 Minute Ausfall in einer Woche, bzw. circa 50 Minuten Ausfall in einem Jahr.

- “Build a system used by millions of people that is always available – out less than 1 second per 100 years = 8 9’s of availability” (J. Gray: 1998 Turing Award Lecture)

Jim Gray: We have added three 9s in 45 years (starting with 90%), or about 15 years per order-of-magnitude improvement in availability. We should aim for five more 9s: an expectation of one second outage in a century. This is an extreme goal, but it seems achievable if hardware is very cheap and bandwidth is very high. One can replicate the services in many places, use transactions to manage the data consistency, use design diversity to avoid common mode failures, and quickly repair nodes when they fail. Again, this is not something you will be able to test: so achieving this goal will require careful analysis and proof.



Quelle: bild.de, 27.01.2015

- Ausfall hat (angeblich) ca. 1 Stunde gedauert
- Nehmen wir an, dass dies ein Mal im Jahr geschieht.
- Wie groß ist die Availability? Wie viele "Neunen"?

Wie kann man annähernd $A = 1,0$ erreichen?

- MTTF $\rightarrow \infty$?
- MTTR $\ll$ MTTF!

Es gibt Fehler die man nur sehr schwer oder gar nicht durch testen in den Griff bekommen kann. Sie treten nichtdeterministisch auf, oft aufgrund von Nebenläufigkeit in Threads, unter hoher Last, oder in anderen seltenen Fällen.

Solche Probleme werden auch “Heisenbugs” genannt (Jim Gray); nach Heisenbergs Unschärferelation.

D.h. ein schneller Wiederanlauf (Recovery) ist essentiell für hohe Verfügbarkeit (Availability); da diese “Heisenbugs” die MTTF in der Praxis einschränken.

Grundlagen der DB-Recovery

Aufgabe des DBMS

- Automatische Behandlung aller erwarteten Fehler

Was sind erwartete Fehler?

- DB-Operation wird zurückgewiesen, Commit wird nicht akzeptiert, ...
- Stromausfall, DBMS-Probleme
- Geräte funktionieren nicht (Spur, Zylinder, Platte defekt)
- Beliebiges Fehlverhalten der Gerätesteuerung
- ...

Fehlermodelle von (zentralisierten) DBMS

- Transaktionsfehler
- Systemfehler
- Gerätefehler
- Katastrophen

Recovery-Arten

Transaktions-Recovery

- Zurücksetzen einzelner (noch nicht abgeschlossener) TA im laufenden Betrieb (TA-Fehler, Deadlock, etc.)
- Vollständiges Zurücksetzen auf BOT (TA-Undo)
- Partielles Zurücksetzen auf Rücksetzpunkt (Savepoint) innerhalb der Transaktion

Crash-Recovery nach Systemfehler

- Wiederherstellen des jüngsten transaktionskonsistenten DB-Zustands:
- (partiell) Redo für erfolgreiche TA (Wiederholung verlorengegangener Änderungen)
- Undo aller durch Ausfall unterbrochenen TA (Entfernung der Änderungen aus der DB)

Recovery-Arten (2)

Medien-Recovery nach Gerätefehler

- Spiegelplatten, bzw.
- vollständiges Wiederholen (Redo) aller Änderungen auf einer Archivkopie

Katastrophen-Recovery

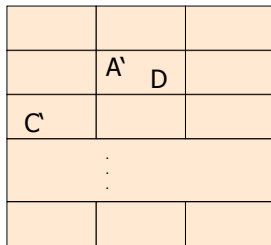
- Nutzung einer aktuellen DB-Kopie in einem “entfernten” System oder
- stark verzögerte Fortsetzung der DB-Verarbeitung mit repariertem/neuem System auf Basis gesicherter Archivkopien

Betrachten im Folgenden kurz Überlegungen zu Crash-Recovery nach Systemfehler.

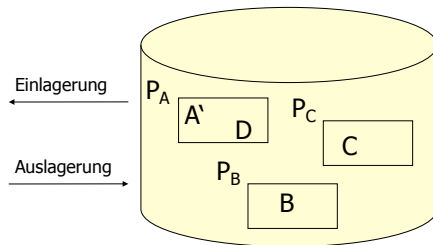
Zweistufige Speicherhierarchie

- Seiten P_i
- Datensätze $A, B, C, D, ..$

DBMS-Puffer,
z.B. Hauptspeicher



Hintergrundspeicher,
z.B. Festplatte



Force und Steal

Ersetzung von Puffer-Seiten:

- **¬steal:** Ersetzung von Seiten, die von einer noch aktiven Transaktion modifiziert wurden, ausgeschlossen $\Rightarrow$ “dreckige” Seiten (dirty pages) müssen im Puffer bleiben.
- **steal:** Jede nicht fixierte Seite ist prinzipiell ein Kandidat für die Ersetzung, falls neue Seiten eingelagert werden müssen, auch “dreckige” Seiten.

Einbringen von Änderungen abgeschlossener TAs:

- **force:** Änderungen werden bei commit auf den Hintergrundspeicher geschrieben (flush).
- **¬force:** geänderte Seiten können auch nach commit im Puffer verbleiben.

dirty page = Inhalt der Seite im HS $\neq$ Inhalt der Seite auf FS

Auswirkungen auf Recovery

Undo: entfernt ungültige Einträge aus der DB.

Redo: fügt gültige Einträge in die DB sein.

	force	$\neg$force
$\neg$steal		
steal		

- Was ist besser? steal oder $\neg$ steal?
- Was ist besser? force oder $\neg$ force?
- Annahme: Schreiboperationen von Seiten müssen atomar sein.

Auswirkungen auf Recovery

Undo: entfernt ungültige Einträge aus der DB.

Redo: fügt gültige Einträge in die DB sein.

	force	$\neg$force
$\neg$steal	• kein Undo • kein Redo $\Rightarrow$ keine Recovery	• Redo • kein Undo
steal	• kein Redo • Undo	• Redo • Undo

- Was ist besser? steal oder $\neg$ steal?
- Was ist besser? force oder $\neg$ force?
- Annahme: Schreiboperationen von Seiten müssen atomar sein.

WAL-Prinzip und Commit-Regel bei Log-basierter Recovery

Write-Ahead-Log-Prinzip (WAL)

Bevor eine modifizierte Seite ausgelagert werden darf, müssen alle Log-Einträge, die zu dieser Seite gehören, in die Log-Datei und das Log-Archiv ausgeschrieben werden.

Commit-Regel (Force-Log-at-Commit)

Bevor eine Transaktion festgeschrieben (**committed**) wird, müssen alle “zu ihr gehörenden” Log-Einträge auf stabilen Storage ausgeschrieben werden.

Details zur Realisierung von Recovery werden in der VL
Datenbanksysteme besprochen.

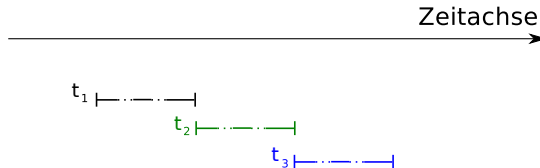
C. Mohan et al. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using

Mehrbenutzersynchronisation

Das “I” in ACID.

Ausführung der drei Transaktionen t_1 , t_2 und t_3 :

(a) im Einzelbetrieb



(b) im (verzahnten) Mehrbenutzerbetrieb



Man möchte, dass die verzahnte Ausführung “semantisch” äquivalent zu einer seriellen Ausführung (d.h. alle Transaktionen schön getrennt nacheinander, eine nach der anderen, z.B. $t_1 t_3 t_2$) ist.

Theorie der Serialisierbarkeit: Transaktion

“Formale” Definition einer Transaktion

Operationen einer Transaktion t_i :

$r_i(x)$	= Lesen des Datenobjekts x
$w_i(x)$	= Schreiben des Datenobjekts x
a_i	= abort
c_i	= commit

Lese- und Schreiboperationen sowie mehrfache Schreiboperationen auf demselben Datenobjekt sind geordnet

Das lost-update Problem

t_1	Time	t_2
	$/*\ x = 100\ */$	
$r(x)$	1	
	2	$r(x)$
$/*\text{update } x := x + 30\ */$	3	
	4	$/*\ \text{update } x := x + 20\ */$
$w(x)$	5	
	$/*\ x = 130\ */$	
	6	$w(x)$
	$/*\ x = 120\ */$	

Das lost-update Problem / Notation

- Die Essenz dieses Problems kann durch folgende Sequenz von Lese- und Schreiboperationen ausgedrückt werden:

$$r_1(x) \ r_2(x) \ w_1(x) \ w_2(x)$$

Das inconsistent-read Problem

Beispiel aus Anwendung in Bank. Aktueller Stand $x = y = 50$, also $x + y = 100$. Transaktion t_1 berechnet die Summe von x und y , während t_2 einen Wert von 10 von x nach y transferiert.

t_1	Time	t_2
	1	$r(x)$
	2	$/^* x := x - 10 */$
	3	$w(x)$
$/^* sum := 0 */$	4	
$r(x)$	5	
$r(y)$	6	
$/^* sum := sum + x */$	7	
$/^* sum := sum + y */$	8	
	9	$r(y)$
	10	$/^* y := y + 10 */$
	11	$w(y)$

Das inconsistent-read Problem (2)

- Offensichtlich ist auch hier wieder das Problem, dass Lese- und Schreiboperationen der einzelnen Transaktionen gemischt ablaufen

$$r_2(x) \ w_2(x) \ r_1(x) \ r_1(y) \ r_2(y) \ w_2(y)$$

Das dirty-read Problem

t_1	Time	t_2
$r(x)$	1	
$/* x := x + 100 */$	2	
$w(x)$	3	
	4	$r(x)$
	5	$/* x := x - 100 */$
failure & rollback	6	
	7	$w(x)$

Konsistenzanforderung einer Transaktion t_i

- Entweder **abort** oder **commit** aber nicht beides!
- Falls t_i ein **abort** durchführt, müssen alle anderen Operationen $p_i(x)$ vor a_i ausgeführt werden, also $p_i(x) <_i a_i$.
- Analoges gilt für das **commit**, d.h. $p_i(x) <_i c_i$ falls t_i “**committed**”.
- Wenn eine Transaktion t_i ein Datum x liest und auch schreibt, dann muss die Reihenfolge festgelegt werden, also entweder $r_i(x) <_i w_i(x)$ oder $w_i(x) <_i r_i(x)$.

Beispiel Transaktion:

$r_1(x) \ w_1(x) \ r_1(y) \ w_1(y) \ c_1$

Historien

- Historie enthält alle Operationen aller Transaktionen
- Historie benötigt eine Terminierungsoperation für jede TA
- Historie bewahrt alle Ordnungen innerhalb der TA
- Terminierungsoperation ist letzte Operation in jeder TA
- Konfliktoperationen sind geordnet.

Beispiel Historie:

$$H = r_1(x) \ r_2(x) \ w_1(x) \ r_3(x) \ w_3(x) \ w_2(y) \ c_3 \ c_2 \ w_1(y) \ c_1$$

Serialisierbare Historie

Eine Historie ist serialisierbar, wenn sie äquivalent zu einer seriellen Historie H_s ist.

- Was bedeutet nun “äquivalent”?
- Wie entscheidet man effizient, ob eine Historie serialisierbar ist?
- Wie entscheidet man, in welcher Reihenfolge die einzelnen Operationen ausgeführt werden?

Konfliktoperationen

- Gegeben zwei Transaktionen t_i und t_j
- $r_i(x)$ und $r_j(x)$:
 - Reihenfolge der Ausführungen irrelevant, da beide Transaktionen in jedem Fall denselben Zustand lesen.
 - Operationen **stehen nicht im Konflikt** zueinander
- $r_i(x)$ und $w_j(x)$:
 - **Konflikt**, da Transaktion t_i entweder den alten **oder** den neuen Wert von x liest.
 - entweder $r_i(x)$ **vor** $w_j(x)$ oder: $w_j(x)$ **vor** $r_j(x)$ spezifizieren.
- $w_i(x)$ und $r_j(x)$: analog
- $w_i(x)$ und $w_j(x)$:
 - Reihenfolge der Ausführung entscheidend für den Zustand der Datenbasis.
 - **Konflikt**, für den die Reihenfolge festzulegen ist.

Konfliktschritte-Graph

Gegeben eine Historie H , z.B.

$$H = r_1(x) \ r_2(x) \ w_1(x) \ r_3(x) \ w_3(x) \ w_2(y) \ c_3 \ c_2 \ w_1(y) \ c_1$$

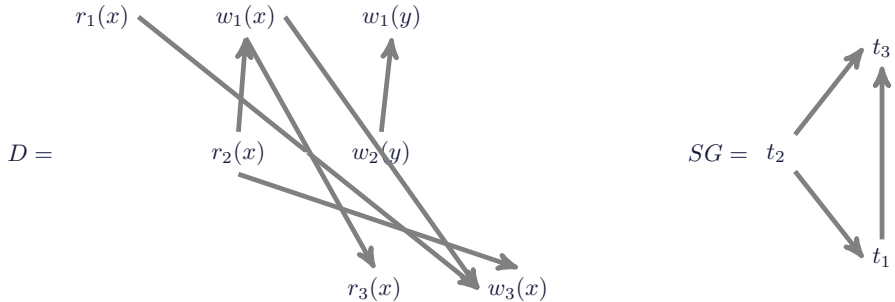
Wir definieren einen Graphen D , den sogenannten

Konfliktschritte-Graph, wie folgt: Als Knoten haben wir alle Operationen der in der Historie beteiligten Transaktionen. D hat gerichtete Kanten zwischen den einzelnen Konfliktoperationen. Für das Beispiel oben:

$$D(H) =$$

Zwei Historien s und s' heißen äquivalent, wenn ihre Konfliktschritte-Graphen identisch sind.

Serialisierbarkeitsgraph



- **Konfliktschritte-Graph** D beschreibt einzelne Konflikte zwischen Operationen $p_i(x_i)$
- Kante $w_1(x) \rightarrow r_3(x)$ aus D führt zur Kante $t_1 \rightarrow t_3$ des **Serialisierbarkeitsgraphen** (aka. Konfliktgraph) SG
- Weitere Kanten analog
- **Ist die zugrunde liegende Historie serialisierbar?**

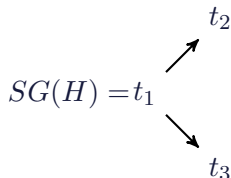
Serialisierbarkeitstheorem

Eine Historie H ist genau dann serialisierbar, wenn der zugehörige Serialisierbarkeitsgraph $SG(H)$ azyklisch ist.

Historie

$$H = w_1(x) \ w_1(y) \ c_1 \ r_2(x) \ r_3(y) \ w_2(x) \ c_2 \ w_3(y) \ c_3$$

Serialisierbarkeitsgraph:



Topologische Ordnung(en):

$$H_s^1 = t_1 | t_2 | t_3$$

$$H_s^2 = t_1 | t_3 | t_2$$

$$H \equiv H_s^1 \equiv H_s^2$$

Eigenschaften von Historien bezüglich Recovery

Wir sagen, dass in der Historie H Transaktion t_i von t_j liest, wenn folgendes gilt:

1. t_j schreibt mindestens ein Datum x , das t_i nachfolgend liest, also:
 $w_j(x) <_H r_i(x)$
2. t_j wird (zumindest) nicht vor dem Lesevorgang von t_i zurückgesetzt, also $a_j \not<_H r_i(x)$
3. Alle anderen zwischenzeitlichen Schreibvorgänge auf x durch andere Transaktionen t_k werden **vor** dem Lesen durch t_i zurückgesetzt. Falls also ein $w_k(x)$ mit $w_j(x) <_H w_k(x) <_H r_i(x)$ existiert, so muss es auch ein $a_k <_H r_i(x)$ geben.

Intuition: t_i liest ein Datum in genau dem Zustand, den t_j geschrieben hat.

Rücksetzbare Historien (RC)

Eine Historie heißt zurücksetzbar (recoverable, RC), falls immer die schreibende Transaktion (in unserer Notation t_j) vor der lesenden Transaktion (t_i genannt) ihr commit durchführt, also: $c_j <_H c_i$.

- Anders ausgedrückt: Eine Transaktion darf erst dann ihr **commit** durchführen, wenn alle Transaktionen, von denen sie gelesen hat, beendet sind.
- Ansonsten ist diese TA womöglich nicht rücksetzbar, da sie mit einem Wert gerechnet hat, der nie existiert hat
- Nicht rücksetzbar $\Rightarrow$ verletzt ACID
- Warum?

Merkregel: TA darf erst committen, wenn alle TAs, von denen sie gelesen hat, beendet sind.

Historien mit kaskadierendem Rücksetzen

Schritt	t_1	t_2	t_3	t_4	t_5
0.	...				
1.	$w_1(A)$				
2.		$r_2(A)$			
3.		$w_2(B)$			
4.			$r_3(B)$		
5.			$w_3(C)$		
6.				$r_4(C)$	
7.				$w_4(D)$	
8.					$r_5(D)$
9.	$a_1(abort)$				

Warum ist das hier rücksetzbar?

Historien ohne kaskadierendes Rücksetzen (ACA)

Historien ohne kaskadierendes Rücksetzen (avoids cascading aborts: ACA): Eine Historie H vermeidet kaskadierendes Rücksetzen, wenn für je zwei TAs t_i und t_j gilt: $c_j <_H r_i(x)$ gilt, wann immer t_i ein Datum x von t_j liest.

Merkregel: Es dürfen nur Änderungen von abgeschlossenen TAs gelesen werden.

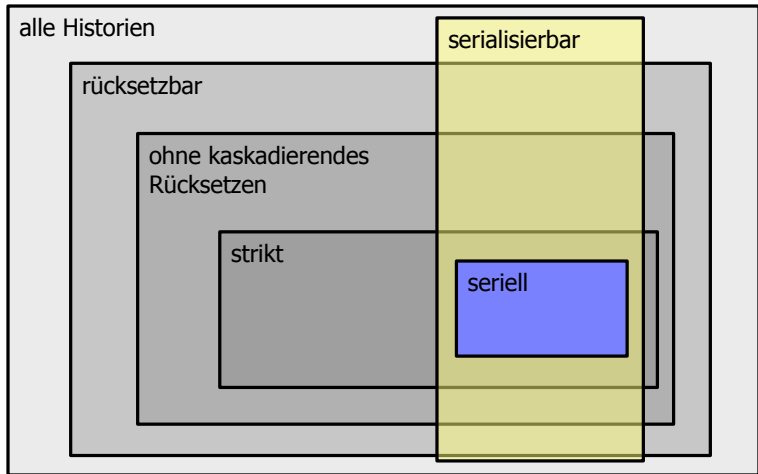
Strikte Historien

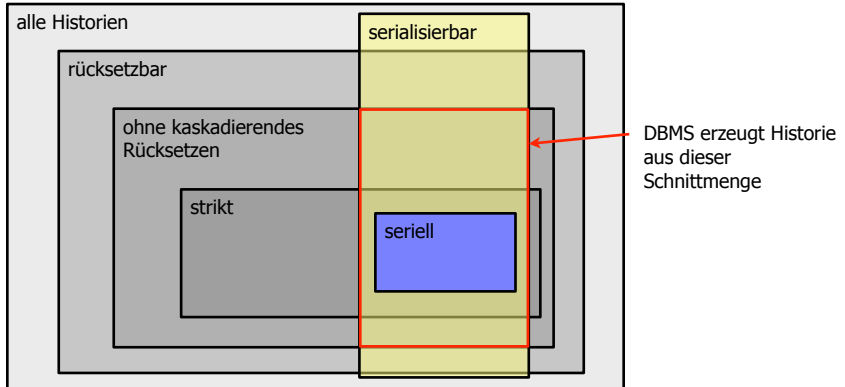
Eine Historie H heißt strikt, wenn für je zwei Transaktionen t_i und t_j gilt:

Wenn $w_j(A) <_H o_i(A)$ mit $o_i = r_i$ oder $o_i = w_i$, dann muss gelten:

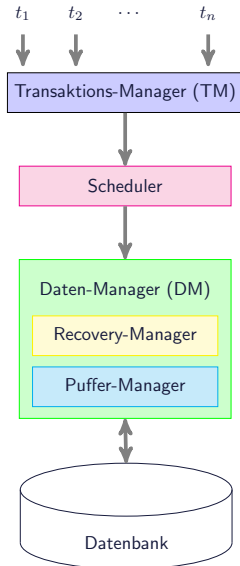
- $a_j <_H o_i(A)$ oder
- $c_j <_H o_i(A)$

Merkregel: Geänderte Daten nicht-abgeschlossener TAs dürfen weder gelesen noch überschrieben werden.





Der Datenbank-Scheduler



- TM: Informationen über TA, welche Schritte als nächstes ausgeführt werden können.
- Scheduler: Bekommt Schedules vom TM und muss diese in einen serialisierbaren Schedule umwandeln.
- Welcher verarbeitet wird von Daten-Manager (DM)

Sperrprotokolle - Allgemeines

Allgemeine Idee

- **Zugriff auf gemeinsam genutzte Daten wird durch Sperren synchronisiert**
- Hier: Ausschließlich konzeptionelle Sichtweise und gleichförmige Granulate wie Seiten (keine Implementierungstechnik, keine multiplen Granulate usw.)

Allgemeine Vorgehensweise

- Scheduler fordert für die betreffende TA für jeden ihrer Schritte eine Sperre an
- Jede Sperre wird in einem spezifischen Modus angefordert (read oder write)
- Falls das Datenelement noch nicht in einem unverträglichen Modus gesperrt ist, wird die Sperre gewährt; sonst ergibt sich ein Sperrkonflikt und die TA wird blockiert, bis die Sperre freigegeben wird.

Sperrbasierte Synchronisation

Zwei Sperrmodi:

- S (shared, read lock, Lesesperre)
- X (exclusive, write lock, Schreibsperre)

Verträglichkeitsmatrix (auch Kompatibilitätsmatrix genannt) und neuer Modus:

aktueller Modus des
Objekts x

	NL	R	X
$rl(x)$	+	+	-
$wl(x)$	+	-	-

neuer Modus des
Objekts x

	NL	R	X
$rl(x)$	R	R	-
$wl(x)$	X	-	-

angeforderte
Sperre

Zwei-Phasen Sperrprotokoll (2PL): Definition

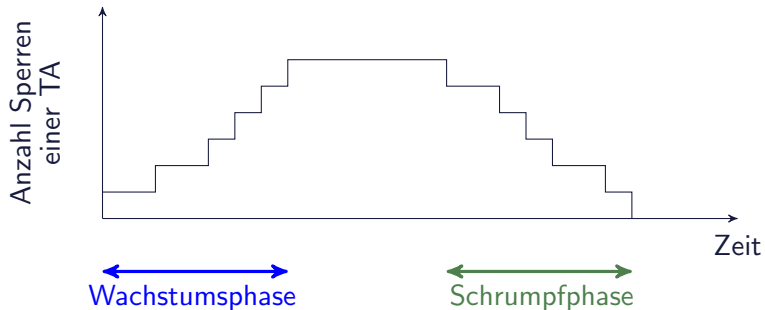
1. Jedes Objekt, das von einer Transaktion benutzt werden soll, muss vorher entsprechend gesperrt werden.
2. Eine Transaktion fordert eine Sperre, die sie schon benutzt, nicht erneut an.
3. Eine Transaktion muss die Sperren anderer Transaktionen auf dem von ihr benötigten Objekt gemäß der Verträglichkeitstabelle beachten. Wenn die Sperre nicht gewährt werden kann, wird die Transaktion in eine entsprechende Warteschlange eingereiht - bis die Sperre gewährt werden kann.
4. Jede Transaktion durchläuft zwei Phasen:
 - Eine **Wachstumsphase**, in der sie Sperren anfordern, aber keine freigeben darf und
 - eine **Schrumpfphase**, in der sie ihre bisher erworbenen Sperren freigibt, aber keine weiteren anfordern darf.
5. Bei EOT (Transaktionsende) muss eine Transaktion alle ihre Sperren zurückgeben.

Zwei-Phasen Sperrprotokoll (2PL): Grafik

Definition: 2PL

Ein Sperrprotokoll ist **zweiphasig** (2PL), wenn für jede TA t_i kein Lock (d.h. ql_i Schritt) dem ersten Unlock (d.h. ou_i Schritt) folgt ($o, q \in \{r, w\}$).

2PL erzeugt nur serialisierbare Historien $\Rightarrow$ 2PL garantiert Serialisierbarkeit

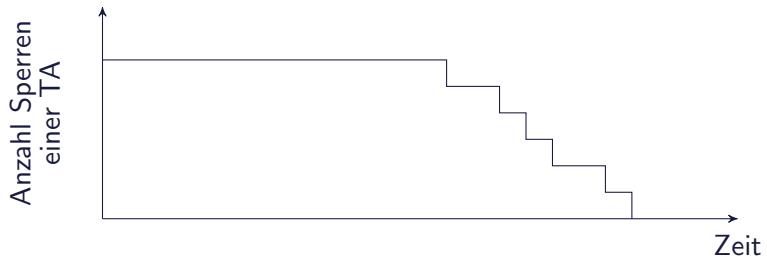


Konservatives 2PL (C2PL)

Definition: Konservatives 2PL

Unter konservativem 2PL (C2PL) fordert jede TA alle Sperren an, bevor sie den erste Read- oder Write-Schritt ausführt (**Preclaiming**).

In der Praxis nur eingeschränkt anwendbar, da alle Sperren schon zu Beginn der TA bekannt sein müssen.

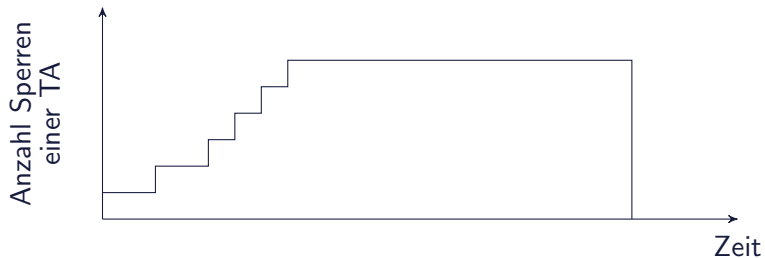


Striktes 2PL (S2PL) und Starkes 2PL (SS2PL)

Definition: Striktes 2PL

Unter striktem 2PL (S2PL) werden **alle exklusiven Sperren** (wl) einer TA bis zur ihrer Terminierung gehalten.

Wird in praktischen Implementierungen am häufigsten eingesetzt. Erzeugt **serialisierbare und strikte Historien**



Definition: Starkes 2PL

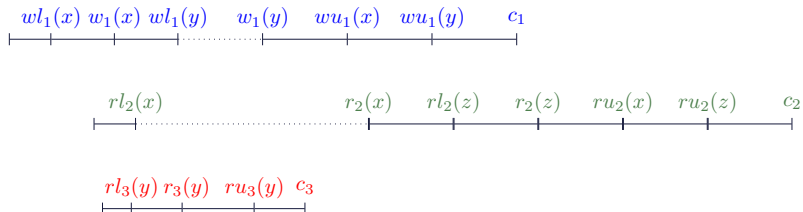
Unter starkem 2PL (strong strict 2PL, SS2PL) werden **alle Sperren** (wl , rl) einer TA bis zur ihrer Terminierung gehalten.

Sperrprotokoll 2PL

Beispiel: Eingabe-Schedule

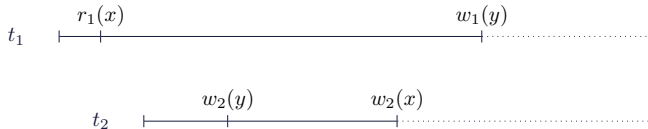
$$s_1 = w_1(x) \ r_2(x) \ r_3(y) \ r_2(z) \ w_1(y) \ c_3 \ c_1 \ c_2$$

2PL Scheduler transformiert s_1 z.B. in folgende Ausgabe-Historie:



Deadlocks

- ... werden verursacht durch zyklisches Warten auf Sperren
- Beispiel



Deadlock-Erkennung

Aufbau eines dynamischen **Wait-for-Graph** (WfG) mit aktiven TAs als Knoten und Wartebeziehungen als Kanten: Eine Kante von t_i nach t_j ergibt sich, wenn t_i auf eine von t_j gehaltene Sperre wartet.

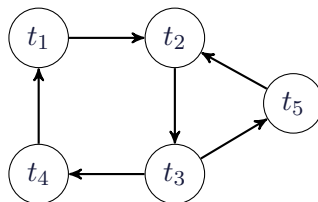
Testen des WfG zur Zyklenerkennung

- kontinuierlich (bei jedem Blockieren)
- periodisch (z.B. einmal pro Sekunde)

Erkennung von Verklemmung

Wartegraph mit zwei Zyklen

- $t_1 \rightarrow t_2 \rightarrow t_3 \rightarrow t_4 \rightarrow t_1$
- $t_2 \rightarrow t_3 \rightarrow t_5 \rightarrow t_2$



- Beide Zyklen können durch Rücksetzen von t_3 “gelöst” werden.
- Zyklenerkennung durch Tiefensuche im Wartegraphen.
- Verschiedene Strategien welche TA zurückgesetzt werden soll (jüngste, älteste, nach $\#$ Sperren etc.)

Transaktionsverwaltung in SQL92

Problem:

Serialisierbarkeit kann zu Performanzproblemen führen.

Idee:

Isolationsgarantien abschwächen, um höhere Performanz zu erreichen.

Vier sogenannte Isolationlevel:

set transaction

[read only, | read write,]

[**isolation level**

read uncommitted, |

read committed, |

repeatable read, |

serializable,] |

[**diagnostics size ...**,]

Isolationsstufen

read uncommitted:

- Schwächste Konsistenzstufe
- Liest möglicherweise inkonsistente/uncommitted data
- Nur für read-only TAs
- Geeignet für “browsing”
- Keine locks für read-Operationen

read committed:

- Nur committed Werte dürfen gelesen werden
- Auch für Schreib-TAs erlaubt
- Locks zum Schreiben im strikten 2PL
- Locks zum Lesen nur kurzzeitig gehalten
- Non-repeatable read möglich
- Phantom-Problem möglich

Isolationsstufen (2)

repeatable read:

- Locks zum Schreiben mit striktem 2PL
- Locks zum Lesen mit striktem 2PL
- Non-repeatable read ausgeschlossen
- Phantom-Problem möglich

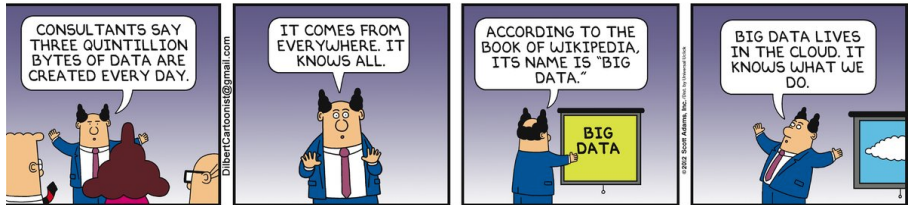
serializable:

- Volle Serialisierbarkeit
- Nicht immer default
- Locks zum Schreiben mit striktem 2PL
- Locks zum Lesen mit striktem 2PL
- Zusätzlich wird der Zugriff über Indexe/Prädikate gelockt

Probleme mit Locking

- Transaktionen halten locks länger als zum Lesen/Schreiben eigentlich notwendig (wegen 2PL)
- Deadlock-Erkennung aufwendig
- Pessimistischer Ansatz: versteckte Annahme, dass es schief gehen wird (Murphy's Law)
- Was, wenn es hauptsächlich Lesetransaktionen gibt?
- Dann blockiert jedes Write eine große Zahl von Lesetransaktionen
- $\Rightarrow$ Lesetransaktionen müssen warten
- $\Rightarrow$ Schlechte Performance

Mehr dazu in der VL Datenbanksysteme im Winter.



source:dilbert.com

Motivation: Big Data Analytics

Algorithmen zur Datenanalyse

- Wie häufig kommt ein Wort in den HTML Dokumenten vor?
- Wie häufig treten Worte zusammen auf?
- Was sind die einflussreichsten Webseiten?
- Was waren die Twitter-Trends der vergangenen Woche?
- Welche Suchbegriffe sind am populärsten?

Paradigma und Ziel

- Sammle Daten und analysiere sie später
- Ziel: Gewinnung von Erkenntnissen/Informationen!
- Teilweise hoher materieller Wert (Platzierung von Werbung, Empfehlung von Produkten)

The 4th Paradigm

Erkenntnisgewinn in der Wissenschaft, traditionell durch

- **Experimente** (seit tausenden von Jahren)
- **Theorie** (seit hundertern von Jahren)
- **Berechnungen und Simulation** (seit wenigen Jahrzehnten)

Nun: Erkenntnisgewinn durch Datenanalyse.

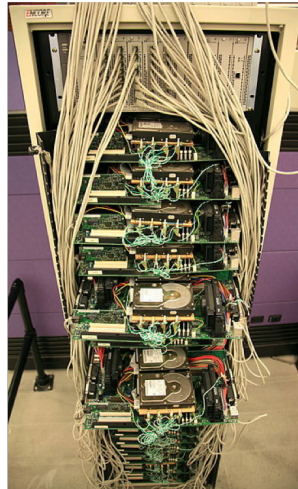
Literatur:

<http://research.microsoft.com/en-us/collaboration/fourthparadigm/>

Große Datenmengen

Beispiel: Google

- Viele Milliarden Webseiten
- Terabytes an Daten
- Nicht nur Webseiten
- Auch Videos (Youtube), Bilder, Benutzerprofile, Emails
- Interne Daten: HTTP (etc.) Access-Logs



Google server, circa, 1999. source:

<http://flickr.com/photos/jurvetson/157722937/>

Geschätzte Datenmengen

- Google: 15 000 PB (=15 Exabytes)
- Facebook: 300 PB
- Ebay: 90 PB
- Spotify: 10 PB

Verarbeitete Datenmenge pro Tag

- Google: 100 PB
- Ebay: 100 PB
- NSA: 29 PB
- Facebook: 600 TB
- Twitter: 100 TB
- Spotify: 2,2 TB

Quelle: <https://followthedata.wordpress.com/2014/06/24/data-size-estimates/>

MB = 10^6 Bytes

GB = 10^9 Bytes

TB (Terabyte) = 10^{12} Bytes

PB (Petabyte) = 10^{15} Bytes

EB (Exabyte) = 10^{18} Bytes

Gigabyte, Terabyte,
Petabyte

*Aus Platzgründen
nur teilweise dargestellt*



The Big Data Challenge: Die 4 V

Volume

- Es gibt sehr viele Daten.

Velocity

- Daten ändern sich und Datenmenge wächst rasant.

Variety

- Daten und Datenquellen sind heterogen.

Verity

- Sind die Informationen wahr oder inkorrekt?

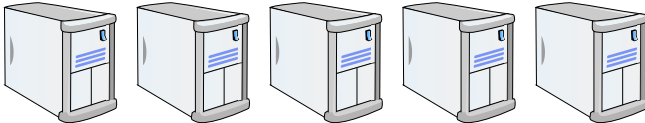
Problem und Konsequenzen

Beispiel: Lesen von 10TB von einer Festplatte

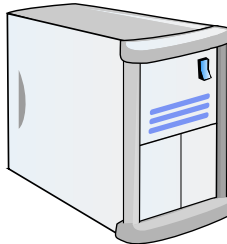
- Nehmen wir an wir haben eine 10 TB große Datei auf der Festplatte
- Wir möchten die Daten (z.B. Twitter tweets) nun analysieren
- Mit einer Festplatte mit 100MB/s Lesegeschwindigkeit (sequentielles Lesen) brauchen wir alleine für das Lesen an sich
 - 100000 Sekunden
 - bzw. 1666 Minuten
 - bzw. 27 Stunden

Horizontale vs. Vertikale Skalierung

- **Horizontale Skalierung (scale out)**: Viele Maschinen (hunderte, tausende) in Rechenzentren



- **Vertikale Skalierung (scale up)**: Aufrüsten eines Servers; mehr RAM, mehr/bessere CPU, mehr Festplattenspeicher, ...



Data Centers



source:Google

Tour durch ein Google-Data-Center via Google-Street-View.

<http://www.google.com/about/datacenters/inside/streetview/>

Hardware Fehler

- **Viele Maschinen, also viel Hardware die kaputt gehen kann.**
- D.h. Hardwarefehler treten häufig auf und sind keine seltene Ausnahme.

Sagen wir z.B. eine bestimmte Maschine fällt ein Mal im Jahr aus, also

$$P[\text{Maschine fällt heute aus}] = \frac{1}{365}$$

Wir haben n Maschinen:

$$P[\text{Heute fällt mindestens eine Maschine aus}] = 1 - (1 - P[\text{Maschine fällt heute aus}])^n$$

für $n=1$: 0.0027

für $n=10$: 0.02706

für $n=100$: 0.239

für $n=1000$: 0.9356

für $n=10\ 000$: ~ 1.0 (!!!)

HOW TO WRITE A CV



Leverage the NoSQL boom

NoSQL

Was steckt hinter NoSQL?

- Beobachtung/Hypothese: Es gibt kein one-size-fits-all Datenbanksystem!
- NoSQL = Not Only SQL (nicht unbedingt “no” SQL)
- Steht als Bezeichner für eine Vielzahl von nicht traditionellen Datenmanagement-Systemen, die stark auf die Anwendung zugeschnitten sind:

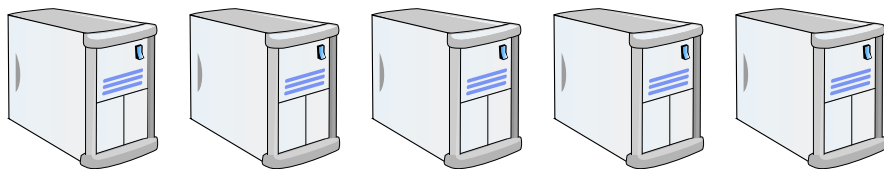
- Key-Value-Datenbanken
- Graph-Datenbanken
- Dokument-Datenbanken



Überblick gibt es unter: <http://nosql-database.org/>

Charakteristika von NoSQL Systemen

- (Meist) kein relationales Datenmodell
- System sind ausgelegt horizontal zu skalieren (scale out), also Daten und Datenverarbeitung über mehrere Maschinen zu verteilen.
- Kein Schema oder nur sehr lose beschrieben.
- Einfache API (normalerweise keine Unterstützung von SQL): CRUD (create, read, update, delete).
- Üblicherweise keine ACID Semantik. Stattdessen: BASE ;)
- Oftmals sind diese System open-source.



NoSQL: Key/Value Datenbanken

- Speichern von Key-Value Paaren
- Values können komplexe(re) Datentypen sein
- Beispiele von Systemen: Amazon Dynamo, Redis, Voldemort
- Zugriff via **CRUD**-Operationen: **C**reate, **R**ead, **U**ppdate, **D**eleate
- Einige Systeme unterstützen auch mächtigere/komplexere Anfragetypen.

Beispiel: Key-Value-Store: Redis

- Online Tutorial: <http://try.redis.io>

Get und Set

SET name "Informationssysteme"

GET name → Informationssysteme

Operationen auf Listen

LPUSH meineListe "a"

LPUSH meineListe "b"

LLENGTH → 2

LRANGE meineListe 0 1 → "b", "a"

Dokumenten Datenbanken

- Speichern JSON (Javascript Object Notation), siehe Beispiel links, oder XML Dokumente
- Systeme: MongoDB oder CouchDB

```
{
  "firstName": "John",
  "lastName": "Smith",
  "age": 25,
  "address": {
    "street": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": 10021
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "fax",
      "number": "646 555-4567"
    }
  ]
}
```

Beispiel: MongoDB

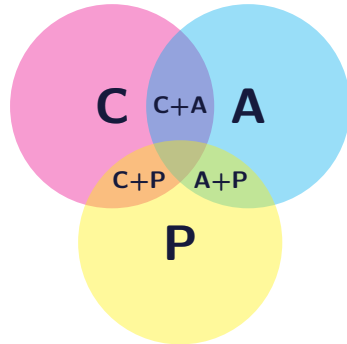
```
var student = {name:'Jim', scores:[75,99,87.2]};  
db.lecture.store(student);
```

```
db.lecture.find();           --liefert alle Eintraege  
db.lecture.find({name:'Jim'}); --spezielle Suche  
db.users.update({name:'Johnny'},{name:'Cash',  
                                languages:['english']});
```

- **MongoDB unterstützt MapReduce** <http://docs.mongodb.org/manual/tutorial/map-reduce-examples/>

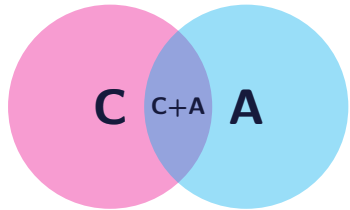
CAP Theorem

- Ein System kann nicht gleichzeitig die folgenden drei Eigenschaften unterstützen:
 - **Consistency** (Konsistenz)
 - **Availability** (Verfügbarkeit)
 - **Partition Tolerance**
(Daten/verarbeitung verteilt auf mehrere Maschinen, funktioniert auch wenn System in mehrere Partitionen zerfällt)



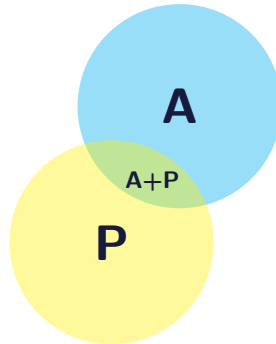
Consistent + Available

- Beispiel: Traditionelle (zentralisierte) Datenbanksysteme



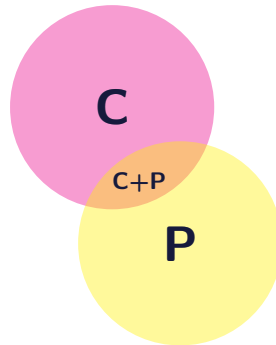
Partition Tolerant + Available

- Beispiel: Domain Name Service (DNS)



Consistent + Partition Tolerant

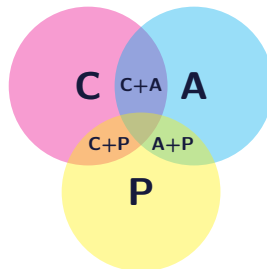
- Beispiel: Verteilte Datenbanken mit verteiltem Locking/Commit



Und nun?

Ohne "P" geht es nicht

- Es sind große Datenmengen zu verarbeiten
⇒ Horizontale Skalierung, viele Knoten.
D.h. das System muss damit umgehen können, dass Knoten oder Netzwerkkomponenten teilweise ausfallen bzw. nicht erreichbar sind.
- Also ist "P" gegeben. Was ist nun zu tun?

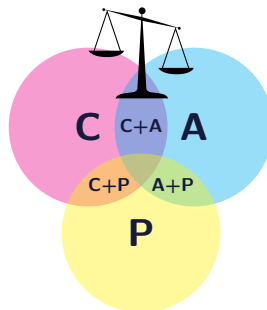


Und nun?

Ohne "P" geht es nicht

- Es sind große Datenmengen zu verarbeiten
⇒ Horizontale Skalierung, viele Knoten.
D.h. das System muss damit umgehen können, dass Knoten oder Netzwerkkomponenten teilweise ausfallen bzw. nicht erreichbar sind.
- Also ist "P" gegeben. Was ist nun zu tun?

Abwägung (Tradeoff) zwischen Konsistenz und Verfügbarkeit.



BASE

Basically Available

Soft State

Eventual Consistency

http://www.allthingsdistributed.com/2008/12/eventually_consistent.html

Tradeoff zwischen Consistency und Availability.

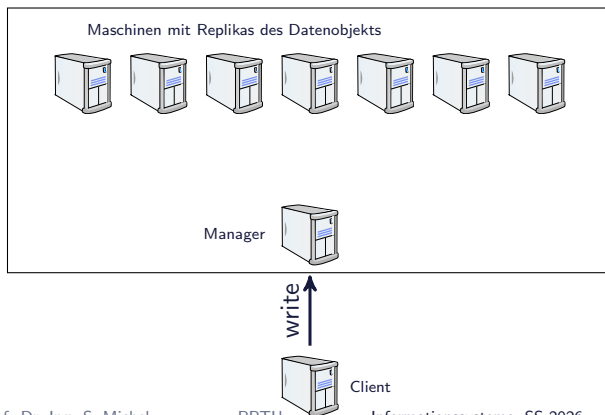
- Repliziere Daten, d.h. mehrere Versionen pro Datensatz
- Replikate werden auf Maschinen verteilt
- Sende Updates an alle Replikate aber warte nicht auf Acknowledgement.
- Lesen Daten von Teilmenge der Replikate.
- D.h. sehr effizient aber nicht unbedingt garantiert konsistent (man kann alte Antworten erhalten)
- Erst nach einiger Zeit konsistent (wenn alle Replikate aktualisiert wurden): **Eventual Consistency**

Bemerkung zu Consistency

- Consistency hier anders definiert als im DB-Kontext (in ACID).
- Hier, generell um Konsistenz von Replikaten (Kopien) einzelner Datenobjekte; dem Erzwingen bzw. nicht Erzwingen von garantierter Synchronisation.

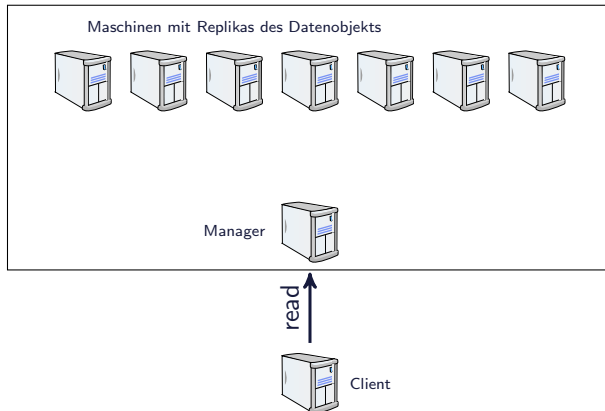
Veranschaulichung Inkonsistenz

- Client schickt **Schreibanweisung** an Manager
- Dieser schickt Schreibanweisung **an alle** Replikate.
- **Zeitstempel** (im einfachsten Fall) beschreibt Zeitpunkt des Schreibens.
- Und schickt Acknowledgement zurück an Client **sobald garantiert**
W Replikas aktualisiert wurden.



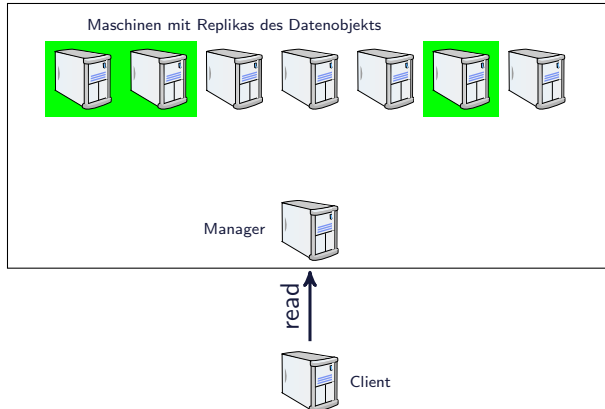
Veranschaulichung Inkonsistenz (2)

- Client schickt **Leseanweisung** an Manager.
- Dieser leitet Anweisung an R Replikas. D.h. **von N existierenden Replikaten werden R gelesen.**
- Antworten werden an Client weiter geleitet.



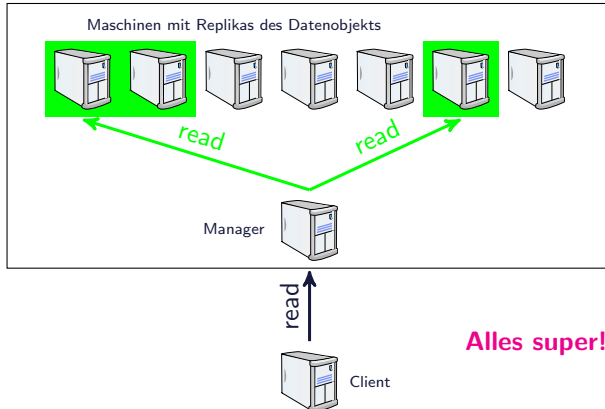
Veranschaulichung Inkonsistenz: Read

- $N = 7$ Replikate
- $W = 3$ und $R = 2$
- Grün markiert sind Replikate, die die neue Version des Objekts besitzen



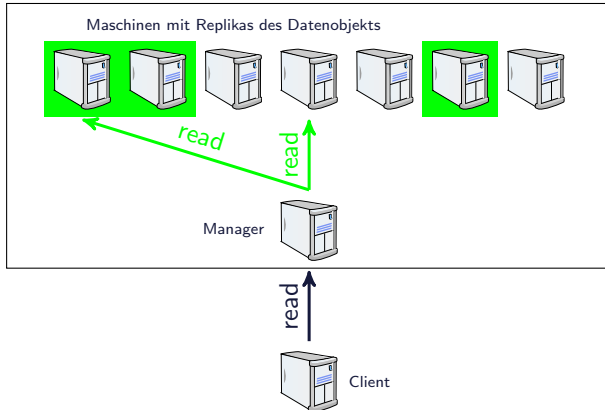
Veranschaulichung Inkonsistenz: Read - OK

- $N = 7$ Replikate
- $W = 3$ und $R = 2$
- Grün markiert sind Replikate, die die neue Version des Objekts besitzen



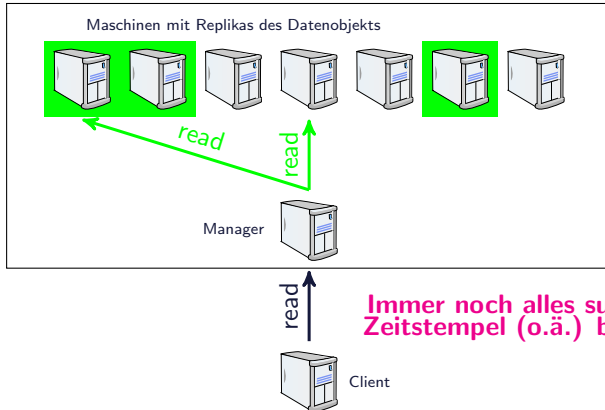
Veranschaulichung Inkonsistenz: Read - OK

- $N = 7$ Replikate
- $W = 3$ und $R = 2$
- Grün markiert sind Replikate, die die neue Version des Objekts besitzen



Veranschaulichung Inkonsistenz: Read - OK

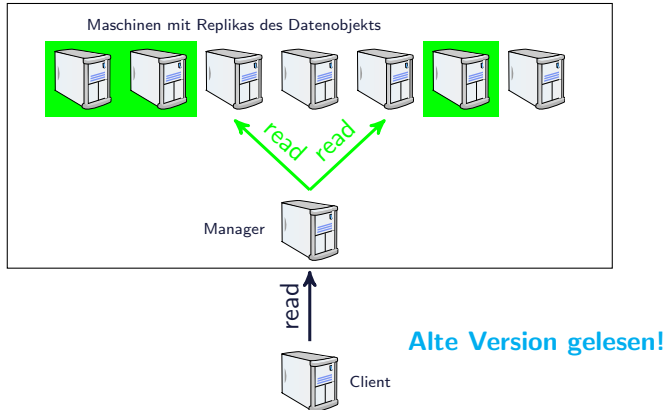
- $N = 7$ Replikate
- $W = 3$ und $R = 2$
- Grün markiert sind Replikate, die die neue Version des Objekts besitzen



**Immer noch alles super!
Zeitstempel (o.ä.) bestimmt neueste Version.**

Veranschaulichung Inkonsistenz: Read - Nicht Korrekt

- $N = 7$ Replikate
- $W = 3$ und $R = 2$
- Grün markiert sind Replikate, die die neue Version des Objekts besitzen



Write- bzw. Read-Optimized Strong Consistency

Wir möchten ein System, wie zuvor beschrieben, so konfigurieren, dass es jederzeit konsistent ist.

Write-Optimized

Read-Optimized

Write- bzw. Read-Optimized Strong Consistency

Wir möchten ein System, wie zuvor beschrieben, so konfigurieren, dass es jederzeit konsistent ist.

Write-Optimized

- $W=1, R=N$
- Beim Lesen finden wir garantiert die zuletzt geschriebene Version.

Read-Optimized

- $R=1, W=N$
- Da alle Knoten die aktuelle Version haben, reicht es von einem Knoten zu lesen.

Eventual Consistency und Konfigurationen

Konfiguration: $R + W > N$

- In diesem Fall kann garantiert werden, dass immer die aktuelle Version gelesen wird.
- Da sich die Mengen der aktualisierten Replikate und die der angefragten Replikate **überlappen müssen!**

Konfiguration: $R + W \leq N$

- In diesem Fall liegt Eventual Consistency vor.

Eventual Consistency

- Eventual (auf Deutsch: letztendlich) Consistency **beschreibt, dass nach einer gewissen Zeit alle Replikate aktualisiert sind.** Aber ab dem Schreibvorgang bis zu diesem Zeitpunkt ist nicht garantiert, dass Lesevorgänge die zuvor geschriebene Version sehen.

